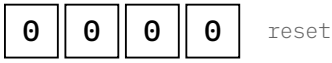
STATI



STATI

Dentro un processore RISC-V:
il datapath stato per stato
e i primi passi in assembly

MOBIUS

STATI · Dentro un processore RISC-V

© 2026 Mobius. Tutti i diritti riservati.
Nessuna parte di questo libro può essere riprodotta, archiviata o trasmessa in alcuna forma senza il permesso scritto dell'autore, a eccezione di brevi citazioni accompagnate dall'indicazione della fonte.

Prima edizione: 2026
ISBN: 979-8161139660

RISC-V è un marchio di RISC-V International. Intel, AMD, Arm, Apple e gli altri nomi di prodotti citati sono marchi dei rispettivi proprietari e compaiono solo a scopo descrittivo.

Gli esempi di codice sono forniti così come sono, senza garanzie. Prima della stampa, i programmi degli stati sono stati eseguiti nel simulatore RARS 1.6 e quelli dei laboratori nel simulatore QtRvSim 0.9.9.

Segnalazioni, errori e domande:
`s7627567@studenti.unige.it`
`soncesare@gmail.com`

Composto in Source Serif 4, IBM Plex Sans e IBM Plex Mono.

INDICE

Gli stati del libro

Ogni cerchio è uno stato; il filo è la strada che li collega.

Come leggere questo libro xiii

PARTE I · ACCENSIONE

0 Reset: che cos'è un calcolatore 3

0.1 Il reset 3

0.2 Gli organi del calcolatore 4

0.3 Una memoria o due? 6

0.4 Livelli, interfacce, virtualizzazione 8

0.5 ISA, ABI e API 9

0.6 Il bus 11

0.7 Il ciclo di un'istruzione 12

1 La lingua del processore: l'ISA RISC-V 15

1.1 RISC e CISC 15

1.2 Il caso RISC-V 18

1.3 I registri 19

1.4 La memoria vista dall'ISA 20

1.5 Calcolare 22

1.6 Decidere: i salti 23

1.7 Costanti grandi e indirizzi 25

1.8 Le pseudo-istruzioni 25

1.9 I formati delle istruzioni 26

1.10 Codificare e decodificare a mano 27

1.11 Chiedere aiuto al sistema operativo 29

1.12 Le estensioni 29

1.13 Un primo programma 30

PARTE II · IL VIAGGIO DI UN'ISTRUZIONE

2 **IF: prelevare l'istruzione** 35
2.1 Il datapath e il controllo 36
2.2 Due famiglie di circuiti 36
2.3 Il clock 38
2.4 Il program counter 39
2.5 La memoria delle istruzioni 40
2.6 Avanti di quattro 42
2.7 Quando non si va avanti: il multiplexer 42
2.8 Il prelievo, passo per passo 43

3 **ID: decodificare** 45
3.1 Tagliare l'istruzione 46
3.2 Il banco dei registri 46
3.3 Il generatore degli immediati 48
3.4 L'unità di controllo 50
3.5 Leggere prima di sapere 52
3.6 Esempio: la decodifica di `xor` 53

4 **EX: eseguire** 55
4.1 L'ALU vista dall'esterno 56
4.2 Chi sceglie l'operazione 57
4.3 Da dove arrivano gli operandi 58
4.4 L'ALU che calcola indirizzi 59
4.5 Decidere un salto 60
4.6 Overflow: il segnale che RISC-V non usa 62
4.7 EX al lavoro 64

5 **Dentro l'ALU: l'aritmetica** 67
5.1 Sommare un bit alla volta 68
5.2 Un'ALU da un bit 69
5.3 Il problema del riporto 71

5.4 Moltiplicare 72
5.5 Dividere 74
5.6 I numeri reali: la virgola mobile 75

6 **MEM: accedere alla memoria** 81
6.1 La memoria dati 82
6.2 Byte, mezze parole e parole 82
6.3 L'allineamento 84
6.4 Perché MEM è la fase più lenta 85
6.5 Una load e una store, passo per passo 86

7 **WB: chiudere il cerchio** 89
7.1 Che cosa si scrive 90
7.2 Il datapath completo 91
7.3 La tabella del controllo 91
7.4 Il viaggio completo di un'istruzione 93
7.5 Quanto dura un ciclo 94

8 **Multiciclo: il controllo come macchina a stati** 97
8.1 Spezzare l'istruzione in passi 98
8.2 Riusare i componenti 98
8.3 Il datapath multiciclo 98
8.4 Il controllo diventa una macchina a stati 100
8.5 Quanto è veloce 103
8.6 Cablato o microprogrammato 103
8.7 Verso la pipeline 104

PARTE III · TANTE ISTRUZIONI INSIEME

9 **Pipeline: la catena di montaggio** 109
9.1 Cinque fasi, cinque istruzioni 110
9.2 Latenza e throughput 111
9.3 Il datapath a pipeline 113
9.4 Il controllo viaggia con l'istruzione 115

9.5 Seguire le istruzioni 116
9.6 Quando le cose si complicano: i conflitti 117
9.7 Verso i conflitti 119

a **Conflitti sui dati: propagare e aspettare** **121**
a.1 Dipendenze e conflitti 122
a.2 La soluzione più semplice: aspettare 123
a.3 Propagare il risultato 124
a.4 Quando propagare non basta 126
a.5 L'unità di rilevamento dei conflitti 127
a.6 Il compilatore può aiutare 130

b **Salti ed eccezioni: indovinare e rimediare** **133**
b.1 Il prezzo di un salto 134
b.2 Annullare le istruzioni sbagliate 135
b.3 Decidere prima 135
b.4 Ritardare il salto 137
b.5 Indovinare: la previsione statica 138
b.6 Imparare dal passato: la previsione dinamica 139
b.7 Le eccezioni 141

c **Oltre la pipeline: più istruzioni per ciclo** **145**
c.1 La corsa alla frequenza 146
c.2 Più istruzioni per ciclo 147
c.3 Emissione statica: il compilatore decide 148
c.4 Emissione dinamica: l'hardware decide 150
c.5 Il multithreading 153
c.6 Un processore vero: Intel Nehalem 154
c.7 L'altro estremo: una sola istruzione 155

PARTE IV · LA MEMORIA

d **La cache e la memoria principale** **161**
d.1 Il divario e la gerarchia 162

d.2 La località 163
d.3 La cache a mappatura diretta 164
d.4 Quanto è grande una cache 166
d.5 Più libertà: le cache associative 168
d.6 Le scritture 170
d.7 Perché si sbaglia: le tre C 171
d.8 Quanto costano i miss 172
d.9 Cache a più livelli 173
d.10 Dentro la DRAM 175
d.11 Quando un bit si inverte 177

e **La memoria virtuale** 183
e.1 Tanti programmi, una memoria 184
e.2 Partizioni e registro base 184
e.3 Le pagine 185
e.4 Le pagine in RISC-V: Sv32 187
e.5 Il page fault 190
e.6 Il TLB 191
e.7 Il TLB e la cache insieme 193
e.8 Protezione e condivisione 194
e.9 Il viaggio di una `lw` 195

f **Più core: coerenza e sincronizzazione** 197
f.1 Più processori sullo stesso chip 198
f.2 La legge di Amdahl 198
f.3 Il problema della coerenza 200
f.4 Osservare il bus 201
f.5 Il protocollo MESI 202
f.6 La falsa condivisione 203
f.7 Sincronizzazione 204
f.8 La coerenza in Nehalem 207

PARTE V · IL MONDO ESTERNO

10 **Ingresso e uscita: il processore e il mondo** 211
10.1 Un mondo di dispositivi 212
10.2 Collegare i dispositivi 213
10.3 L'architettura di un calcolatore moderno 215
10.4 Comunicare con un dispositivo 216
10.5 Aspettare o essere avvisati 217
10.6 Il DMA 219
10.7 Il disco, da vicino 221
10.8 Il cerchio si chiude 222

PARTE VI · LABORATORIO

L0 **Gli strumenti** 227
L0.1 Simulare, emulare, compilare 228
L0.2 QtRvSim 229
L0.3 Una panoramica della finestra 230
L0.4 La prova del banco 232
L0.5 Le regole dell'assemblatore 234
L0.6 Gli altri strumenti 236
L0.7 Un programma per tutti 240

L1 **Primo programma** 245
L1.1 Il programma dello stato 0x1 246
L1.2 Un'istruzione alla volta 246
L1.3 La mappa della memoria 247
L1.4 Comunicare con il terminale 249
L1.5 Riconoscere gli errori 250
L1.6 Esercizi 250

L2 **If e cicli** 253
L2.1 Scegliere 253
L2.2 Ripetere 255
L2.3 Caratteri e stringhe 257

L2.4 Stampare un numero 258
L2.5 Esercizi 259

L3 **Vettori e matrici** 263
L3.1 Indice o puntatore 263
L3.2 Il prodotto scalare 265
L3.3 Matrici 265
L3.4 Le diagonali: versione lenta e versione veloce 266
L3.5 Esercizi 269

L4 **Funzioni e stack** 271
L4.1 Chiamare e tornare 271
L4.2 Chi salva che cosa 272
L4.3 Lo stack e il record di attivazione 274
L4.4 La ricorsione 276
L4.5 Il merge sort 278
L4.6 Esercizi 282

L5 **Vedere la macchina** 285
L5.1 La pipeline senza protezioni 286
L5.2 Stalli e riordino 287
L5.3 Il prezzo dei salti 288
L5.4 La cache 289
L5.5 I dispositivi 291
L5.6 Esercizi 294

L6 **Dal C all'assembly** 297
L6.1 Tradurre a mano 297
L6.2 Il lavoro del compilatore 299
L6.3 Misurare le ottimizzazioni 300
L6.4 Il linker 302
L6.5 Dall'assembly al C 304
L6.6 Il C e il sistema operativo 305
L6.7 Esercizi 306

Appendice A · Numeri binari, esadecimali e in complemento a 2 309

A.1 Basi e notazione posizionale 309
A.2 Esadecimale 310
A.3 Interi senza segno 310
A.4 Complemento a 2 311
A.5 Scorrimenti 312
A.6 Operazioni bit a bit e maschere 312

Appendice B · Scheda di riferimento RISC-V 313

B.1 Registri e formati 313
B.2 Le istruzioni 314
B.3 Pseudo-istruzioni 316
B.4 Chiamate di sistema 317
B.5 Registri di controllo 317

Appendice C · Soluzioni 319

Appendice D · Letture consigliate 365

D.1 I manuali 365
D.2 RISC-V 366
D.3 Approfondimenti 366
D.4 Per il piacere di leggere 366

Glossario 369

Ringraziamenti 373

Indice analitico 375



Come leggere questo libro

Questo libro nasce da un corso universitario di Architettura dei calcolatori e ha due obiettivi. Il primo è mostrarti che cosa accade all'interno di un processore mentre esegue un'istruzione, un componente alla volta. Il secondo è metterti in condizione di scrivere i tuoi primi programmi nel linguaggio che quel processore capisce davvero: l'assembly RISC-V. Per seguirlo basta sapere che cos'è un programma e avere qualche nozione di C; il binario e il complemento a 2 sono riassunti nell'appendice A.

Un processore è una macchina a stati

Una **macchina a stati finiti** è un sistema che in ogni istante si trova in uno fra pochi stati possibili e passa da uno all'altro seguendo regole precise, chiamate **transizioni**. Un semaforo è una macchina a stati: verde, giallo, rosso, e di nuovo verde.

Anche un processore è una macchina a stati, su scala molto più grande. In ogni istante il suo stato è fotografato da pochi elementi: l'indirizzo della prossima istruzione, il contenuto dei registri, il contenuto della memoria. A ogni colpo di clock avviene una transizione, e lo stato cambia. Tutto il resto, dai circuiti che eseguono le somme ai fili che trasmettono i segnali, serve solo a calcolare quale sarà lo stato successivo. Capire un processore significa capire queste transizioni.

Per questo il libro è organizzato nello stesso modo: ogni capitolo è uno *stato*, e si passa al successivo solo quando si è pronti.

Perché gli stati sono in esadecimale

Gli stati sono numerati in esadecimale, come gli indirizzi che incontrerai in ogni pagina: `0x0`, `0x1`, e così via fino a `0xf`. Sono sedici stati, e sedici valori

riempiono esattamente un registro a 4 bit: all'inizio di ogni capitolo vedi quel registro, con i bit dello stato corrente.

C'è però un diciassettesimo stato, `0x10`, dedicato all'Input/Output. Per scriverlo servono 5 bit: è un *overflow* voluto, perché l'I/O è proprio il punto in cui il processore esce dai propri confini e comunica con il mondo esterno.

Com'è fatto uno stato

Ogni stato si apre con il suo registro, il titolo e una striscia di cerchi: è la mappa dell'intero libro, con lo stato in cui ti trovi annerito. Subito sotto si trovano due elenchi.

Ingressi
: Che cosa serve sapere prima di cominciare, con il rimando allo stato in cui è spiegato.

Uscite
: Che cosa saprai fare alla fine.

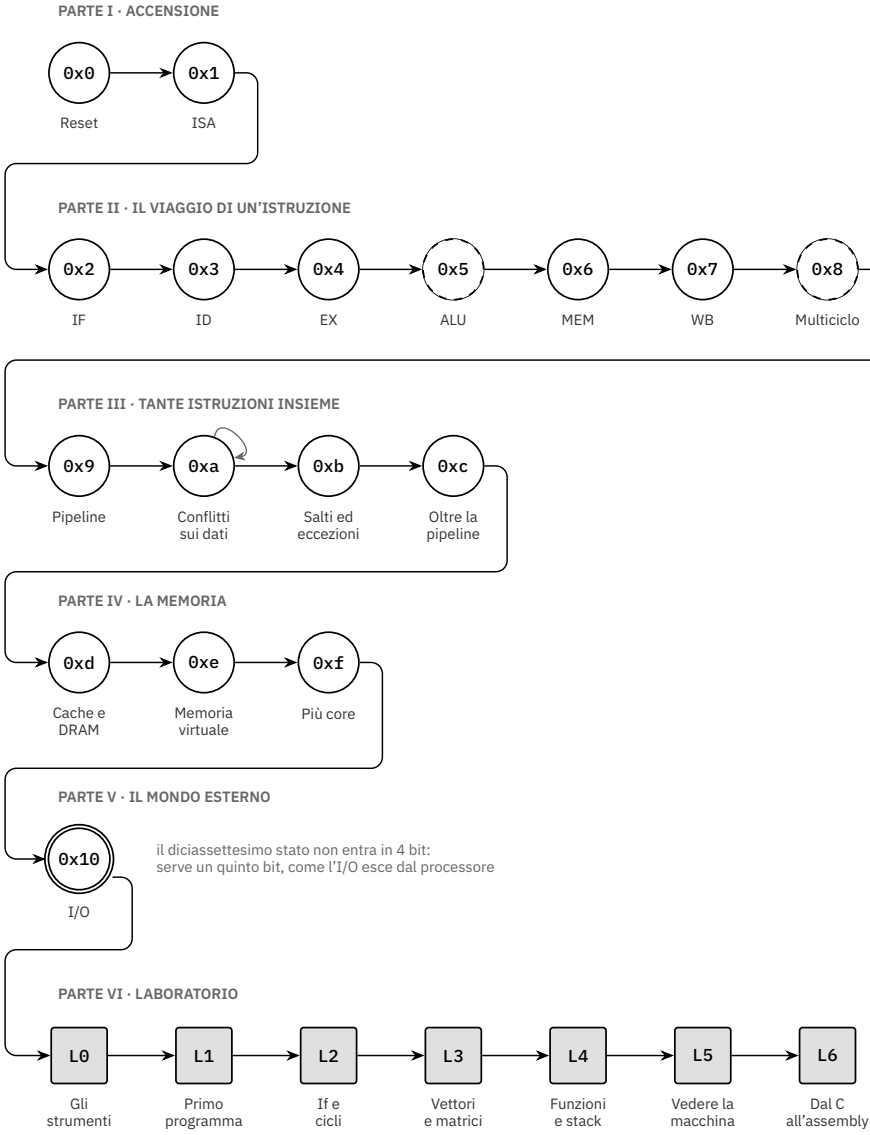
Ogni stato si chiude con una *condizione di transizione*: alcune domande di autoverifica. Se sai rispondere a tutte, passa allo stato successivo. Se una domanda ti mette in difficoltà, accanto c'è una freccia circolare, ↺ § 1.3 per esempio, che ti riporta alla sezione da rileggere: è la transizione di un automa che resta nello stesso stato finché la condizione non è soddisfatta. Le soluzioni sono in fondo al libro.

I due percorsi

Le parti dalla I alla V costruiscono il processore, dal linguaggio che interpreta (`0x1`) ai componenti che eseguono le istruzioni, fino alla memoria e all'I/O. La parte VI, il *Laboratorio*, è dedicata alla programmazione: un capitolo sugli strumenti, L0, e sei laboratori in cui si scrive, si esegue e si osserva codice assembly. Ogni laboratorio indica quali stati occorre aver letto, quindi puoi leggerlo anche in parallelo alla teoria. Se ti interessa soprattutto programmare, una strada possibile è `0x0`, `0x1`, poi L0 e i laboratori da L1 a L4, e solo dopo il resto. La mappa degli stati, poche pagine più avanti, mostra l'intera macchina.

Convenzioni

- Il codice, i nomi dei registri e le istruzioni sono in carattere a spaziatura fissa: `add t0, t1, t2`.



La mappa degli stati. Il libro come macchina a stati. I cerchi sono gli stati, le frecce le transizioni; i cerchi tratteggiati (`0x5` e `0x8`) si possono saltare alla prima lettura; i quadrati sono i laboratori. L'anello sopra `0xa` è una condizione non superata: si resta nello stato e si rilegge.

- I numeri esadecimali hanno il prefisso `0x` (`0x00524433`); quelli binari, quando servono, il pedice 2 (0110011_2).
- I registri sono indicati con il nome convenzionale (`a0`, `t1`, `sp`); il numero (`x10`) compare quando aiuta a capire la codifica.
- I termini inglesi sono in corsivo la prima volta che compaiono, accanto all'equivalente italiano; da lì in poi si usa quello più diffuso fra chi lavora nel campo.
- I riquadri con il punto esclamativo segnalano un *errore frequente*; quelli grigi sono *note* di approfondimento; quelli con un anno in nero, i *retroscena*, raccontano da dove viene un'idea e qualche episodio poco noto della storia del calcolo.

Gli strumenti

Per eseguire il codice basta un simulatore. I laboratori usano QtRvSim, il simulatore adottato nel corso, che mostra anche la pipeline e la cache. Il capitolo L0 spiega come installarlo e presenta gli altri strumenti, con le loro differenze: RARS, usato in alcuni esperimenti degli stati, Ripes, che anima il datapath, il compilatore GCC con l'emulatore QEMU, e Compiler Explorer, che mostra l'assembly generato da un compilatore C.

Se trovi un errore

Un libro tecnico contiene sempre qualche errore, per quanto accuratamente venga controllato. Se ne trovi uno, o se una spiegazione non ti è chiara, scrivimi a uno di questi indirizzi:

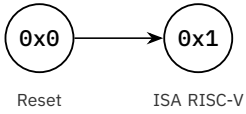
s7627567@studenti.unige.it
soncesare@gmail.com

PARTE

I

Accensione

Prima di aprire il processore, serve sapere che cosa stiamo guardando: da quali parti è fatto un calcolatore, come si separano i livelli di un sistema e quale lingua parla la macchina.



STATO

0x0

b3	b2	b1	b0
0	0	0	0

Reset: che cos'è un calcolatore



→ INGRESSI

- Nessuno: è lo stato iniziale.
- Sapere, a grandi linee, che cos'è un programma.

USCITE →

- Descrivere le parti di un calcolatore secondo von Neumann.
- Spiegare la differenza fra modello di von Neumann e Harvard.
- Distinguere astrazione e virtualizzazione.
- Indicare quali livelli separano ISA, ABI e API.
- Descrivere il ciclo che ogni istruzione percorre.

0.1 Il reset

Quando si accende un processore, la prima cosa che succede non è un calcolo. Un segnale chiamato **reset** porta la macchina in una condizione nota: il registro che indica la prossima istruzione da eseguire viene caricato con un indirizzo fissato dal costruttore, e da quel momento il processore comincia a leggere istruzioni dalla memoria, una dopo l'altra, senza fermarsi più. Prima del reset il contenuto dei circuiti è casuale; dopo, la macchina è in uno *stato* preciso e ogni passo successivo è determinato.

1978 **Ogni PC si sveglia nel 1978**

Anche il reset porta con sé la sua storia. Un processore x86 di oggi, appena acceso, si comporta come l'Intel 8086 del 1978: parte in una modalità a 16 bit, chiamata *modalità reale*. L'8086 cominciava dall'indirizzo `0xffff0`, sedici byte prima della fine del suo unico mebibyte di memoria; il processore di oggi va a cercare la prima istruzione a `0xfffffff0`, sedici byte prima della fine dei primi 4 GiB: lo stesso accorgimento, applicato a una memoria più grande. Solo in seguito il firmware lo porta, passo dopo passo, nelle modalità a 32 e a 64 bit. Perfino il nome dell'avvio, *boot*, è una battuta: viene da *bootstrap*, la linguetta sul retro degli stivali, e dall'idea impossibile di sollevarsi da terra tirandosi su per i propri stivali. Un calcolatore spento non sa fare nulla, nemmeno caricare il programma che gli insegnerebbe a caricare i programmi.

Questo libro parte nello stesso modo. Prima di aprire il processore e guardare i suoi componenti uno per uno, occorre stabilire che cosa sia un calcolatore, da quali parti sia composto e come si organizzi la complessità di un sistema che contiene miliardi di transistor. È il nostro reset.

0.2 Gli organi del calcolatore

Nel giugno 1945 John von Neumann scrisse un documento rimasto famoso, *First Draft of a Report on the EDVAC*, in cui descriveva come avrebbe dovuto essere organizzato un calcolatore elettronico a programma memorizzato. Lo chiamava «dispositivo» e lo divideva in *organi*, ognuno con una funzione precisa. Più di ottant'anni dopo, quella divisione descrive ancora qualunque calcolatore.

Unità aritmetica
: Esegue le operazioni elementari: somme, sottrazioni, moltiplicazioni, confronti. Oggi la chiamiamo **ALU** (*Arithmetic Logic Unit*).

Unità di controllo
: Decide quale operazione eseguire e quando, leggendo le istruzioni del programma e comandando le altre parti. È il «direttore d'orchestra»: da sola non calcola nulla, ma senza di essa nessuno saprebbe che cosa suonare. La sigla moderna è **CU** (*Control Unit*).

Memoria
: Conserva tutto ciò che serve al calcolo.

Ingresso e uscita
: Mettono il dispositivo in contatto con il mondo esterno: ricevono i dati da elaborare e restituiscono i risultati.

Unità aritmetica e unità di controllo, insieme, formano quello che oggi chia-

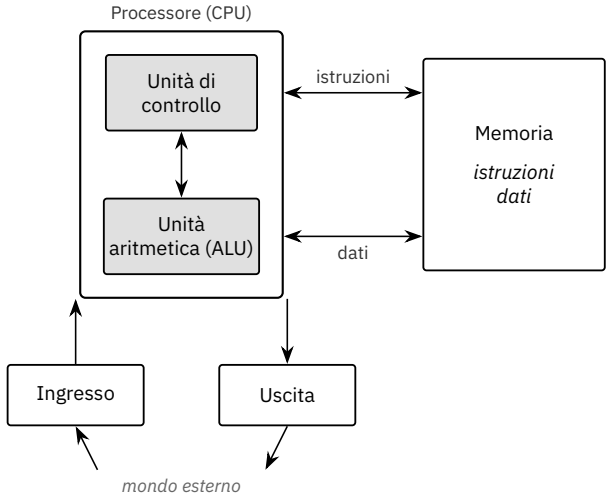


Figura 0.1 Gli organi di un calcolatore secondo von Neumann. Istruzioni e dati abitano la stessa memoria.

miamo **processore** o **CPU** (*Central Processing Unit*). La figura 0.1 mostra come sono collegati.

1945 Un rapporto, un nome solo

Il rapporto sull'EDVAC era una bozza interna, ma Herman Goldstine lo fece circolare con il solo nome di von Neumann, anche se molte idee venivano dal gruppo che a Filadelfia stava costruendo l'ENIAC, guidato da J. Presper Eckert e John Mauchly. La polemica non si è mai spenta del tutto, e il nome «architettura di von Neumann» è rimasto. Il rapporto ha anche un tratto sorprendente: descrive il calcolatore come un sistema nervoso. Calcolo, controllo e memoria corrispondono, scrive von Neumann, ai neuroni «associativi»; ingresso e uscita a quelli sensoriali e motori.

Che cosa deve ricordare la memoria

La memoria non contiene un solo tipo di informazione. Durante un calcolo deve conservare almeno quattro cose diverse:

- i *dati di partenza*, cioè il problema da risolvere;
- i *risultati intermedi*, valori provvisori che servono per arrivare alla fine e che spesso vengono riusati subito;
- i *risultati finali*;
- il *programma*, cioè la sequenza di istruzioni che dice come trasformare i

dati di partenza nei risultati.

L'idea davvero nuova del rapporto di von Neumann è l'ultima: il programma risiede in memoria *insieme* ai dati, codificato in binario come qualunque altro numero. È il concetto di **programma memorizzato**. Von Neumann ammetteva che le funzioni della memoria fossero molto diverse fra loro, ma scriveva che era «allettante trattare l'intera memoria come un unico organo». Da quella scelta discendono due conseguenze che accompagneranno tutto il libro. La prima è positiva: un programma può leggere e scrivere altri programmi come se fossero dati, ed è così che funzionano compilatori, caricatori e sistemi operativi. La seconda è un problema, e merita una sezione a parte.

1946 Programmare con i cavi

Prima del programma memorizzato, programmare significava ricablare. L'ENIAC, presentato nel 1946, conteneva circa 18 000 valvole, pesava quasi trenta tonnellate e si programmava collegando cavi e girando interruttori: passare da un problema all'altro poteva richiedere giorni. Quel lavoro era svolto da sei matematiche, Kay McNulty, Betty Jennings, Betty Snyder, Marlyn Wescoff, Fran Bilas e Ruth Lichterman, il cui contributo è stato riconosciuto pubblicamente solo decenni dopo.

0.3 Una memoria o due?

Una prima caratteristica accomuna qualunque calcolatore: il processore è veloce e la memoria è lenta. Nel modello di von Neumann, inoltre, la memoria è una sola, e attraverso di essa devono passare sia le istruzioni sia i dati. Per eseguire una somma con gli operandi in memoria, come nelle prime macchine, il processore deve leggere l'istruzione, poi leggere i due operandi, poi scrivere il risultato: quattro accessi in fila alla stessa memoria lenta, uno alla volta. Il canale fra processore e memoria diventa una strettoia, che dal 1977 si chiama **collo di bottiglia di von Neumann**: il nome si deve a John Backus, il padre del linguaggio FORTRAN, che lo usò nella lezione per il suo premio Turing.

La soluzione più diretta è usare due memorie separate, una per le istruzioni e una per i dati, ciascuna con il proprio collegamento: mentre si legge un'istruzione si può leggere o scrivere un dato. È il **modello Harvard**, così chiamato dall'Harvard Mark I, una macchina degli anni Quaranta che leggeva le istruzioni da un nastro di carta perforata e conservava i dati in contatori separati. La figura 0.2 mette a confronto i due modelli.

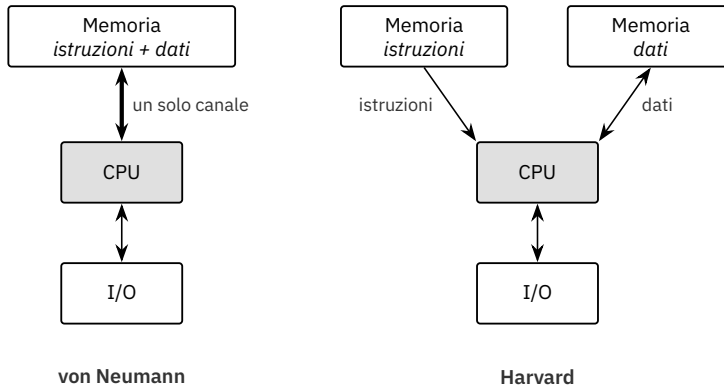


Figura 0.2 A sinistra il modello di von Neumann: istruzioni e dati passano dallo stesso canale. A destra il modello Harvard: due memorie, due canali, due accessi nello stesso momento.

1947 Il primo bug vero

L'Harvard Mark I, in funzione dal 1944, era lungo oltre quindici metri, pesava più di quattro tonnellate ed eseguiva circa tre somme al secondo. Sul suo successore, il Mark II, il 9 settembre 1947 il gruppo di Grace Hopper trovò una falena incastrata in un relè che bloccava i calcoli. La falena fu incollata nel registro di bordo con l'annotazione «First actual case of bug being found», il primo caso reale di un *bug* trovato. La parola *bug* per indicare un difetto esisteva già, ed è proprio per questo che la battuta funzionava. Il registro, falena compresa, è conservato allo Smithsonian di Washington.

La risposta dei processori moderni

Quale dei due modelli usano i processori di oggi? Entrambi. Visto dal programmatore, un processore moderno è una macchina di von Neumann: c'è un unico spazio di indirizzi, e un programma può trattare il codice come dati. Questa scelta non è dettata da pigrizia: a livello di programmazione separare dati e istruzioni non ha senso, ed è stata proprio la programmazione a guidare lo sviluppo delle architetture. Dentro il processore, però, nelle piccole memorie velocissime più vicine ai circuiti di calcolo (le *cache*, che studieremo nello stato 0xd), istruzioni e dati sono tenuti separati come nel modello Harvard. In breve: *von Neumann per tutto, tranne che nella memoria più interna, che è Harvard*.

> **Una semplificazione utile.** Nei capitoli sul datapath disegneremo spesso due memorie distinte, una per le istruzioni e una per i dati. Non è una contraddizione: sono le due cache di primo livello, che il programmatore non vede. Quando parleremo di «memoria» senza altri aggettivi, ricordati che le memorie sono due, ma per semplicità le trattiamo come se fossero una sola.

0.4 Livelli, interfacce, virtualizzazione

Un processore moderno contiene miliardi di transistor, un sistema operativo milioni di righe di codice. Nessuno può progettare o capire un oggetto del genere guardandolo tutto insieme. Il metodo che lo rende possibile si chiama **astrazione**: si divide il sistema in *livelli*, e si ragiona su un livello alla volta, partendo da quello più alto. Non si comincia dal transistor. Si comincia da «processore, memoria, I/O», poi si scende di un livello dentro un componente, senza dimenticare che tutti gli altri esistono.

Fra un livello e l'altro c'è un'**interfaccia**: l'insieme delle regole con cui il livello superiore usa quello inferiore. Un'interfaccia *ben definita* permette di usare un sistema senza sapere che cosa c'è dentro, come una scatola nera di cui si conoscono soltanto gli ingressi e le uscite. È un approccio che si ritrova ovunque in informatica: i protocolli di rete come TCP/IP sono livelli di astrazione sopra i cavi e le onde radio.

Un esempio quotidiano è il *file system*. Un disco è organizzato in settori e tracce, ma il sistema operativo li nasconde e mostra ai programmi soltanto dei *file*: sequenze di byte con un nome, organizzate in cartelle. Chi scrive un programma crea, legge e scrive file senza sapere nulla della costruzione fisica del disco (figura 0.3, a sinistra).

Il prezzo delle interfacce

Le interfacce hanno un costo: legano i livelli fra loro. Un programma scritto per una certa interfaccia funziona solo con quella; se l'interfaccia cambia, il sistema finisce in uno stato ignoto, e il programma smette di funzionare. Il problema si chiama mancanza di **interoperabilità**.

La risposta è la **virtualizzazione**: far comportare un sistema come se fosse un altro, offrendo l'interfaccia che il livello superiore si aspetta. Torniamo al disco: un unico disco fisico può essere diviso in più *dischi virtuali*, ciascuno dei quali è in realtà un file sul disco vero (figura 0.3, a destra). Chi lo usa vede un disco completo; un software si occupa di tradurre ogni accesso.

Lo stesso principio vale per sistemi interi. Una **macchina virtuale** è un programma che imita un calcolatore completo, processore compreso, e permette di eseguire un sistema operativo dentro un altro: è così, per esempio,

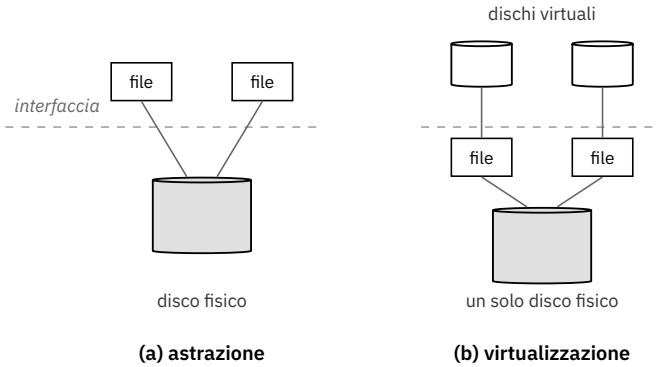


Figura 0.3 (a) L'astrazione offre un'interfaccia più semplice verso la stessa risorsa: i file al posto di settori e tracce. (b) La virtualizzazione offre, allo stesso livello, un'interfaccia uguale a quella di una risorsa diversa: due dischi che in realtà sono due file.

che si può usare Linux all'interno di Windows. Un simulatore come quelli che useremo nei laboratori è una macchina virtuale che imita un processore RISC-V su un calcolatore che RISC-V non è.

1972 Macchine virtuali prima dei PC

La virtualizzazione sembra un'idea recente, ma è più vecchia del personal computer. Nel 1972 IBM mise in vendita VM/370, un sistema che permetteva a un solo mainframe di eseguire insieme molte macchine virtuali, ciascuna con il proprio sistema operativo. Nasceva da un prototipo della fine degli anni Sessanta, CP-67, ed è l'antenato diretto delle macchine virtuali che oggi fanno funzionare il cloud.

0.5 ISA, ABI e API

Applichiamo l'idea dei livelli a un calcolatore completo. Dall'alto verso il basso troviamo i programmi applicativi, le librerie, il sistema operativo e infine l'hardware. Fra questi strati ci sono tre interfacce che conviene saper distinguere fin da subito (figura 0.4).

ISA, *Instruction Set Architecture*

È il confine fra software e hardware: l'elenco delle istruzioni che il processore sa eseguire, i registri che il programmatore vede, il formato dei dati, la dimensione degli indirizzi. Per chi programma è l'unica cosa che conta del processore; per chi lo progetta è un contratto da rispettare alla lettera. Una parte dell'ISA è *privilegiata*: alcune operazioni, come configurare la memoria o comunicare direttamente con i dispositivi, sono riservate al

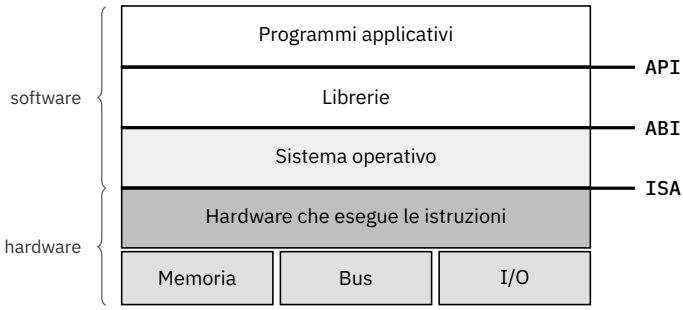


Figura 0.4 I livelli di un sistema di calcolo e le tre interfacce. L'API separa le applicazioni dalle librerie; l'ABI è ciò che vede un programma già compilato (le istruzioni non privilegiate più le chiamate al sistema operativo); l'ISA separa il software dall'hardware.

sistema operativo, che funge da gestore dell'hardware.

ABI, *Application Binary Interface*
: È ciò che vede un programma già compilato: le istruzioni non privilegiate dell'ISA più il modo di chiedere servizi al sistema operativo, oltre a convenzioni come l'uso dei registri nelle chiamate a funzione. Un eseguibile compilato non funziona su qualunque macchina: serve la stessa ISA *e* la stessa ABI, che in pratica vuol dire anche lo stesso sistema operativo.

API, *Application Programming Interface*
: È l'interfaccia delle librerie, a livello di codice sorgente: un insieme di funzioni già pronte che si usano chiamandole, senza sapere come sono fatte né in che linguaggio sono state scritte.

L'ISA è il livello che ci interessa di più, ed è anche il punto in cui le architetture si differenziano. Intel, AMD e RISC-V hanno ISA diverse; a volte cambiano perfino fra generazioni di processori della stessa casa. Ma la stessa ISA può avere molte realizzazioni hardware diverse, più o meno veloci, più o meno costose: Intel e AMD, per esempio, progettano processori molto diversi che eseguono lo stesso insieme di istruzioni x86. La realizzazione concreta di un'ISA si chiama **microarchitettura**, ed è ciò che studieremo dallo stato 0x2 in poi.

1964 La parola «architettura»

Fu IBM, nel 1964, a usare per la prima volta la parola *architettura* per un calcolatore. Gene Amdahl, Gerrit Blaauw e Fred Brooks, progettisti del System/360, la definirono come «gli attributi di un sistema visti dal programmatore», distinti dalla loro realizzazione fisica: è esattamente la nostra ISA. Il System/360 fu una famiglia di macchine di prezzo e velocità molto diversi che eseguivano tutte gli stessi programmi, e quella scelta è ancora viva: i mainframe IBM di oggi eseguono programmi scritti per il System/360 negli anni Sessanta.

Per capire come funziona un processore conviene scegliere un caso concreto. Il manuale dell'ISA di Intel supera le quattromila pagine, troppe per un primo studio. Useremo RISC-V, un'ISA moderna, aperta e abbastanza piccola da poter essere studiata per intero. Lo stato 0x1 è interamente dedicato a questa ISA.

0.6 Il bus

I componenti di un calcolatore comunicano attraverso un **bus**: un insieme di fili condivisi su cui viaggiano le informazioni. Un bus di sistema si divide in tre parti (figura 0.5).

- Il *bus dati* trasporta i dati veri e propri. Spesso ha la stessa larghezza dei registri del processore, quella che si intende quando si dice «a 32 bit»; ma non sempre: l'Intel 80386SX, un processore a 32 bit, aveva un bus dati da 16 bit, per costare meno.
- Il *bus indirizzi* indica dove si trova il dato: quale cella di memoria o quale dispositivo.
- Il *bus di controllo* trasmette i segnali che regolano lo scambio: se l'operazione è una lettura o una scrittura, quando i dati sono pronti, chi ha il diritto di usare il bus in quel momento.

La condivisione dei fili impone delle regole: i componenti non possono trasmettere tutti insieme, e serve un meccanismo di arbitraggio che faccia da semaforo. C'è anche un problema di velocità. Se componenti molto veloci e molto lenti fossero collegati allo stesso bus, tutto il sistema andrebbe alla velocità del più lento. Per questo i calcolatori usano più bus a velocità diverse, collegati da ponti: per anni l'organizzazione classica è stata quella con un *Northbridge*, per i componenti veloci come memoria e scheda video, e un *Southbridge*, per quelli lenti come USB e dischi. Oggi il Northbridge non esiste più come chip separato: dagli anni Duemila il controllore della memoria e il collegamento alla scheda video sono stati portati dentro il pro-

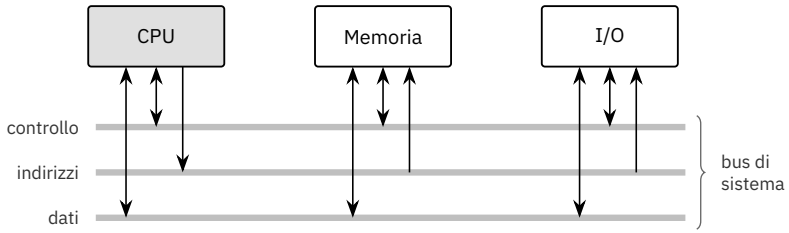


Figura 0.5 Il bus di sistema: dati, indirizzi e segnali di controllo viaggiano su tre gruppi di fili condivisi. Gli indirizzi partono dalla CPU; i dati vanno in entrambe le direzioni.

cessore. Riprenderemo l'argomento nello stato 0x10; fino ad allora possiamo concentrarci sui collegamenti dei dati e trascurare i bus.

0.7 Il ciclo di un'istruzione

Mettiamo insieme i pezzi. Il programma è in memoria, un'istruzione dopo l'altra. Come fa il processore a sapere quale eseguire? L'unità di controllo, da sola, non dispone di questa informazione: è in grado di interpretare un'istruzione, ma non ne conserva l'indirizzo. Serve un registro apposito, il **program counter** (**PC**), che contiene l'indirizzo della prossima istruzione da eseguire.

Il processore ripete sempre lo stesso ciclo:

1. legge dalla memoria l'istruzione che si trova all'indirizzo contenuto nel PC: è la fase di **prelievo**, in inglese *fetch*;
2. la **decodifica**: riconosce che tipo di istruzione è (una somma, una lettura, un salto) e prepara gli operandi;
3. la esegue;
4. se serve, accede alla memoria dei dati;
5. scrive il risultato nella destinazione prevista e, intanto, aggiorna il PC.

Nel caso più semplice le istruzioni si eseguono in sequenza, e il PC avanza ogni volta della lunghezza di un'istruzione: 4 byte nell'ISA che useremo. Ma un programma ha bisogno anche di scegliere e di ripetere, con costrutti come `if` e i cicli `for`. Per questo esistono istruzioni che scrivono nel PC un indirizzo diverso: i *salti*.

La figura 0.6 disegna questo ciclo come una macchina a stati. Non è un caso che le cinque fasi abbiano le sigle inglesi IF, ID, EX, MEM e WB: sono i nomi degli stati 0x2–0x7 di questo libro, e ognuna avrà il suo capitolo.

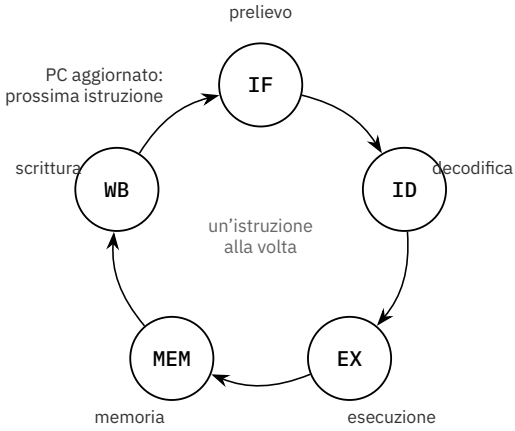


Figura 0.6 Il ciclo di un'istruzione disegnato come macchina a stati. Ogni fase è uno stato; dopo la scrittura si ricomincia con l'istruzione indicata dal PC. Nella parte II ogni fase avrà il suo capitolo.

Che cosa forma lo stato di un calcolatore

Durante tutto il ciclo i circuiti lavorano senza sosta: i segnali elettrici si propagano, i valori cambiano, per un attimo le uscite sono perfino sbagliate. Nulla di tutto questo conta finché il risultato non viene *scritto*. Lo **stato** di un calcolatore, in un dato istante, è costituito soltanto dagli elementi di memoria: il PC, i registri e la memoria. Se fotografi questi tre, sai esattamente dove si trova il programma e puoi farlo ripartire da lì.

È proprio ciò che fa il sistema operativo quando esegue tanti programmi «contemporaneamente» con un solo processore: li esegue a turno e, quando ne interrompe uno, ne salva lo stato, come in una fotografia, per ripristinarlo al turno successivo. Questa idea, *lo stato è ciò che è stato scritto*, tornerà in quasi tutti i capitoli. Ma prima di aprire il processore occorre conoscere il linguaggio in cui sono scritte le istruzioni che esegue.

Condizione di transizione uscita verso `0x1`

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Quali sono gli organi di un calcolatore nel modello di von Neumann, e che cosa fa ciascuno? ↺ § 0.2

2 Che cosa intendeva von Neumann con «programma memorizzato», e perché è un'idea potente? ↺ § 0.2

3 Perché una memoria unica per istruzioni e dati diventa un collo di bottiglia? Come lo risolve il modello Harvard? ↺ § 0.3

4 Un processore moderno è una macchina di von Neumann o Harvard? ↺ § 0.3

5 Qual è la differenza fra astrazione e virtualizzazione? Fai un esempio per ciascuna. ↺ § 0.4

6 Un programma compilato per Linux su un processore RISC-V funziona su un altro sistema operativo con lo stesso processore? Perché? ↺ § 0.5

7 A che cosa servono il bus dati, il bus indirizzi e il bus di controllo? ↺ § 0.6

8 Che cosa contiene il PC, e che cosa forma lo stato di un calcolatore? ↺ § 0.7

STATO

0x1

0 0 0 1

b3 b2 b1 b0

La lingua del processore: l'ISA RISC-V



→ INGRESSI

- Che cos'è un'ISA e dove si colloca (stato 0x0, § 0.5).
- Il ciclo di un'istruzione e il ruolo del PC (§ 0.7).
- Numeri binari, esadecimali e complemento a 2 (appendice A).

USCITE →

- Spiegare che cosa distingue un'architettura RISC da una CISC.
- Leggere e scrivere le istruzioni di base di RISC-V.
- Usare i registri con i loro nomi e ruoli.
- Codificare e decodificare a mano un'istruzione a 32 bit.
- Comprendere il funzionamento di un primo programma assembly completo.

1.1 RISC e CISC

Ogni processore esegue soltanto le istruzioni della propria ISA, ma non tutte le ISA sono pensate allo stesso modo. La storia delle architetture si divide in due grandi filosofie, che conviene conoscere prima di studiarne una.

CISC: istruzioni ricche, programmi corti

Negli anni Settanta la memoria era costosissima e lenta, e i compilatori erano ancora rudimentali. Conveniva quindi che un programma occupasse pochi byte e che fosse facile da scrivere a mano. Nacquero così le architetture **CISC**

(*Complex Instruction Set Computer*), in cui la complessità risiede nell'hardware.

- Una singola istruzione può svolgere più operazioni: leggere un valore dalla memoria, sommarlo a un altro e riscrivere il risultato in memoria, tutto insieme.
- Le istruzioni hanno lunghezza variabile: alcune occupano un byte, altre molti. Il processore deve analizzare ogni istruzione per determinare dove termina, e la decodifica diventa lenta e difficile da progettare.
- Molte istruzioni richiedono più cicli di clock.

L'esempio classico è la famiglia x86 di Intel e AMD, che si trova nella maggior parte dei PC fissi e portatili.

RISC: istruzioni semplici, hardware veloce

All'inizio degli anni Ottanta David Patterson a Berkeley e John Hennessy a Stanford osservarono che i processori CISC usavano raramente le loro istruzioni più complesse: i compilatori preferivano quelle semplici. Proposero allora di spostare la complessità dall'hardware al software, cioè al compilatore, e di tenere il processore il più semplice, snello e veloce possibile. È la filosofia **RISC** (*Reduced Instruction Set Computer*).

- Ogni istruzione svolge una sola operazione elementare; un'operazione complessa diventa una sequenza di istruzioni semplici.
- Le operazioni aritmetiche operano solo sui registri. L'accesso alla memoria avviene soltanto tramite due tipi di istruzioni: *load* (carica un dato dalla memoria in un registro) e *store* (salva un registro in memoria). Per questo un'architettura RISC si dice **load/store**.
- Tutte le istruzioni hanno la stessa lunghezza, per esempio 32 bit: il processore individua subito dove termina un'istruzione e la decodifica è immediata.
- Servono molti registri, perché tutti i calcoli transitano per i registri.
- La semplicità permette di eseguire, in media, circa un'istruzione per ciclo di clock (vedremo come nella parte III).

Tabella 1.1 RISC e CISC a confronto.

	RISC	CISC
Istruzioni	semplici, una sola operazione	complesse, più operazioni insieme
Lunghezza	fissa (32 bit in RISC-V)	variabile, da 1 a molti byte
Memoria	solo con load e store	anche dentro le istruzioni di calcolo
Registri	molti (32 in RISC-V)	pochi (16 in x86-64)
Decodifica	semplice e veloce	complessa
Esempi	RISC-V, Arm, MIPS	x86 (Intel, AMD)

1980 Dal laboratorio IBM al premio Turing

Il primo processore costruito secondo questa filosofia non nacque in un'università ma in IBM. Dal 1974 il gruppo di John Cocke lavorava a un controllore per centrali telefoniche, e nel 1980 il prototipo, chiamato IBM 801, dimostrò che istruzioni semplici eseguite in un ciclo battevano quelle complesse; Cocke ricevette il premio Turing nel 1987. Il nome RISC arrivò poco dopo da Berkeley, dove Patterson guidava il progetto Berkeley RISC, mentre Hennessy a Stanford lavorava al MIPS. Le loro idee sono oggi dentro miliardi di dispositivi, e insieme hanno ricevuto il premio Turing 2017. Sono anche gli autori dei manuali su cui hanno studiato generazioni di ingegneri.

Il problema del legacy

Se RISC è più efficiente, perché i nostri PC usano ancora x86? Per il **legacy**: la necessità che l'hardware nuovo continui a eseguire i programmi e i sistemi operativi vecchi. Due storie degli anni Duemila mostrano quanto pesi.

- Nel 2001 Intel, insieme a HP, lanciò Itanium (IA-64), un'architettura a 64 bit completamente nuova. Non eseguiva in modo efficiente il vasto software x86 a 32 bit già esistente, ed era difficile da programmare: non ebbe successo, tanto che la stampa specializzata lo soprannominò «Itanic», e le ultime consegne risalgono al 2021.
- Nel 2003 AMD scelse la strada opposta: estese l'x86 a 32 bit già esistente fino a 64 bit, aggiungendo funzioni senza togliere nulla. L'architettura AMD64 (o x86-64) è diventata lo standard del mercato, adottata anche da Intel.

La conseguenza è che oggi la distinzione netta fra RISC e CISC è quasi scomparsa nei processori di fascia alta. Un processore Intel o AMD è CISC «all'esterno», per eseguire tutto il software x86; ma «all'interno» un decodificatore molto sofisticato spezza ogni istruzione complessa in **micro-operazioni** semplici, del tutto simili a istruzioni RISC, che vengono poi eseguite da un motore

interno di tipo RISC. Si ottiene la velocità del mondo RISC senza perdere la compatibilità del mondo CISC. Vedremo questo meccanismo da vicino nello stato 0xc.

1985 Il processore che girava senza corrente

Fra i primi a credere nel RISC ci fu una piccola azienda inglese, Acorn, che progettò un processore per i propri computer: l'ARM, da *Acorn RISC Machine*, opera di Sophie Wilson e Steve Furber. Quando il primo chip arrivò dalla fabbrica, il 26 aprile 1985, Furber lo collegò alla scheda di prova e il multimetro segnò un consumo pari a zero. La scheda aveva un guasto e l'alimentazione non arrivava affatto: il processore funzionava con le sole correnti di dispersione dei circuiti collegati. Quel consumo minuscolo, nato quasi per caso, è il motivo per cui oggi quasi tutti gli smartphone contengono un processore Arm.

1.2 Il caso RISC-V

RISC-V (si pronuncia «risk five») nasce nel 2010 all'Università della California a Berkeley, nel gruppo di Krste Asanović e David Patterson, come ISA per la ricerca e la didattica. Il «V» ricorda che è la quinta generazione di progetti RISC di Berkeley. Due caratteristiche la rendono diversa da tutte le ISA che l'hanno preceduta.

È aperta
: Le specifiche sono pubbliche e chiunque può progettare un processore RISC-V senza pagare licenze. Oggi sono gestite da una fondazione, RISC-V International. È il primo grande successo di un'ISA non proprietaria.

È modulare
: C'è una base minima di istruzioni intere, a cui si aggiungono *estensioni* opzionali: moltiplicazione e divisione, virgola mobile, operazioni atomiche e altre (§ 1.12). Un microcontrollore per una lavatrice può usare solo la base; un processore per server, tutte le estensioni.

2010 Un progetto da tre mesi

RISC-V doveva essere un lavoro breve. Nel 2010 Krste Asanović e alcuni suoi dottorandi, fra cui Yunsup Lee e Andrew Waterman, cercavano un'ISA semplice e libera da usare nella ricerca e nei corsi. Quelle commerciali erano troppo complicate o vincolate da licenze, e il gruppo decise di scriverne una propria in un progetto di tre mesi, durante l'estate. Il progetto estivo è diventato uno standard internazionale: nel 2020 la fondazione che lo gestisce si è trasferita in Svizzera ed è diventata RISC-V International.

Proprio la libertà dal legacy ha fatto la fortuna di RISC-V nei *sistemi embed-*

ded, dai sensori ai controllori industriali: in quel mondo hardware e software vengono spesso sviluppati su misura per una singola applicazione, e non c'è software precedente da eseguire. Da lì RISC-V si è esteso a processori sempre più potenti.

In questo libro useremo **RV32I**, la base a 32 bit, più l'estensione M per moltiplicazione e divisione: è ciò che offrono i simulatori del corso. «32 bit» significa che i registri e gli indirizzi sono larghi 32 bit; la variante RV64I ha registri da 64 bit e qualche istruzione in più, ma la logica è identica.

> **Quanto è lunga un'istruzione?** In RV32I ogni istruzione occupa 32 bit, cioè 4 byte. Ma l'ISA è pensata per crescere: i due bit meno significativi indicano subito al processore la lunghezza dell'istruzione. Se valgono entrambi 1, l'istruzione è lunga 32 bit; altri valori indicano le istruzioni compresse a 16 bit dell'estensione C. Il processore la determina prima ancora di decodificare il resto.

1.3 I registri

Proviamo a immaginare un'ISA senza registri, in cui ogni istruzione opera direttamente sulla memoria. Una somma dovrebbe indicare tre indirizzi: i due operandi e la destinazione. Con indirizzi da 32 bit servirebbero 96 bit solo per gli indirizzi, oltre al codice dell'operazione. E ogni somma richiederebbe tre accessi alla memoria, che è lenta.

La soluzione è il **banco dei registri** (*register file*): una memoria piccolissima, costruita con circuiti velocissimi dentro il processore, capace di lavorare alla stessa velocità dell'ALU. È veloce proprio perché è piccola. RISC-V ha 32 registri da 32 bit, chiamati da `x0` a `x31`. Per indicarne uno bastano 5 bit, perché $2^5 = 32$: un'istruzione con tre registri impiega 15 bit invece di 96.

C'è un compromesso. Più registri renderebbero i programmi più veloci, ma servirebbero più bit per indicarli, e in un'istruzione di lunghezza fissa resterebbe meno spazio per il resto: il tipo di operazione, le costanti. Trentadue è il punto di equilibrio scelto da RISC-V.

Nomi e ruoli

I registri sono tutti uguali per l'hardware, con un'eccezione: `x0` vale sempre zero. Si può leggere, e restituisce 0; si può scrivere, e la scrittura viene ignorata. Sembra uno spreco, ma vedremo che è utilissimo.

Per tutti gli altri, una *convenzione* stabilisce chi li usa e per che cosa, così che il codice scritto da persone e compilatori diversi possa collaborare. Ogni registro ha quindi anche un nome simbolico, riportato nella tabella 1.2, e

Tabella 1.2 I 32 registri di RISC-V e i loro nomi convenzionali. I registri `gp` e `tp` sono inizializzati una volta per tutte dall'ambiente di esecuzione, e il codice ordinario non li modifica.

Registro	Nome	Uso	Chi lo preserva
`x0`	`zero`	costante zero	—
`x1`	`ra`	indirizzo di ritorno	chi chiama
`x2`	`sp`	puntatore allo stack	chi è chiamato
`x3`	`gp`	puntatore ai dati globali	non si modifica
`x4`	`tp`	puntatore al thread	non si modifica
`x5`–`x7`	`t0`–`t2`	temporanei	chi chiama
`x8`	`s0` / `fp`	salvato / puntatore al frame	chi è chiamato
`x9`	`s1`	salvato	chi è chiamato
`x10`–`x11`	`a0`–`a1`	argomenti e valori restituiti	chi chiama
`x12`–`x17`	`a2`–`a7`	argomenti	chi chiama
`x18`–`x27`	`s2`–`s11`	salvati	chi è chiamato
`x28`–`x31`	`t3`–`t6`	temporanei	chi chiama

in assembly si usano quasi sempre i nomi simbolici. Il significato di «chi lo preserva» sarà chiaro nel laboratorio L4, dedicato alle funzioni.

Il PC non fa parte dei 32 registri: è un registro a sé, che le istruzioni non possono leggere o scrivere direttamente, ma solo attraverso i salti.

1.4 La memoria vista dall'ISA

Per l'ISA la memoria è un lunghissimo vettore di *byte*, ciascuno con il suo indirizzo: con indirizzi da 32 bit ce ne sono 2^{32}, cioè 4 GiB. Ogni cella contiene 8 bit, ma i dati che il processore usa di solito sono più grandi, quindi occupano celle consecutive:

- un *byte*: 8 bit, una cella;
- una *mezza parola* (*halfword*): 16 bit, due celle;
- una **parola** (*word*): 32 bit, quattro celle.

Leggere una parola significa leggere 4 byte alla volta. Per questo, nell'esecuzione sequenziale, il PC avanza di 4: ogni istruzione occupa una parola.

L'ordine dei byte

Se una parola occupa quattro celle, in quale ordine vi sono collocati i suoi byte? Esistono due convenzioni (figura 1.1). Nella **little-endian** all'indirizzo più basso è collocato il byte meno significativo; nella **big-endian** vi è collocato il più significativo. RISC-V è little-endian: la specifica ammette anche accessi ai dati big-endian, come opzione che quasi nessun processore realizza, ma

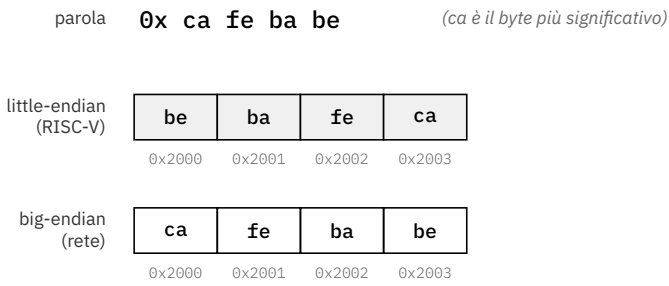


Figura 1.1 La parola `0xcafebabe` memorizzata a partire dall'indirizzo `0x2000`. In RISC-V, little-endian, la prima cella contiene il byte meno significativo.

le istruzioni sono sempre memorizzate in little-endian, e little-endian sono tutti i simulatori di questo libro. Dentro il processore la differenza non si nota mai; diventa importante quando i dati escono dalla macchina, per esempio in rete, dove per tradizione si usa la big-endian: se mittente e destinatario non sono d'accordo, i byte arrivano invertiti.[1]

1726 Le uova di Lilliput

I nomi *big-endian* e *little-endian* vengono da un romanzo. Nei *Viaggi di Gulliver* di Jonathan Swift, pubblicati nel 1726, Lilliput e Blefuscu combattono una guerra lunghissima, nata da una disputa su quale estremità si debba rompere un uovo, la grande o la piccola. Il 1º aprile 1980 l'informatico Danny Cohen prese in prestito quei nomi in un articolo intitolato *On Holy Wars and a Plea for Peace*, «sulle guerre sante e un appello alla pace», per sostenere che anche la disputa sull'ordine dei byte era, in fondo, una questione di gusti.

Allineamento

Le parole sono memorizzate di solito a indirizzi multipli di 4, le mezze parole a indirizzi pari: si dice che sono **allineate**. La specifica di RISC-V lascia libero chi costruisce il processore di gestire o meno gli accessi non allineati; molti simulatori, RARS compreso, li rifiutano con un errore. Conviene quindi considerarli vietati.

Load e store

Essendo un'architettura load/store, RISC-V usa istruzioni dedicate per spostare i dati fra registri e memoria (tabella 1.3). Tutte calcolano l'indirizzo nello

[1] Il valore della figura non è casuale: `0xcafebabe` è il «numero magico» con cui comincia ogni file compilato di Java. James Gosling lo scelse cercando parole esadecimali che cominciassero per `cafe`, in ricordo di un locale di Palo Alto dove il suo gruppo andava a pranzo.

Tabella 1.3 Le istruzioni di trasferimento fra memoria e registri.

Istruzione	Effetto
`lw rd, off(rs1)`	carica una parola
`lh rd, off(rs1)`	carica una mezza parola ed estende il segno
`lhu rd, off(rs1)`	carica una mezza parola e riempie di zeri
`lb rd, off(rs1)`	carica un byte ed estende il segno
`lbu rd, off(rs1)`	carica un byte e riempie di zeri
`sw rs2, off(rs1)`	salva in memoria la parola contenuta in `rs2`
`sh rs2, off(rs1)`	salva i 16 bit meno significativi di `rs2`
`sb rs2, off(rs1)`	salva gli 8 bit meno significativi di `rs2`

stesso modo: il contenuto di un registro *base* più una costante con segno da 12 bit, lo *spiazzamento* (*offset*). La scrittura

```
lw   t1, 8(t0)       # t1 ← parola all'indirizzo t0 + 8
```

si legge «carica in `t1` la parola che si trova 8 byte dopo l'indirizzo contenuto in `t0`». È lo stesso calcolo che il C fa per trovare l'elemento di un vettore: indirizzo di partenza più spiazzamento. L'istruzione non ha abbastanza bit per contenere un indirizzo completo da 32 bit, e così l'indirizzo viene calcolato a partire da un registro.

Quando si carica un dato più piccolo di 32 bit, i bit rimanenti del registro devono essere riempiti. Le versioni normali (`lb`, `lh`) considerano il dato un numero con segno e copiano il bit del segno in tutti i bit liberi a sinistra: è l'**estensione del segno**, e grazie al complemento a 2 non cambia il valore del numero. Le versioni *unsigned* (`lbu`, `lhu`) riempiono invece di zeri. Se in memoria si trova il byte `0x80`:

```
lb   t1, 0(t0)       # t1 = 0xffffff80  (cioè -128)
lbu  t2, 0(t0)       # t2 = 0x00000080  (cioè 128)
```

1.5 Calcolare

Le istruzioni aritmetiche e logiche operano su due operandi e producono un risultato in un registro. Il secondo operando può essere un registro oppure una costante scritta dentro l'istruzione stessa, detta **immediato**: le istruzioni il cui nome finisce per `i`, come `addi` o `xori`, usano un immediato da 12 bit con segno, cioè un valore da −2048 a +2047 (tabella 1.4).

Seguono alcune osservazioni su istruzioni che a prima vista possono sembrare insolite.

Tabella 1.4 Le istruzioni aritmetiche e logiche di RV32I.

Istruzione	Effetto
`add rd, rs1, rs2`	$rd \leftarrow rs1 + rs2$
`sub rd, rs1, rs2`	$rd \leftarrow rs1 - rs2$
`and`, `or`, `xor`	operazioni logiche bit a bit
`sll rd, rs1, rs2`	scorrimento a sinistra di `rs2` posizioni
`srl rd, rs1, rs2`	scorrimento a destra, entrano zeri
`sra rd, rs1, rs2`	scorrimento a destra, entra il bit del segno
`slt rd, rs1, rs2`	$rd \leftarrow 1$ se $rs1 < rs2$ con segno, altrimenti 0
`sltu rd, rs1, rs2`	come `slt`, ma senza segno
`addi`, `andi`, `ori`, `xori`, `slti`, `sltiu`	come sopra, con un immediato da 12 bit
`slli`, `srli`, `srai`	scorrimenti di una quantità fissa (0–31)

- Non esiste `subi`: per sottrarre una costante si somma il suo opposto, `addi t0, t0, -1`.
- Con AND, OR e XOR si può costruire qualunque altra funzione logica: la NOT, per esempio, è uno XOR con tutti 1.
- Gli **scorrimenti** (*shift*) fanno scorrere i bit di un registro. Far scorrere un valore a sinistra di k posizioni equivale a moltiplicare per 2^k, ed è molto più veloce di una moltiplicazione. Esistono anche in C, con gli operatori « e », perché il C nacque per scrivere un sistema operativo, Unix, dove manipolare i singoli bit era essenziale.
- `slt` e `sltu` (*set less than*) scrivono nel registro di destinazione un valore booleano, 0 o 1. Servono a calcolare le condizioni di `if` e cicli composti. La versione *unsigned* interpreta i registri come numeri sempre non negativi: il bit che prima indicava il segno diventa un bit di valore.

1.6 Decidere: i salti

Un programma che esegue le istruzioni sempre in sequenza non può fare scelte. I **salti** cambiano il flusso scrivendo nel PC un indirizzo diverso da quello dell'istruzione successiva. RISC-V ne ha due famiglie.

Salti condizionati

I **salti condizionati** (*branch*) confrontano due registri e saltano solo se la condizione è vera; altrimenti si prosegue con l'istruzione successiva, quella a $PC + 4$ (tabella 1.5).

Come si indica la destinazione? Un indirizzo completo non trova posto nell'istruzione. Si potrebbe usare un indirizzo corto completato con zeri,

Tabella 1.5 I salti condizionati. L'etichetta indica la destinazione.

Istruzione	Salta se
`beq rs1, rs2, et`	$rs1 = rs2$
`bne rs1, rs2, et`	$rs1 \neq rs2$
`blt rs1, rs2, et`	$rs1 < rs2$, con segno
`bge rs1, rs2, et`	$rs1 \geq rs2$, con segno
`bltu rs1, rs2, et`	$rs1 < rs2$, senza segno
`bgeu rs1, rs2, et`	$rs1 \geq rs2$, senza segno

ma allora si potrebbe saltare solo nelle zone più basse della memoria. La soluzione di RISC-V è il salto *relativo al PC*: l'istruzione contiene la *distanza* fra sé e la destinazione, e il processore la somma al PC. Funziona perché i salti di un `if` o di un ciclo `for` portano quasi sempre a poche decine di istruzioni di distanza.

Per tornare indietro, per esempio per ripetere un ciclo, la distanza deve essere negativa. Il complemento a 2 rende la cosa naturale: l'immediato viene esteso con il segno fino a 32 bit e sommato al PC, e la somma di un numero negativo porta all'indietro. La distanza è sempre pari, perché le istruzioni (anche quelle compresse da 16 bit) iniziano sempre a un indirizzo pari; il bit meno significativo non viene memorizzato, e così i 12 bit disponibili coprono un intervallo di circa ±4 KiB.

!

Non confondere RISC-V con MIPS. Molti libri e molti materiali didattici mostrano figure dell'architettura MIPS, la «sorella maggiore» di RISC-V. Si somigliano, ma non sono uguali. In MIPS l'offset dei salti è espresso in multipli di 4 byte e viene fatto scorrere a sinistra di due posizioni; in RISC-V è espresso in multipli di 2 byte e scorre di una sola posizione. In MIPS i registri salvati sono `s0`–`s7`; in RISC-V sono `s0`–`s11`. In MIPS i registri sorgente si chiamano `rs` e `rt`; in RISC-V `rs1` e `rs2`.

Salti incondizionati

I salti incondizionati avvengono sempre e servono soprattutto a chiamare le funzioni. Chiamare una funzione significa saltare al suo codice, ma anche memorizzare il punto di partenza, per potervi tornare alla fine. Per questo l'istruzione si chiama **jump and link**:

- `jal rd, et` salva in `rd` l'indirizzo dell'istruzione successiva ($PC + 4$), il *link*, e salta all'etichetta. Anche qui la distanza è relativa al PC, con un intervallo di circa ±1 MiB.
- `jalr rd, off(rs1)` (*jump and link register*) esegue la stessa operazione, ma

salta all'indirizzo *rs1* + *off*: la destinazione può essere qualunque indirizzo contenuto in un registro. È il modo per tornare da una funzione, saltando all'indirizzo salvato in `ra`.

Se non occorre memorizzare il link, basta usare `x0` come destinazione: il link viene scritto in un registro che lo ignora. Ecco il primo uso del registro zero.

1.7 Costanti grandi e indirizzi

Un immediato da 12 bit basta per molte costanti, non per tutte. Per costruire un valore qualunque a 32 bit servono due istruzioni.

- `lui rd, imm` (*load upper immediate*) scrive i 20 bit dell'immediato nella parte alta del registro, i 20 bit più significativi, e azzera i 12 bit bassi.
- Un'`addi` successiva aggiunge i 12 bit bassi.

C'è una sottigliezza. L'immediato di `addi` è con segno, quindi se il bit 11 della costante vale 1 l'`addi` sottrae invece di sommare. Per compensare si aggiunge 1 alla parte alta. Per caricare `0x12345fff`, per esempio, l'assemblatore produce:

```
lui  t0, 0x12346      # t0 = 0x12346000
addi t0, t0, -1       # t0 = 0x12345fff
```

Per gli indirizzi c'è `auipc rd, imm` (*add upper immediate to PC*), che somma la parte alta al valore del PC invece di partire da zero. Seguita da un `addi`, permette di calcolare l'indirizzo di un dato a partire dalla posizione dell'istruzione stessa, qualunque sia l'indirizzo a cui il programma viene caricato. Vale la stessa sottigliezza: se il bit 11 della distanza vale 1, la parte alta di `auipc` va aumentata di 1.

1.8 Le pseudo-istruzioni

Nessuno scrive a mano `lui` e `addi` per caricare una costante. L'assemblatore, il programma che traduce l'assembly in codice macchina, accetta delle **pseudo-istruzioni**: istruzioni che il processore non possiede, ma che vengono tradotte automaticamente in una o più istruzioni vere. Rendono il codice molto più leggibile (tabella 1.6).

Il registro zero rende possibili molte di queste traduzioni. La `nop` merita un commento: un processore non si ferma mai, e «non fare nulla» va tradotto in un'istruzione vera che non cambia lo stato. Sommare zero a `x0` e scrivere il risultato in `x0` è perfetto, perché la scrittura in `x0` viene ignorata. Le `nop`

Tabella 1.6 Le pseudo-istruzioni più usate e la loro traduzione.

Pseudo	Traduzione	Significato
`nop`	`addi x0, x0, 0`	non fare nulla
`mv rd, rs`	`addi rd, rs, 0`	copia un registro
`li rd, cost`	`addi` o `lui+addi`	carica una costante
`la rd, et`	`auipc+addi`	carica un indirizzo
`not rd, rs`	`xori rd, rs, -1`	complementa i bit
`neg rd, rs`	`sub rd, x0, rs`	cambia segno
`j et`	`jal x0, et`	salta e basta
`jal et`	`jal ra, et`	chiama una funzione
`jr rs`	`jalr x0, 0(rs)`	salta all'indirizzo in `rs`
`ret`	`jalr x0, 0(ra)`	torna da una funzione
`beqz rs, et`	`beq rs, x0, et`	salta se zero
`bnez rs, et`	`bne rs, x0, et`	salta se diverso da zero
`bgt rs, rt, et`	`blt rt, rs, et`	salta se maggiore
`ble rs, rt, et`	`bge rt, rs, et`	salta se minore o uguale

torneranno molto utili quando parleremo di pipeline.

1.9 I formati delle istruzioni

Ogni istruzione RV32I è una parola di 32 bit, numerati da 0 (il meno significativo, a destra) a 31. I bit sono raggruppati in *campi*, e ogni gruppo ha un significato. Esistono sei disposizioni dei campi, i **formati**, mostrati nella figura 1.2.

opcode
: I 7 bit meno significativi indicano a quale famiglia appartiene l'istruzione, e quindi quale formato usa.

funct3 e funct7
: Precisano l'operazione dentro la famiglia: per esempio `add` e `sub` hanno lo stesso opcode e lo stesso funct3, e si distinguono solo per funct7.

rd, rs1, rs2
: I tre registri: destinazione e due sorgenti, 5 bit ciascuno.

imm
: L'immediato, a volte spezzato in più pezzi.

Perché i campi sono disposti così

La disposizione sembra capricciosa, soprattutto negli immediati di B e J, ma è progettata con cura per semplificare l'hardware.

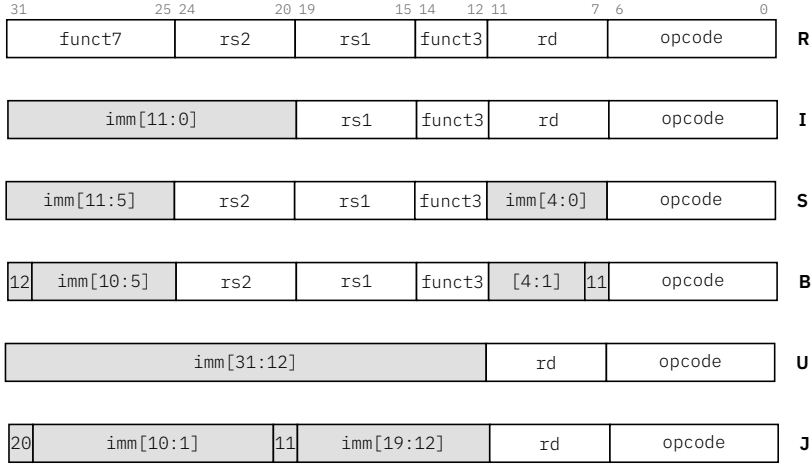


Figura 1.2 I sei formati di RISC-V. In grigio i bit dell'immediato, con l'indicazione di quali bit del valore contengono. R: operazioni fra registri. I: immediati, load e `jalr`. S: store. B: salti condizionati. U: `lui` e `auipc`. J: `jal`.

- `rs1`, `rs2` e `rd` occupano sempre gli stessi bit, in tutti i formati in cui compaiono. Il processore può cominciare a leggere i registri prima ancora di avere determinato quale istruzione sta eseguendo: se non servono, i valori letti vengono semplicemente ignorati.
- Il bit del segno di ogni immediato è sempre il bit 31 dell'istruzione. L'estensione del segno può partire subito, in parallelo con la decodifica.
- Nei formati S e B l'immediato è suddiviso per lasciare `rs1` e `rs2` al loro posto; i pezzi sono distribuiti in modo che quasi tutti i bit dell'immediato occupino la stessa posizione in tutti i formati, e così servono meno multiplexer per ricomporli.

Ritroveremo questi vantaggi quando costruiremo la fase di decodifica, nello stato 0x3.

1.10 Codificare e decodificare a mano

Saper passare a mano dall'assembly al codice macchina, e viceversa, è il modo migliore per capire i formati; è anche un esercizio frequente agli esami.

Dall'assembly all'esadecimale

Codifichiamo `xor x8, x4, x5`, che calcola lo XOR bit a bit di `x4` e `x5` e scrive il risultato in `x8`. È un'operazione fra registri, quindi usa il formato R;

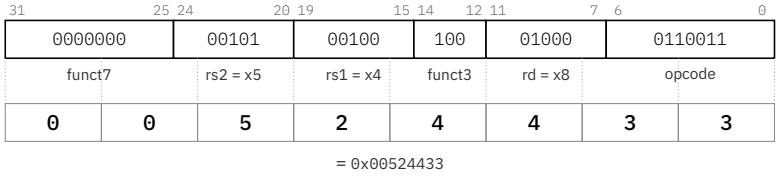


Figura 1.3 La codifica di `xor x8, x4, x5`. I campi si leggono in alto; in basso gli stessi 32 bit raggruppati a quattro danno le cifre esadecimali.

dalla tabella delle istruzioni (appendice B) si ricava che lo XOR ha opcode 0110011_2, funct3 100_2 e funct7 0000000_2. I registri si scrivono in binario su 5 bit: $\texttt{x5} = 00101_2$, $\texttt{x4} = 00100_2$, $\texttt{x8} = 01000_2$. Basta collocare ogni campo nella sua posizione e raggruppare i bit a quattro a quattro (figura 1.3).

Il risultato è `0x00524433`. Con lo stesso procedimento `sub x6, x15, x1` dà `0x40178333`: oltre ai registri cambia funct7, che per la sottrazione vale 0100000_2, ed è da quel 4 iniziale che si riconosce una `sub`.

Dall'esadecimale all'assembly

Il percorso inverso parte dall'opcode. Decodifichiamo `0x003110b3`.

1. Si scrive in binario: `0000 0000 0011 0001 0001 0000 1011 0011`.
2. I 7 bit meno significativi, 0110011_2, indicano che si tratta di un'operazione fra registri, formato R.
3. Si separano gli altri campi: $\text{rd} = 00001_2 = 1$, $\text{funct3} = 001_2$, $\text{rs1} = 00010_2 = 2$, $\text{rs2} = 00011_2 = 3$, $\text{funct7} = 0000000_2$.
4. $\text{funct3} = 001_2$ con funct7 nullo è lo scorrimento a sinistra: l'istruzione è `sll x1, x2, x3`.

Un salto, con l'immediato a pezzi

I salti sono il caso più insidioso. Prendiamo `beq a0, a1, avanti`, dove `avanti` si trova 16 byte più avanti. La distanza è $16 = 0000000010000_2$ su 13 bit; il bit 0 non si memorizza, e gli altri si distribuiscono come prescrive il formato B: il bit 12 nel bit 31 dell'istruzione, i bit 10–5 nei bit 30–25, i bit 4–1 nei bit 11–8, il bit 11 nel bit 7. Con $\texttt{a0} = \texttt{x10}$ e $\texttt{a1} = \texttt{x11}$, $\text{funct3} = 000_2$ e opcode 1100011_2 si ottiene `0x00b50863`. Un assemblatore svolge questo lavoro al nostro posto; ma vedere almeno una volta come l'immediato viene distribuito aiuta a capire perché, nel datapath, servirà un circuito apposito per ricomporlo.

1.11 Chiedere aiuto al sistema operativo

Un programma utente non può, e non deve, accedere direttamente alle risorse della macchina: il disco, la rete, lo schermo, la memoria degli altri programmi. Se potesse farlo, un programma sbagliato o malevolo metterebbe in crisi l'intero sistema. Per questo i processori prevedono dei **livelli di privilegio**: RISC-V ne definisce tre, *machine*, *supervisor* (in cui opera il sistema operativo) e *user* (in cui sono eseguiti i programmi). Un'istruzione privilegiata eseguita a livello utente provoca un errore.

Quando un programma ha bisogno di un servizio, per esempio stampare un messaggio o terminare, lo chiede al sistema operativo con l'istruzione **ecall** (*environment call*).

1. Il programma scrive in `a7` il numero del servizio richiesto e negli altri registri `a` gli eventuali argomenti.
2. Esegue `ecall`: la sua esecuzione si sospende e il controllo passa al sistema operativo, a un livello di privilegio più alto.
3. Il sistema operativo legge il numero, esegue il servizio, scrive un eventuale risultato in `a0` e restituisce il controllo al programma, all'istruzione successiva.

I numeri dei servizi non sono fissati dall'ISA ma dall'ambiente, cioè dall'ABI. QtRvSim usa quelli di Linux: `93` termina il programma (con il codice di uscita in `a0`), `64` scrive dei byte su un file, `63` li legge. RARS accetta sia questi, sia una serie di servizi propri pensati per la didattica, come `1` per stampare un intero e `10` per terminare. Li useremo nei laboratori.

1.12 Le estensioni

La base RV32I contiene solo ciò che è indispensabile. Il resto arriva con le estensioni, indicate da una lettera.

M Moltiplicazione e divisione intere (`mul`, `div`, `rem`…). In sua assenza, queste operazioni devono essere realizzate in software con lenti cicli di somme e sottrazioni; nello stato 0x5 vedremo come le esegue l'hardware.

A Operazioni *atomiche*: leggono, modificano e riscrivono un dato in memoria in un unico passo indivisibile. Sono indispensabili nei sistemi in cui più programmi condividono dati, come i database: interrompere un aggiornamento a metà lascerebbe i dati corrotti.

F e D
Virgola mobile in precisione singola e doppia (stato 0x5).

C Istruzioni compresse da 16 bit, per programmi più compatti.

V Istruzioni vettoriali, che applicano la stessa operazione a molti dati insieme.

Un processore si descrive elencando le estensioni: un RV32IMC, per esempio, è un processore a 32 bit con base intera, moltiplicazione e istruzioni compresse.

1.13 Un primo programma

Mettiamo insieme quasi tutto. Il programma seguente legge due numeri dalla memoria, li somma, scrive il risultato in memoria e termina. Le righe che iniziano con un punto sono **direttive**: non sono istruzioni ma indicazioni per l'assemblatore. Le parole seguite da due punti sono **etichette**: nomi simbolici per indirizzi che chi scrive il programma non conosce, e che l'assemblatore sostituirà con i valori veri.

```
        .text                # da qui in poi: istruzioni
        .globl main          # rende visibile l'etichetta main
main:
        la   t0, valore      # t0 ← indirizzo del primo dato
        lw   t1, 0(t0)       # t1 ← 10
        lw   t2, 4(t0)       # t2 ← 20, la parola successiva
        add  t3, t1, t2      # t3 ← 30
        sw   t3, 8(t0)       # terza parola ← 30
        li   a7, 93          # servizio 93: termina
        li   a0, 0           # codice di uscita 0
        ecall                # passa il controllo al sistema operativo

        .data                # da qui in poi: dati
valore: .word 10, 20, 0      # tre parole, inizializzate a 10, 20 e 0
```

Seguiamolo riga per riga.

- `.text` e `.data` dividono il programma in una zona per il codice e una per i dati; `.word` riserva una parola di memoria per ogni valore elencato.
- `la` è una pseudo-istruzione: diventa una coppia `auipc` e `addi` che calcola l'indirizzo di `valore` partendo dal PC.
- Le due `lw` usano lo stesso registro base con spiazzamenti diversi: 0 e 4, perché ogni parola occupa 4 byte. Lo spiazzamento non modifica `t0`: l'indirizzo calcolato vale solo per quell'istruzione.
- La `add` opera solo sui registri: è l'architettura load/store all'opera.
- `li` carica una costante, e la coppia `a7` = 93 ed `ecall` chiede al sistema operativo di terminare.

Eseguito in un simulatore, il programma lascia in memoria le tre parole 10,

20 e 30. Lo riprenderemo nel laboratorio L1, dove vedremo come caricarlo in QtRvSim ed eseguirlo un'istruzione alla volta, osservando i registri e la memoria che cambiano.

Due modi di scrivere la stessa cosa. Per leggere tre elementi consecutivi si può usare un unico registro base con spiazzamenti 0, 4 e 8, come sopra, oppure far avanzare il registro con `addi t0, t0, 4` prima di ogni `lw t1, 0(t0)`. Il risultato è lo stesso, ma la seconda versione esegue più istruzioni. Il dibattito fra RISC e CISC nasce proprio qui: un'istruzione più ricca svolge più lavoro, ma rende più difficile costruire un hardware veloce. Il progettista cerca il compromesso che massimizza la velocità complessiva.



Condizione di transizione uscita verso **0x2**

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Indica due differenze fra un'architettura RISC e una CISC, e spiega perché oggi la distinzione si è sfumata. ↺ § 1.1

2 Perché bastano 5 bit per indicare un registro? Che cosa ha di speciale `x0`? ↺ § 1.3

3 In memoria, all'indirizzo `0x2000`, è memorizzata la parola `0xcafebabe`. Quale byte si trova all'indirizzo `0x2000`? ↺ § 1.4

4 In memoria si trova il byte `0xf0`. Che valore ha `t0` dopo `lb t0, 0(a0)`? E dopo `lbu`? ↺ § 1.4

5 Perché la destinazione di un salto condizionato è relativa al PC, e come si salta all'indietro? ↺ § 1.6

6 Come si carica in un registro la costante `0x12345678`? ↺ § 1.7

7 In che cosa viene tradotta la pseudo-istruzione `ret`? ↺ § 1.8

8 Codifica in esadecimale `add x5, x6, x7`. ↺ § 1.10

9 Decodifica `0x007067b3`. ↺ § 1.10

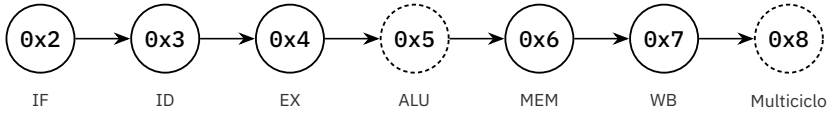
10 Perché un programma non può scrivere direttamente sul disco? Che cosa fa `ecall`? ↺ § 1.11

PARTE

II

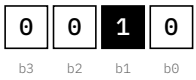
Il viaggio di un’istruzione

Apriamo il processore. Seguiamo un’istruzione dal momento in cui viene letta dalla memoria a quello in cui scrive il suo risultato, e costruiamo, una fase alla volta, il circuito che la esegue. Alla fine di questa parte sapremo come un processore esegue ogni istruzione di base di RISC-V, e perché conviene spezzarne il lavoro in più cicli di clock.



STATO

0x2



IF: prelevare l'istruzione

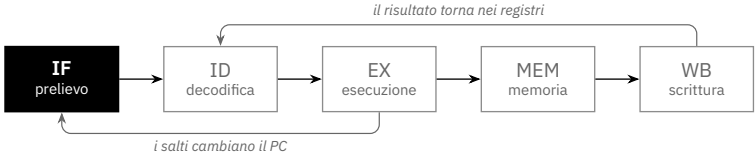


→ INGRESSI

- Il ciclo di un'istruzione e che cosa forma lo stato di un calcolatore (stato 0x0, § 0.7).
- Il modello Harvard (§ 0.3).
- Come un'istruzione occupa 4 byte di memoria, in ordine little-endian (stato 0x1, § 1.4).

USCITE →

- Distinguere un circuito combinatorio da un elemento di stato.
- Spiegare che cosa succede durante un ciclo di clock e passare da periodo a frequenza.
- Descrivere i componenti della fase IF: PC, memoria istruzioni, sommatore e multiplexer.
- Seguire il prelievo di un'istruzione, valore per valore.



Con questo stato apriamo il processore. Nella parte II seguiremo una sola istruzione alla volta lungo il suo viaggio, e a ogni tappa costruiremo i circuiti che servono: alla fine avremo disegnato un processore completo, capace di eseguire tutte le istruzioni di base di RISC-V. La striscia qui sopra indica

in quale delle cinque fasi ci troviamo. Si comincia dalla prima: il prelievo dell'istruzione.

2.1 Il datapath e il controllo

Dentro un processore si distinguono due parti, che conviene tenere separate fin da subito.

- Il **datapath** (*percorso dei dati*) è l'insieme dei componenti che i dati attraversano: i registri, la memoria, l'ALU, i sommatori e i fili che li collegano.
- Il **controllo** (*control path*) è l'insieme dei segnali che indicano al datapath quali operazioni svolgere sull'istruzione corrente: quale operazione deve eseguire l'ALU, se un registro deve essere scritto oppure no, quale percorso deve seguire un dato.

Un'immagine utile è quella di una rete ferroviaria. Il datapath corrisponde ai binari e alle stazioni, i dati ai treni, e il controllo è il ferroviere che, per ogni treno, sposta gli scambi nella posizione giusta. I binari sono gli stessi per tutti; cambia soltanto la posizione degli scambi.

Il processore che costruiamo esegue ogni istruzione in un solo ciclo di clock, dall'inizio alla fine: è un processore **a ciclo singolo**. Non è il modo in cui lavorano i processori moderni, ma è il modello più semplice e completo, e tutto quello che verrà dopo, dal multiciclo (0x8) alla pipeline (0x9), nasce da qui. Faremo una semplificazione: le memorie rispondono in un tempo paragonabile a quello degli altri componenti. E terremo presente un vincolo: ogni componente può essere usato al più una volta per ciclo, perché in un ciclo un circuito combinatorio produce un solo risultato e non dispone di alcun elemento in cui conservarne uno intermedio. Nella parte IV vedremo che cosa succede quando la memoria è lenta, come accade nella realtà.

2.2 Due famiglie di circuiti

Tutti i circuiti digitali di un processore appartengono a una di due famiglie.

Circuiti combinatori

Un **circuito combinatorio** calcola le sue uscite soltanto a partire dagli ingressi presenti in quel momento: non ricorda nulla. Un sommatore è combinatorio: dati due numeri, produce la loro somma; se gli ingressi cambiano, cambia anche la somma. Sono combinatori l'ALU, i multiplexer, i decodificatori e tutta la logica che interpreta le istruzioni.

La risposta, però, non è istantanea. Quando un ingresso cambia, il segnale deve attraversare le porte logiche, e ognuna impiega qualche decina di pico-

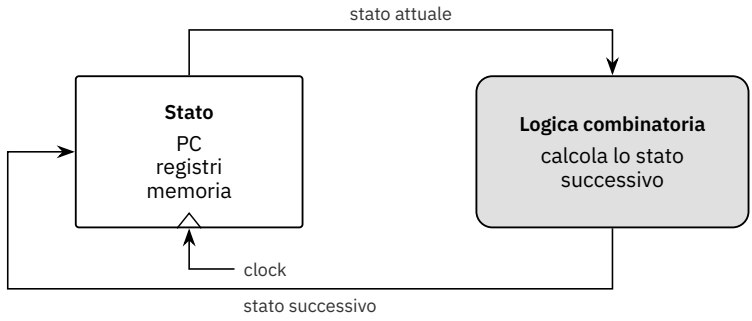


Figura 2.1 Lo schema di ogni macchina a stati, e quindi anche di un processore. A sinistra gli elementi che ricordano, a destra quelli che calcolano. Il triangolo indica che lo stato si aggiorna con il clock.

secondi. Per un breve intervallo l'uscita oscilla e può assumere valori senza senso; poi si *stabilizza* sul risultato corretto. Il tempo che serve si chiama **ritardo di propagazione**.

Elementi di stato

Un **elemento di stato** invece ricorda. Il più semplice è il **flip-flop**, che conserva un bit; affiancandone 32 si ottiene un **registro** da 32 bit. Un registro non cambia quando cambia il suo ingresso: copia l'ingresso al suo interno solo in un istante preciso, scandito dal clock, e per il resto del tempo mantiene il valore che ha. Sono elementi di stato il PC, i 32 registri e la memoria.

Ecco perché lo stato di un calcolatore, che nello stato 0x0 abbiamo definito come «ciò che è stato scritto», è costituito proprio dal PC, dai registri e dalla memoria: sono gli unici componenti capaci di ricordare. L'ALU, per quanto importante, non ne fa parte, perché non trattiene niente da un ciclo all'altro.

Il processore come macchina a stati

Mettendo insieme le due famiglie si ottiene la struttura di *qualunque* macchina a stati finiti: un gruppo di elementi di stato, che contiene lo stato attuale, e un circuito combinatorio che, a partire dallo stato attuale, calcola lo stato successivo. A ogni colpo di clock lo stato successivo prende il posto di quello attuale, e il giro ricomincia (figura 2.1).

È proprio la macchina a stati da cui questo libro prende il nome, con una differenza di scala. Un semaforo ha tre stati. Il nostro processore, contando soltanto il PC e i 31 registri che si possono scrivere, ha $32 + 31 \times 32 = 1024$ bit di stato, cioè 2^{1024} stati possibili: un numero di 309 cifre, mentre gli atomi

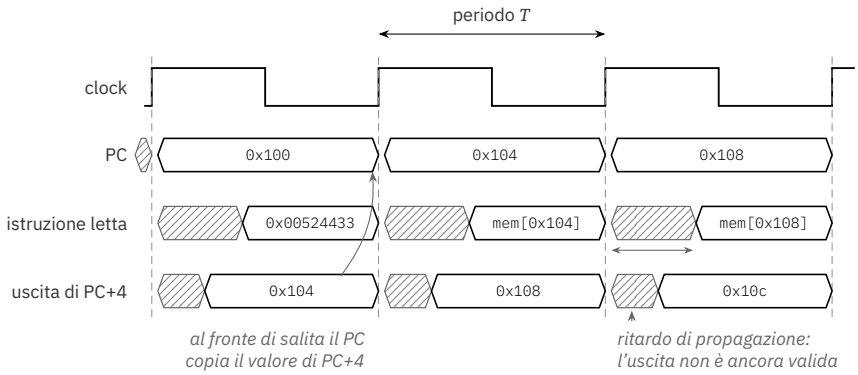


Figura 2.2 Tre cicli della fase IF. A ogni fronte di salita (le linee tratteggiate) il PC prende un nuovo valore; la memoria e il sommatore impiegano un po' di tempo a rispondere (zone a tratteggio obliquo), poi le loro uscite restano stabili fino al fronte successivo. `mem[0x104]` è la parola che si trova all'indirizzo `0x104`.

dell'universo osservabile si stimano in circa 10^{80}. E non abbiamo ancora contato la memoria. Una macchina a stati *finiti*, ma solo in teoria.

2.3 Il clock

Il **clock** è un segnale che oscilla di continuo fra 0 e 1, come un'onda quadra, e funge da metronomo a tutto il processore. Ogni oscillazione completa è un **ciclo di clock**; la sua durata è il **periodo** T, e il numero di cicli al secondo è la **frequenza** f:

$$f = \frac{1}{T}.$$

Un processore a 1 GHz ha un periodo di 1 ns; uno a 3 GHz di circa 0,33 ns, cioè 330 picosecondi. In quel tempo la luce percorre una decina di centimetri: le dimensioni di un chip non sono un dettaglio.

L'istante che conta è il **fronte di salita**, il passaggio da 0 a 1. In quel momento tutti gli elementi di stato copiano il proprio ingresso, insieme: è la fotografia che fissa il nuovo stato. Subito dopo il nuovo stato comincia a propagarsi nei circuiti combinatori, come un'onda che attraversa il processore da sinistra a destra; le uscite oscillano, poi si stabilizzano. Al fronte di salita successivo tutti i valori devono essere già stabili, pronti per essere fotografati. La figura 2.2 mostra questo andamento per la fase che studiamo in questo stato.

Da qui discendono due conseguenze, che torneranno spesso.

- *Il periodo deve bastare per il percorso più lento.* In un processore a ciclo singolo, in un periodo l'istruzione deve attraversare tutto il datapath, dalla lettura alla scrittura del risultato. Il clock va quindi regolato sull'istruzione che impiega più tempo, anche se quasi tutte le altre finirebbero prima. Faremo i conti nello stato 0x7.
- *Un'istruzione esiste solo quando scrive.* Tutto ciò che accade durante il ciclo è provvisorio. Se, prima del fronte di salita, si impedisce all'istruzione di scrivere (nei registri, in memoria e nel PC), è come se non fosse mai stata eseguita: non lascia traccia nello stato. Sembra un dettaglio, ma è il meccanismo con cui la pipeline annullerà le istruzioni avviate per errore, nello stato 0xb.

1927 Il cristallo che batte il tempo

Il metronomo di ogni calcolatore moderno è un cristallo di quarzo. Il quarzo è piezoelettrico: se lo si deforma produce una tensione, e se gli si applica una tensione si deforma. Tagliato nel modo giusto, vibra a una frequenza stabilissima. Il primo orologio al quarzo fu costruito nel 1927 da Warren Marrison e J. W. Horton ai Bell Telephone Laboratories di New York: il cristallo oscillava 50 000 volte al secondo, e un divisore di frequenza riduceva quelle oscillazioni a un ritmo abbastanza lento da muovere le lancette. Oggi il principio è lo stesso, ma rovesciato: un cristallo vibra a qualche decina di megahertz e un circuito apposito ne *moltiplica* la frequenza fino ai gigahertz del processore.

2.4 Il program counter

Il **PC** è un registro da 32 bit. A ogni fronte di salita copia il valore presente al suo ingresso, e per tutto il ciclo lo mantiene sulla sua uscita. Durante il ciclo, quindi, il PC contiene l'indirizzo dell'istruzione *in esecuzione*; al fronte di salita riceve quello della prossima. «Istruzione corrente» e «prossima istruzione», le parole usate nello stato 0x0, sono la stessa cosa guardata da due istanti diversi.

Quando il processore si accende, il PC viene caricato con un indirizzo fisso, il *vettore di reset*, deciso da chi ha progettato il processore: è l'indirizzo della prima istruzione che verrà eseguita. Nei simulatori si parte direttamente dal programma: RARS, per esempio, comincia dall'indirizzo `0x00400000`.

Il PC ha una caratteristica: deve sempre contenere un multiplo di 4, perché le istruzioni sono allineate (con l'estensione C, che aggiunge istruzioni da 16 bit, basta un multiplo di 2). Se un salto porta a un indirizzo non allineato, il processore genera un'eccezione invece di prelevare l'istruzione.

Perché il PC non è un registro come gli altri? In RISC-V il PC non è uno dei 32 registri e nessuna istruzione può scriverci direttamente: il suo valore viene modificato soltanto dai salti e letto soltanto da `auipc` (oltre che dai salti, che lo usano per calcolare la destinazione e, nel caso di `jal` e `jalr`, l'indirizzo di ritorno). Non tutte le architetture hanno fatto questa scelta. Nell'ARM a 32 bit il PC è il registro `r15`, e una qualunque istruzione che scrive in `r15` è di fatto un salto: comodo per il programmatore, scomodo per chi progetta il processore, che deve trattare ogni scrittura in un registro come un possibile salto. Nella versione a 64 bit di ARM il PC è uscito dal banco dei registri, proprio come in RISC-V.

1948 Il PC che arrivava sempre prima

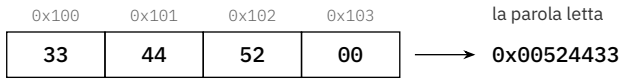
Il 21 giugno 1948, a Manchester, un calcolatore sperimentale chiamato *Baby* eseguì il primo programma memorizzato della storia: 17 istruzioni scritte da Tom Kilburn per trovare il più grande divisore proprio di 2^{18}, provando tutti i numeri da $2^{18} - 1$ in giù. Impiegò 52 minuti e circa 3,5 milioni di operazioni. La memoria conteneva 32 parole da 32 bit, e la macchina eseguiva in hardware soltanto sottrazioni e cambi di segno. Il suo program counter si chiamava CI (*control instruction*) e aveva una particolarità curiosa: veniva incrementato *prima* di prelevare l'istruzione, non dopo. Per saltare alla riga 6, quindi, bisognava scrivere nel CI il numero 5. Il Baby aveva anche un salto relativo, che sommava una distanza al CI: la stessa idea che RISC-V usa per i suoi salti condizionati.

2.5 La memoria delle istruzioni

La **memoria istruzioni** riceve un indirizzo da 32 bit e restituisce la parola da 32 bit che si trova a quell'indirizzo. Durante l'esecuzione il programma non cambia, quindi questa memoria viene solo letta: nel nostro modello si comporta come un circuito combinatorio, che risponde all'indirizzo dopo un certo ritardo.

Separare la memoria istruzioni da quella dei dati, come nel modello Harvard (§ 0.3), qui diventa una necessità: nello stesso ciclo il processore deve leggere un'istruzione e, se si tratta di una load o di una store, leggere o scrivere un dato. Il programmatore continua a vedere un unico spazio di indirizzi; la separazione è un dettaglio dell'implementazione, e nei processori reali corrisponde alle due cache di primo livello che vedremo nella parte IV.

L'istruzione occupa 4 byte consecutivi, e l'ordine è little-endian (§ 1.4). Se l'istruzione `xor x8, x4, x5` si trova all'indirizzo `0x100`, in memoria si trovano questi byte:



La memoria li restituisce già ricomposti nella parola `0x00524433`, con il byte dell'indirizzo più basso nella posizione meno significativa.

Che le istruzioni siano numeri in memoria, come i dati, non è un modo di dire: è l'idea del programma memorizzato (§ 0.2). Lo si può verificare con un programma che legge le proprie istruzioni.

```
        .text
main:   la   t0, main        # t0 = indirizzo della prima istruzione
        li   t1, 4           # quante parole leggere
ciclo:  mv   a0, t0          # stampa l'indirizzo
        li   a7, 34
        ecall
        li   a0, 32          # uno spazio
        li   a7, 11
        ecall
        lw   a0, 0(t0)       # legge la parola a quell'indirizzo
        li   a7, 34          # e la stampa in esadecimale
        ecall
        li   a0, 10          # a capo
        li   a7, 11
        ecall
        addi t0, t0, 4       # avanti di 4, come fa il PC
        addi t1, t1, -1
        bnez t1, ciclo
        li   a7, 10
        ecall
```

In RARS il programma va eseguito con l'opzione *Self-modifying code* attiva (nel menu *Settings*), perché normalmente il simulatore impedisce di leggere la zona del codice con una `lw`. Il risultato è questo:

```
0x00400000 0x00000297
0x00400004 0x00028293
0x00400008 0x00400313
0x0040000c 0x00500533
```

Le prime due parole sono la pseudo-istruzione `la`, tradotta in `auipc` e `addi` (§ 1.8); la terza è `li t1, 4`, cioè `addi t1, zero, 4`. Prova a decodificare la quarta: è la `mv`. La traduzione standard della tabella 1.6 è `addi rd, rs, 0`, ma RARS sceglie `add rd, zero, rs`: le due istruzioni sono equivalenti.

2.6 Avanti di quattro

Quasi sempre la prossima istruzione è quella che segue in memoria, 4 byte più avanti. Per ottenerne l'indirizzo serve un sommatore che calcoli $PC + 4$. Il valore 4 viene dal fatto che la memoria è indirizzata al byte e ogni istruzione occupa 4 byte; con l'estensione C il sommatore dovrebbe aggiungere 2 o 4, a seconda della lunghezza dell'istruzione appena letta.

Perché un sommatore apposito, quando nel processore c'è già un'ALU capace di sommare? Perché nel ciclo singolo ogni componente può essere usato una sola volta per ciclo, e l'ALU è occupata a eseguire l'istruzione stessa. Un sommatore da 32 bit costa qualche centinaio di porte logiche, una spesa minima rispetto al resto del chip, e permette di calcolare l'indirizzo successivo *in parallelo* a tutto il resto. Lo stesso ragionamento, nello stato 0x4, giustificherà un secondo sommatore per i salti.

Il sommatore opera per ogni istruzione, anche per quelle che poi salteranno altrove: in hardware un calcolo inutile non costa nulla, mentre aspettare di sapere se serve costerebbe tempo. Se l'istruzione è un salto, il valore $PC + 4$ verrà semplicemente scartato (o, come vedremo, usato come indirizzo di ritorno).

2.7 Quando non si va avanti: il multiplexer

Il PC non riceve sempre $PC + 4$. Per i salti, l'indirizzo successivo proviene da altre parti del processore, e un componente deve selezionare il valore da caricare nel PC. Questo compito è svolto dal componente più diffuso di tutto il datapath: il **multiplexer**, o *mux*.

Un multiplexer ha diversi ingressi di dati, un'uscita e un ingresso di *selezione*: il valore della selezione determina quale ingresso viene copiato sull'uscita. È lo scambio ferroviario del nostro esempio: i binari in arrivo sono tanti, quello in uscita è uno solo, e la posizione dello scambio è decisa dal controllo. Con 2 ingressi basta un bit di selezione; con 3 o 4 ne servono 2; in generale, con n ingressi ne servono $\lceil \log_2 n \rceil$.

La figura 2.3 mostra il simbolo e l'interno di un multiplexer. Davanti al PC ne serve uno a tre ingressi, comandato da un segnale di due bit che chiameremo **PCSel** (tabella 2.1).

Gli ingressi 1 e 2 vengono calcolati nella fase di esecuzione, e anche PCSel viene determinato in quella fase, quando si conosce l'esito di un salto condizionato: li costruiremo nello stato 0x4. Per ora è sufficiente sapere che il multiplexer è presente e che il PC conserva il proprio valore fino al fronte di salita: la selezione dispone quindi di tutto il ciclo per essere determinata.

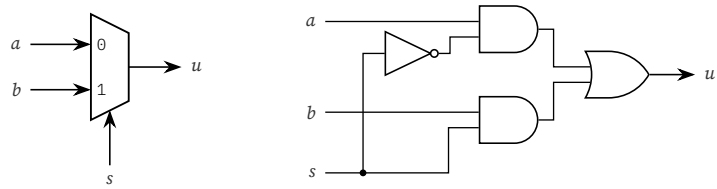


Figura 2.3 Un multiplexer a due ingressi. A sinistra il simbolo usato nelle figure del datapath, con il numero di ogni ingresso; a destra lo stesso circuito per un singolo bit, realizzato con porte logiche. Un multiplexer da 32 bit è costituito da 32 copie di questo circuito, comandate dallo stesso segnale di selezione.

Tabella 2.1 I tre ingressi del multiplexer del PC.

PCSel	Ingresso	Quando
0	$PC + 4$	Tutte le istruzioni che non saltano, e i salti condizionati quando la condizione è falsa.
1	$PC + imm$	Salti condizionati con condizione vera; `jal`.
2	$rs1 + imm$, calcolato dall'ALU, con il bit 0 azzerato	`jalr`.

2.8 Il prelievo, passo per passo

Mettiamo insieme i pezzi (figura 2.4) e seguiamo il prelievo dell'istruzione `xor x8, x4, x5`, che si trova all'indirizzo `0x100`.

1. Al fronte di salita il PC prende il valore `0x100`. Il nuovo valore compare sulla sua uscita e raggiunge, in parallelo, la memoria istruzioni e il sommatore.
2. La memoria legge i quattro byte da `0x100` a `0x103` e, dopo il suo tempo di accesso, presenta sull'uscita la parola `0x00524433`. L'istruzione parte verso la fase di decodifica.
3. Nello stesso tempo il sommatore calcola `0x104`.
4. `xor` non è un salto, quindi PCSel vale 0 e all'ingresso del PC arriva `0x104`.
5. Al fronte di salita successivo il PC copia `0x104`, e il ciclo ricomincia con l'istruzione seguente.

Nella fase IF il contenuto della parola prelevata non viene interpretato: la fase svolge sempre lo stesso lavoro, che l'istruzione sia una somma, un salto o una lettura dalla memoria. Interpretare il contenuto di quei 32 bit è il

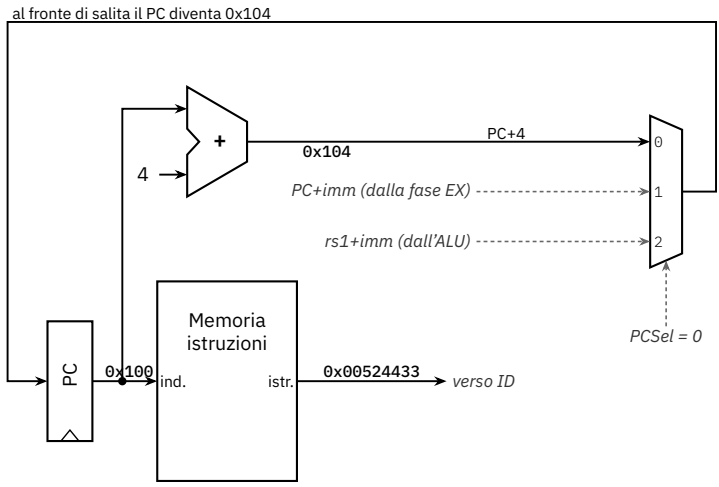


Figura 2.4 La fase IF al completo, con i valori del prelievo di `xor x8, x4, x5`. Tutti i fili trasportano 32 bit; quelli tratteggiati sono segnali di controllo o valori che arrivano dalle fasi successive.

compito della fase successiva.

Condizione di transizione uscita verso **0x3**

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1. Perché l'ALU non fa parte dello stato del processore, mentre il PC sì? ↺ § 2.2
2. Quanto dura un ciclo di clock in un processore a 2,5 GHz? ↺ § 2.3
3. Che cosa succede a un'istruzione a cui viene impedito di scrivere prima del fronte di salita? ↺ § 2.3
4. Il PC vale `0x0040001c`. Quali byte legge la memoria istruzioni, e che valore avrà il PC al ciclo dopo se l'istruzione non è un salto? ↺ § 2.8
5. Perché per calcolare $PC + 4$ si usa un sommatore apposito invece dell'ALU? ↺ § 2.6
6. Quanti bit di selezione servono a un multiplexer con 3 ingressi? E con 5? ↺ § 2.7
7. In ARM a 32 bit il PC è il registro `r15`. Che cosa succede se un'istruzione aritmetica scrive in `r15`? ↺ § 2.4

STATO

0x3

b3	b2	b1	b0
0	0	1	1

ID: decodificare

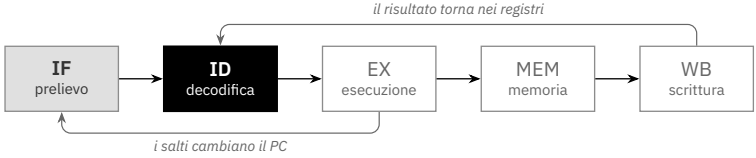


→ INGRESSI

- I formati delle istruzioni e il significato dei campi (stato 0x1, § 1.9).
- L'estensione del segno (§ 1.4) e la codifica degli immediati spezzati (§ 1.10).
- Elementi di stato, clock e multiplexer (stato 0x2).

USCITE →

- Spiegare come si estraggono i campi di un'istruzione.
- Descrivere il banco dei registri, dentro e fuori.
- Ricostruire l'immediato di qualunque formato a partire dai bit dell'istruzione.
- Spiegare quale funzione svolge l'unità di controllo e quali segnali produce.



Alla fine della fase IF disponiamo di 32 bit. Per il processore non sono ancora un'istruzione: sono soltanto 32 fili, ciascuno al valore 0 o 1. La fase di **decodifica** (*instruction decode*, ID) li trasforma in tutto ciò che serve alle fasi successive:

- i *valori* dei registri sorgente, letti dal banco dei registri;
- l'*immediato*, ricostruito ed esteso a 32 bit;

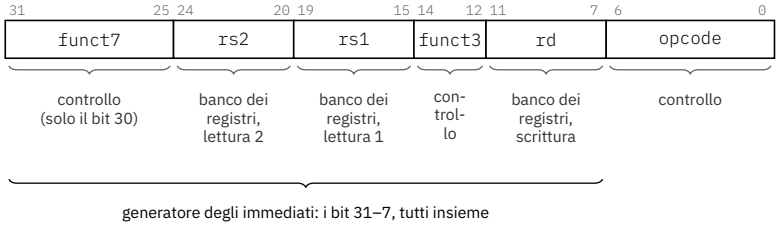


Figura 3.1 La destinazione dei bit di un'istruzione. I campi sono quelli del formato R; negli altri formati gli stessi bit possono contenere parti dell'immediato, e per questo i bit 31–7 raggiungono anche il generatore degli immediati. Non c'è nessuna porta logica: solo fili.

- i *segnali di controllo*, che indicheranno a ogni componente quale operazione svolgere per questa istruzione.

Le tre operazioni avvengono nello stesso momento, in parallelo. Vedremo che non è un caso: è la conseguenza diretta del modo in cui RISC-V dispone i campi.

3.1 Tagliare l'istruzione

Il primo passo è il più semplice di tutti, perché non richiede nessun circuito. Separare i campi di un'istruzione significa soltanto instradare ciascun gruppo di fili verso la destinazione corretta: i 5 fili dei bit 19–15, quelli di `rs1`, raggiungono l'ingresso del banco dei registri che seleziona il primo registro da leggere; i 7 fili dell'opcode raggiungono l'unità di controllo; e così via (figura 3.1). Un filo può anche trasmettere lo stesso segnale a più componenti: il bit 30, per esempio, raggiunge sia il generatore degli immediati sia l'unità di controllo.

Qui si vede il frutto della disposizione dei campi descritta nello stato 0x1. Poiché `rs1`, `rs2` e `rd` occupano sempre gli stessi bit, i fili che li collegano al banco dei registri sono fissi, uguali per tutte le istruzioni. Se in un formato `rs1` occupasse i bit 19–15 e in un altro i bit 24–20, servirebbe un multiplexer per selezionarli, e il multiplexer potrebbe commutare solo dopo il riconoscimento del formato. Così, invece, la lettura dei registri parte subito.

3.2 Il banco dei registri

Il **banco dei registri** (*register file*) contiene i 32 registri dell'ISA. Visto dall'esterno ha tre *porte*, cioè tre gruppi di ingressi e uscite che operano in modo indipendente:

- due *porte di lettura*: si presenta il numero di un registro su 5 bit (`rs1` o `rs2`) e si ottiene, sulle uscite `rd1` e `rd2`, il suo contenuto su 32 bit;
- una *porta di scrittura*: il numero del registro di destinazione (`rd`), il valore da scrivere (`wd`) e un segnale di controllo, **RegWriteEn**, che autorizza la scrittura.

Due letture e una scrittura non sono un numero scelto a caso: è esattamente ciò che serve all'istruzione più esigente, un'operazione di tipo R come `add rd, rs1, rs2`, che legge due registri e ne scrive uno.

Leggere

La lettura è combinatoria. Appena `rs1` cambia, dopo un breve ritardo su `rd1` compare il contenuto del registro indicato; non serve aspettare il clock. Internamente, ogni porta di lettura è un multiplexer molto ampio: 32 ingressi da 32 bit, uno per registro, e `rs1` come selezione (figura 3.2).

Scrivere

La scrittura invece avviene solo al fronte di salita del clock, e solo se RegWriteEn vale 1. Per selezionare il registro da scrivere si usa un **decodificatore** (*decoder*): un circuito con n ingressi e 2^n uscite, in cui è attiva sempre e soltanto l'uscita il cui numero è scritto sugli ingressi. Con 5 ingressi ha 32 uscite: se `rd` $= 01000_2$, si attiva l'uscita 8 e tutte le altre restano a 0. Ogni uscita, combinata in AND con RegWriteEn, abilita la scrittura di un solo registro, mentre il valore `wd` arriva a tutti, ma soltanto il registro abilitato lo memorizza.

Il registro che non c'è

Il registro `x0` vale sempre zero e ignora le scritture. Realizzarlo è semplicissimo, ed è quasi un artificio: `x0` *non esiste*. All'ingresso 0 dei multiplexer di lettura è collegata la costante 0, e l'uscita 0 del decodificatore non è collegata ad alcun registro. Una scrittura in `x0` abilita un registro inesistente e svanisce. Il banco, quindi, contiene davvero soltanto 31 registri: $31 \times 32 = 992$ flip-flop.

Leggere e scrivere lo stesso registro

Che cosa succede con `addi sp, sp, -16`, che legge e scrive lo stesso registro? Durante il ciclo la lettura restituisce il valore *vecchio* di `sp`; il nuovo valore viene scritto solo al fronte di salita, quando l'istruzione è finita. È esattamente il comportamento che ci si aspetta, e nel ciclo singolo si ottiene senza alcun costo aggiuntivo. Nella pipeline, dove due istruzioni diverse leggono e scrivono il banco nello stesso ciclo, la questione diventerà più delicata (stato 0xa).

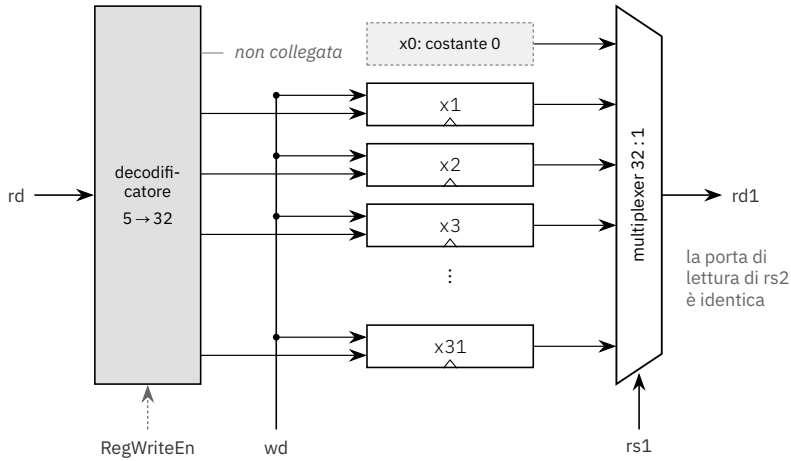


Figura 3.2 L'interno del banco dei registri, semplificato. A sinistra la porta di scrittura: il decodificatore abilita un solo registro, che al fronte di salita copia `wd`. A destra una delle due porte di lettura: un multiplexer a 32 ingressi. Il registro `x0` non esiste: al suo posto c'è la costante 0.

1918 Due valvole che ricordano

Il primo circuito elettronico capace di ricordare un bit fu descritto nel 1918 da due fisici inglesi, William Eccles e Frank Wilfred Jordan. Usava due valvole termoioniche collegate in modo che ciascuna tenesse spenta l'altra: il circuito restava stabile in uno dei due stati finché un impulso non lo faceva scattare nell'altro. Il brevetto, intitolato «Improvements in ionic relays», fu pubblicato nel 1920. Venticinque anni dopo lo stesso circuito lavorava dentro Colossus, il calcolatore britannico che decifrava i messaggi dell'alto comando tedesco. Oggi lo chiamiamo flip-flop, e il banco dei registri di un processore ne contiene centinaia, fatti di transistor invece che di valvole.

3.3 Il generatore degli immediati

Nello stato 0x1 abbiamo visto che l'immediato di un'istruzione è lungo 12 o 20 bit, e nei formati S, B e J è spezzato in più pezzi. Le fasi successive, però, lavorano su 32 bit. Il **generatore degli immediati** ricompone i pezzi ed estende il risultato a 32 bit. Siccome la posizione dei pezzi dipende dal formato, ha bisogno di un segnale di controllo, **ImmSel**, che gli indichi il formato.

La figura 3.3 è la «ricetta» completa, e vale la pena di leggerla in due direzioni.

- *Per righe*, indica come si ottiene l'immediato di un formato. Nel formato I,

bit	31	30–20	19–12	11	10–5	4–1	0
I	i31	i31	i31	i31	i[30:25]	i[24:21]	i20
S	i31	i31	i31	i31	i[30:25]	i[11:8]	i7
B	i31	i31	i31	i7	i[30:25]	i[11:8]	0
U	i31	i[30:20]	i[19:12]	0	0	0	0
J	i31	i31	i[19:12]	i20	i[30:25]	i[24:21]	0

Figura 3.3 Come il generatore degli immediati compone i 32 bit per ogni formato. `i31` indica il bit 31 dell'istruzione, `i[30:25]` i bit da 30 a 25, e così via. In grigio le copie del bit 31, cioè l'estensione del segno. Leggendo una colonna dall'alto in basso si vede fra quante sorgenti deve scegliere ogni bit del risultato: al più quattro.

per esempio, i 12 bit dell'immediato sono i bit 31–20 dell'istruzione, e i 20 bit più alti del risultato sono tutti copie del bit 31: è l'estensione del segno.

- *Per colonne*, indica quanto lavoro deve svolgere l'hardware. Il bit 31 del risultato è *sempre* il bit 31 dell'istruzione, in tutti i formati; i bit 10–5 sono sempre i bit 30–25 (oppure 0 nel formato U); i bit 4–1 provengono da due sole posizioni (più lo 0 del formato U). Ogni bit del risultato sceglie fra al più quattro sorgenti, con un piccolo multiplexer comandato da ImmSel.

Ecco il motivo della disposizione «capricciosa» degli immediati di B e J: i progettisti di RISC-V hanno distribuito i bit in modo che ognuno occupasse il più spesso possibile la stessa posizione, così da ridurre al minimo i multiplexer del generatore. Inoltre il bit del segno occupa sempre il bit 31, così l'estensione non deve attendere di conoscere il formato.

L'estensione del segno non costa nulla

Estendere il segno sembra un'operazione, ma in hardware è solo un filo: il bit 31 dell'istruzione viene collegato a tutti i bit alti del risultato. Non c'è calcolo e non c'è ritardo. Le istruzioni che estendono con zeri invece che con il segno (come le versioni *unsigned* delle load) non coinvolgono il generatore: lì si estende il dato letto dalla memoria, come vedremo nello stato 0x6.

Due esempi

Prendiamo `addi a0, a0, -1`, codificata `0xfff50513`. È di formato I, quindi i bit 31–20, cioè `0xfff`, sono l'immediato; il bit 31 vale 1, e le sue 20 copie riempiono la parte alta. Il risultato è `0xffffffff`, cioè −1.

Ora un salto all'indietro: `bne t0, zero, -12`, codificata `0xfe029ae3`. In

binario:

```
1111111 00000 00101 001 1010 1 1100011
```

con i campi del formato B separati da spazi. Seguendo la riga B della figura 3.3: il bit 31 (1) è copiato in tutti i bit da 31 a 12; il bit 7 (1) diventa il bit 11; i bit 30–25 (111111_2) diventano i bit 10–5; i bit 11–8 (1010_2) i bit 4–1; il bit 0 vale 0. Il risultato è `0xfffffff4`, cioè −12: il salto torna indietro di tre istruzioni.

!

Lo scorrimento che non c'è. Nel datapath del Patterson e Hennessy, il libro di testo più diffuso, fra il generatore degli immediati e il sommatore dei salti c'è un blocco «Shift left 1»: in quella versione il generatore produce l'immediato *senza* il bit 0, e lo scorrimento lo aggiunge. Nel nostro datapath il generatore inserisce direttamente lo 0 nel bit 0 (righe B e J della figura), quindi lo scorrimento non serve: è già realizzato dal cablaggio. Le figure su MIPS mostrano invece «Shift left 2», perché lì l'offset è espresso in parole da 4 byte. Sono tre modi di disegnare la stessa idea.

3.4 L'unità di controllo

L'**unità di controllo** è il ferroviere del nostro esempio: esamina l'istruzione e determina la posizione degli scambi. Nel processore a ciclo singolo è un circuito puramente combinatorio, senza memoria: i suoi ingressi sono alcuni bit dell'istruzione, le sue uscite sono i segnali di controllo. (Nello stato 0x8, quando l'esecuzione si dividerà in più cicli, l'unità di controllo diventerà una vera macchina a stati.)

Che cosa guarda

Il campo più importante è l'opcode, che indica la *famiglia* dell'istruzione e quindi il suo formato (tabella 3.1). Per riconoscere la famiglia bastano i bit 6–2; i bit 1–0, che nelle istruzioni da 32 bit valgono sempre 11_2 (stato 0x1), servono solo a scartare le parole che non sono istruzioni valide: la parola `0x00000000`, per esempio, per specifica è illegale. Per distinguere le operazioni dentro una famiglia servono anche funct3 e il bit 30, l'unico bit di funct7 che in RV32I cambia: vale 1 in `sub`, `sra` e `srai`, 0 in tutte le altre. (Con l'estensione M anche il bit 25 diventa importante, perché distingue `mul` da `add`.) Funct3 e il bit 30 servono soprattutto al controllo dell'ALU e alla logica dei salti, che costruiremo nello stato 0x4.

Manca soltanto `fence` (opcode `0001111`), che serve a ordinare gli accessi alla memoria quando più processori la condividono: in un processore a ciclo singolo, che esegue un accesso alla volta, si può eseguire come una `nop`.

Tabella 3.1 Le famiglie di istruzioni di RV32I riconosciute dall'opcode.

Opcode	Famiglia	Formato	Istruzioni
`0110011`	OP	R	`add sub sll slt sltu xor srl sra or and`
`0010011`	OP-IMM	I	`addi slti sltiu xori ori andi slli srli srai`
`0000011`	LOAD	I	`lb lh lw lbu lhu`
`0100011`	STORE	S	`sb sh sw`
`1100011`	BRANCH	B	`beq bne blt bge bltu bgeu`
`1101111`	JAL	J	`jal`
`1100111`	JALR	I	`jalr`
`0110111`	LUI	U	`lui`
`0010111`	AUIPC	U	`auipc`
`1110011`	SYSTEM	I	`ecall ebreak`

Tabella 3.2 I segnali di controllo del processore a ciclo singolo. La colonna «Fase» indica in quale fase vengono usati.

Segnale	Fase	Che cosa decide
ImmSel	ID	Il formato dell'immediato: I, S, B, U o J.
Op1Sel	EX	Il primo operando dell'ALU: `rs1`, il PC oppure 0.
Op2Sel	EX	Il secondo operando dell'ALU: `rs2` oppure l'immediato.
ALUOp	EX	Il tipo di operazione richiesto all'ALU; insieme a funct3 e al bit 30 diventa l'operazione vera e propria.
Branch, Jal, Jalr	EX	Il tipo di salto dell'istruzione (al più uno vale 1): condizionato, `jal`, `jalr`. Servono a calcolare PCSel.
MemWrite	MEM	Se la memoria dati va scritta.
WBSel	WB	Che cosa scrivere nel registro di destinazione: il risultato dell'ALU (0), il dato letto dalla memoria (1) o $PC + 4$ (2).
RegWriteEn	WB	Se il registro di destinazione va scritto.

Se l'opcode non corrisponde a nessuna famiglia, l'istruzione è *illegale*: il processore non deve eseguirla, ma generare un'eccezione e passare il controllo al sistema operativo. Lo stesso accade, in modo voluto, con `ecall` ed `ebreak`, che hanno lo stesso opcode e lo stesso funct3 e si distinguono per il bit 20. Esamineremo il meccanismo delle eccezioni nello stato 0xb.

Che cosa decide

La tabella 3.2 elenca i segnali di controllo del nostro processore. Molti comandano componenti che non abbiamo ancora costruito: per ora basta sapere che esistono, e che sono tutti determinati qui, nella fase ID, a partire dall'istruzione.

Per i segnali che riguardano questa fase, la decisione si legge direttamente dalla tabella 3.1: ImmSel è il formato della famiglia, e RegWriteEn vale 1 per tutte le famiglie che hanno un `rd`, cioè tutte tranne STORE, BRANCH e (nel nostro processore) SYSTEM. Completeremo la tabella della verità dell'unità di controllo fase dopo fase, fino alla versione finale nello stato 0x7.

> **«Non importa».** Per alcune istruzioni certi segnali sono indifferenti. In un'operazione di tipo R l'immediato non serve, e il valore di ImmSel non ha alcun effetto; in una store non si scrive nessun registro, e WBSel può valere qualunque cosa, perché RegWriteEn vale 0. Nelle tabelle questi casi si indicano con un trattino, e chi progetta il circuito può sceglierne il valore che rende la logica più semplice. I segnali che autorizzano una scrittura, invece, non sono mai indifferenti: RegWriteEn e MemWrite devono valere 0 ogni volta che non si deve scrivere.

3.5 Leggere prima di sapere

C'è un dettaglio che a prima vista sembra uno spreco. Il banco dei registri legge `rs1` e `rs2` per *ogni* istruzione, anche quando non servono. In una `lui`, per esempio, i bit 24–15 fanno parte dell'immediato: il banco legge comunque i due registri indicati da quei bit, cioè due registri arbitrari, e i valori letti vengono semplicemente scartati, perché nessun componente li userà.

È una scelta deliberata. Attendere che l'unità di controllo abbia riconosciuto il formato, e solo dopo leggere i registri, allungherebbe la fase ID di tutto il tempo della decodifica. Leggere sempre, invece, permette di svolgere le tre operazioni in parallelo: mentre il controllo decide, i valori dei registri e l'immediato sono già pronti, e i componenti che non li usano li ignorano. In hardware un lavoro inutile non costa tempo; un'attesa sì. Ritroveremo spesso questo principio, che è lo stesso che ci ha fatto calcolare $PC + 4$ anche per i salti.

1971 Un processore con le iniziali

Nel novembre del 1971 Intel cominciò a pubblicizzare il 4004, il primo microprocessore in commercio: un processore intero su un solo chip. Era nato per una calcolatrice da tavolo della giapponese Busicom: Ted Hoff e Stan Mazor ne avevano ridotto il progetto a pochi chip, Masatoshi Shima aveva lavorato alla logica, e Federico Faggin, un fisico di Vicenza arrivato in Intel dall'Italia passando per la Fairchild, lo aveva reso realtà sul silicio. Aveva 2300 transistor e sedici registri da 4 bit: tutto ciò che in questa parte disegniamo con blocchi separati era contenuto in pochi millimetri quadrati. Faggin lasciò una firma che pochi hanno visto: le sue iniziali, «F.F.», incise sul bordo di ogni 4004 prodotto.

3.6 Esempio: la decodifica di `xor`

Riprendiamo l'istruzione prelevata nello stato 0x2: `xor x8, x4, x5`, cioè `0x00524433`. Supponiamo che `x4` = `0x12345678` e `x5` = `0x0000ffff`.

1. I fili separano i campi: opcode 0110011_2, rd = 8, funct3 = 100_2, rs1 = 4, rs2 = 5, bit 30 = 0.
2. Il banco dei registri legge `x4` e `x5`: su `rd1` compare `0x12345678`, su `rd2` `0x0000ffff`.
3. Il generatore degli immediati produce un valore qualunque, che nessun componente utilizzerà: un'operazione di tipo R non ha immediato.
4. L'unità di controllo riconosce la famiglia OP e imposta i segnali: RegWriteEn = 1, perché alla fine si dovrà scrivere `x8`; ALUOp indica «operazione di tipo R»; Op2Sel seleziona `rs2`; WBSel seleziona il risultato dell'ALU; MemWrite = 0; nessun salto.

Tutto questo accade insieme, in una frazione del ciclo di clock. Alla fine della fase ID l'istruzione non è più un numero: si è trasformata in due operandi pronti e in un insieme di comandi. La figura 3.4 mostra il datapath costruito finora.

Condizione di transizione uscita verso **`0x4`**

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Perché separare i campi di un'istruzione non richiede porte logiche? Che cosa servirebbe se `rs1` cambiasse posizione da un formato all'altro? ↺ § 3.1

2 Quante porte ha il banco dei registri, e perché proprio quelle? ↺ § 3.2

3 Come si ottiene che `x0` valga sempre zero? ↺ § 3.2

4 Durante l'esecuzione di `add t0, t0, t1`, quale valore di `t0` legge il banco dei registri? ↺ § 3.2

5 Ricostruisci l'immediato di `0xfe62d6e3`, una `bge`. ↺ § 3.3

6 Perché il bit 0 dell'immediato è sempre 0 nei formati B e J, e che cosa sostituisce lo «Shift left 1» di altri libri? ↺ § 3.3

7 Che cosa legge il banco dei registri durante una `lui`, e perché non è un problema? ↺ § 3.5

8 Quali bit dell'istruzione esamina l'unità di controllo in RV32I? ↺ § 3.4

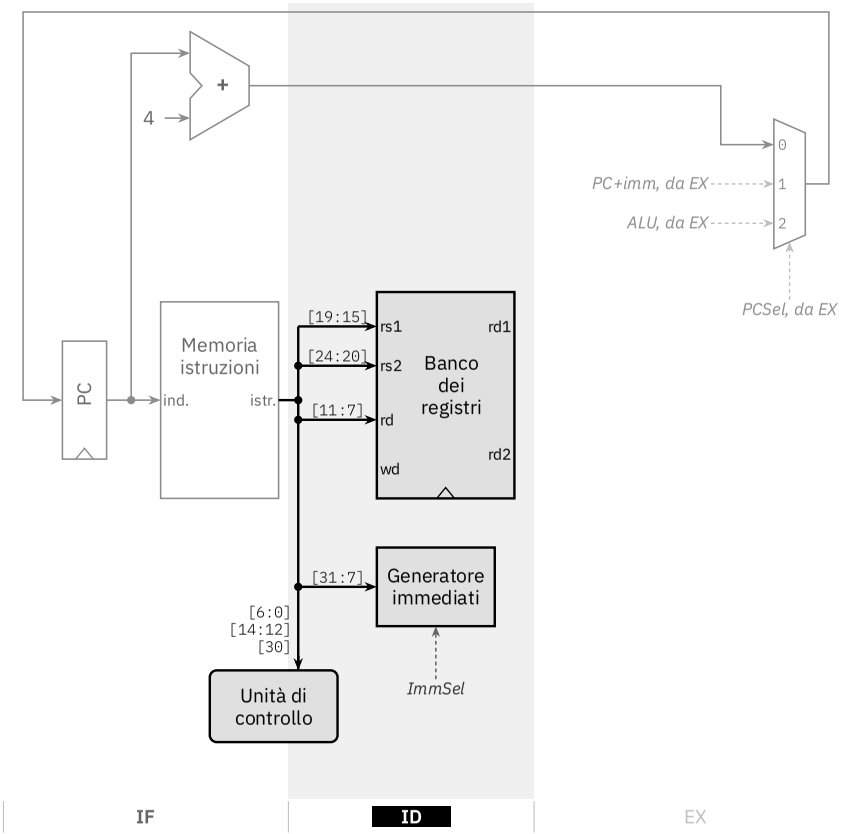


Figura 3.4 Il datapath finora. In nero la fase ID appena costruita, in grigio la fase IF. Le linee tratteggiate sono segnali di controllo o valori che arriveranno dalla fase EX; i fili che partono dall'unità di controllo non sono disegnati, per non appesantire la figura. Il banco dei registri ha anche una porta di scrittura, i cui fili arriveranno con la fase WB.

STATO

0x4

b3	b2	b1	b0
0	1	0	0

EX: eseguire

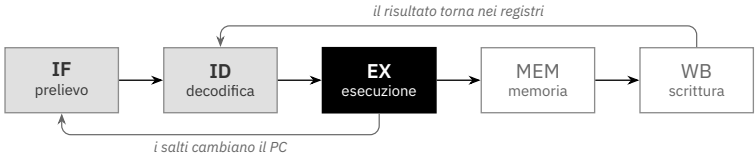


→ INGRESSI

- Le istruzioni aritmetiche, logiche e di salto (stato 0x1, § 1.5 e § 1.6).
- Il complemento a 2 (appendice A).
- Il banco dei registri, il generatore degli immediati e i segnali di controllo (stato 0x3).

USCITE →

- Descrivere l'ALU vista dall'esterno e spiegare come riceve l'operazione da eseguire.
- Indicare da dove provengono gli operandi per ogni famiglia di istruzioni.
- Spiegare come il processore determina l'esito di un salto condizionato e ne calcola la destinazione.
- Riconoscere un overflow e controllarlo via software, sapendo che RISC-V non lo segnala.



Alla fine della fase ID l'istruzione si è trasformata in operandi e comandi. Nella fase di **esecuzione** (*execute*, EX) finalmente si calcola. Che cosa si calcola dipende dall'istruzione:

- il risultato di un'operazione aritmetica o logica;
- l'indirizzo di memoria di una load o di una store;

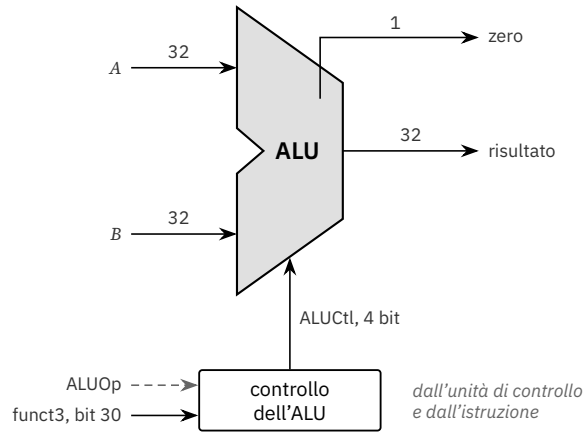


Figura 4.1 L'ALU vista dall'esterno, con il piccolo circuito che ne seleziona l'operazione. I numeri sui fili indicano quanti bit trasportano.

- l'esito del confronto di un salto condizionato, e la sua destinazione;
- l'indirizzo di destinazione di `jal` e di `jalr`, e il valore di `lui` e `auipc`.

Per tutto questo bastano due componenti: l'ALU e un sommatore. Il resto della fase è costituito da multiplexer che inviano all'ALU gli operandi corretti e da una piccola rete logica che determina se saltare.

4.1 L'ALU vista dall'esterno

L'**ALU** riceve due operandi da 32 bit, che chiameremo *A* e *B*, e un codice di 4 bit, **ALUCtl**, che specifica l'operazione da eseguire. Produce un risultato da 32 bit e un segnale da un bit, **zero**, che vale 1 quando il risultato è zero (figura 4.1). È un circuito combinatorio: non ricorda nulla, e il risultato è pronto dopo il suo ritardo di propagazione.

Il nostro processore usa le operazioni della tabella 4.1. I codici non sono scelti a caso: per le istruzioni di tipo R sono proprio i 4 bit {bit 30, funct3} dell'istruzione, così che in quel caso non ci sia nulla da tradurre. Ogni processore reale ha i suoi codici, ma in ogni processore RV32I l'elenco delle operazioni è sempre questo, perché lo impone l'ISA.

Negli scorrimenti contano solo i 5 bit meno significativi di *B*: far scorrere un registro da 32 bit di 32 posizioni o più non avrebbe senso. Nello stato 0x5 vedremo come l'ALU realizzi queste operazioni, dalla somma bit per bit al calcolo anticipato dei riporti; qui è sufficiente saperla utilizzare.

Tabella 4.1 Le operazioni dell'ALU del nostro processore.

ALUCtl	Operazione	Risultato
`0000`	ADD	$A + B$
`1000`	SUB	$A - B$
`0001`	SLL	A fatto scorrere a sinistra di B posizioni
`0010`	SLT	1 se $A < B$ come numeri con segno, altrimenti 0
`0011`	SLTU	1 se $A < B$ come numeri senza segno, altrimenti 0
`0100`	XOR	A XOR B, bit a bit
`0101`	SRL	A fatto scorrere a destra, entrano zeri
`1101`	SRA	A fatto scorrere a destra, entra il bit del segno
`0110`	OR	A OR B, bit a bit
`0111`	AND	A AND B, bit a bit

1970 Un'ALU in un chip

All'inizio del 1970 Texas Instruments mise in vendita il 74181, il primo circuito integrato che conteneva un'ALU completa: 4 bit, sedici operazioni logiche e sedici aritmetiche. Per lavorare su 16 o 32 bit bastava affiancarne quattro o otto, e per anni le ALU dei minicomputer furono realizzate in questo modo: lo usavano il PDP-11 di Digital, lo Xerox Alto, il VAX-11/780. Fra le sue operazioni ce ne sono alcune dall'aspetto bizzarro, come «A più (A AND B)»: nessuno le aveva richieste, ma derivavano senza costi aggiuntivi dalla struttura del circuito, che sommava ad A una qualunque funzione logica di A e B. E per essere veloce calcolava tutti i riporti in anticipo, invece di aspettarli bit dopo bit: è il *carry lookahead* dello stato 0x5.

4.2 Chi sceglie l'operazione

ALUCtl si calcola in due passi, con una divisione dei compiti che si ritrova in quasi tutti i processori didattici e in molti reali.

1. L'unità di controllo principale esamina soltanto l'opcode e produce un segnale di 2 bit, **ALUOp**, che indica il tipo di operazione richiesta (tabella 4.2).
2. Un piccolo circuito, il *controllo dell'ALU*, combina ALUOp con funct3 e con il bit 30 dell'istruzione e produce ALUCtl.

Nelle figure del datapath, per non affollarle, continueremo a disegnare un solo riquadro, l'unità di controllo, che comprende anche il controllo dell'ALU e la logica dei salti che costruiremo fra poco: per questo il riquadro riceve, oltre all'opcode, anche funct3 e il bit 30.

L'ultima riga merita attenzione. Nelle istruzioni con immediato il bit 30 fa parte dell'immediato, e può valere 1 per caso. Prendiamo `addi t0, t1,`

Tabella 4.2 Dal tipo di lavoro (ALUOp) all'operazione (ALUCtl). b_{30} è il bit 30 dell'istruzione.

ALUOp	Famiglie	ALUCtl
`00` somma	LOAD, STORE, JALR, LUI, AUIPC	sempre ADD
`01` confronto	BRANCH	SUB se funct3 è `00x`, SLT se è `10x`, SLTU se è `11x`
`10` tipo R	OP	$\{b_{30}$, funct3$\}$
`11` tipo I	OP-IMM	$\{b_{30}$, funct3$\}$ se funct3 = `101`; altrimenti $\{0$, funct3$\}$

`1024`: l'immediato 1024 ha il bit 10 a 1, che nella codifica `0x40030293` occupa proprio il bit 30. Se il controllo dell'ALU considerasse il bit 30 anche qui, interpreterebbe la somma come una sottrazione. Per questo nelle istruzioni di tipo I il bit 30 conta solo negli scorrimenti a destra (funct3 = `101`), dove distingue `srli` da `srai`: in quelle istruzioni l'immediato è lungo solo 5 bit, e i bit 31–25 svolgono la stessa funzione di funct7.

!

Non esiste `subi`. Nello stato 0x1 abbiamo visto che per sottrarre una costante si somma il suo opposto. Ora si vede anche il motivo hardware: nel formato I non c'è spazio per distinguere somma e sottrazione, perché il bit che le distingue nelle istruzioni di tipo R lì appartiene all'immediato. E una `subi` sarebbe comunque inutile, visto che l'immediato può essere negativo.

4.3 Da dove arrivano gli operandi

A monte di ciascun ingresso dell'ALU è posto un multiplexer.

- **Op2Sel** seleziona il secondo operando: il valore di `rs2` per le operazioni fra registri e per i confronti dei salti, oppure l'immediato per tutte le altre istruzioni.
- **Op1Sel** seleziona il primo: quasi sempre `rs1`, ma il PC per `auipc` e la costante 0 per `lui`.

Le ultime due scelte trattano `lui` e `auipc` come due facce della stessa operazione: entrambe sommano un immediato di tipo U a un altro valore. `auipc` lo somma al PC; `lui` lo somma a zero, e quindi lo copia così com'è. È lo stesso accorgimento del registro `x0`: sommare zero è il modo più economico per lasciare invariato un valore. (Non si può usare `x0` come primo operando, perché nelle istruzioni di tipo U i bit che indicherebbero `rs1` fanno parte dell'immediato.)

Tabella 4.3 Il lavoro della fase EX per ogni famiglia di istruzioni. Il trattino indica che l'ALU non serve e il suo risultato viene ignorato.

Famiglia	A (Op1Sel)	B (Op2Sel)	Operazione	Il risultato è...
OP	`rs1`	`rs2`	dall'istruzione	il valore per `rd`
OP-IMM	`rs1`	imm (I)	dall'istruzione	il valore per `rd`
LOAD	`rs1`	imm (I)	ADD	l'indirizzo da leggere
STORE	`rs1`	imm (S)	ADD	l'indirizzo da scrivere
BRANCH	`rs1`	`rs2`	SUB, SLT, SLTU	il confronto
JAL	–	–	–	–
JALR	`rs1`	imm (I)	ADD	la destinazione del salto
LUI	0	imm (U)	ADD	il valore per `rd`
AUIPC	PC	imm (U)	ADD	il valore per `rd`

Numerando gli ingressi come nella figura del datapath, Op1Sel vale 0 per `rs1`, 1 per il PC e 2 per la costante 0; Op2Sel vale 0 per `rs2` e 1 per l'immediato. La tabella 4.3 riassume, famiglia per famiglia, che cosa entra nell'ALU e che cosa esce.

4.4 L'ALU che calcola indirizzi

Una load o una store non devono eseguire alcun calcolo: devono trasferire un dato. Eppure passano dall'ALU, che calcola il loro indirizzo come somma di un registro base e di un immediato. È un uso «opportunistico» del componente: nel ciclo singolo l'ALU sarebbe rimasta comunque inutilizzata, e il registro letto deve comunque attraversare il processore per arrivare alla memoria. Sfruttarla non costa nulla, e dà al programmatore un modo di indirizzare la memoria potente e naturale. È lo stesso calcolo che esegue il C per trovare un campo in una struttura o un elemento di un vettore con indice costante: il registro contiene l'indirizzo di partenza, l'immediato lo spiazzamento.

Per esempio, con `sp` = `0x7fffeff0`, l'istruzione `lw t1, 8(sp)` fa calcolare all'ALU `0x7fffeff0` + 8 = `0x7fffeff8`: è l'indirizzo che arriverà alla memoria dati nella fase successiva.

Anche `jalr` usa l'ALU per sommare `rs1` e l'immediato. Il risultato, però, non è destinato né alla memoria né a un registro, ma al multiplexer del PC, attraverso l'ingresso 2 della tabella 2.1. Lungo il percorso il suo bit 0 viene forzato a zero. Lo prescrive la specifica: semplifica un po' l'hardware e lascia libero il bit meno significativo dei puntatori a funzione, che alcuni sistemi usano per memorizzare un'informazione in più. In hardware non costa nulla: il filo del bit 0 semplicemente non viene collegato, e al suo posto c'è la costante 0.

Tabella 4.4 Quando un salto condizionato viene eseguito. r_0 è il bit 0 del risultato dell'ALU.

funct3	Istruzione	ALU	Salta se	Cioè
`000`	`beq`	SUB	zero = 1	$rs1 = rs2$
`001`	`bne`	SUB	zero = 0	$rs1 \neq rs2$
`100`	`blt`	SLT	$r_0 = 1$	$rs1 < rs2$
`101`	`bge`	SLT	$r_0 = 0$	$rs1 \geq rs2$
`110`	`bltu`	SLTU	$r_0 = 1$	$rs1 < rs2$ senza segno
`111`	`bgeu`	SLTU	$r_0 = 0$	$rs1 \geq rs2$ senza segno

4.5 Decidere un salto

Un salto condizionato richiede alla fase EX due operazioni nello stesso ciclo: *confrontare* due registri e *calcolare* l'indirizzo di destinazione, $PC + imm$. Sono due calcoli diversi, e l'ALU può eseguirne uno solo. Le soluzioni possibili sono due: un sommatore apposito per la destinazione, oppure un comparatore apposito per il confronto. Il nostro processore sceglie la prima.

Il sommatore dei salti

Il **sommatore dei salti** calcola $PC + imm$ per ogni istruzione, proprio come il sommatore della fase IF calcola $PC + 4$. Il risultato serve ai salti condizionati (immediato di tipo B) e a `jal` (immediato di tipo J), e arriva all'ingresso 1 del multiplexer del PC. Non è necessario alcuno scorrimento: come abbiamo visto nello stato 0x3, il generatore degli immediati fornisce già lo spiazzamento in byte.

Il confronto nell'ALU

Per il confronto l'ALU riceve `rs1` e `rs2` ed esegue una delle tre operazioni della riga `01` della tabella 4.2:

- per `beq` e `bne` una sottrazione: i due registri sono uguali se e solo se la differenza è zero, cioè se il segnale zero vale 1;
- per `blt` e `bge` un SLT: il bit 0 del risultato vale 1 se e solo se $rs1 < rs2$ con segno;
- per `bltu` e `bgeu` un SLTU, lo stesso confronto senza segno.

A quel punto la condizione si legge da uno di due segnali: zero oppure il bit 0 del risultato, che chiameremo r_0. La tabella 4.4 mostra come, e rivela una regolarità nascosta nei codici funct3.

Leggi la tabella colonna per colonna. Il bit 2 di funct3 seleziona il segnale da esaminare: zero se vale 0, r_0 se vale 1. Il bit 1 seleziona, per l'ALU, fra

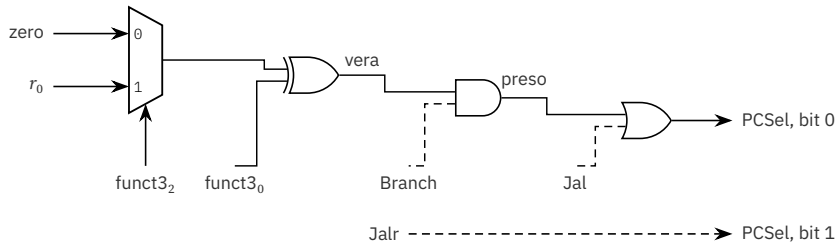


Figura 4.2 La logica dei salti. Il multiplexer seleziona il segnale da esaminare, lo XOR lo nega quando serve, l'AND lascia passare il risultato solo se l'istruzione è davvero un salto condizionato. Le linee tratteggiate arrivano dall'unità di controllo.

confronto con segno e senza. Il bit 0 *nega* la condizione: `bne` è `beq` al contrario, `bge` è `blt` al contrario, e così via. Nessuno dei sei salti richiede un caso speciale. La condizione si calcola quindi così:

$$vera = (\,\text{funct3}_2\ ?\ r_0\ :\ zero\,) \oplus \text{funct3}_0,$$

dove $\oplus$ è lo XOR, che inverte il valore quando funct3_0 vale 1.

La logica dei salti

Resta da determinare il valore di PCSel. Questo compito è svolto da un piccolo circuito, la **logica dei salti**, che riceve dall'unità di controllo tre segnali che indicano il tipo di salto dell'istruzione (Branch, Jal e Jalr: al più uno vale 1) e calcola i due bit di PCSel (figura 4.2):

- il bit 1 vale 1 solo per `jalr`, che riceve la destinazione dall'ALU;
- il bit 0 vale 1 se l'istruzione è una `jal`, oppure se è un salto condizionato e la condizione è vera: in entrambi i casi la destinazione è $PC + imm$.

Se nessuno dei due bit vale 1, PCSel vale 0 e il PC avanza di 4.

Seguiamo due salti. Sia `beq a0, a1, avanti` all'indirizzo `0x00400010`, con `avanti` 16 byte più avanti e `a0` = `a1` = 7. L'ALU calcola $7 - 7 = 0$, quindi zero = 1; funct3 = `000`, quindi si esamina zero senza negarlo: la condizione è vera. Branch vale 1, PCSel vale 1, e al fronte di salita il PC prende il valore del sommatore dei salti, `0x00400010` + 16 = `0x00400020`.

Ora `blt t0, t1, et` con `t0` = −1 e `t1` = 1. L'ALU esegue SLT: $-1 < 1$, quindi $r_0 = 1$ e il salto viene eseguito. Con `bltu` gli stessi registri darebbero il risultato opposto: senza segno, `t0` vale `0xffffffff`, il numero più grande che 32 bit possano contenere, e il salto non verrebbe eseguito.

> **Un'altra strada: il comparatore.** Nella figura del «datapath completo» che compare in molti materiali didattici, il confronto non è eseguito dall'ALU, ma da un blocco separato, la *Branch Logic*, che riceve direttamente `rs1` e `rs2`. Accanto c'è comunque un sommatore per la destinazione. È una scelta più costosa ma più flessibile: l'ALU resta libera, e il confronto, che non deve aspettare i multiplexer degli operandi, può essere anticipato. Nella pipeline sarà proprio questo a permettere di decidere i salti con un ciclo di anticipo (stato 0xb).

4.6 Overflow: il segnale che RISC-V non usa

Con 32 bit si rappresentano i numeri senza segno da 0 a $2^{32} - 1$ e quelli con segno da -2^{31} a $2^{31} - 1$. Il risultato di una somma può non essere rappresentabile.

- *Senza segno*, il risultato non è rappresentabile quando la somma produce un riporto oltre il bit 31: `0xffffffff` + 1 dà 0.
- *Con segno* si ha un **overflow** quando due numeri dello stesso segno danno un risultato di segno opposto: `0x7fffffff` + 1, cioè $2^{31} - 1$ più 1, dà `0x80000000`, che con segno vale -2^{31}. In una somma, due numeri di segno diverso non possono mai dare overflow; in una sottrazione, invece, l'overflow è possibile solo quando i due numeri hanno segno diverso. In hardware lo si riconosce confrontando il riporto che entra nel bit 31 con quello che ne esce: se sono diversi, c'è overflow.

Molte ALU hanno un'uscita per l'overflow, e molte architetture la usano. In x86 quasi ogni operazione aritmetica aggiorna un registro speciale di *flag*, bit che ricordano se il risultato era zero, negativo, se c'è stato un riporto o un overflow; in ARM i flag sono aggiornati dalle istruzioni che lo richiedono (con il suffisso S, come `adds`) e dai confronti. In entrambe le architetture i salti condizionati non confrontano registri, ma leggono quei bit. In MIPS l'istruzione `add` genera un'eccezione in caso di overflow (e per chi non la vuole esiste `addu`, che lo ignora).

RISC-V non adotta nessuna di queste soluzioni: non ha flag, e `add` non genera mai eccezioni. I flag costituiscono uno stato aggiuntivo, condiviso da tutte le istruzioni: ogni operazione lo modifica e ogni salto lo legge, e due istruzioni che non hanno nulla in comune finiscono per dipendere l'una dall'altra. Nella pipeline, e ancora di più nei processori che eseguono le istruzioni fuori ordine (*out-of-order*), è una complicazione costosa. RISC-V preferisce integrare il confronto nell'istruzione di salto, e lascia il controllo dell'overflow al software, che lo esegue solo quando serve.

!

In RISC-V l'overflow non genera eccezioni. Slide e dispense derivate da MIPS indicano spesso l'overflow fra le cause di eccezione delle istruzioni aritmetiche. In RISC-V non lo è: `add`, `addi` e `sub` producono sempre i 32 bit meno significativi del risultato, senza segnalare nulla. Il segnale di overflow, se l'ALU lo calcola, non è collegato ad alcun componente.

Quando serve, il controllo si scrive con un'istruzione in più per la somma senza segno, con tre per quella con segno. Per una somma senza segno basta verificare che il risultato non sia diventato più piccolo di un addendo:

```
        add  t0, t1, t2
        bltu t0, t1, overflow      # riporto: il risultato non sta in 32 bit
```

Per una somma con segno il ragionamento è questo: se `t2` è negativo la somma deve essere minore di `t1`, se è positivo o nullo deve essere maggiore o uguale. Se le due condizioni non coincidono, c'è stato overflow.

```
        add  t0, t1, t2
        slti t3, t2, 0             # t3 = 1 se t2 < 0
        slt  t4, t0, t1            # t4 = 1 se la somma è minore di t1
        bne  t3, t4, overflow      # devono coincidere
```

Sono le sequenze suggerite dalla specifica di RISC-V; il file `overflow.s` che accompagna il libro le prova su sei coppie di numeri, fra cui `0x7fffffff` + 1 e `0x80000000` + (−1).

1996 Trentasette secondi

Il 4 giugno 1996 il primo Ariane 5 decollò dalla Guyana francese. Il volo fu regolare per circa 37 secondi; poi il razzo deviò bruscamente e si distrusse. La commissione d'inchiesta trovò la causa in poche righe di software riprese dall'Ariane 4: una funzione del sistema di riferimento inerziale convertiva un numero in virgola mobile a 64 bit, legato alla velocità orizzontale, in un intero con segno a 16 bit. Sull'Ariane 4 quel valore era sempre rimasto abbastanza piccolo; l'Ariane 5, con velocità orizzontali molto più alte nei primi secondi di volo, lo fece traboccare. La conversione generò un errore che nessuno gestiva, il sistema si spense, e il suo gemello di riserva, che eseguiva lo stesso programma, si era spento un istante prima per lo stesso motivo. La funzione, per di più, dopo il decollo non serviva a nulla. Qui l'overflow non passò inosservato, come accadrebbe in RISC-V: generò un'eccezione. Ma un'eccezione che nessuno gestisce può fare più danni di un risultato sbagliato.

Tabella 4.5 Otto istruzioni nella fase EX. Valori di partenza: `x4` = `0x12345678`, `x5` = `0x0000ffff`, `a0` = `a1` = 7, `t0` = −1, `t1` = 1, `sp` = `0x7fffeff0`, `ra` = `0x00400024`; il PC è indicato fra parentesi quando serve.

Istruzione	A	B	ALU	Risultato e uso
`xor x8, x4, x5`	`12345678`	`0000ffff`	XOR	`1234a987`, per `x8`
`addi a0, a0, -1`	7	−1	ADD	6, per `a0`
`lw t1, 8(sp)`	`7fffeff0`	8	ADD	`7fffeff8`, indirizzo
`beq a0, a1, +16` (PC `00400010`)	7	7	SUB	0, zero = 1: salta a `00400020`
`blt t0, t1, et`	−1	1	SLT	1, $r_0 = 1$: salta
`lui t0, 0x12345`	0	`12345000`	ADD	`12345000`, per `t0`
`auipc t0, 0x10` (PC `00400008`)	`00400008`	`00010000`	ADD	`00410008`, per `t0`
`jalr x0, 0(ra)`	`00400024`	0	ADD	`00400024`, al PC

4.7 EX al lavoro

La tabella 4.5 segue otto istruzioni attraverso la fase EX. Conviene coprire le ultime due colonne e provare a compilarle autonomamente.

Due osservazioni. Nella riga di `addi`, l'operando B è −1 perché il generatore degli immediati lo ha già esteso a `0xffffffff`: l'ALU non distingue gli immediati, ma riceve soltanto due numeri. E nella riga di `beq` il risultato della sottrazione non viene utilizzato: conta solo il segnale zero. Il valore calcolato dall'ALU passerà comunque alle fasi successive, ma non verrà scritto in alcun elemento di stato, perché per un salto condizionato RegWriteEn e MemWrite valgono 0.

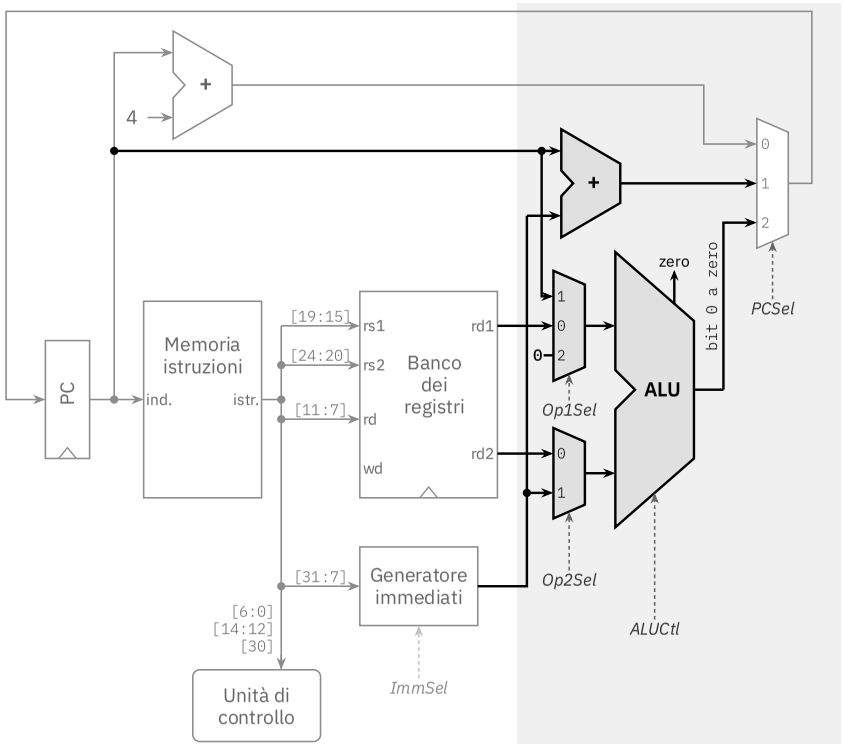


Figura 4.3 Il datapath finora. In nero la fase EX: i multiplexer degli operandi, l'ALU e il sommatore dei salti, che invia $PC + imm$ al multiplexer del PC. Il risultato dell'ALU raggiunge anche il multiplexer del PC, per `jalr`. Il segnale PCSel è prodotto dalla logica dei salti della figura 4.2.

Condizione di transizione uscita verso **0x5**

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Quali uscite ha l'ALU del nostro processore, e quanti bit trasporta ciascuna? ↺ § 4.1

2 Perché nelle istruzioni di tipo I il controllo dell'ALU considera il bit 30 solo quando funct3 vale `101`? ↺ § 4.2

3 Che valori assumono Op1Sel e Op2Sel per `lw`, `sw`, `add`, `beq` e `lui`? ↺ § 4.3

4 Durante una `sw`, che cosa calcola l'ALU? ↺ § 4.4

5 Perché la destinazione di un salto condizionato non è calcolata dall'ALU? ↺ § 4.5

6 Per `bgeu`, quale operazione esegue l'ALU e quale segnale determina il salto? ↺ § 4.5

7 Che cosa produce `add` in RISC-V se `t1` vale `0x7fffffff` e `t2` vale `1`? Viene segnalato qualcosa? ↺ § 4.6

8 Scrivi le istruzioni che controllano se la somma senza segno di `t1` e `t2` è traboccata. ↺ § 4.6

STATO

0x5

0	1	0	1
b3	b2	b1	b0

Dentro l'ALU: l'aritmetica

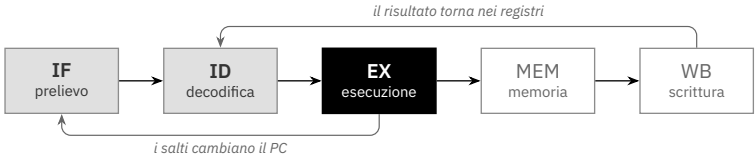


→ INGRESSI

- Numeri binari e complemento a 2 (appendice A).
- L'ALU vista dall'esterno e l'overflow (stato 0x4, § 4.1 e § 4.6).
- Il multiplexer (stato 0x2, § 2.7).

USCITE →

- Costruire un sommatore a partire dalle porte logiche e spiegare perché è lento.
- Spiegare il calcolo anticipato dei riporti.
- Moltiplicare e dividere in binario come fa l'hardware, anche con l'algoritmo di Booth.
- Rappresentare un numero reale in IEEE 754 e prevederne gli errori di arrotondamento.



Nello stato 0x4 abbiamo usato l'ALU come una scatola chiusa: due numeri entrano, il risultato esce. Qui la apriamo. È una deviazione dal viaggio dell'istruzione, e per questo nella mappa degli stati il cerchio è tratteggiato: alla prima lettura puoi passare direttamente allo stato 0x6 senza perdere il filo.

Tabella 5.1 La tabella della verità di un full adder: la somma di tre bit, scritta come riporto e cifra.

a_i	b_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Ma se ti sei mai chiesto come una rete di porte logiche riesca a sommare, moltiplicare o rappresentare un numero come 0,1, è qui che lo scopri.

5.1 Sommare un bit alla volta

La somma in binario si esegue come la somma in colonna studiata a scuola: da destra verso sinistra, una cifra alla volta, con il riporto che si trasmette alla cifra successiva. Per ogni colonna servono tre ingressi, i due bit a_i e b_i e il riporto in arrivo c_i, e due uscite, il bit della somma s_i e il riporto c_{i+1} verso la colonna successiva. Il circuito che svolge questa funzione si chiama **full adder** (sommatore completo), e la sua tabella della verità è la tabella 5.1.

Dalla tabella si ricavano due equazioni. Il bit della somma vale 1 quando gli ingressi a 1 sono in numero dispari, cioè è lo XOR dei tre; il riporto vale 1 quando almeno due ingressi valgono 1:

$$s_i = a_i \oplus b_i \oplus c_i, \qquad c_{i+1} = a_i b_i + (a_i \oplus b_i)\, c_i,$$

dove il prodotto indica l'AND e la somma l'OR. Bastano cinque porte logiche, perché $a_i \oplus b_i$ serve sia alla somma sia al riporto.

Il sommatore a propagazione del riporto

Per sommare due numeri da 32 bit si collegano in cascata 32 full adder, e il riporto in uscita da ciascuno diventa il riporto in ingresso del successivo (figura 5.1). È il **sommatore a propagazione del riporto** (*ripple carry adder*): semplice, piccolo, e lento.

Il problema è il tempo. Il bit s_{31} non può essere calcolato finché non arriva c_{31}, che dipende da c_{30}, che dipende da c_{29}... fino a c_0. Ogni full adder aggiunge il ritardo di un paio di porte logiche, quindi nel caso peggiore il riporto attraversa una sessantina di porte prima che il risultato sia stabile.

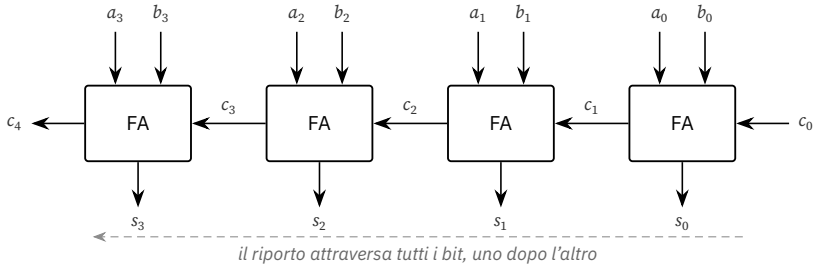


Figura 5.1 Un sommatore da 4 bit a propagazione del riporto. Il bit meno significativo è a destra, come nei numeri scritti. Per la versione da 32 bit servono 32 blocchi FA.

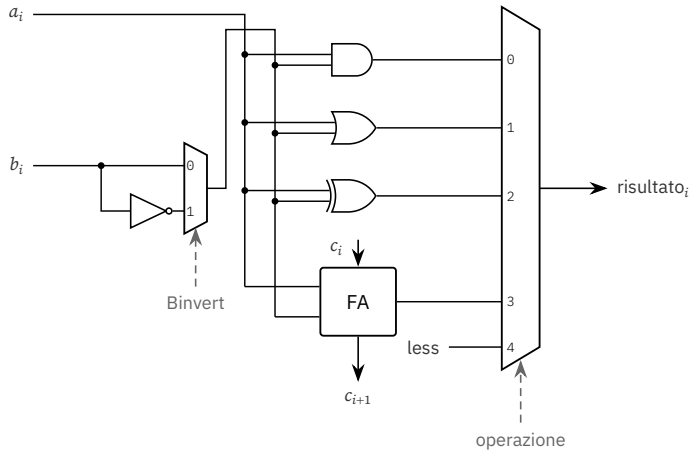


Figura 5.2 Una cella da un bit (*bit-slice*) dell'ALU. Binvert seleziona b_i o il suo negato; il multiplexer di destra seleziona il risultato. L'ingresso *less* serve a SLT: vale 0 in tutte le celle tranne la prima, come spiegato nel testo.

È il caso, per esempio, di `0xffffffff` + 1, in cui il riporto generato nel bit 0 deve propagarsi fino al bit più significativo. Nel ciclo singolo il periodo di clock deve essere abbastanza lungo da comprendere questo ritardo.

5.2 Un'ALU da un bit

Un'ALU completa si costruisce allo stesso modo: si progetta il circuito per un solo bit, una *cella da un bit* (*bit-slice*), e se ne collegano 32 in cascata. La figura 5.2 mostra una cella capace di eseguire AND, OR, XOR, somma e sottrazione: le operazioni logiche sono semplici porte, la somma è un full adder, e un multiplexer seleziona il risultato da presentare in uscita.

La sottrazione costa un invertitore

Per sottrarre non serve un circuito nuovo. In complemento a 2 l'opposto di b si ottiene invertendo tutti i bit e sommando 1, quindi

$$a - b = a + \overline{b} + 1.$$

L'ALU inverte i bit di b con il multiplexer comandato da *Binvert* e aggiunge l'1 come riporto entrante del primo full adder, $c_0 = 1$. Lo stesso sommatore esegue somme e sottrazioni.

La numerazione degli ingressi del multiplexer è interna alla cella, e non coincide con il codice ALUCtl della tabella 4.1: un piccolo circuito traduce ALUCtl nei tre segnali Binvert, c_0 e *operazione*. ADD diventa 0, 0, 3; SUB diventa 1, 1, 3; AND, OR e XOR usano gli ingressi 0, 1 e 2 con Binvert = 0. Gli scorrimenti non coinvolgono le celle: sono eseguiti da un circuito separato, come vedremo fra poco.

Zero, overflow e confronti

Collegando in cascata 32 celle si ottiene l'ALU intera, e i segnali che ne escono si calcolano con poca logica in più.

- *zero* è un NOR dei 32 bit del risultato: vale 1 solo se sono tutti 0.
- L'*overflow* si calcola nell'ultima cella, confrontando il riporto che entra nel bit 31 con quello che ne esce, come visto nello stato 0x4.
- SLTU ($a < b$ senza segno) è il riporto finale della sottrazione negato: $a + \overline{b} + 1$ produce un riporto oltre il bit 31 se e solo se $a \geq b$.
- SLT ($a < b$ con segno) esamina il bit del segno della sottrazione. Il risultato del confronto, un solo bit, viene scritto nel bit 0 del risultato attraverso l'ingresso *less* della prima cella; tutte le altre celle ricevono 0.

!

SLT non è solo il segno di $a - b$. Molti testi riassumono SLT con «$a < b \Rightarrow a - b < 0$», ma il segno della sottrazione basta solo se non c'è overflow. Prendiamo $a =$ `0x7fffffff`, il più grande positivo, e $b = -1$. La differenza esatta è 2^{31}, che non è rappresentabile in 32 bit con segno: il risultato è `0x80000000`, con il bit del segno a 1, e chi esaminasse soltanto quel bit concluderebbe $a < b$. La regola corretta è $less = s_{31} \oplus V$, dove V è l'overflow: quando c'è overflow, il segno va letto al contrario.

Gli scorrimenti

Gli scorrimenti sono eseguiti da un circuito separato, lo **shifter a barile** (*barrel shifter*). Ha cinque livelli di multiplexer: il primo fa scorrere il dato di 16 posizioni oppure lo lascia invariato, il secondo di 8, poi di 4, di 2 e di 1, e

ogni livello è comandato da uno dei cinque bit di B che indicano di quanto scorrere. Per far scorrere di $13 = 01101_2$ posizioni si attivano i livelli da 8, da 4 e da 1. In cinque passi si ottiene qualunque scorrimento da 0 a 31.

5.3 Il problema del riporto

Il sommatore a propagazione è lento perché ogni riporto attende il precedente. Ma si può scrivere ogni riporto direttamente in funzione degli ingressi, senza attendere i riporti precedenti. Si definiscono per ogni bit due segnali:

$$g_i = a_i\, b_i \qquad\qquad p_i = a_i + b_i.$$

Il bit *i genera* un riporto ($g_i = 1$) se a_i e b_i valgono entrambi 1, e *propaga* un riporto in arrivo ($p_i = 1$) se almeno uno dei due vale 1. (Si può usare anche $p_i = a_i \oplus b_i$, come nel full adder: i riporti non cambiano.) L'equazione del riporto diventa $c_{i+1} = g_i + p_i c_i$, e sostituendo una dentro l'altra:

$$\begin{aligned}
c_1 &= g_0 + p_0 c_0 \\
c_2 &= g_1 + p_1 g_0 + p_1 p_0 c_0 \\
c_3 &= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \\
c_4 &= g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0
\end{aligned}$$

Ogni riporto è un OR di AND: due soli livelli di porte, qualunque sia la sua posizione. Questo è il **carry lookahead**, il calcolo anticipato dei riporti. Le equazioni si leggono a parole: c'è un riporto in ingresso al bit 4 se il bit 3 lo genera, oppure se lo genera il bit 2 e il bit 3 lo propaga, e così via.

Facciamo una prova con 0110 + 0111 (6 + 7). I segnali valgono $g = 0110$ e $p = 0111$ (letti dal bit 3 al bit 0); con $c_0 = 0$ si ottiene $c_1 = 0$, $c_2 = 1$, $c_3 = 1$, $c_4 = 0$, e la somma è 1101 = 13.

C'è un prezzo. L'ultimo termine di c_4 è già un AND a cinque ingressi, e per c_{32} servirebbero porte con decine di ingressi, che nella pratica non si possono costruire: ogni porta reale accetta pochi ingressi prima di diventare a sua volta lenta. La soluzione è a livelli. Si raggruppano i bit a quattro a quattro, si calcolano per ogni gruppo un «genera» e un «propaga» di gruppo, e si applica la stessa idea ai gruppi. Era esattamente l'organizzazione dei minicomputer degli anni Settanta: il chip 74181 dello stato 0x4 conteneva un gruppo da 4 bit, e un chip compagno, il 74182, calcolava in anticipo i riporti fra quattro 74181.

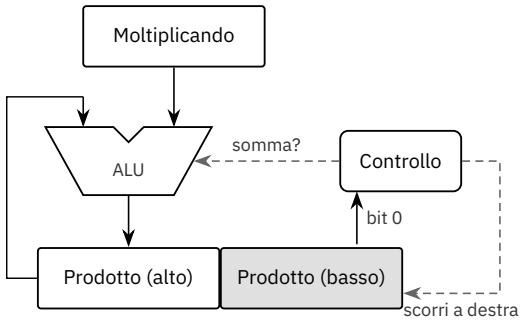


Figura 5.3 Il moltiplicatore «somma e scorri». Il registro del prodotto è lungo il doppio degli operandi; la sua metà bassa contiene all'inizio il moltiplicatore, che scompare un bit alla volta mentre entra il prodotto.

Tabella 5.2 6×5 con l'algoritmo somma e scorri, su 4 bit: moltiplicando `0110`, moltiplicatore `0101`.

Passo	Bit 0	Azione	Prodotto (alto, basso)
–		valori iniziali	`0000 0101`
1	1	somma, poi scorri	`0110 0101` → `0011 0010`
2	0	solo scorri	`0001 1001`
3	1	somma, poi scorri	`0111 1001` → `0011 1100`
4	0	solo scorri	`0001 1110` = 30

5.4 Moltiplicare

In binario la moltiplicazione in colonna è più facile che in decimale, perché ogni cifra del moltiplicatore vale 0 o 1: il prodotto parziale è zero oppure il moltiplicando stesso, fatto scorrere di qualche posizione. Moltiplicare si riduce a sommare e far scorrere. Il prodotto di due numeri da n bit può occupare fino a $2n$ bit: due numeri da 32 bit danno un risultato da 64.

Somma e scorrimento

Il circuito più semplice (figura 5.3) usa un registro del prodotto lungo il doppio. All'inizio la metà destra contiene il moltiplicatore e la metà sinistra è azzerata. A ogni passo si esamina il bit meno significativo: se vale 1 si somma il moltiplicando alla metà sinistra, poi si fa scorrere tutto il registro di una posizione a destra. Dopo n passi il registro contiene il prodotto. La tabella 5.2 segue 6×5 su 4 bit.

In generale la somma può produrre un riporto oltre la metà alta: per questo il registro del prodotto ha un bit in più a sinistra, e quel bit entra nella metà alta con lo scorrimento. Con 32 bit servono 32 passi, cioè 32 cicli di clock:

Tabella 5.3 $2 \times (-3)$ con l'algoritmo di Booth, su 4 bit: moltiplicando $M = \texttt{0010}$, moltiplicatore `1101`. q_{-1} è il bit appena scartato.

Passo	$q_0\, q_{-1}$	Azione	Dopo l'azione	Dopo lo scorrimento
–				`0000 1101`, $q_{-1} = 0$
1	`10`	sottrai M	`1110 1101`	`1111 0110`, $q_{-1} = 1$
2	`01`	somma M	`0001 0110`	`0000 1011`, $q_{-1} = 0$
3	`10`	sottrai M	`1110 1011`	`1111 0101`, $q_{-1} = 1$
4	`11`	niente	`1111 0101`	`1111 1010` $= -6$

troppi per un processore progettato per eseguire un'istruzione per ciclo. I moltiplicatori reali calcolano tutti i prodotti parziali insieme e li sommano con un albero di sommatori, in pochi cicli, al prezzo di molti più transistor.

L'algoritmo di Booth

Nel 1950 Andrew Booth, che al Birkbeck College di Londra si occupava di cristallografia, notò che una fila di 1 nel moltiplicatore si può trattare con una sola sottrazione e una sola somma. Per esempio $0110 = 1000 - 0010$: invece di sommare il moltiplicando due volte, lo si sottrae una volta in corrispondenza del primo 1 e lo si somma una volta dopo l'ultimo. In pratica a ogni passo si esaminano due bit: quello corrente del moltiplicatore e quello appena scartato (all'inizio uno 0 fittizio).

- $\texttt{10}_2$: inizia una fila di 1, si *sottrae* il moltiplicando;
- $\texttt{01}_2$: la fila è finita, si *somma* il moltiplicando;
- $\texttt{00}_2$ o $\texttt{11}_2$: dentro una fila di 0 o di 1, non si esegue alcuna operazione.

Poi si fa scorrere a destra il registro, ma in modo *aritmetico*, cioè copiando il bit del segno. La tabella 5.3 moltiplica $2 \times (-3)$ e mostra il vero vantaggio dell'algoritmo: funziona direttamente in complemento a 2, senza dover prima rendere positivi gli operandi e poi correggere il segno.

Attenzione a un luogo comune: la versione appena vista non dimezza il numero di passi, che restano uno per bit. Riduce le somme quando nel moltiplicatore ci sono lunghe file di 1, e può perfino aumentarle con bit alternati come `0101`. La versione usata nei moltiplicatori veloci esamina tre bit alla volta e avanza di due posizioni per passo: quella sì dimezza i prodotti parziali.

Moltiplicare in RISC-V

Le istruzioni di moltiplicazione e divisione non fanno parte di RV32I: sono introdotte dall'estensione M (§ 1.12). Siccome un registro contiene solo 32 bit,

Tabella 5.4 7 : 2 con la divisione con ripristino, su 4 bit.

Passo	Dopo lo scorrimento	Resto − 2	Azione	Resto, quoziente
–			valori iniziali	`0000 0111`
1	`0000 1110`	−2	ripristina, bit 0	`0000 1110`
2	`0001 1100`	−1	ripristina, bit 0	`0001 1100`
3	`0011 1000`	1	tieni, bit 1	`0001 1001`
4	`0011 0010`	1	tieni, bit 1	`0001 0011`

il prodotto da 64 si ottiene in due metà: `mul` restituisce i 32 bit bassi, `mulh` quelli alti, considerando entrambi gli operandi con segno, `mulhu` senza segno e `mulhsu` con il primo con segno e il secondo senza. La parte bassa è la stessa con o senza segno, per questo basta una sola `mul`.

```
        li    t1, 100000
        li    t2, 300000
        mul   a0, t1, t2      # a0 = 0xfc23ac00, i 32 bit bassi
        mulhu a1, t1, t2      # a1 = 0x00000006, i 32 bit alti
```

Il prodotto esatto è $3 \times 10^{10} = \texttt{0x6fc23ac00}$: non è rappresentabile in 32 bit, e `mul` da sola ne restituirebbe soltanto la parte bassa, senza alcun avviso.

5.5 Dividere

La divisione è l'operazione più complessa: anche qui si fa scorrere e si sottrae, ma non si sa in anticipo se la sottrazione si potrà eseguire. Il metodo più semplice, la **divisione con ripristino**, usa un registro doppio: la metà sinistra contiene il resto parziale, la destra all'inizio il dividendo e alla fine il quoziente. A ogni passo:

1. si fa scorrere tutto il registro di una posizione a sinistra;
2. si sottrae il divisore dal resto parziale;
3. se il risultato è negativo, il divisore «non ci stava»: si ripristina il resto sommando di nuovo il divisore, e il nuovo bit del quoziente è 0; altrimenti si conserva la differenza e il bit è 1.

Alla fine (tabella 5.4) il quoziente è `0011` = 3 e il resto `0001` = 1. La variante *senza ripristino* evita la somma di correzione: se la sottrazione ha dato un risultato negativo, al passo successivo invece di sottrarre si somma, e il risultato finale è comunque corretto. I divisori veloci, come quello dei Pentium, producono più bit del quoziente per passo, aiutandosi con una tabella che suggerisce la cifra successiva.

Dividere in RISC-V

L'estensione M offre `div` e `divu` per il quoziente, `rem` e `remu` per il resto. La divisione con segno arrotonda verso zero, e il resto ha il segno del dividendo: $-7 \div 2$ dà quoziente -3 e resto -1. Ma il caso più interessante è un altro: che cosa succede dividendo per zero?

In RISC-V non accade nulla di grave: non viene sollevata alcuna eccezione. Il quoziente ha tutti i bit a 1 (cioè -1 per `div`, $2^{32}-1$ per `divu`) e il resto è uguale al dividendo. La specifica spiega il motivo: sarebbe l'unica eccezione aritmetica dell'intero set di istruzioni, e ai linguaggi che la richiedono basta un salto, `beqz` sul divisore, prima della divisione. Anche l'unico caso di overflow, $-2^{31} \div (-1)$, il cui risultato esatto 2^{31} non è rappresentabile in 32 bit, ha un risultato fissato: quoziente -2^{31} e resto 0. Il file `muldiv.s` prova tutti questi casi con RARS.

1994 L'errore da 475 milioni di dollari

Nell'estate del 1994 Thomas Nicely, un professore di matematica che studiava i reciproci dei numeri primi gemelli, si accorse che il suo Pentium nuovo sbagliava: il reciproco di 824 633 702 441 usciva leggermente diverso da quello calcolato dai suoi vecchi computer. A ottobre, dopo mesi di verifiche, lo rese pubblico. Il divisore in virgola mobile del Pentium usava un algoritmo, detto SRT, che produceva due bit del quoziente per passo consultando una tabella di 2048 caselle; a causa di un errore nello script che l'aveva caricata nel circuito, alcune caselle erano rimaste vuote. L'errore si verificava circa una volta su nove miliardi di divisioni casuali, e Intel all'inizio lo considerò trascurabile. La stampa non fu d'accordo: a dicembre l'azienda accettò di sostituire tutti i processori difettosi, con una spesa di 475 milioni di dollari.

5.6 I numeri reali: la virgola mobile

Gli interi non bastano: servono numeri con la virgola, molto grandi o molto piccoli. L'idea è la notazione scientifica, $6{,}02 \times 10^{23}$, ma in base 2: un numero si scrive come

$$(-1)^s \times 1{,}f \times 2^e,$$

con un *segno* s, una *mantissa* $1{,}f$ e un *esponente* e. Lo standard che stabilisce come codificarli nei bit è l'**IEEE 754**, adottato da praticamente ogni processore.

Il formato

Un numero in *singola precisione* (il `float` del C) occupa 32 bit: 1 per il segno, 8 per l'esponente e 23 per la parte frazionaria; uno in *doppia precisione*



Figura 5.4 I formati IEEE 754 in singola e in doppia precisione. Nella seconda riga la scala non è quella dei bit: i campi sono ristretti per rientrare nella pagina.

(il `double`) 64 bit: 1, 11 e 52 (figura 5.4). Tre scelte rendono il formato più ingegnoso di quanto sembri.

- *Il bit nascosto.* Un numero normalizzato in base 2 comincia sempre con un 1 prima della virgola: allora è inutile memorizzarlo. Si salva solo la parte frazionaria f, e la precisione guadagna un bit senza alcun costo.
- *L'esponente con eccesso.* L'esponente non è in complemento a 2: si memorizza $e + 127$ (in doppia precisione $e + 1023$). Il numero 127 si chiama *bias*. Così il campo è sempre un numero senza segno.
- *Il segno davanti, poi l'esponente.* Con l'esponente in eccesso, collocato prima della parte frazionaria, due numeri positivi si confrontano come se fossero interi: il numero con l'esponente più grande ha il campo più grande, e a parità di esponente decide la parte frazionaria. Il confronto fra numeri reali può usare lo stesso circuito degli interi, con una piccola correzione per il segno.

Un esempio: $-0{,}75 = -0{,}11_2 = -1{,}1_2 \times 2^{-1}$. Il segno è 1, l'esponente memorizzato $-1 + 127 = 126 = 01111110_2$, la parte frazionaria $1000\ldots0_2$:

$$\texttt{1 01111110 10000000000000000000000} = \texttt{0xbf400000}.$$

I valori speciali

Due configurazioni dell'esponente sono riservate (tabella 5.5). L'esponente con tutti i bit a 1 indica l'infinito, che si ottiene per esempio dividendo per zero un numero diverso da zero, oppure un *NaN* (*Not a Number*), il risultato di operazioni senza senso come $0/0$ o $\sqrt{-1}$. L'esponente con tutti i bit a 0 indica lo zero e i numeri *denormalizzati*: valori così vicini a zero che non si possono normalizzare, per i quali il bit nascosto vale 0 e l'esponente è fissato a -126. Riempiono, con precisione sempre minore, il vuoto fra lo zero e il più piccolo numero normalizzato, 2^{-126}.

I numeri normalizzati vanno quindi da $2^{-126} \approx 1{,}2 \times 10^{-38}$ a $(2 - 2^{-23}) \times 2^{127} \approx 3{,}4 \times 10^{38}$, con circa sette cifre decimali significative; in doppia precisione con circa sedici.

Tabella 5.5 Che cosa rappresenta un numero in singola precisione, a seconda dell'esponente memorizzato e della parte frazionaria.

Esponente	Parte frazionaria	Valore
0	0	± 0
0	diversa da 0	denormalizzato: $\pm 0{,}f \times 2^{-126}$
da 1 a 254	qualunque	normalizzato: $\pm 1{,}f \times 2^{e-127}$
255	0	$\pm\infty$
255	diversa da 0	NaN

Arrotondare

La maggior parte dei numeri reali non ha una rappresentazione esatta. Anche un numero innocuo come 0,1, in binario, è periodico: $0{,}0\overline{0011}_2$, e in singola precisione diventa `0x3dcccccd`, cioè 0,100000001490116 … Ogni operazione deve quindi *arrotondare* il risultato esatto a un numero rappresentabile. Lo standard del 1985 prevedeva quattro modi: verso il più vicino (quello predefinito), verso zero, verso $+\infty$ e verso $-\infty$; la revisione del 2008 ne aggiunge un quinto, e RISC-V li offre tutti. Nel primo, quando il risultato cade esattamente a metà fra due numeri, si sceglie quello con l'ultimo bit pari, così gli errori non si accumulano sempre nella stessa direzione.

Per arrotondare bene, durante il calcolo l'hardware mantiene tre bit aggiuntivi a destra della parte frazionaria: il bit di *guardia*, quello di *arrotondamento* e lo *sticky bit*, che vale 1 se almeno un bit diverso da zero è uscito a destra durante gli scorrimenti. I tre bit insieme indicano se il valore scartato è meno della metà, esattamente la metà o più della metà di un'unità dell'ultimo bit.

Sommare e moltiplicare

Sommare due numeri in virgola mobile è molto più laborioso che sommare due interi. Vediamolo con $0{,}5 + (-0{,}4375)$, usando 4 bit di mantissa.

1. *Si ripristina il bit nascosto*: $1{,}000 \times 2^{-1}$ e $-1{,}110 \times 2^{-2}$.
2. *Si allineano gli esponenti*, facendo scorrere a destra la mantissa del numero con l'esponente minore: $-1{,}110 \times 2^{-2} = -0{,}111 \times 2^{-1}$.
3. *Si sommano le mantisse*: $1{,}000 - 0{,}111 = 0{,}001$, quindi $0{,}001 \times 2^{-1}$.
4. *Si normalizza* il risultato, controllando che l'esponente non esca dai limiti: $1{,}000 \times 2^{-4}$.
5. *Si arrotonda* (qui non serve) e, se l'arrotondamento ha alterato la normalizzazione, si normalizza di nuovo.
6. *Si rimuove il bit nascosto* e si memorizza: 0,0625.

La moltiplicazione è più semplice: si sommano gli esponenti, si moltiplicano le mantisse, si normalizza e si arrotonda; il segno è lo XOR dei due segni. L'unica insidia è il bias. Sommando i due esponenti memorizzati si conta il bias due volte, e quindi va sottratto una volta: per $0{,}5 \times (-0{,}4375)$ si ottiene $126 + 125 - 127 = 124$, cioè 2^{-3}, e il risultato è $-1{,}110 \times 2^{-3} = -0{,}21875$.

La virgola mobile in RISC-V

I numeri reali hanno in RISC-V registri propri: l'estensione F (singola precisione) aggiunge 32 registri `f0`–`f31`, che con l'estensione D (doppia precisione) diventano da 64 bit, e le istruzioni per operare su di essi, da `flw` e `fsw` per la memoria a `fadd.s`, `fmul.s`, `fdiv.s` e `fsqrt.s`. I confronti `feq.s`, `flt.s` e `fle.s` scrivono 0 o 1 in un registro intero, così i salti restano quelli che già conosciamo. E anche qui nessuna operazione genera eccezioni: overflow, divisione per zero e risultati inesatti vengono annotati in un registro di controllo, il `fcsr`, che il programma può consultare quando vuole.

Ecco un esperimento che ogni programmatore dovrebbe fare almeno una volta: sommare 0,1 e 0,2 e confrontare il risultato con 0,3, in singola e in doppia precisione.

```
        .data
sing:   .float  0.1, 0.2, 0.3
dopp:   .double 0.1, 0.2, 0.3
        .text
main:   la     t0, sing
        flw    ft0, 0(t0)
        flw    ft1, 4(t0)
        flw    ft2, 8(t0)
        fadd.s ft3, ft0, ft1
        feq.s  a0, ft3, ft2       # 1 se 0.1 + 0.2 == 0.3
        # ...stampa a0, poi la stessa cosa in doppia precisione
        la     t0, dopp
        fld    fa0, 0(t0)
        fld    fa1, 8(t0)
        fld    fa2, 16(t0)
        fadd.d fa3, fa0, fa1
        feq.d  a0, fa3, fa2
```

Il programma completo, `virgola.s`, stampa che in singola precisione la somma è uguale a 0,3, e in doppia no: vale 0,30000000000000004. Ciò non significa che la doppia precisione sia peggiore: in singola gli errori di arrotondamento di 0,1 e 0,2 si sono compensati per caso. La lezione è che due numeri in virgola mobile non si confrontano mai con l'uguaglianza esatta, ma controllando che la loro differenza sia abbastanza piccola.

1985 **Un solo modo di sbagliare**

Fino agli anni Settanta ogni costruttore aveva la sua virgola mobile: binaria, ottale o esadecimale, come quella dei mainframe IBM, con esponenti, arrotondamenti e casi limite diversi. Lo stesso programma dava risultati diversi su macchine diverse, e a volte risultati sbagliati su tutte. Nel 1976 Intel chiese a William Kahan, matematico di Berkeley, di aiutarla a progettare l'aritmetica del suo coprocessore 8087; Kahan spinse perché quel lavoro diventasse uno standard per tutti. La proposta, scritta nel 1978 con Jerome Coonen e Harold Stone, fu adottata nel 1985 come IEEE 754. Nel 1989 Kahan ricevette il premio Turing. È grazie a quello standard che 0,1 + 0,2 dà lo stesso risultato, leggermente sbagliato, su ogni calcolatore del mondo.

Condizione di transizione uscita verso `0x6`

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1. Scrivi le equazioni della somma e del riporto di un full adder. ↺ § 5.1
2. Perché un sommatore a propagazione del riporto da 32 bit è lento? Qual è il caso peggiore? ↺ § 5.1
3. Come fa l'ALU a sottrarre usando un sommatore? ↺ § 5.2
4. Perché SLT non può limitarsi a esaminare il bit del segno di $a - b$? ↺ § 5.2
5. Calcola g, p e i riporti di 1011 + 0110, con $c_0 = 0$. ↺ § 5.3
6. Quanti bit occupa il prodotto di due numeri da 32 bit? Con quali istruzioni RISC-V si ottiene tutto? ↺ § 5.4
7. Nell'algoritmo di Booth, che cosa si fa quando la coppia di bit è `10`? E quando è `01`? ↺ § 5.4
8. Che cosa restituisce `div` in RISC-V se il divisore è zero? ↺ § 5.5
9. Codifica 6,5 in singola precisione. ↺ § 5.6
10. Perché non si confrontano due numeri in virgola mobile con l'uguaglianza esatta? ↺ § 5.6

STATO

0x6

0	1	1	0
b3	b2	b1	b0

MEM: accedere alla memoria

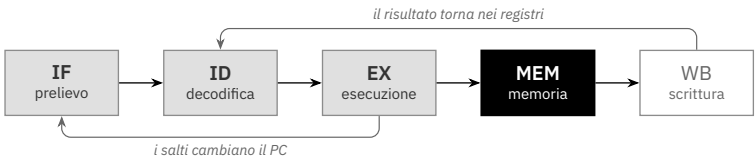


→ INGRESSI

- Le istruzioni di load e store, l'ordine little-endian e l'allineamento (stato 0x1, § 1.4).
- Il calcolo dell'indirizzo nell'ALU (stato 0x4, § 4.4).
- L'immediato di tipo S (stato 0x3, § 3.3).

USCITE →

- Descrivere la memoria dati e i segnali che la comandano.
- Spiegare come si leggono e si scrivono byte e mezze parole in una memoria organizzata a parole.
- Spiegare perché gli accessi non allineati sono costosi.
- Spiegare perché MEM è la fase più lenta, e che cosa cambia nei processori reali.



La quarta fase è la più semplice da descrivere e la più difficile da rendere veloce. Delle nove famiglie di istruzioni che il nostro processore esegue, solo due la usano davvero: le load, che leggono un dato dalla memoria, e le store, che vi scrivono un dato. Per tutte le altre la fase MEM è un corridoio da attraversare senza fermarsi. Ma in un programma reale le load e le store

sono una parte consistente delle istruzioni eseguite, una su tre o quattro, e il tempo che vi trascorrono incide sulle prestazioni complessive.

6.1 La memoria dati

La **memoria dati** ha la stessa forma della memoria istruzioni, con in più la possibilità di scrivere.

- L'*indirizzo* arriva dall'ALU, che lo ha calcolato nella fase EX sommando `rs1` e l'immediato.
- Il *dato da scrivere* arriva direttamente dalla seconda porta di lettura del banco dei registri, `rd2`: in una store `rs2` non indica un operando ma il registro da salvare. È il lungo filo che nel datapath passa sotto l'ALU.
- **MemWrite** autorizza la scrittura, che come quella dei registri avviene al fronte di salita del clock, alla fine dell'istruzione.
- Il campo funct3 specifica la *dimensione* dell'accesso (byte, mezza parola o parola) e, per le letture, come estendere il dato.
- Il *dato letto* esce verso la fase WB.

La lettura, come per la memoria istruzioni, nel nostro modello è combinatoria: avviene sempre, per ogni istruzione, e i componenti che non hanno bisogno del risultato lo ignorano. Per una `add` la memoria legge quindi un indirizzo qualunque, quello prodotto dall'ALU, e il valore letto si perde. In un sistema reale questa disinvoltura non è ammessa: certi indirizzi corrispondono a dispositivi di ingresso e uscita, per i quali una lettura ha degli effetti (stato 0x10), e un indirizzo arbitrario può generare un'eccezione di memoria virtuale (stato 0xe). Per questo i processori reali hanno anche un segnale *MemRead*, che autorizza la lettura solo alle load. Nel nostro modello possiamo farne a meno.

6.2 Byte, mezze parole e parole

La memoria è indirizzata al byte, ma è costruita a parole: a ogni accesso legge o scrive una parola intera, quattro byte allineati. Le load e le store che operano su un byte o su una mezza parola devono quindi selezionare i byte corretti all'interno della parola. A guidare la scelta sono due informazioni: i due bit meno significativi dell'indirizzo, che indicano *dove* si trova il dato all'interno della parola, e funct3, che ne specifica *la dimensione* (tabella 6.1).

Ancora una volta funct3 nasconde una regolarità, come per i salti condizionati: i bit 1–0 valgono 00_2, 01_2 o 10_2 per 1, 2 o 4 byte, e il bit 2 vale 1 per le versioni *unsigned*. L'hardware non ha bisogno di riconoscere le istruzioni

Tabella 6.1 funct3 nelle load e nelle store. I due bit bassi specificano la dimensione; quello alto indica, solo nelle load, se l'estensione è senza segno.

funct3	Istruzioni	Dimensione	Estensione (solo load)
`000`	`lb`, `sb`	byte	con il segno
`001`	`lh`, `sh`	mezza parola	con il segno
`010`	`lw`, `sw`	parola	–
`100`	`lbu`	byte	con zeri
`101`	`lhu`	mezza parola	con zeri

Tabella 6.2 Quattro load sulla parola `0xcafebabe`, che in memoria occupa i byte `be ba fe ca` a partire da `0x10010000`. `s0` contiene l'indirizzo della parola. Risultati verificati con RARS.

Istruzione	Byte letti	Estensione	Risultato
`lb a0, 1(s0)`	`ba`	segno (bit 7 a 1)	`0xffffffba`
`lbu a0, 1(s0)`	`ba`	zeri	`0x000000ba`
`lh a0, 2(s0)`	`fe ca`	segno (bit 15 a 1)	`0xffffcafe`
`lhu a0, 2(s0)`	`fe ca`	zeri	`0x0000cafe`

una per una: esamina i bit.

Leggere una parte della parola

Per una load la memoria legge la parola intera che contiene il dato; un multiplexer ne seleziona il byte o la mezza parola indicati dai bit bassi dell'indirizzo, e un circuito di estensione completa i 32 bit con copie del segno oppure con zeri (figura 6.1, a sinistra). Riprendiamo la parola `0xcafebabe` dello stato 0x1, questa volta memorizzata all'indirizzo `0x10010000`, cioè all'inizio della zona dati di RARS: la tabella 6.2 mostra che cosa caricano le varie istruzioni.

Scrivere una parte della parola

La scrittura è più delicata: `sb` deve cambiare un solo byte e lasciare intatti gli altri tre. Per questo la memoria non ha un unico segnale di scrittura, ma quattro, uno per ogni byte della parola, detti *abilitazioni di byte* (*byte enable*). Il byte da scrivere viene copiato in tutte e quattro le posizioni, e la logica di controllo della memoria abilita solo quella indicata dall'indirizzo (figura 6.1, a destra). Per `sh` si abilitano due byte, per `sw` tutti e quattro.

Il programma `memoria.s` che accompagna il libro ripete gli esempi: dopo `sb t1, 3(s0)` la parola vale `0x11febabe`, e dopo `sh` con `0x2233` all'offset 0 vale `0x11fe2233`.

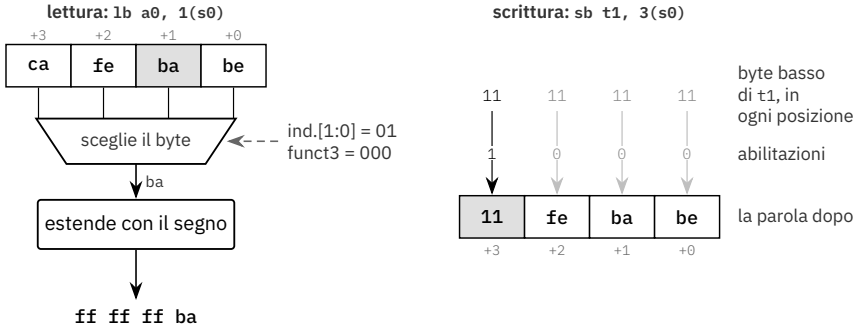


Figura 6.1 Leggere e scrivere un byte in una memoria a parole. A sinistra, `lb` seleziona il byte all'offset 1 della parola `0xcafebabe` e lo estende. A destra, `sb` con `t1` = `0x11`: il byte arriva a tutte le posizioni, ma solo quella all'offset 3 è abilitata, e la parola diventa `0x11febabe`.

6.3 L'allineamento

Lo stato 0x1 raccomandava di collocare le parole a indirizzi multipli di 4 e le mezze parole a indirizzi pari. Ora se ne vede il motivo hardware. Una parola all'indirizzo `0x10010002` occupa due byte della parola che comincia a `0x10010000` e due di quella che comincia a `0x10010004`: per leggerla servirebbero *due* accessi alla memoria, e poi un circuito che ricomponga le parti. Con una memoria che legge una sola parola per accesso, in un processore a ciclo singolo ciò non è possibile.

Per questo la specifica lascia la scelta a chi costruisce il processore: gestire gli accessi non allineati in hardware, magari lentamente, oppure generare un'eccezione e affidare il lavoro al software. RARS sceglie l'eccezione:

```
        .data
dato:   .word 0x11223344, 0x55667788
        .text
main:   la   s0, dato
        lw   a0, 2(s0)       # Load address not aligned to word boundary
```

Il nostro processore farà lo stesso quando avrà le eccezioni (stato 0xb); per ora supponiamo che gli accessi siano sempre allineati. In ogni caso un compilatore non genera mai accessi non allineati, se non gli si chiede esplicitamente di farlo.

6.4 Perché MEM è la fase più lenta

Nel calcolo della durata del ciclo che svolgeremo nello stato 0x7, la memoria ha lo stesso tempo di accesso dell'ALU: 200 picosecondi. È un'ipotesi didattica, e generosa. Una memoria principale reale, realizzata con DRAM, risponde in qualche decina di nanosecondi: per un processore a 3 GHz sono *centinaia* di cicli di clock.

- Una cella di **DRAM** (*Dynamic RAM*) è un condensatore con un transistor che funge da interruttore. È piccolissima, e per questo se ne integrano miliardi in un chip; ma la carica del condensatore si disperde, e ogni cella va riletta e riscritta di continuo, ogni poche decine di millisecondi. Leggerla è un processo lento e delicato.
- Una cella di **SRAM** (*Static RAM*) è costituita da sei transistor: quattro formano due invertitori che si mantengono reciprocamente in uno stato stabile, come le valvole di Eccles e Jordan dello stato 0x3, e due collegano la cella ai fili di lettura e scrittura. È molto più veloce, ma occupa molto più spazio e costa di più.

Il divario cresce da decenni: le prestazioni dei processori sono migliorate molto più in fretta della velocità delle memorie. Nel 1995 William Wulf e Sally McKee lo chiamarono, in un articolo diventato famoso, il *muro della memoria* (*memory wall*). La soluzione, che vedremo nella parte IV, è una *gerarchia*: piccole memorie SRAM veloci, le cache, vicino al processore, che contengono i dati usati più spesso, e la grande DRAM più lontano. Il nostro processore a ciclo singolo, con la sua memoria ideale che risponde istantaneamente, è un modo di rimandare il problema.

1966 Un condensatore al posto di un anello

Nell'autunno del 1966, all'IBM, Robert Dennard assistette alla presentazione di una memoria a nuclei magnetici: ogni bit era un anellino di ferrite attraversato da fili, e la sua magnetizzazione indicava 0 o 1. Tornò a casa chiedendosi se si potesse sostituire l'anello con un piccolo condensatore carico o scarico. Dopo qualche mese arrivò alla soluzione più semplice possibile: un condensatore e un solo transistor, collegato a un filo dati che serve sia a leggere sia a scrivere. Il brevetto fu concesso nel 1968; IBM però non lo usò subito, e il primo chip DRAM in commercio fu l'Intel 1103, nel 1970, con 1024 bit e una cella a tre transistor. Le memorie di oggi usano ancora la cella di Dennard, ripetuta miliardi di volte.

6.5 Una load e una store, passo per passo

Seguiamo due istruzioni attraverso le quattro fasi costruite finora. Il registro `sp` vale `0x7fffeff0`.

`lw t1, 8(sp)`

1. IF: si preleva `0x00812303`.
2. ID: rs1 = `sp`, rd = `t1`; l'immediato di tipo I vale 8. L'unità di controllo riconosce LOAD: RegWriteEn = 1, MemWrite = 0.
3. EX: Op1Sel seleziona `rs1`, Op2Sel l'immediato, l'ALU somma: `0x7fffeff8`.
4. MEM: la memoria legge la parola a `0x7fffeff8`; funct3 = `010`, quindi la parola viene trasferita intera. Il valore letto prosegue verso la fase WB, che lo scriverà in `t1`.

`sw t1, -8(sp)`

1. IF: si preleva `0xfe612c23`.
2. ID: rs1 = `sp`, rs2 = `t1`. L'immediato è di tipo S, suddiviso nei bit 31–25 (1111111_2) e 11–7 (11000_2): il generatore lo ricompone in `0xfffffff8`, cioè -8. Il controllo riconosce STORE: RegWriteEn = 0, MemWrite = 1.
3. EX: l'ALU calcola `0x7fffeff0` $-$ 8 = `0x7fffefe8`.
4. MEM: all'indirizzo `0x7fffefe8` arriva il valore di `t1`, dalla porta `rd2`; al fronte di salita la memoria lo scrive. La fase WB non svolge alcuna operazione.

L'esempio mostra la divisione dei compiti fra le due porte di lettura del banco dei registri. In una store `rs1` è la *base* dell'indirizzo e attraversa l'ALU; `rs2` è il *dato* e raggiunge direttamente la memoria.

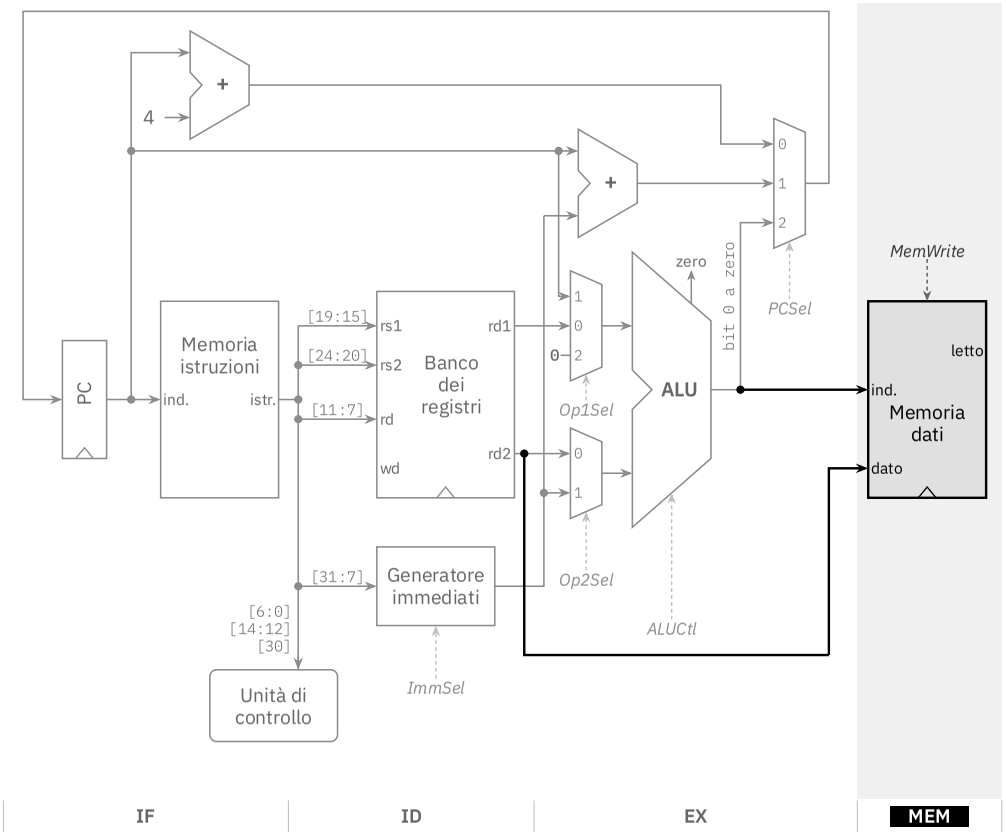


Figura 6.2 Il datapath finora. In nero la fase MEM: la memoria dati riceve l'indirizzo dall'ALU e il dato da scrivere da `rd2`, attraverso il filo che passa sotto l'ALU. MemWrite, al fronte di salita, autorizza la scrittura; il campo funct3, che specifica quanti byte leggere o scrivere, le giunge dall'istruzione con un filo non disegnato. Il dato letto e il risultato dell'ALU proseguono verso la fase WB.

Condizione di transizione uscita verso **0x7**

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1. Quali ingressi e quali uscite ha la memoria dati? Quando avviene la scrittura? ↺ § 6.1
2. Perché nei processori reali serve anche un segnale MemRead? ↺ § 6.1
3. All'indirizzo `0x2000` è memorizzata la parola `0xcafebabe`. Che cosa caricano `lh` da `0x2000` e `lbu` da `0x2003`? ↺ § 6.2
4. Che cosa indicano i tre bit di funct3 in una load? ↺ § 6.2
5. Come fa `sb` a modificare un solo byte di una parola? ↺ § 6.2
6. Perché un accesso non allineato è difficile da gestire in hardware? ↺ § 6.3
7. In una store, da dove arrivano l'indirizzo e il dato da scrivere? ↺ § 6.5
8. Perché nei processori reali MEM è la fase più lenta? Come si rimedia? ↺ § 6.4

STATO

0x7

b3	b2	b1	b0
0	1	1	1

WB: chiudere il cerchio

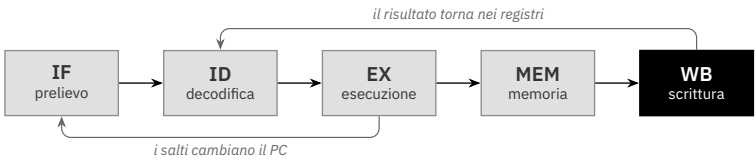


→ INGRESSI

- Il banco dei registri e la sua porta di scrittura (stato 0x3, § 3.2).
- I segnali di controllo (tabella 3.2).
- Le fasi IF, ID, EX e MEM (stati 0x2–0x6).

USCITE →

- Indicare che cosa scrive la fase WB, dove e quando.
- Leggere il datapath completo e seguirvi il percorso di qualunque istruzione di RV32I.
- Compilare la tabella dei segnali di controllo.
- Calcolare il periodo di clock di un processore a ciclo singolo e spiegare perché spreca tempo.



L'ultima fase è quella in cui l'istruzione produce il suo effetto permanente. Finora tutto è stato provvisorio: valori letti, somme calcolate, dati letti dalla memoria. Nella fase di **scrittura** (*write back*, WB) il risultato viene scritto nel registro di destinazione e, da quel momento, fa parte dello stato del processore. Con questa fase il datapath è completo: alla fine dello stato avremo un

processore capace di eseguire tutte le istruzioni di base di RISC-V, e potremo chiederci quanto è veloce.

7.1 Che cosa si scrive

Il valore da scrivere in `rd` può provenire da tre sorgenti, e un multiplexer comandato da **WBSel** seleziona quella corretta:

- 0, il *risultato dell'ALU*: per le famiglie OP e OP-IMM, per `lui` e per `auipc`;
- 1, il *dato letto dalla memoria*: per le load;
- 2, *PC* + 4: per `jal` e `jalr`, che salvano l'indirizzo di ritorno, il *link*.

Il valore selezionato torna indietro, lungo il filo che nel datapath passa sotto tutti gli altri, fino all'ingresso `wd` del banco dei registri. Al fronte di salita, se RegWriteEn vale 1, il registro indicato da `rd` lo memorizza. Le store e i salti condizionati non scrivono alcun registro: per queste istruzioni RegWriteEn vale 0, e il valore selezionato da WBSel non ha importanza.

Le frecce che tornano indietro

Guardando il datapath completo (figura 7.1) si nota che quasi tutti i fili sono orientati da sinistra a destra, come l'onda che attraversa il processore. Fanno eccezione soltanto due percorsi: il filo che riporta il risultato al banco dei registri e l'anello che riporta il nuovo indirizzo al PC. Non è un caso. Sono due delle tre scritture dello stato, che avvengono in un elemento che si trova *prima* della fase in cui il valore è pronto. La terza scrittura, quella nella memoria dati, non ha bisogno di tornare indietro: la memoria si trova proprio nella fase in cui il dato arriva. Tutto il resto del datapath serve a calcolare, durante il ciclo, che cosa scrivere al fronte di salita. Ricordati di questi due fili: nella pipeline saranno l'origine di quasi tutti i problemi.

Il link

`jal ra, funzione` scrive in `ra` il valore di *PC*+4, cioè l'indirizzo dell'istruzione che segue la chiamata; nello stesso ciclo il PC riceve l'indirizzo della funzione. Il valore *PC* + 4 proviene da lontano: è quello calcolato dal sommatore della fase IF, trasportato fino alla fase WB dal lungo filo che percorre la parte alta del datapath.

C'è un caso che sembra un paradosso: `jalr ra, 0(ra)`, che salta all'indirizzo contenuto in `ra` e insieme scrive in `ra` un nuovo indirizzo di ritorno. Funziona perché, come abbiamo visto nello stato 0x3, la lettura avviene durante il ciclo e la scrittura solo al fronte di salita: l'ALU calcola la destinazione con il valore *vecchio* di `ra`, e il nuovo valore arriva quando il salto è già deciso.

Tabella 7.1 La tabella della verità dell'unità di controllo. Il trattino indica un valore indifferente. Le colonne sono, nell'ordine: RegWriteEn, ImmSel, Op1Sel, Op2Sel, ALUOp, MemWrite, WBSel, Branch, Jal, Jalr.

Famiglia	RegW	Imm	Op1	Op2	ALUOp	MemW	WB	Br	Jal	Jalr
OP	1	–	0	0	10	0	0	0	0	0
OP-IMM	1	I	0	1	11	0	0	0	0	0
LOAD	1	I	0	1	00	0	1	0	0	0
STORE	0	S	0	1	00	1	–	0	0	0
BRANCH	0	B	0	0	01	0	–	1	0	0
JAL	1	J	–	–	–	0	2	0	1	0
JALR	1	I	0	1	00	0	2	0	0	1
LUI	1	U	2	1	00	0	0	0	0	0
AUIPC	1	U	1	1	00	0	0	0	0	0

7.2 Il datapath completo

La figura 7.1 è il punto di arrivo della parte II. Vale la pena di contarne i componenti:

- tre elementi di stato, il PC, il banco dei registri e la memoria dati, più la memoria istruzioni, che conserva il programma ma durante l'esecuzione si legge come un circuito combinatorio (§ 2.5);
- tre sommatori: quello di $PC + 4$, quello dei salti e l'ALU;
- quattro multiplexer nel disegno, davanti al PC, ai due ingressi dell'ALU e al banco dei registri, più quelli nascosti dentro i blocchi, come le porte di lettura del banco o il generatore degli immediati;
- una modesta quantità di logica di controllo: l'unità di controllo, il controllo dell'ALU e la logica dei salti.

Con questi pochi blocchi si eseguono una quarantina di istruzioni diverse. Restano fuori solo le istruzioni di sistema, come `ecall`, che richiedono il meccanismo delle eccezioni. `fence`, l'unica della famiglia MISC-MEM, si esegue come una `nop`: tutti i segnali che autorizzano una scrittura (RegWriteEn, MemWrite, Branch, Jal e Jalr) valgono 0.

7.3 La tabella del controllo

Ora possiamo completare la tabella della verità dell'unità di controllo, cominciata nello stato 0x3. Per ogni famiglia di istruzioni, la tabella 7.1 indica il valore di ogni segnale.

Qualche osservazione sulla tabella.

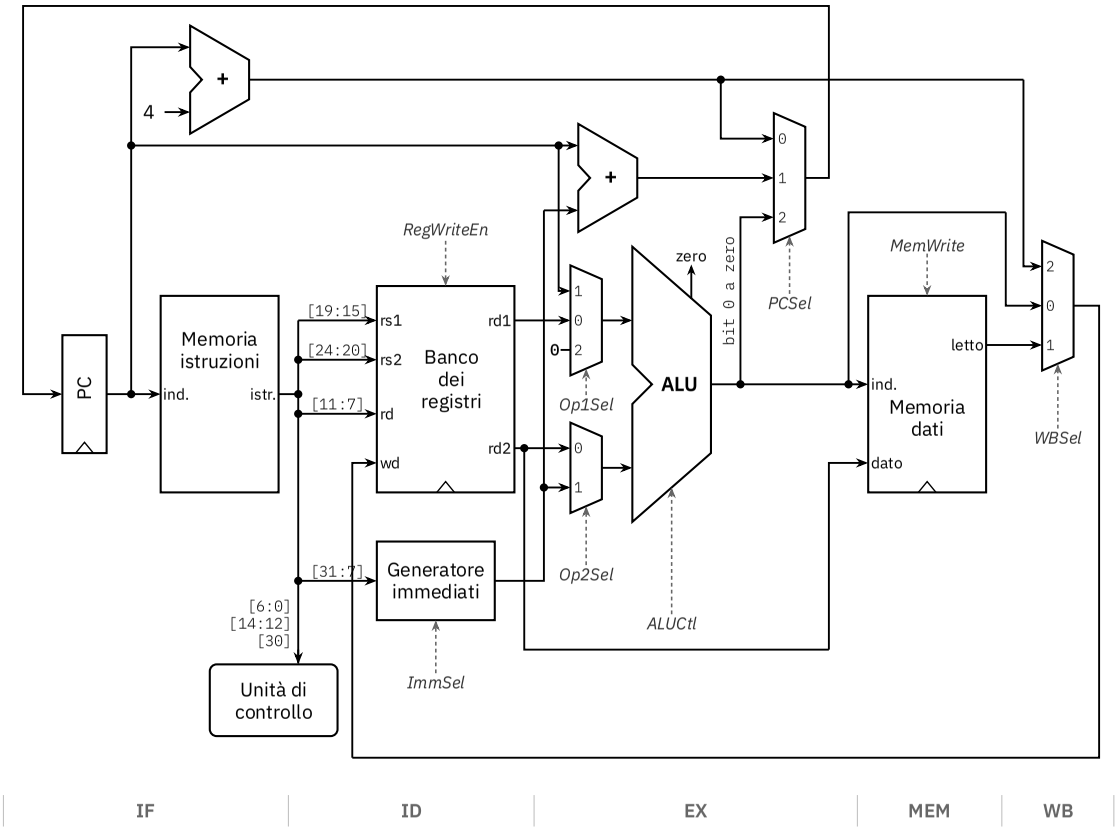


Figura 7.1 Il datapath completo del processore a ciclo singolo. Esegue tutte le istruzioni di RV32I tranne quelle di sistema. Le linee tratteggiate sono segnali di controllo. Il riquadro dell'unità di controllo comprende anche il controllo dell'ALU (figura 4.1), che produce ALUCtl da funct3 e dal bit 30, e la logica dei salti (figura 4.2), che produce PCSel. Per non affollare il disegno non sono tracciati i fili che le trasmettono zero e r_0 dall'ALU, né quello di funct3 verso la memoria dati. I numeri dentro i multiplexer sono i valori dei segnali di selezione.

Tabella 7.2 Il viaggio di `xor x8, x4, x5`, all'indirizzo `0x100`, con `x4` = `0x12345678` e `x5` = `0x0000ffff`. Tutto avviene in un solo ciclo di clock.

Fase	Che cosa succede
IF	Il PC vale `0x100`; la memoria istruzioni restituisce `0x00524433`; il sommatore calcola `0x104`.
ID	I campi raggiungono le rispettive destinazioni; il banco legge `0x12345678` e `0x0000ffff`; il controllo riconosce OP: RegWriteEn = 1, ALUOp = `10`, Op1Sel = Op2Sel = 0, WBSel = 0.
EX	Il controllo dell'ALU produce XOR ({0, `100`}); l'ALU calcola `0x1234a987`. Non è un salto: PCSel = 0.
MEM	Nessuna operazione: MemWrite = 0. La memoria legge un indirizzo inutile e il valore si perde.
WB	WBSel seleziona il risultato dell'ALU, che torna all'ingresso `wd`.
Fronte	Il registro `x8` diventa `0x1234a987` e il PC diventa `0x104`. Lo stato è cambiato: l'istruzione è eseguita.

- Per BRANCH l'immediato serve anche se Op2Sel seleziona `rs2`: non è destinato all'ALU, ma al sommatore dei salti. Per questo ImmSel non è indifferente.
- Per JAL l'ALU non serve, e i suoi tre segnali sono indifferenti: il sommatore dei salti calcola la destinazione, e il link proviene dal sommatore di $PC + 4$.
- Le colonne RegWriteEn e MemWrite non contengono trattini. Un valore indifferente in una di queste colonne significherebbe lasciare al caso se un'istruzione modifica lo stato, ed è proprio ciò che non deve mai succedere.

L'unità di controllo principale è quindi un circuito combinatorio con pochi ingressi (i 7 bit dell'opcode) e una quindicina di uscite, e la tabella è la sua descrizione completa. Si può realizzare con porte logiche ottenute dalla tabella, oppure scrivendo la tabella in una piccola memoria a sola lettura in cui l'opcode funge da indirizzo: ogni riga della tabella è una parola della memoria. Nello stato 0x8 vedremo che cosa cambia quando il controllo deve memorizzare lo stato di avanzamento dell'esecuzione.

7.4 Il viaggio completo di un'istruzione

Possiamo finalmente seguire `xor x8, x4, x5` dall'inizio alla fine. La tabella 7.2 raccoglie quello che abbiamo visto, fase per fase, negli stati 0x2–0x7.

L'ultima riga è quella che conta. Tutto il resto, dalla lettura della memoria al calcolo dell'ALU, è stato un lungo preparativo per due scritture che avvengono nello stesso istante: il registro di destinazione e il PC.

Tabella 7.3 Il percorso critico di ogni famiglia di istruzioni, in picosecondi. I componenti che operano in parallelo contano una volta sola: per un salto condizionato, il sommatore dei salti opera mentre l'ALU confronta. *Per JAL i 200 ps sono quelli del sommatore dei salti: l'ALU non è utilizzata.

Famiglia	Mem. istr.	Lettura reg.	ALU*	Mem. dati	Scrittura reg.	Totale
LOAD	200	100	200	200	100	**800**
STORE	200	100	200	200		**700**
OP, OP-IMM	200	100	200		100	**600**
BRANCH, JALR	200	100	200			**500**
LUI, AUIPC	200		200		100	**500**
JAL	200		200			**400**

7.5 Quanto dura un ciclo

In un processore a ciclo singolo il periodo di clock deve bastare all'istruzione più lenta. Per sapere quale sia, si somma il ritardo dei componenti che ogni istruzione attraversa uno dopo l'altro: è il suo **percorso critico**. Adottiamo le ipotesi classiche dei testi didattici: memorie da 200 ps, ALU e sommatori da 200 ps, lettura e scrittura dei registri da 100 ps, e ritardo trascurabile per multiplexer, controllo, generatore degli immediati e fili.

La load è l'istruzione più lenta (tabella 7.3): attraversa l'intero datapath, dalla memoria istruzioni alla scrittura del registro, passando per la memoria dati. Il periodo di clock deve quindi essere di almeno 800 ps, cioè una frequenza di al più 1,25 GHz. Per `jal` e `jalr` la scrittura del link non allunga il percorso: $PC + 4$ è pronto dopo 200 ps e scritto dopo 300, mentre la destinazione del salto arriva dopo 400 o 500.

Il tempo sprecato

Il tempo di esecuzione di un programma si scrive come prodotto di tre fattori:

$$\text{tempo} = N \times CPI \times T,$$

dove N è il numero di istruzioni eseguite, CPI il numero medio di cicli di clock per istruzione e T il periodo. Nel ciclo singolo $CPI = 1$, il valore più basso possibile per un processore che esegue un'istruzione alla volta, ma T è il più lungo possibile.

Quanto tempo si spreca? Prendiamo una combinazione di istruzioni (*mix*) tipica dei programmi interi, ispirata alle misure SPECint: 25% di load, 10% di store, 45% di operazioni aritmetiche e logiche, 15% di salti condizionati e

5% di salti incondizionati, per i quali consideriamo in media 450 ps. Se ogni istruzione potesse durare solo il suo tempo, la media sarebbe

$$0{,}25 \times 800 + 0{,}10 \times 700 + 0{,}45 \times 600 + 0{,}15 \times 500 + 0{,}05 \times 450 \approx 640 \text{ ps},$$

contro gli 800 imposti dal clock: un quinto del tempo va perso in attesa. Un clock di durata variabile non è una soluzione: gli oscillatori al quarzo oscillano a una frequenza fissa, e soprattutto bisognerebbe conoscere l'istruzione prima di averla letta.

I limiti del ciclo singolo

Lo spreco del 20% è solo l'inizio.

- *Le istruzioni lente rallentano tutte le altre.* Una moltiplicazione eseguita con l'algoritmo dello stato 0x5 richiede 32 passi; una divisione in virgola mobile ancora di più. In un processore a ciclo singolo il periodo dovrebbe allungarsi per tutte le istruzioni, anche per la più semplice delle somme.
- *L'hardware si duplica.* Poiché ogni componente si usa una volta per ciclo, servono due memorie, perché una load legge l'istruzione e il dato nello stesso ciclo, e tre sommatori, perché un salto condizionato usa insieme l'ALU, il sommatore dei salti e quello di $PC + 4$.
- *Gran parte del circuito resta inattiva per la maggior parte del tempo.* Mentre la memoria dati è in funzione, la memoria istruzioni ha terminato da tempo il suo lavoro e attende il ciclo successivo.

Le prossime due soluzioni affrontano proprio questi limiti. Il *multiciclo* (stato 0x8) suddivide l'istruzione in passi più brevi e riusa gli stessi componenti in passi diversi. La *pipeline* (parte III) fa operare tutte le fasi insieme, su istruzioni diverse.

1957 Il percorso critico, prima dei chip

Il nome «percorso critico» è più antico dei processori moderni. Fra il 1956 e il 1959 Morgan Walker, ingegnere della DuPont, e James Kelley, matematico della Remington Rand, misero a punto un metodo per pianificare la costruzione degli impianti chimici, e poi le loro fermate per manutenzione: si scompone il lavoro in attività, si annota quali attività devono attendere il completamento di altre, e si cerca la catena più lunga di attività che dipendono l'una dall'altra. Quella catena determina la durata dell'intero progetto, e accorciare qualunque altra attività non serve a niente. I calcoli venivano eseguiti su un UNIVAC; nello stabilimento di Louisville le fermate per manutenzione si accorciarono di circa un quarto. È esattamente il ragionamento della tabella 7.3: i componenti sono le attività, e il clock attende la catena più lunga.

Condizione di transizione uscita verso `0x8`

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Da quali tre sorgenti può arrivare il valore scritto in `rd`, e per quali istruzioni? ↺ §7.1

2 Quali fili del datapath procedono «all'indietro», e perché la scrittura in memoria non ne ha bisogno? ↺ §7.1

3 Come fa `jalr ra, 0(ra)` a usare il vecchio valore di `ra` e a scriverne uno nuovo? ↺ §7.1

4 Perché RegWriteEn e MemWrite non possono mai essere indifferenti? ↺ §7.3

5 Scrivi i segnali di controllo di `sw t1, -8(sp)`. ↺ §7.3

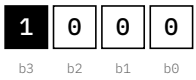
6 Qual è il percorso critico di una load, e quanto vale con le ipotesi del capitolo? ↺ §7.5

7 Se la memoria dati impiegasse 300 ps invece di 200, quale sarebbe il periodo di clock? ↺ §7.5

8 Perché non si può usare un clock di durata variabile, adattata a ogni istruzione? ↺ §7.5

STATO

0x8



Multiciclo: il controllo come macchina a stati



→ INGRESSI

- Il datapath a ciclo singolo e la tabella del controllo (stato 0x7, § 7.2 e § 7.3).
- Il percorso critico e i limiti del ciclo singolo (§ 7.5).
- La struttura di una macchina a stati (stato 0x2, § 2.2).

USCITE →

- Spiegare come si divide un'istruzione in passi e perché servono registri fra un passo e l'altro.
- Descrivere il datapath multiciclo, con una sola memoria e una sola ALU.
- Disegnare e leggere l'unità di controllo come macchina a stati.
- Confrontare i tempi del ciclo singolo e del multiciclo, e capire da dove nasce la pipeline.

Lo stato 0x7 si è chiuso con un conto deludente: nel processore a ciclo singolo ogni istruzione dura quanto la più lenta, e gran parte del circuito resta inattiva per la maggior parte del tempo. Questo stato esplora una strada diversa. Non fa parte del programma di molti corsi, compreso quello da cui nasce il libro, e per questo nella mappa degli stati il suo cerchio è tratteggiato: alla prima lettura puoi leggere solo l'ultima sezione, § 8.7, e passare alla parte III. Ma la deviazione merita di essere percorsa, per due motivi. Mostra in modo concreto che cosa significhi riusare l'hardware. E soprattutto

trasforma l'unità di controllo in una vera macchina a stati finiti, come quelle che danno il nome a questo libro.

8.1 Spezzare l'istruzione in passi

L'idea è semplice: invece di eseguire un'istruzione in un ciclo lungo, la si esegue in più cicli corti, uno per ogni passo. Il periodo di clock non deve più contenere l'intera istruzione, ma soltanto il passo più lungo. Con le ipotesi dello stato 0x7 il passo più lungo dura 200 ps, quanto un accesso alla memoria o un'operazione dell'ALU: la frequenza di clock può essere quattro volte più alta. In cambio, un'istruzione richiede più cicli, e non tutte lo stesso numero: una load ne usa cinque, un salto condizionato tre.

C'è però una conseguenza importante. Nel ciclo singolo i valori intermedi, come l'istruzione letta o il risultato dell'ALU, transitano sui fili e vengono usati nello stesso ciclo. Ora invece un passo produce un valore che servirà al passo successivo, cioè al ciclo seguente: e al fronte di salita le uscite dei circuiti combinatori cambiano. Ogni valore che deve sopravvivere al passaggio da un ciclo all'altro va quindi salvato in un registro. Il datapath si arricchisce di **registri intermedi**, invisibili al programmatore, che separano un passo dall'altro.

8.2 Riusare i componenti

Spezzare l'istruzione ha un secondo vantaggio: un componente può essere utilizzato in passi diversi per scopi diversi.

- *Una sola memoria.* L'istruzione si legge nel primo passo, il dato in un passo successivo: non c'è più bisogno di due memorie separate. Torniamo, senza conflitti, alla memoria unica di von Neumann dello stato 0x0.
- *Una sola ALU.* Nel primo passo l'ALU calcola $PC + 4$; nel secondo la destinazione di un eventuale salto; nei passi successivi il risultato dell'istruzione. I due sommatori aggiunti nel ciclo singolo non servono più.

Il prezzo sono multiplexer più grandi, perché ogni ingresso condiviso deve poter selezionare fra più sorgenti, e i registri intermedi.

8.3 Il datapath multiciclo

La figura 8.1 mostra il datapath che ne risulta. Lo schema è quello proposto da Harris e Harris nel loro libro di testo, adattato alle convenzioni di questo libro e completato con `jalr`, `lui` e `auipc`. I registri intermedi sono sei.

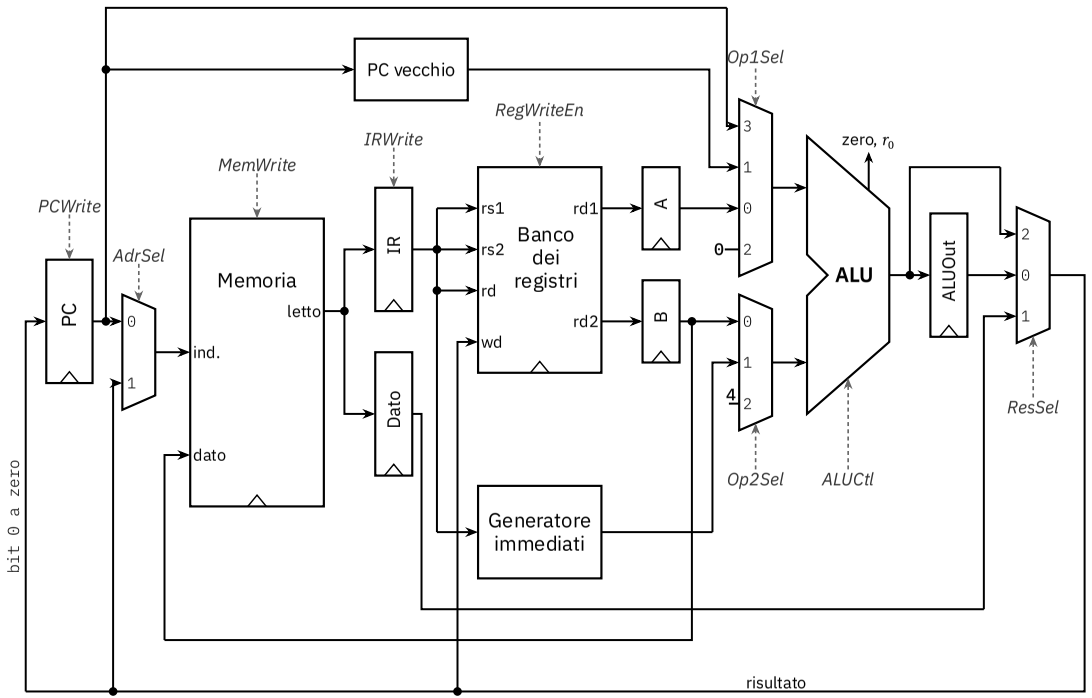


Figura 8.1 Il datapath multiciclo. Una sola memoria per istruzioni e dati, una sola ALU, e i registri intermedi (IR, Dato, A, B, ALUOut, PC vecchio) che conservano i valori da un passo all'altro. IRWrite abilita anche la scrittura del PC vecchio; il generatore degli immediati riceve ImmSel come nel ciclo singolo. Il risultato selezionato da ResSel può diventare il nuovo PC, un indirizzo di memoria o il valore da scrivere in un registro.

IR
: Il *registro istruzione* conserva l'istruzione letta nel primo passo, perché nei passi successivi la memoria sarà impiegata per altri accessi.

PC vecchio
: Conserva l'indirizzo dell'istruzione. Il PC, infatti, viene aggiornato a *PC* + 4 già nel primo passo, ma i salti e `auipc` hanno bisogno dell'indirizzo originale.

Dato
: Conserva la parola letta dalla memoria da una load.

A, B
: Conservano i valori letti dal banco dei registri.

ALUOut
: Conserva il risultato dell'ALU.

IR e PC vecchio si scrivono solo nel primo passo, abilitati dal segnale *IRWrite*; il PC solo quando *PCWrite* vale 1. Gli altri registri intermedi si riscrivono a ogni ciclo, e il loro contenuto è valido solo per il ciclo successivo.

I multiplexer sono quattro. *AdrSel* seleziona l'indirizzo della memoria: il PC per prelevare l'istruzione, il risultato selezionato da *ResSel* (in pratica ALUOut) per accedere ai dati. *Op1Sel* seleziona il primo operando dell'ALU fra `A`, il PC vecchio, la costante 0 e il PC; *Op2Sel* il secondo fra `B`, l'immediato e la costante 4. Infine *ResSel* seleziona il *risultato* del passo: il contenuto di ALUOut, il dato letto oppure l'uscita diretta dell'ALU. Il risultato può raggiungere tre destinazioni: il PC, l'ingresso degli indirizzi della memoria, il banco dei registri.

8.4 Il controllo diventa una macchina a stati

Nel ciclo singolo l'unità di controllo era combinatoria: esaminava l'istruzione e determinava tutti i segnali contemporaneamente. Ora i segnali devono cambiare da un passo all'altro, e il controllo deve *memorizzare* il passo raggiunto. Servono un registro di stato e una logica che calcoli lo stato successivo: la macchina a stati della figura 2.1, questa volta dentro l'unità di controllo.

La figura 8.2 disegna la macchina, e la tabella 8.1 descrive che cosa avviene in ogni stato. I primi due passi sono uguali per tutte le istruzioni, perché il controllo non ha ancora identificato l'istruzione in esecuzione.

- *S0, prelievo.* La memoria legge l'istruzione all'indirizzo contenuto nel PC, e l'istruzione viene memorizzata in IR; il registro PC vecchio conserva l'indirizzo; intanto l'ALU calcola *PC* + 4, che al fronte di salita diventa il nuovo PC.

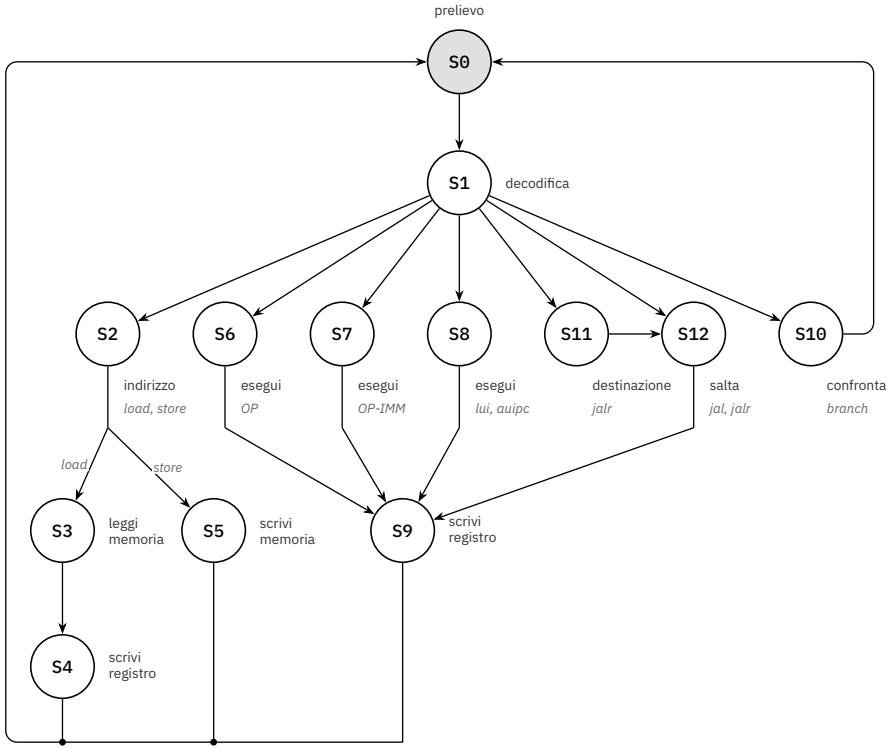


Figura 8.2 L'unità di controllo del processore multiciclo come macchina a stati. Ogni stato è un passo, cioè un ciclo di clock. Da S1 si prosegue a seconda della famiglia dell'istruzione, scritta in corsivo sotto ogni stato; quando l'istruzione è finita si torna a S0 per prelevare la successiva.

- *S1, decodifica.* Il banco dei registri legge `rs1` e `rs2`, e i due valori vengono memorizzati in A e B. L'ALU, che altrimenti rimarrebbe inutilizzata, calcola in anticipo $PCv + imm$, con il PC vecchio (quello nuovo vale già $PC + 4$), la destinazione di un eventuale salto, e la scrive in ALUOut. Se l'istruzione non salta, il calcolo è stato inutile ma gratuito: lo stesso principio che nello stato 0x3 ci faceva leggere i registri prima di sapere se servivano.

Da S1 la macchina segue percorsi diversi a seconda dell'opcode, e ogni percorso termina tornando a S0.

Due esempi

Una load attraversa cinque stati: S0 e S1, poi S2 calcola l'indirizzo, S3 legge la memoria, S4 scrive il dato nel registro. Sono le cinque fasi IF, ID, EX, MEM e WB degli stati 0x2–0x7, finalmente separate nel tempo.

Tabella 8.1 Che cosa avviene in ogni stato del controllo multiciclo. La freccia indica una scrittura al fronte di salita; *PCv* è il PC vecchio.

Stato	Azioni	Poi
S0	IR ← Mem[PC], PCv ← PC, PC ← PC + 4	S1
S1	A ← rs1, B ← rs2, ALUOut ← PCv + imm	secondo l'opcode
S2	ALUOut ← A + imm	S3 o S5
S3	Dato ← Mem[ALUOut]	S4
S4	rd ← Dato	S0
S5	Mem[ALUOut] ← B	S0
S6	ALUOut ← A *op* B	S9
S7	ALUOut ← A *op* imm	S9
S8	ALUOut ← 0 + imm (`lui`) oppure PCv + imm (`auipc`)	S9
S9	rd ← ALUOut	S0
S10	confronta A e B; se la condizione è vera, PC ← ALUOut	S0
S11	ALUOut ← A + imm	S12
S12	PC ← ALUOut con il bit 0 a zero, ALUOut ← PCv + 4	S9

`jal` è trattata in modo più ingegnoso. In S1 l'ALU ha già calcolato la destinazione, *PCv* + *imm*. In S12 avvengono due aggiornamenti nello stesso fronte di salita: il PC riceve il valore di ALUOut, e ALUOut riceve il nuovo valore calcolato dall'ALU, *PCv* + 4, cioè il link. I registri si aggiornano tutti insieme, quindi il PC riceve il valore *vecchio* di ALUOut, prima che venga sovrascritto. In S9 il link viene scritto in `rd`. Anche `jalr` passa da S12, dopo aver calcolato in S11 la sua destinazione, *rs1* + *imm*. Sul filo che entra nel PC il bit 0 viene forzato a zero, come nel ciclo singolo: per `jalr` serve, per tutte le altre destinazioni, che sono sempre pari, non cambia nulla.

Un lettore attento noterà che per `auipc` il risultato è già in ALUOut alla fine di S1, perché *PCv* + *imm* è proprio ciò che S1 calcola: si potrebbe passare direttamente a S9. Lo stato S8 la tratta come `lui` solo per semplicità.

Quanti bit servono per ricordare

La macchina ha 13 stati, quindi il suo registro di stato ha 4 bit, come quello che apre ogni capitolo di questo libro. Il resto del controllo è logica combinatoria: un circuito calcola lo stato successivo dallo stato attuale e dall'opcode, un altro calcola i segnali di controllo dallo stato attuale e, per alcuni segnali, dall'opcode, da funct3 e dall'esito del confronto, cioè dai segnali zero e r_0 dell'ALU, come nella logica dei salti dello stato 0x4: ImmSel dipende dal formato, l'operazione dell'ALU da funct3, la scrittura del PC in S10 dalla condizione del salto. È esattamente lo schema della figura 2.1, in miniatura.

Tabella 8.2 Cicli di clock per famiglia di istruzioni nel processore multiciclo.

Famiglia	Stati attraversati	Cicli
LOAD	S0, S1, S2, S3, S4	5
STORE	S0, S1, S2, S5	4
OP, OP-IMM, LUI, AUIPC	S0, S1, S6 (o S7, S8), S9	4
BRANCH	S0, S1, S10	3
JAL	S0, S1, S12, S9	4
JALR	S0, S1, S11, S12, S9	5

8.5 Quanto è veloce

Ogni passo dura un ciclo di 200 ps, e il numero di cicli dipende dall'istruzione (tabella 8.2). Per confrontare il multiciclo con il ciclo singolo usiamo la stessa combinazione di istruzioni dello stato 0x7.

Contando 4,5 cicli in media per `jal` e `jalr`, il numero medio di cicli per istruzione è

$$CPI = 0{,}25 \times 5 + 0{,}10 \times 4 + 0{,}45 \times 4 + 0{,}15 \times 3 + 0{,}05 \times 4{,}5 = 4{,}125,$$

e il tempo medio per istruzione è $4{,}125 \times 200 = 825$ ps. Il ciclo singolo ne impiegava 800. Il multiciclo, con questi numeri, *non* è più veloce.

È una sorpresa istruttiva. I passi non hanno tutti la stessa durata: una lettura dei registri dura 100 ps ma occupa un ciclo intero da 200, e lo stesso vale per la scrittura. Ogni istruzione paga il periodo pieno per ognuno dei suoi passi, anche per quelli corti. E nella realtà la situazione è perfino peggiore, perché ogni registro intermedio aggiunge un piccolo ritardo a ogni ciclo.

Perché allora il multiciclo ha avuto tanto successo per decenni? Per due motivi. Il primo è il *costo*: una sola memoria e una sola ALU, in un'epoca in cui ogni componente era costoso. Il secondo sono le istruzioni *lunghe*. Una moltiplicazione da 32 passi, nel ciclo singolo, costringerebbe il clock ad allungarsi per tutte le istruzioni; nel multiciclo occupa semplicemente più cicli, e le istruzioni semplici restano veloci. Le architetture CISC, con istruzioni di durata molto diversa, sono state per anni multiciclo.

8.6 Cablato o microprogrammato

La macchina a stati del controllo si può costruire in due modi. Il primo è quello che abbiamo descritto: un registro di stato e due circuiti combinatori ricavati dalle tabelle. È il controllo **cablato**, veloce ma rigido: per aggiungere un'istruzione bisogna riprogettare la logica.

Il secondo è il **microprogramma**. Le righe della tabella 8.1, con tutti i loro segnali, si scrivono come parole in una piccola memoria, la *memoria di controllo*; ogni parola è una *microistruzione*. Il controllo diventa un minuscolo processore che, per eseguire un'istruzione della macchina, esegue un breve programma di microistruzioni. Cambiare il comportamento significa cambiare il contenuto di quella memoria. I processori x86 di oggi usano ancora un microprogramma, detto *microcodice*, per le loro istruzioni più complicate, e i produttori lo aggiornano per correggere errori anche dopo che il processore è stato venduto.

1951 Un calcolatore dentro il calcolatore

Maurice Wilkes, che a Cambridge aveva costruito l'EDSAC, uno dei primi calcolatori a programma memorizzato, si era reso conto di quanto fosse difficile progettarne l'unità di controllo: un insieme intricato di circuiti progettati su misura, diverso per ogni istruzione. Nel 1951 propose di trasformarla in un «calcolatore in miniatura», guidato da un programma scritto in una memoria. L'idea era in anticipo sui tempi, perché richiedeva memorie veloci che ancora non esistevano; il suo EDSAC 2, nel 1958, fu il primo calcolatore con il controllo microprogrammato. Il trionfo arrivò nel 1964 con il System/360 di IBM, la famiglia dello stato 0x0: quasi tutti i modelli, di potenza diversissima, eseguivano la stessa ISA grazie a microprogrammi diversi, e alcuni potevano perfino imitare via microcodice i vecchi calcolatori IBM. Per questo nacque una parola nuova: *emulazione*.

8.7 Verso la pipeline

Guardiamo il multiciclo con gli occhi del ciclo singolo. Durante il passo S3 di una load è attiva soltanto la memoria; l'ALU, il banco dei registri e gli altri componenti sono inattivi. Durante S1 la memoria è inattiva. In ogni ciclo è attiva solo una piccola parte del processore.

E se, mentre la load è in S3, un'altra istruzione usasse l'ALU, e una terza stesse già leggendo i registri? Se ogni passo operasse su un'istruzione diversa, come le postazioni di una catena di montaggio, il processore completerebbe un'istruzione *a ogni ciclo* da 200 ps, quattro volte meglio degli 800 ps del ciclo singolo. È la *pipeline*, e per farla funzionare dovremo affrontare problemi nuovi: che cosa succede se un'istruzione ha bisogno del risultato di una che non ha ancora finito? E se un salto modifica il flusso del programma quando le istruzioni successive sono già state avviate? La parte III comincia da qui.

Condizione di transizione uscita verso `0x9`

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Perché nel multiciclo servono registri intermedi fra un passo e l'altro? ↺ §8.1

2 Quali componenti del ciclo singolo scompaiono nel multiciclo, e perché si possono eliminare? ↺ §8.2

3 A che cosa serve il PC vecchio? ↺ §8.3

4 Quale operazione svolge l'ALU durante il passo S1, e perché? ↺ §8.4

5 Quali stati attraversa una `sw`, e quanti cicli impiega? ↺ §8.4

6 Quanti bit servono al registro di stato del controllo, e perché? ↺ §8.4

7 Con le ipotesi del capitolo il multiciclo non è più veloce del ciclo singolo. Perché? In quali casi conviene comunque? ↺ §8.5

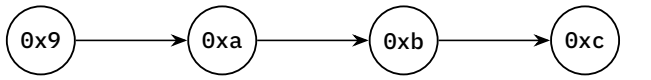
8 Che differenza c'è fra controllo cablato e microprogrammato? ↺ §8.6

PARTE

III

Tante istruzioni insieme

Il processore della parte II esegue un'istruzione alla volta. Ora lo trasformiamo in una catena di montaggio, in cui cinque istruzioni lavorano insieme, ognuna in una fase diversa. Il guadagno è grande, ma le istruzioni cominciano a disturbarsi a vicenda: vedremo come il processore si accorge dei conflitti e come li risolve, e poi come i processori moderni arrivano a eseguire più istruzioni per ciclo.



STATO

0x9

1	0	0	1
b3	b2	b1	b0

Pipeline: la catena di montaggio

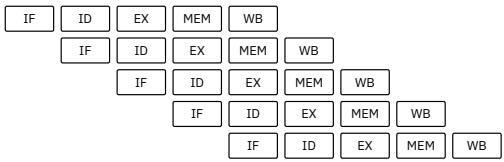


→ INGRESSI

- Il datapath a ciclo singolo e le sue cinque fasi (stato 0x7, § 7.2).
- I tempi dei componenti e il percorso critico (§ 7.5).
- Perché si spezza un'istruzione in passi (stato 0x8, almeno § 8.7).

USCITE →

- Spiegare perché la pipeline aumenta il numero di istruzioni completate al secondo ma non accorcia la singola istruzione.
- Calcolare il periodo di clock e l'accelerazione di una pipeline.
- Descrivere il contenuto di ogni registro di pipeline, compresi i segnali di controllo.
- Leggere e disegnare i diagrammi della pipeline.
- Riconoscere i tre tipi di conflitto.



Lo stato 0x8 si è chiuso con un'immagine: mentre una load legge la memoria dati, il resto del processore è inattivo. In questo stato facciamo lavorare contemporaneamente tutte le parti del processore. L'idea non è nuova: è quella della catena di montaggio, in cui ogni operaio esegue sempre la stes-

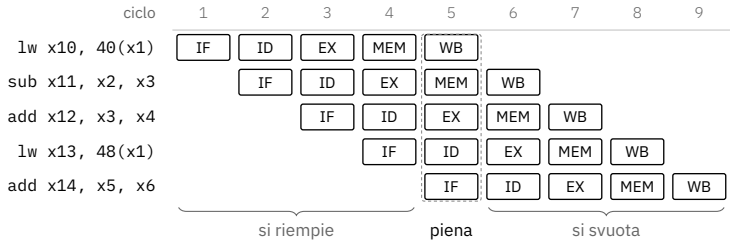


Figura 9.1 Cinque istruzioni indipendenti nella pipeline. Ogni istruzione impiega cinque cicli, ma ne entra una nuova a ogni ciclo. Al ciclo 5, nel riquadro tratteggiato, tutte le fasi sono attive, ognuna su un'istruzione diversa.

sa operazione e consegna il pezzo al vicino. Nessun pezzo viene montato più in fretta, ma dalla fine della catena escono molti più pezzi all'ora. Nel processore i pezzi sono le istruzioni, e la catena si chiama **pipeline**, letteralmente «condotto»: un tubo in cui le istruzioni entrano da una parte ed escono dall'altra, una dietro l'altra.

9.1 Cinque fasi, cinque istruzioni

Le cinque fasi che abbiamo costruito nella parte II sono già le postazioni della catena:

1. IF, prelievo dell'istruzione dalla memoria istruzioni;
2. ID, decodifica e lettura dei registri;
3. EX, calcolo nell'ALU;
4. MEM, accesso alla memoria dati;
5. WB, scrittura del risultato nel banco dei registri.

Nel processore a ciclo singolo un'istruzione le attraversa tutte in un ciclo lungo; nella pipeline ogni fase occupa un ciclo corto, e appena un'istruzione lascia una fase, la fase accoglie l'istruzione successiva. La figura 9.1 mostra cinque istruzioni nella pipeline. Le righe sono le istruzioni, le colonne i cicli di clock: la casella all'incrocio indica in quale fase si trova l'istruzione in quel ciclo.

Il diagramma va letto in due direzioni. Lungo una riga si segue un'istruzione nel tempo: `sub` è in IF al ciclo 2, in ID al ciclo 3, e così via. Lungo una colonna si vede una fotografia del processore: al ciclo 5 la prima `lw` sta scrivendo il risultato, `sub` è in MEM, `add` nell'ALU, la seconda `lw` legge i registri e l'ultima `add` viene prelevata. Nei primi quattro cicli la pipeline *si riempie*, negli ultimi quattro *si svuota*; in mezzo, in un programma reale che dura

milioni di cicli, è sempre piena, e a ogni ciclo un'istruzione entra e una esce.

Una conseguenza è che ogni istruzione attraversa tutte e cinque le fasi, anche quando una fase non le serve. Una `add` non usa la memoria dati, ma trascorre comunque un ciclo in MEM senza svolgere alcuna operazione. Se la saltasse, arriverebbe in WB nello stesso ciclo dell'istruzione che la precede, e le due vorrebbero scrivere nel banco dei registri insieme, attraverso l'unica porta di scrittura. Tenere tutte le istruzioni allo stesso passo rende la catena regolare: a ogni fase, in ogni ciclo, c'è al più un'istruzione.

1913 Il nastro di Highland Park

Nella primavera del 1913, nella fabbrica Ford di Highland Park, vicino a Detroit, il magnete del volano della Model T smise di essere montato da un operaio solo: un gruppo di operai in fila lungo un banco aggiungeva ciascuno un piccolo pezzo e faceva scivolare il lavoro al vicino. Il lavoro necessario per un magnete scese da circa 20 minuti a 13, e con qualche ritocco a cinque. L'idea veniva dai macelli di Chicago e Cincinnati, dove le carcasse avanzavano appese a un binario da un operaio all'altro. In pochi mesi il metodo fu esteso a tutta l'automobile: il lavoro per montare una Model T passò da dodici ore e mezza a circa un'ora e mezza, 93 minuti. Ma il numero più impressionante è un altro: dalla fine della linea usciva un'automobile ogni tre minuti circa. Novanta minuti per costruirne una, tre minuti fra una e la successiva: è esattamente la distinzione della prossima sezione.

9.2 Latenza e throughput

Per misurare il guadagno della pipeline servono due grandezze. La **latenza** è il tempo che un'istruzione impiega dall'inizio alla fine. Il **throughput**, o *produttività*, è il numero di istruzioni completate nell'unità di tempo.

Riprendiamo i tempi dello stato 0x7: memorie, ALU e sommatori da 200 ps, lettura e scrittura dei registri da 100 ps. Nel ciclo singolo il periodo era di 800 ps, il tempo della load. Nella pipeline il periodo deve bastare alla fase *più lenta*, perché tutte le fasi avanzano insieme a ogni fronte di clock: 200 ps. Le fasi ID e WB, che sono attive solo per 100 ps, restano inattive per il resto del ciclo. La figura 9.2 mette a confronto le due soluzioni.

La figura mette in evidenza due fatti che sembrano in contrasto.

- *La latenza peggiora.* Una load impiegava 800 ps; ora impiega cinque cicli da 200, cioè 1000 ps. Ogni istruzione paga il tempo morto delle fasi corte, e nella realtà anche un piccolo ritardo per attraversare ogni registro fra una fase e l'altra.
- *Il throughput migliora.* A pipeline piena esce un'istruzione ogni 200 ps invece che ogni 800: il throughput è quattro volte maggiore.

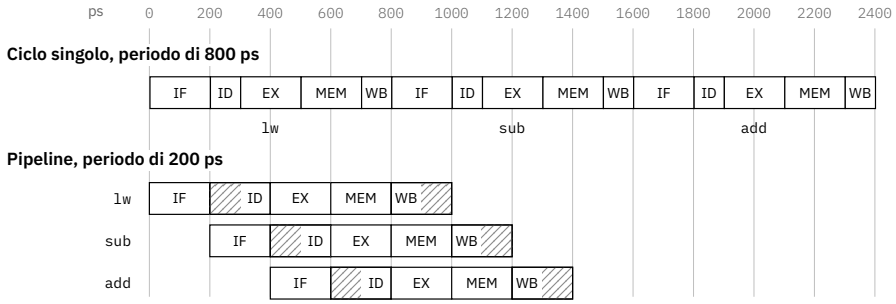


Figura 9.2 Tre istruzioni nel processore a ciclo singolo e nella pipeline. Nella pipeline ogni fase dura un ciclo da 200 ps; nelle fasi ID e WB la parte tratteggiata è il tempo in cui la fase è inattiva. WB scrive nella prima metà del ciclo e ID legge nella seconda, per un motivo che vedremo in § 9.6. Le tre istruzioni finiscono dopo 2400 ps nel ciclo singolo e dopo 1400 ps nella pipeline.

Per un programma conta il throughput. Con la formula dello stato 0x7, tempo $= N \times CPI \times T$, la pipeline lascia il CPI a 1, perché a regime si completa un'istruzione per ciclo, e divide per quattro il periodo T.

Contiamo con precisione. Con k fasi, la prima istruzione esce dopo k cicli e ognuna delle altre un ciclo dopo la precedente: N istruzioni richiedono $k + N - 1$ cicli. Per le tre istruzioni della figura sono $5 + 3 - 1 = 7$ cicli, 1400 ps contro 2400. Per un milione di istruzioni sono 1 000 004 cicli, poco più di 200 µs contro 800: i quattro cicli per riempire la pipeline diventano trascurabili, e l'**accelerazione**, il rapporto fra i due tempi, tende a $800/200 = 4$.

Perché quattro e non cinque

Spesso si legge che una pipeline a cinque fasi è cinque volte più veloce. È vero solo se le fasi sono *bilanciate*, cioè durano tutte lo stesso tempo: se ogni fase durasse 160 ps, il periodo scenderebbe da 800 a 160 ps. Le nostre fasi durano 200, 100, 200, 200 e 100 ps, e il ciclo deve adeguarsi alle più lunghe. L'accelerazione massima è il rapporto fra il tempo totale dell'istruzione e quello della fase più lenta: $800/200 = 4$.

Progettare una pipeline significa quindi soprattutto bilanciarla. Se una fase è troppo lenta, si può suddividere in due. Supponiamo che la memoria dati impieghi 400 ps: invece di allungare il ciclo per tutti, si divide l'accesso in due fasi da 200 ps, DM1 e DM2, e la pipeline passa a sei fasi (figura 9.3). Lo stesso si fa per le operazioni lunghe, come la moltiplicazione: invece di assegnarle un ciclo lunghissimo, la si affida a un'unità separata, divisa in più fasi, che opera in parallelo all'ALU. Più fasi significano però più istruzioni

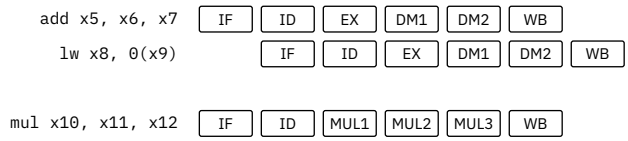


Figura 9.3 Due varianti della pipeline. In alto l'accesso alla memoria dati è diviso in due fasi; in basso la moltiplicazione usa un'unità separata che impiega tre cicli. In entrambi i casi il periodo resta di 200 ps.

contemporaneamente in esecuzione nella pipeline (*in-flight*), e vedremo presto che è proprio questo a creare problemi.

9.3 Il datapath a pipeline

Per far operare le fasi su istruzioni diverse bisogna risolvere un problema che conosciamo dal multiciclo: alla fine di un ciclo, quando un'istruzione passa alla fase successiva, i valori che la riguardano devono avanzare insieme a essa. L'istruzione letta in IF serve ancora in ID, dove però, al ciclo successivo, la memoria istruzioni sta già leggendo un'altra istruzione. La soluzione è la stessa del multiciclo: un registro fra una fase e l'altra. Questi **registri di pipeline**, detti anche *registri di interstadio*, prendono il nome dalle due fasi che separano: IF/ID, ID/EX, EX/MEM e MEM/WB. A ogni fronte di salita ciascuno di essi fotografa le uscite della fase alla sua sinistra, che durante il ciclo successivo diventano gli ingressi della fase alla sua destra.

La figura 9.4 mostra il datapath della parte II con i quattro registri al loro posto. Non c'è un registro dopo WB: la fase WB scrive direttamente nel banco dei registri, che è esso stesso un elemento di stato.

Rispetto al ciclo singolo i componenti sono gli stessi: il PC, le due memorie, il banco dei registri, il generatore degli immediati, l'ALU, i due sommatori, gli stessi multiplexer con la stessa numerazione. Cambiano tre cose.

- *I registri di pipeline* interrompono ogni filo che collega una fase all'altra. Ogni valore che serve più avanti deve attraversarli tutti, anche quando nessuna fase intermedia lo usa: per questo $PC + 4$, che serve solo in WB per il link di `jal` e `jalr`, percorre la parte alta di tutta la figura.
- *Il sommatore dei salti* opera in EX, dove arrivano il PC dell'istruzione e l'immediato; il suo risultato e quello dell'ALU tornano al multiplexer del PC dalla fase MEM, dove la *logica dei salti* prende la decisione. È la scelta adottata da Patterson e Hennessy, che miglioreremo nello stato 0xb.
- *Il numero del registro di destinazione* viaggia insieme all'istruzione fino a MEM/WB, e da lì torna al banco dei registri insieme al valore da scrivere.

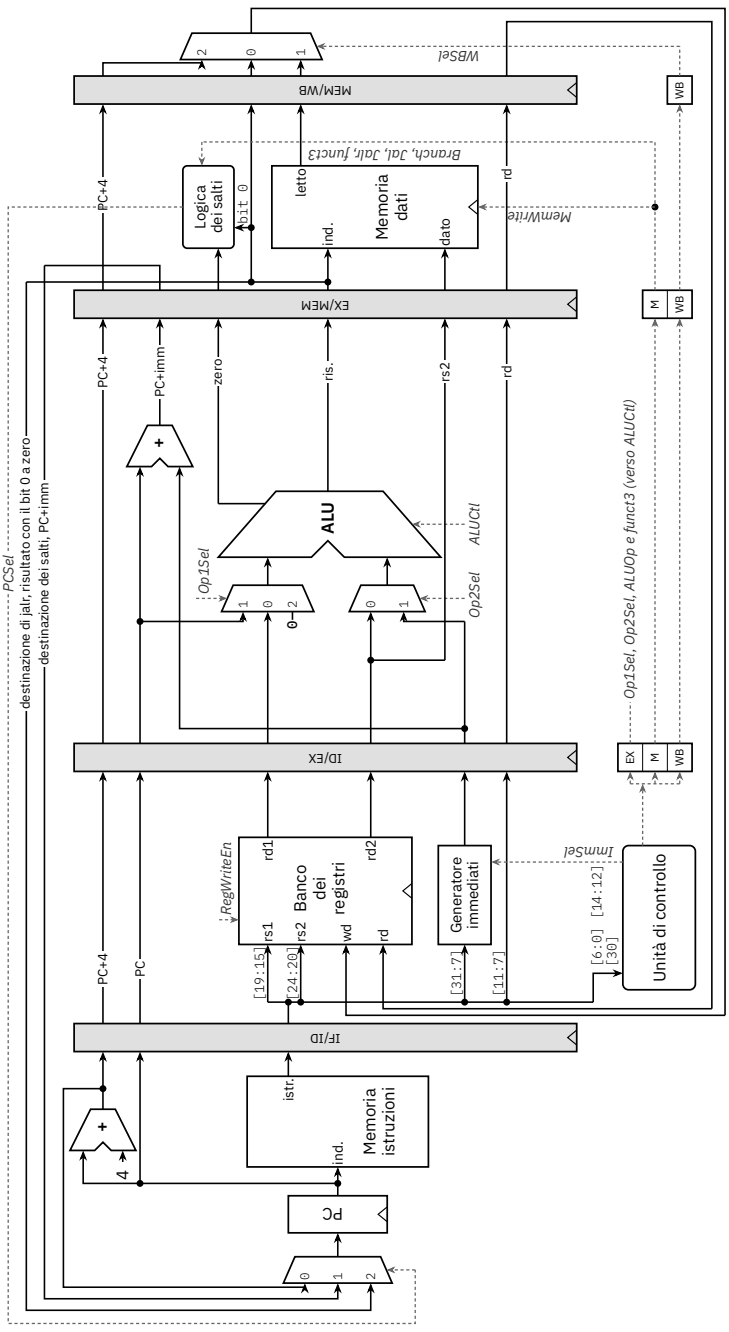


Figura 9.4 Il datapath a pipeline. I quattro registri di pipeline (in grigio) separano le fasi; sotto ognuno, i campi del controllo che viaggiano con l'istruzione. Il controllo M comanda la memoria e la logica dei salti, che in questa versione decide in MEM; il controllo WB comanda il multiplexer finale e, attraverso RegWriteEn, la scrittura nel banco dei registri.

Tabella 9.1 Il contenuto dei registri di pipeline. I segnali di controllo sono descritti nella tabella 9.2.

Registro	Dati	Controllo
IF/ID	istruzione, PC, $PC + 4$	—
ID/EX	valori di `rs1` e `rs2`, immediato, PC, $PC + 4$, numero di `rd`	EX, M, WB
EX/MEM	risultato dell'ALU e segnale zero, valore di `rs2` (il dato da scrivere), $PC + imm$, $PC + 4$, numero di `rd`	M, WB
MEM/WB	risultato dell'ALU, dato letto, $PC + 4$, numero di `rd`	WB

!

Il registro giusto L'ultimo punto merita attenzione. Nel ciclo singolo il campo `rd` era collegato direttamente dall'istruzione al banco dei registri. Nella pipeline, quando una `lw` arriva in WB, l'istruzione in ID è un'altra, tre posti più indietro. Se il numero del registro venisse preso da IF/ID, il dato della `lw` verrebbe scritto nel registro di destinazione di quell'altra istruzione. Il numero `rd` deve quindi attraversare ID/EX, EX/MEM e MEM/WB e arrivare in WB insieme al valore. Lo stesso vale per RegWriteEn: è il segnale dell'istruzione in WB, non di quella in ID.

Che cosa c'è nei registri

La tabella 9.1 elenca il contenuto di ogni registro. Una regola la riassume: un valore entra in un registro se *qualche* fase successiva lo userà, ed esce dalla pipeline appena l'ultima fase che lo usa è passata. Il PC, per esempio, si ferma in ID/EX, perché dopo EX nessuna fase ne ha bisogno; lo stesso vale per l'immediato.

I registri di pipeline sono larghi. IF/ID ha 96 bit, tre parole da 32; ID/EX, il più largo, supera i 170 bit fra dati e controllo. Sono registri invisibili al programmatore, come IR o ALUOut nel multiciclo: non fanno parte dello stato architetturale, ma senza di loro la pipeline non potrebbe funzionare.

9.4 Il controllo viaggia con l'istruzione

L'unità di controllo resta quella della tabella 7.1: esamina l'opcode in ID e produce tutti i segnali. Ma ora i segnali servono in momenti diversi. Op1Sel serve in EX, un ciclo dopo; MemWrite in MEM, due cicli dopo; WBSel e RegWriteEn in WB, tre cicli dopo. Nel frattempo l'unità di controllo sta già decodificando altre istruzioni.

La soluzione è trattare il controllo come un dato. I segnali entrano in ID/EX

Tabella 9.2 I tre campi di controllo. Accanto ai segnali della tabella 7.1 viaggiano alcuni campi dell'istruzione che servono più avanti.

Campo	Usato in	Contenuto
EX	EX	Op1Sel, Op2Sel, ALUOp, e per il controllo dell'ALU funct3 e il bit 30
M	MEM	MemWrite, Branch, Jal, Jalr, e funct3 per la dimensione degli accessi e per la condizione dei salti
WB	WB	RegWriteEn, WBSel

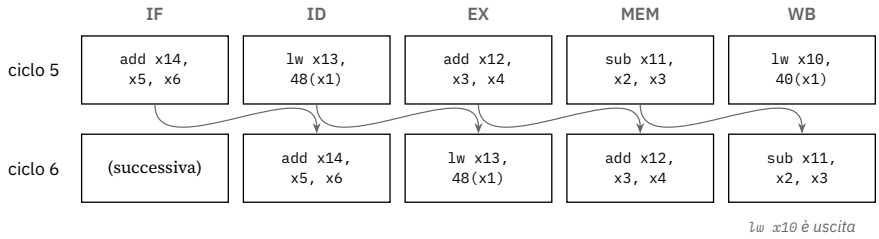


Figura 9.5 Due fotografie della pipeline. Fra il ciclo 5 e il ciclo 6 ogni istruzione avanza di una fase, la prima esce e ne entra una nuova.

divisi in tre gruppi, uno per fase (tabella 9.2). Ogni fase usa il suo gruppo e trasmette in avanti gli altri: EX consuma il gruppo EX e copia M e WB in EX/MEM; MEM consuma M e copia WB in MEM/WB. Nella figura 9.4 sono i riquadri sotto i registri, che si accorciano da sinistra a destra. A differenza del ciclo singolo, i segnali di controllo non si consumano nel ciclo in cui nascono: accompagnano la loro istruzione fino all'ultima fase che li usa.

Due segnali non entrano nei registri. ImmSel serve in ID stesso, nello stesso ciclo in cui viene prodotto: il generatore degli immediati lavora accanto all'unità di controllo. PCSel invece non proviene dall'unità di controllo: è calcolato dalla logica dei salti in MEM, a partire da Branch, Jal, Jalr, funct3 e dall'esito del confronto eseguito dall'ALU.

9.5 Seguire le istruzioni

Per capire la pipeline conviene fotografarla in un ciclo preciso. La figura 9.5 mostra le cinque istruzioni della figura 9.1 ai cicli 5 e 6, come le vedrebbe chi osservasse il datapath della figura 9.4.

Seguiamo in dettaglio la prima `lw x10, 40(x1)`, all'indirizzo `0x1000`.

Ciclo 1, IF

La memoria istruzioni legge l'istruzione all'indirizzo `0x1000`; il sommatore

calcola `0x1004`. Al fronte di salita il PC diventa `0x1004`, e IF/ID memorizza l'istruzione, `0x1000` e `0x1004`.

Ciclo 2, ID

Il banco dei registri legge `x1`; il generatore degli immediati produce 40; il controllo riconosce una load. Intanto IF sta già prelevando `sub`. Al fronte, ID/EX memorizza il valore di `x1`, 40, il numero 10 e i tre campi di controllo.

Ciclo 3, EX

L'ALU somma il valore di `x1` e 40: è l'indirizzo. EX/MEM lo memorizza, insieme a 10 e ai campi M e WB.

Ciclo 4, MEM

La memoria dati legge la parola all'indirizzo calcolato. MEM/WB memorizza il dato letto, 10 e il campo WB.

Ciclo 5, WB

WBSel = 1 seleziona il dato letto; RegWriteEn = 1 e il numero 10 arrivano al banco dei registri. Al fronte di salita `x10` riceve il dato.

Mentre la `lw` scrive `x10`, nello stesso ciclo, in ID, la seconda `lw` legge `x1`: il banco dei registri è usato da due istruzioni nello stesso ciclo, una in scrittura e una in lettura. Con due porte di lettura e una di scrittura indipendenti non c'è problema, purché i registri coinvolti siano diversi. Ma se fossero lo stesso? È il primo segnale di un problema più generale.

9.6 Quando le cose si complicano: i conflitti

Il modello di von Neumann prevede che le istruzioni si eseguano una alla volta, e il programmatore ragiona proprio così: quando un'istruzione comincia, la precedente è terminata. La pipeline viola questo principio, perché cinque istruzioni sono in esecuzione insieme. Finché le istruzioni sono indipendenti, come nella figura 9.1, la sovrapposizione non produce effetti visibili. Ma ci sono tre situazioni in cui la sovrapposizione cambierebbe il risultato, o non sarebbe possibile. Si chiamano **conflitti** (in inglese *hazard*, pericolo), e la figura 9.6 ne mostra un esempio per tipo.

Conflitti strutturali

Due istruzioni richiedono la stessa risorsa nello stesso ciclo. Con una memoria unica alla von Neumann, una load o una store in MEM e l'istruzione in IF si contenderebbero la memoria. Il nostro datapath li evita per costruzione, con le due memorie separate del modello di Harvard (stato 0x0, § 0.3) e con la regola delle cinque fasi per tutti, che impedisce a due istruzioni di arrivare in WB insieme.

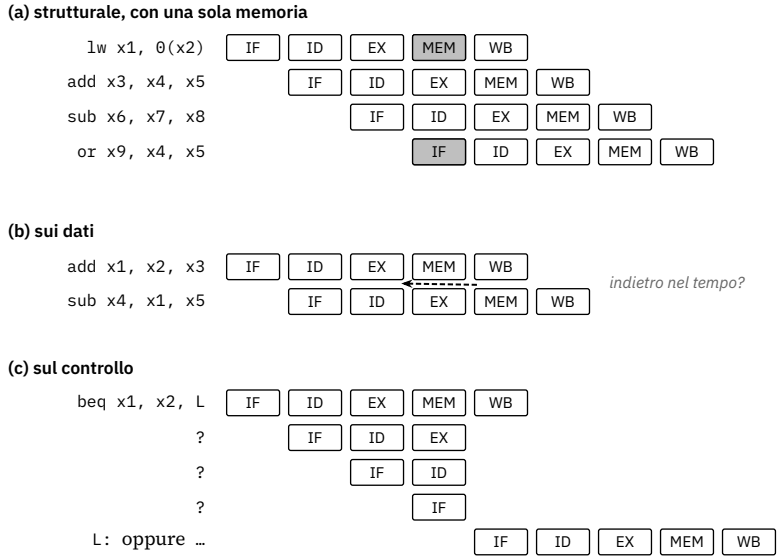


Figura 9.6 I tre tipi di conflitto. (a) Con una memoria unica, al ciclo 4 la `lw` legge il dato mentre `or` dovrebbe essere prelevata. (b) `sub` legge `x1` prima che `add` l'abbia scritto: la freccia dovrebbe andare indietro nel tempo. (c) Prima che il salto sia deciso, la pipeline deve già prelevare tre istruzioni.

Conflitti sui dati

Un'istruzione ha bisogno di un valore che un'istruzione precedente non ha ancora scritto. È il caso più frequente: i programmi sono pieni di istruzioni che usano il risultato della precedente. Lo affronteremo nello stato 0xa.

Conflitti sul controllo

Il PC dell'istruzione successiva dipende da un'istruzione che non ha ancora finito: un salto, che si decide solo in MEM, quando altre tre istruzioni sono già entrate. Ce ne occuperemo nello stato 0xb, insieme alle eccezioni, che sono salti imprevisti.

Un primo rimedio: metà ciclo per scrivere, metà per leggere

C'è un conflitto che si può eliminare subito, con un'idea semplice. Nella figura 9.6(b), se fra `add` e `sub` ci fossero altre due istruzioni, `sub` arriverebbe in ID al ciclo 5, lo stesso ciclo in cui `add` scrive `x1` in WB. La scrittura avviene al fronte di salita alla fine del ciclo, quindi la lettura troverebbe ancora il valore vecchio.

Il banco dei registri, però, è veloce: un accesso dura 100 ps, metà del ciclo. Si può allora organizzarlo in modo che la scrittura avvenga nella *prima*

metà del ciclo e la lettura nella *seconda*. Con questo accorgimento, nel ciclo in cui un'istruzione è in WB, un'istruzione in ID legge già il valore nuovo. Funziona quando fra l'istruzione che scrive e quella che legge ci sono almeno *due* istruzioni; a distanza uno o due il valore non è ancora arrivato in WB, e serviranno le tecniche dello stato 0xa. D'ora in poi il nostro banco dei registri si comporterà sempre così.

!

Un'istruzione in mezzo non basta Si legge a volte che l'accorgimento del mezzo ciclo risolve il conflitto quando le istruzioni dipendenti sono separate da almeno un'istruzione. Contiamo i cicli. Se `add` entra in IF al ciclo 1, scrive in WB al ciclo 5. L'istruzione subito dopo legge in ID al ciclo 3, quella a distanza due al ciclo 4, quella a distanza tre al ciclo 5. Solo quest'ultima, con due istruzioni in mezzo, legge nello stesso ciclo della scrittura e approfitta del mezzo ciclo.

1961 Organizzato come una catena di montaggio

Nel 1956 IBM promise al laboratorio di Los Alamos un calcolatore cento volte più veloce del suo modello di punta, l'IBM 704. Lo chiamò *Stretch*, perché doveva tendere al limite ogni tecnologia del tempo. Il primo esemplare, l'IBM 7030, arrivò a Los Alamos nel 1961. Secondo la descrizione della stessa IBM, l'unità di elaborazione era «organizzata come una catena di montaggio»: ogni parte, finito il suo compito, passava il lavoro alla successiva e cominciava il compito seguente, e un'unità detta di *look-ahead* preparava le istruzioni future mentre si calcolavano le presenti. Stretch fu il calcolatore più veloce del mondo fino al 1964, ma era solo da 30 a 40 volte più veloce del 704. IBM ne abbassò il prezzo da 13,5 a 7,8 milioni di dollari, ne installò soltanto nove e ritirò il prodotto. Commercialmente fu un fallimento; tecnicamente fu una miniera: la sovrapposizione delle istruzioni, la protezione della memoria e le interruzioni generalizzate passarono al System/360, la famiglia dello stato 0x0, e da lì a quasi tutti i processori successivi.

9.7 Verso i conflitti

Riassumiamo il punto in cui siamo. La pipeline ha lo stesso datapath del ciclo singolo, tagliato in cinque parti da quattro registri. Con le nostre ipotesi è quattro volte più veloce, a patto di non incontrare conflitti. Abbiamo evitato i conflitti strutturali in fase di progetto; quelli sui dati e sul controllo, invece, nascono dal programma che stiamo eseguendo, e il processore deve rilevarli durante l'esecuzione. Nello stato 0x7 avevamo notato che solo due fili del datapath tornano indietro: il risultato che torna al banco dei registri e il nuovo indirizzo che torna al PC. Nella pipeline quei due fili attraversano i registri all'indietro, da un'istruzione più avanzata a una più giovane, e sono esattamente l'origine dei due tipi di conflitto che restano.

Condizione di transizione uscita verso `0xa`

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Che differenza c'è fra latenza e throughput? Quale dei due migliora con la pipeline? ↺ § 9.2

2 Con le ipotesi del capitolo, quanto dura il ciclo della pipeline, e perché l'accelerazione è 4 e non 5? ↺ § 9.2

3 Quanto tempo serve per eseguire 100 istruzioni senza conflitti nella pipeline? E nel ciclo singolo? ↺ § 9.2

4 Perché una `add` attraversa la fase MEM anche se non usa la memoria? ↺ § 9.1

5 Che cosa contiene il registro EX/MEM? ↺ § 9.3

6 Perché il numero del registro `rd` deve attraversare tutti i registri di pipeline? ↺ § 9.3

7 In quale campo di controllo viaggiano MemWrite e WBSel? Quali segnali non entrano nei registri di pipeline? ↺ § 9.4

8 Nella figura 9.1, in quale fase si trova `lw x13, 48(x1)` al ciclo 6? ↺ § 9.5

9 Fai un esempio di ciascuno dei tre tipi di conflitto. ↺ § 9.6

STATO

0xa



Conflitti sui dati: propagare e aspettare

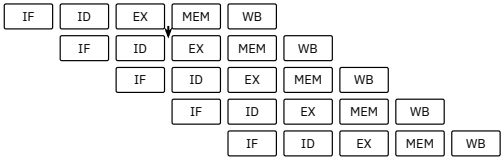


→ INGRESSI

- Il datapath a pipeline e i registri di pipeline (stato 0x9, § 9.3).
- I campi di controllo EX, M e WB (tabella 9.2).
- I tre tipi di conflitto e il banco dei registri che scrive a metà ciclo (§ 9.6).

USCITE →

- Trovare le dipendenze in un frammento di codice e stabilire quali diventano conflitti.
- Risolvere un conflitto aspettando, e calcolarne il costo.
- Spiegare la propagazione dei risultati e scrivere le condizioni dell'unità di propagazione.
- Riconoscere il conflitto fra una load e l'istruzione che ne usa il dato, e descrivere come l'hardware inserisce una bolla.
- Riordinare il codice per evitare gli stalli.



In un programma reale le istruzioni non sono indipendenti come quelle della figura 9.1. Al contrario: quasi ogni istruzione usa un valore calcolato poco prima. Questo stato affronta il problema in tre tappe. Prima lo risolviamo

nel modo più semplice, aspettando. Poi scopriamo che quasi sempre il valore esiste già, da qualche parte nella pipeline, e basta prelevarlo da lì. Infine incontriamo l'unico caso in cui questo non basta, e dotiamo il processore di un meccanismo per fermarsi autonomamente.

a.1 Dipendenze e conflitti

Consideriamo questa sequenza, in cui le ultime quattro istruzioni usano il registro `x2` scritto dalla prima:

```
        sub  x2, x1, x3       # scrive x2
        and  x12, x2, x5      # legge x2
        or   x13, x6, x2      # legge x2
        add  x14, x2, x2      # legge x2 due volte
        sw   x15, 100(x2)     # usa x2 come base dell'indirizzo
```

Diciamo che c'è una **dipendenza** fra due istruzioni quando una legge un registro che l'altra, precedente, scrive. Le dipendenze sono una proprietà del *programma*: esisterebbero anche nel processore a ciclo singolo, dove però non creano problemi, perché ogni istruzione finisce prima che la successiva cominci. Si chiamano anche dipendenze *RAW*, dall'inglese *read after write*: la lettura deve avvenire dopo la scrittura.

Una dipendenza diventa un **conflitto sui dati** quando, nella pipeline, la lettura avverrebbe *prima* della scrittura. La figura a.1 lo mostra. `sub` scrive `x2` in WB, al ciclo 5. `and` lo legge in ID al ciclo 3, `or` al ciclo 4: troppo presto, leggerebbero il valore vecchio. `add` lo legge al ciclo 5, nello stesso ciclo della scrittura, e grazie al banco dei registri che scrive nella prima metà del ciclo e legge nella seconda (§ 9.6) trova già il valore nuovo. Per `sw` non c'è alcun problema. Delle quattro dipendenze, quindi, solo due sono conflitti: quelle a distanza uno e due.

Due precisazioni. La prima: `x0` non crea mai dipendenze. Le scritture in `x0` vengono ignorate e la lettura restituisce sempre zero, quindi un'istruzione che legge `x0` non deve attendere alcun risultato. La seconda: in questa pipeline solo le dipendenze RAW possono diventare conflitti. Un programma contiene anche altre dipendenze, per esempio un'istruzione che scrive un registro letto da una precedente; ma tutte le istruzioni leggono i registri nella stessa fase, ID, e li scrivono nella stessa fase, WB, sempre nell'ordine del programma, quindi nessuna può scrivere un registro prima che una precedente l'abbia letto o scritto. Quelle altre dipendenze diventano un problema solo con l'esecuzione fuori ordine (*out-of-order*), che affronteremo nello stato 0xc.

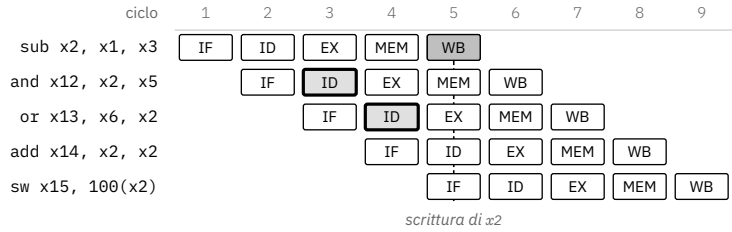


Figura a.1 Le dipendenze da `sub`, che scrive `x2` in WB (in grigio) nella prima metà del ciclo 5. Le quattro istruzioni successive leggono `x2` in ID: `and` e `or` lo leggono prima della scrittura (caselle evidenziate), e sono conflitti; `add` lo legge nella seconda metà del ciclo 5 e `sw` al ciclo 6, e trovano il valore giusto.

a.2 La soluzione più semplice: aspettare

Se `and` legge troppo presto, la soluzione più semplice è avviarla più tardi. Il compilatore, o il programmatore, può inserire fra le due istruzioni delle `nop`, istruzioni che non svolgono alcuna operazione (stato 0x1, § 1.8): `nop` è `addi x0, x0, 0`, che scrive in `x0` e quindi non cambia lo stato. Due `nop` bastano a portare la lettura di `and` al ciclo 5:

```
        sub  x2, x1, x3
        nop
        nop
        and  x12, x2, x5      # legge x2 mentre sub lo scrive
        or   x13, x6, x2
```

Lo stesso effetto si può ottenere in hardware: invece di eseguire `nop` scritte nel programma, il processore rileva il conflitto e trattiene l'istruzione in ID, facendo avanzare nel frattempo istruzioni vuote. Fermare così una parte della pipeline si chiama **stallo**, e le istruzioni vuote che avanzano al posto di quella ferma si chiamano **bolle**: posti vuoti nella catena di montaggio, che arrivano fino in fondo senza produrre nulla.

In entrambi i casi il risultato è corretto, ma si perde tempo. La sequenza di cinque istruzioni, che senza conflitti finirebbe in 9 cicli, ne richiede 11. Se in un programma tipico un'istruzione su tre dipende dalla precedente, gli stalli fanno salire il CPI ben sopra 1, e la pipeline perde buona parte del suo vantaggio. Occorre una soluzione migliore.

1981 **Un processore senza blocchi**

Nel 1981 John Hennessy avviò a Stanford un progetto di processore RISC con i suoi studenti. Il nome era una dichiarazione di intenti: MIPS, *Microprocessor without Interlocked Pipeline Stages*, un microprocessore le cui fasi non si bloccano a vicenda. I circuiti che rilevano i conflitti e fermano la pipeline, i cosiddetti *interlock*, allungavano il percorso critico; Hennessy preferì eliminarli e affidare il problema al compilatore, che doveva ordinare le istruzioni in modo da non creare mai conflitti, inserendo `nop` quando non trovava di meglio. Il progetto diventò un'azienda, MIPS Computer Systems, che nel 1985 annunciò il primo processore commerciale, l'R2000. Col tempo però anche MIPS aggiunse gli interlock: già la seconda versione dell'architettura, MIPS II, tolse l'obbligo di non usare subito il dato di una load. Nel marzo 2021 la società che ne portava il nome annunciò che avrebbe smesso di sviluppare l'architettura MIPS per progettare processori RISC-V. Hennessy e David Patterson, l'altro padre del RISC, avevano ricevuto da poco il premio Turing 2017, annunciato nel marzo 2018.

a.3 Propagare il risultato

Guardiamo meglio la figura a.1. `and` ha bisogno di `x2` non in ID, ma in EX, quando l'ALU lo usa: al ciclo 4. E il nuovo valore di `x2` esiste già alla fine del ciclo 3, quando l'ALU di `sub` l'ha calcolato. Al ciclo 4 si trova nel registro EX/MEM, in attesa di attraversare le ultime due fasi per arrivare nel banco dei registri. Non occorre attenderlo: basta un filo che lo trasmetta da EX/MEM all'ingresso dell'ALU. Allo stesso modo, al ciclo 5 `or` riceve il valore dalla fase WB, all'uscita del multiplexer WBSel.

Questa tecnica si chiama **propagazione**, in inglese *forwarding* o *bypassing*: il risultato viene inoltrato in avanti, a un'istruzione più giovane, scavalcando il banco dei registri. Il termine non va confuso con il ritardo di propagazione dei circuiti (stato 0x2): qui indica l'inoltro di un valore da un'istruzione all'altra, non il tempo di risposta delle porte logiche. Nei diagrammi della pipeline la propagazione si disegna come una freccia che va dalla fine della fase in cui il valore è pronto all'inizio della fase in cui serve (figura a.2). Finché la freccia va avanti nel tempo, o resta sullo stesso confine fra due cicli, la propagazione è possibile.

Il nuovo hardware

Per realizzare la propagazione servono due componenti (figura a.4).

- *Due multiplexer di propagazione*, uno per ciascun registro sorgente, all'inizio della fase EX. Ognuno seleziona fra tre valori: 0, il valore letto dal banco dei registri e arrivato con ID/EX; 1, il valore che sta per essere scritto in WB, preso all'uscita del multiplexer WBSel; 2, il risultato dell'ALU salva-

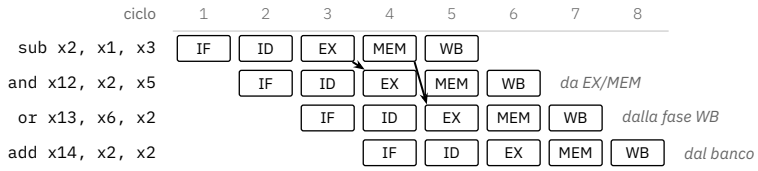


Figura a.2 La propagazione. Il risultato di `sub` arriva all'ALU di `and` dal registro EX/MEM e all'ALU di `or` dall'uscita del multiplexer WBSel, nella fase WB. `add` lo legge normalmente dal banco dei registri.

to in EX/MEM. Le loro uscite entrano nei multiplexer degli operandi della parte II, che restano invariati. L'uscita del multiplexer di `rs2` raggiunge anche la memoria come dato da scrivere: una `sw` può così memorizzare un valore appena calcolato.

- *Un'unità di propagazione*, che comanda i due multiplexer. Per decidere deve conoscere i registri letti dall'istruzione in EX, e per questo i numeri di `rs1` e `rs2` entrano ora anche nel registro ID/EX.

Le condizioni

L'unità di propagazione confronta i registri letti dall'istruzione in EX con i registri scritti dalle due istruzioni che la precedono, quella in MEM e quella in WB. Chiamiamo ForwardA e ForwardB i segnali che comandano i multiplexer di `rs1` e di `rs2`. Per `rs1`:

```
se EX/MEM.RegWriteEn = 1 e EX/MEM.rd != 0
   e EX/MEM.rd = ID/EX.rs1
     allora ForwardA = 2     # dal registro EX/MEM
altrimenti se MEM/WB.RegWriteEn = 1 e MEM/WB.rd != 0
   e MEM/WB.rd = ID/EX.rs1
     allora ForwardA = 1     # dal valore in scrittura in WB
altrimenti   ForwardA = 0    # dal banco dei registri
```

ForwardB si calcola nello stesso modo con `rs2` al posto di `rs1`. Ognuna delle tre condizioni ha un motivo preciso.

- *Il confronto dei numeri* è il cuore della regola: si propaga solo se l'istruzione più vecchia scrive proprio il registro che quella più giovane legge.
- *RegWriteEn* = 1: le store e i salti condizionati non scrivono registri, ma i bit 11:7 della loro codifica esistono lo stesso, e contengono parti dell'immediato (stato 0x1, § 1.9). Senza questo controllo, una `sw` il cui immediato contiene per caso il numero 5 farebbe propagare un valore senza senso al posto di `x5`.
- $rd \neq 0$: un'istruzione che scrive in `x0` non cambia nulla, e un'istruzione che

legge `x0` deve ottenere zero, non il risultato scartato di un'altra istruzione.

Resta un caso particolare: `jal` e `jalr` scrivono in `rd` il link, $PC + 4$, e non il risultato dell'ALU, che EX/MEM propagherebbe. In questo caso, però, non si verifica alcun errore. Il salto si decide in MEM, ed è sempre preso: le istruzioni che lo seguono in pipeline vengono annullate, e la prima istruzione valida arriva in EX quando il link è già nel banco dei registri. Il problema nascerà quando decideremo i salti prima, nello stato 0xb.

Il conflitto ripetuto

Perché la prima condizione ha la precedenza sulla seconda? Consideriamo tre istruzioni che modificano lo stesso registro una dopo l'altra:

```
        add  x1, x1, x2
        add  x1, x1, x3
        add  x1, x1, x4       # quale x1 deve usare?
```

Quando la terza è in EX, la prima è in WB e la seconda in MEM: entrambe scrivono `x1`, ed entrambe le condizioni sono vere. Il valore corretto è quello della seconda, la più recente nell'ordine del programma, che si trova in EX/MEM. Per questo la propagazione da EX/MEM viene controllata per prima, e quella dalla fase WB solo se la prima non è soddisfatta.

!

Una condizione scritta male In alcuni testi la priorità si ottiene aggiungendo alla seconda condizione il termine «EX/MEM.rd ≠ ID/EX.rs1». Sembra equivalente, ma non lo è. Se l'istruzione in MEM è una store, il suo campo rd è una parte dell'immediato che può coincidere per caso con rs1: la seconda condizione verrebbe bloccata, e l'istruzione in EX perderebbe il valore che le serve dalla fase WB. La precedenza va data solo quando la prima condizione è vera *per intero*, compreso RegWriteEn: è proprio ciò che fa la struttura «se … altrimenti».

Con la propagazione e il banco dei registri che scrive a metà ciclo, ogni dipendenza da un'istruzione aritmetica o logica è risolta senza perdere neanche un ciclo: a distanza uno il valore arriva da EX/MEM, a distanza due dall'uscita del multiplexer WBSel nella fase WB, a distanza tre o più dal banco dei registri. Quasi tutte le dipendenze, ma non tutte.

a.4 Quando propagare non basta

Proviamo con una load:

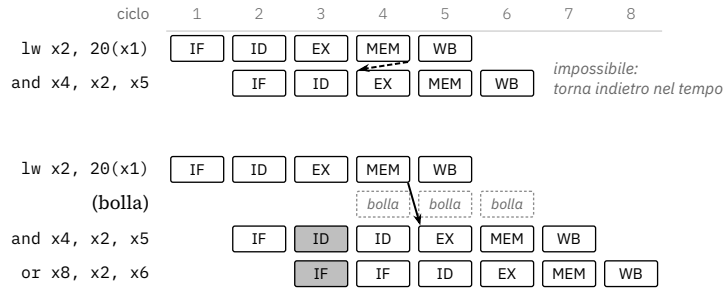


Figura a.3 Il conflitto load-use. In alto: il dato esce dalla memoria alla fine del ciclo 4, ma `and` lo vorrebbe all'inizio del ciclo 4. In basso: `and` resta in ID un ciclo in più (la casella grigia), e `or` resta in IF; al loro posto avanza una bolla. Al ciclo 5 il dato arriva all'ALU dalla fase WB, attraverso l'uscita del multiplexer WBSel.

```
lw   x2, 20(x1)
and  x4, x2, x5       # usa subito il dato caricato
```

Il dato della `lw` non esce dall'ALU: esce dalla memoria dati, alla fine della fase MEM, cioè alla fine del ciclo 4. Ma `and` lo richiede all'inizio della sua fase EX, che è proprio il ciclo 4. La freccia della propagazione dovrebbe tornare indietro nel tempo (figura a.3, in alto), e nessun filo può farlo. Questo è il **conflitto load-use**: una load seguita subito da un'istruzione che usa il suo risultato.

L'unica soluzione è aspettare un ciclo. Dopo lo stallo, `and` è in EX al ciclo 5, e il dato della `lw`, ormai nella fase WB, le arriva con la normale propagazione dall'uscita del multiplexer WBSel. Il costo è un ciclo per ogni conflitto load-use: un costo contenuto, ma non più nullo.

a.5 L'unità di rilevamento dei conflitti

Lo stallo deve essere deciso dall'hardware, e con anticipo: quando la `lw` è in EX e `and` in ID, cioè al ciclo 3. Se la decisione fosse rinviata al ciclo 4, `and` sarebbe già in EX con il valore sbagliato. Se ne occupa una nuova unità nella fase ID, l'**unità di rilevamento dei conflitti** (in inglese *hazard detection unit*). La sua condizione è:

```
se   ID/EX.RegWriteEn = 1 e ID/EX.WBSel = 1   # in EX c'è una load
e    ID/EX.rd != 0
e    (ID/EX.rd = IF/ID.rs1  oppure  ID/EX.rd = IF/ID.rs2)
     allora ferma la pipeline per un ciclo
```

Nei testi derivati da MIPS il primo controllo si chiama *MemRead*, un segnale

che vale 1 per le load. Nel nostro datapath quel segnale non esiste, ma non serve: le load sono le sole istruzioni che scrivono un registro (RegWriteEn = 1) con un dato letto dalla memoria (WBSel = 1). Il controllo su RegWriteEn è necessario, perché nelle store e nei salti condizionati WBSel è indifferente (tabella 7.1) e potrebbe valere 1. Il controllo su `rd` $\neq 0$ ha lo stesso motivo che nella propagazione: una load in `x0` non produce alcun valore da attendere.

Come si ferma una pipeline

Fermare la pipeline comporta tre azioni, tutte nello stesso ciclo (figura a.4):

1. *Il PC non cambia.* Il segnale PCWrite = 0 impedisce al PC di aggiornarsi, e l'istruzione in IF verrà prelevata di nuovo.
2. *IF/ID non cambia.* IF/IDWrite = 0 trattiene in ID l'istruzione dipendente, che al ciclo successivo verrà decodificata una seconda volta.
3. *In EX entra una bolla.* Un multiplexer dopo l'unità di controllo sostituisce tutti i segnali diretti in ID/EX con degli zeri. Con RegWriteEn, MemWrite, Branch, Jal e Jalr a zero, l'istruzione che attraversa EX, MEM e WB non può modificare nulla: è a tutti gli effetti una `nop`, inserita dall'hardware.

Al ciclo successivo la `lw` è in MEM, la bolla in EX, `and` di nuovo in ID. Questa volta la condizione è falsa, perché in EX c'è la bolla, e la pipeline riparte. I registri di pipeline diventano quindi registri con un ingresso di abilitazione, come il PC del multiciclo: scrivono solo quando il segnale di scrittura vale 1.

> **Stalli inutili** L'unità di rilevamento, come quella di propagazione, confronta i bit 19:15 e 24:20 dell'istruzione anche quando l'istruzione non legge `rs1` o `rs2`: in una `lui` o in una `jal` quei bit fanno parte dell'immediato. Per la propagazione non importa, perché il valore propagato viene poi ignorato dai multiplexer degli operandi. Per il rilevamento, invece, una coincidenza casuale produce uno stallo inutile: il risultato resta corretto, ma si perde un ciclo. I processori reali lo evitano facendo indicare all'unità di controllo quali registri l'istruzione legge effettivamente.

> **Copiare un dato** Una `lw` seguita da una `sw` che ne scrive il dato in memoria, come nella copia di una parola, è un conflitto load-use per la nostra unità di rilevamento. In realtà la `sw` ha bisogno del dato solo in MEM, un ciclo dopo; con un percorso di propagazione aggiuntivo, dall'uscita del multiplexer WBSel nella fase WB all'ingresso dei dati della memoria, lo stallo si eviterebbe. È uno dei tanti miglioramenti possibili, che il nostro processore didattico non implementa.

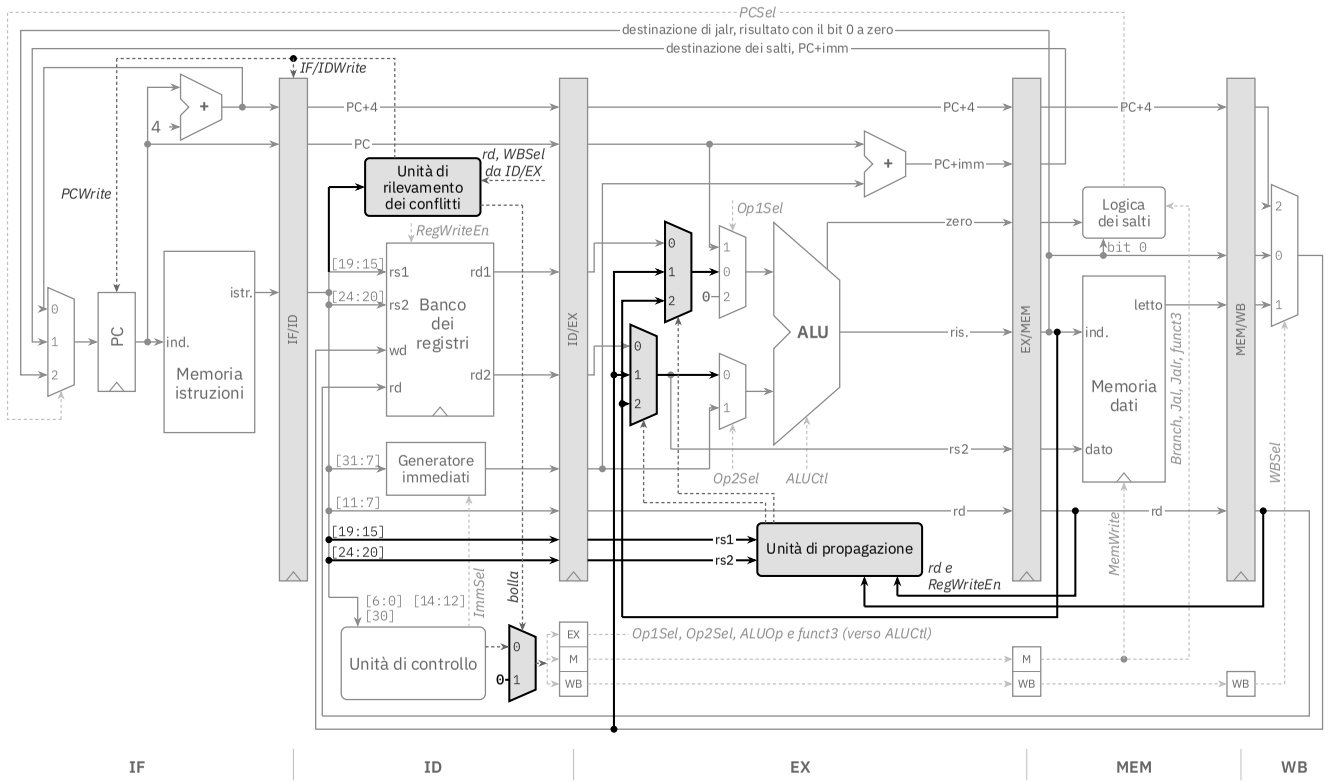


Figura a.4 Il datapath a pipeline con la propagazione e il rilevamento dei conflitti. Le parti nuove sono in nero, quelle dello stato 0x9 in grigio. L'unità di propagazione confronta rs1 e rs2 dell'istruzione in EX con il registro di destinazione delle istruzioni in MEM e in WB, e comanda i due multiplexer davanti a quelli degli operandi. L'unità di rilevamento confronta rs1 e rs2 dell'istruzione in ID con il registro di destinazione di una load in EX, e in caso di conflitto ferma il PC e IF/ID e inserisce una bolla in ID/EX.

a.6 Il compilatore può aiutare

Gli stalli dovuti alle load si possono spesso evitare cambiando l'ordine delle istruzioni, senza cambiare il risultato. Prendiamo le due righe di C

$$\texttt{a} = \texttt{b} + \texttt{e}; \qquad \texttt{c} = \texttt{b} + \texttt{f};$$

con le cinque variabili in memoria, a partire dall'indirizzo in `a0`: `b` a 0, `e` a 4, `f` a 8, `a` a 12, `c` a 16. La traduzione più diretta è quella a sinistra; quella a destra anticipa la terza load.

```
lw   t1, 0(a0)
lw   t2, 4(a0)
add  t3, t1, t2   # stallo
sw   t3, 12(a0)
lw   t4, 8(a0)
add  t5, t1, t4   # stallo
sw   t5, 16(a0)
```

```
lw   t1, 0(a0)
lw   t2, 4(a0)
lw   t4, 8(a0)
add  t3, t1, t2
sw   t3, 12(a0)
add  t5, t1, t4
sw   t5, 16(a0)
```

A sinistra ogni `add` segue subito la load di uno dei suoi operandi: due stalli, e le sette istruzioni finiscono in $7 + 4 + 2 = 13$ cicli. A destra fra ogni load e l'istruzione che ne usa il dato c'è almeno un'altra istruzione: nessuno stallo, 11 cicli. Le `sw` non creano problemi, perché il valore da scrivere arriva dalla `add` precedente per propagazione. Il riordino è lecito perché `lw t4` non dipende da `add t3` e non accede alla locazione di memoria scritta da `sw t3`.

I compilatori svolgono questo lavoro automaticamente, e conoscono la pipeline del processore per cui compilano: è uno dei motivi per cui un programma compilato con le ottimizzazioni attive, per esempio con l'opzione `-O2` di GCC, può essere molto più veloce dello stesso programma compilato senza.

Quanto costano gli stalli

Il CPI della pipeline è 1 più i cicli persi in stalli, in media per istruzione. Riprendiamo la combinazione di istruzioni dello stato 0x7, con il 25% di load, e supponiamo che metà delle load siano seguite subito da un'istruzione che ne usa il dato. Allora

$$CPI = 1 + 0{,}25 \times 0{,}5 \times 1 = 1{,}125,$$

e il tempo medio per istruzione è $1{,}125 \times 200 = 225$ ps, contro gli 800 del ciclo singolo. Senza propagazione la situazione sarebbe molto peggiore: ogni

dipendenza a distanza uno costerebbe due cicli, ogni dipendenza a distanza due un ciclo.

Condizione di transizione uscita verso **0xb**

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1. Che differenza c'è fra una dipendenza e un conflitto sui dati? ↺ § a.1
2. Nella sequenza di § a.1, quali dipendenze sono conflitti e perché? ↺ § a.1
3. Senza propagazione, quante `nop` servono fra `add x1, x2, x3` e `sub x4, x1, x5`? ↺ § a.2
4. Da dove arriva il valore propagato a un'istruzione che dipende dalla precedente? E a una che dipende da quella due posti prima? ↺ § a.3
5. Perché l'unità di propagazione controlla anche RegWriteEn e rd ≠ 0? ↺ § a.3
6. In `add x1, x1, x2`; `add x1, x1, x3`; `add x1, x1, x4`, da dove riceve `x1` la terza istruzione? ↺ § a.3
7. Perché il conflitto load-use richiede uno stallo anche con la propagazione? ↺ § a.4
8. Quali operazioni compie l'hardware, ciclo per ciclo, per inserire una bolla? ↺ § a.5
9. Riordina `lw t0, 0(a0)`; `addi t0, t0, 1`; `lw t1, 4(a0)`; `addi t1, t1, 1` in modo da eliminare gli stalli. ↺ § a.6

STATO

0xb

1	0	1	1
b3	b2	b1	b0

Salti ed eccezioni: indovinare e rimediare

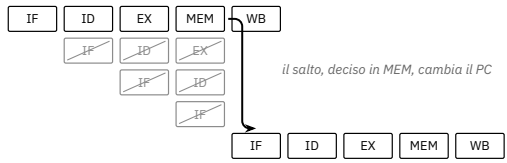


→ INGRESSI

- Il datapath con propagazione e rilevamento dei conflitti (stato 0xa, figura a.4).
- La logica dei salti e il confronto fatto dall'ALU (stato 0x4, § 4.5).
- `ecall` e i livelli di privilegio (stato 0x1, § 1.11).

USCITE →

- Calcolare quanti cicli costa un salto nella pipeline, e quanto pesa sul CPI.
- Spiegare come si annullano le istruzioni prelevate per errore.
- Descrivere i vantaggi e i problemi di decidere i salti in ID.
- Confrontare previsione statica e dinamica, e seguire un predittore a 1 e a 2 bit su un ciclo.
- Descrivere il comportamento della pipeline quando un'istruzione provoca un'eccezione, con i registri `mepc`, `mcause` e `mtvec`.



I conflitti sui dati riguardano il primo dei due fili che tornano indietro nel datapath, quello che trasmette i risultati al banco dei registri. Questo

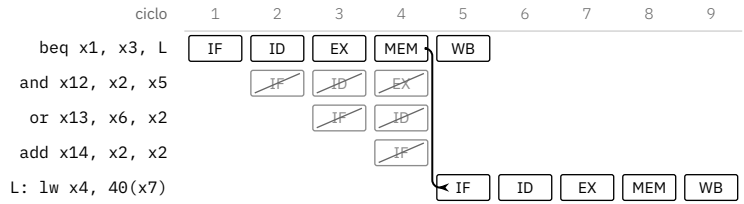


Figura b.1 Un salto preso, deciso in MEM. Le tre istruzioni prelevate dopo la `beq` non dovevano essere eseguite: vengono annullate (caselle barrate), e la destinazione entra in IF al ciclo 5. Il salto costa tre cicli.

stato si occupa del secondo, quello che trasmette il nuovo indirizzo al PC. Il problema è semplice da enunciare: per prelevare un'istruzione occorre conoscerne l'indirizzo, e dopo un salto lo si conosce soltanto quando il salto è stato eseguito. Le soluzioni sono di quattro tipi: attendere, decidere prima, ritardare il salto, prevedere. Le stesse idee servono anche per le eccezioni, che sono salti che nessuno ha scritto nel programma.

b.1 Il prezzo di un salto

Nella pipeline dello stato 0x9 la logica dei salti decide in MEM. Seguiamo un salto condizionato preso (figura b.1). La `beq` viene prelevata al ciclo 1 e arriva in MEM al ciclo 4; solo alla fine del ciclo 4 il PC riceve l'indirizzo della destinazione, e l'istruzione giusta viene prelevata al ciclo 5. Nel frattempo, nei cicli 2, 3 e 4, la fase IF non è rimasta inattiva: ha prelevato le tre istruzioni che seguono la `beq` in memoria, come se il salto non esistesse.

Se il salto non viene preso, quelle tre istruzioni sono proprio quelle giuste, e non si perde nulla. Se viene preso, sono sbagliate e vanno eliminate: si perdono tre cicli, la **penalità del salto**. Per `jal` e `jalr`, che saltano sempre, la penalità si paga ogni volta.

Quanto pesa? Riprendiamo la combinazione di istruzioni dello stato 0x7: 15% di salti condizionati e 5% di salti incondizionati. Supponiamo che metà dei salti condizionati vengano presi. I cicli persi per istruzione sono

$$0{,}15 \times 0{,}5 \times 3 + 0{,}05 \times 3 = 0{,}375,$$

tre volte più di quelli persi per le load nello stato 0xa. Sommati a quelli, il CPI arriva a $1 + 0{,}125 + 0{,}375 = 1{,}5$: la pipeline, che sulla carta era quattro volte più veloce del ciclo singolo, lo è ormai poco più di due volte e mezza. Ridurre la penalità dei salti è quindi la priorità.

b.2 Annullare le istruzioni sbagliate

Prima di ridurre la penalità, vediamo come si eliminano le istruzioni sbagliate. L'operazione si chiama **annullamento** (in inglese *flush*, letteralmente «svuotare con un getto») e sfrutta un fatto già osservato: un'istruzione cambia lo stato solo attraverso RegWriteEn, MemWrite e i segnali dei salti. Se questi segnali sono a zero, l'istruzione attraversa la pipeline senza lasciare traccia, proprio come la bolla dello stato 0xa.

Quando la logica dei salti decide di saltare, quindi, alla fine dello stesso ciclo:

- il registro EX/MEM riceve campi di controllo a zero al posto di quelli dell'istruzione che era in EX;
- il registro ID/EX riceve campi di controllo a zero al posto di quelli dell'istruzione che era in ID;
- il registro IF/ID riceve una `nop` al posto dell'istruzione appena prelevata.

Le tre istruzioni continuano ad avanzare, ma come bolle. Nessuna di esse ha fatto in tempo a cambiare lo stato: la più avanzata era in EX, e le scritture avvengono solo in MEM e in WB.

!

Una parola interamente nulla non è una nop Nei datapath derivati da MIPS l'istruzione in IF/ID si annulla azzerando il registro: in MIPS la parola `0x00000000` è `sll $0, $0, 0`, che non produce alcun effetto. In RISC-V invece la parola interamente nulla è, per scelta esplicita della specifica, un'istruzione *illegale*: se un programma finisce per errore in una zona di memoria azzerata, il processore rileva immediatamente l'errore. Annullando con gli zeri si provocherebbe quindi un'eccezione. In RISC-V si scrive in IF/ID la codifica di `nop`, cioè `0x00000013`, oppure si azzera un bit di validità che accompagna l'istruzione.

b.3 Decidere prima

La penalità è il numero di istruzioni prelevate prima della decisione. Per ridurla occorre decidere prima. La decisione richiede due informazioni: l'indirizzo della destinazione e l'esito del confronto. Entrambe si possono anticipare alla fase ID (figura b.2).

- *La destinazione* è *PC+imm*, e in ID si conoscono già sia il PC sia l'immediato: basta spostare in ID il sommatore dei salti.
- *Il confronto* richiede i valori dei due registri, che in ID sono appena stati letti. Invece di usare l'ALU in EX, si aggiunge in ID un circuito di confronto dedicato.

Con la decisione in ID, un salto preso costa un solo ciclo: quando la `beq` è in

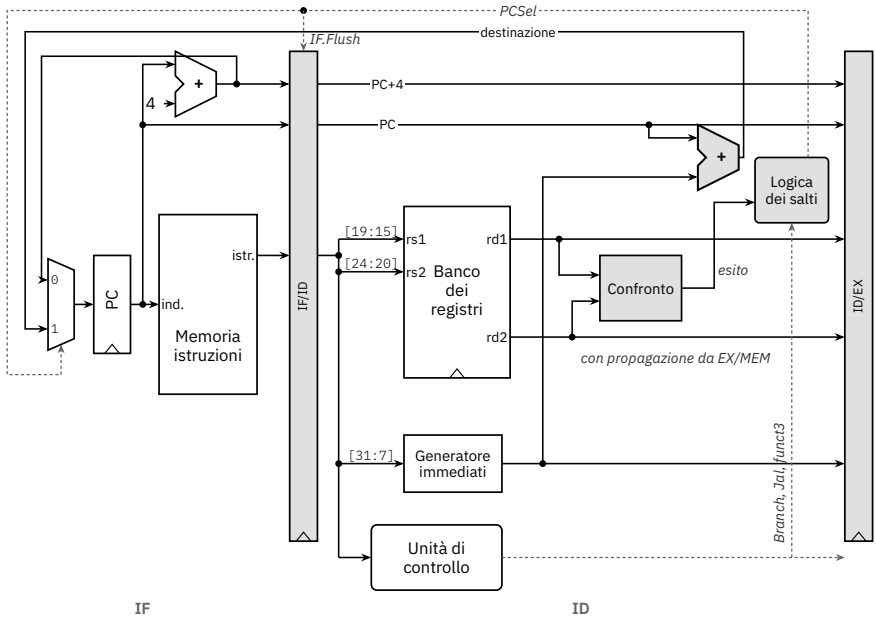


Figura b.2 Le fasi IF e ID con la decisione dei salti anticipata in ID. Il sommatore calcola *PC* + *imm*; il circuito di confronto, alimentato anche da percorsi di propagazione, calcola l'esito; la logica dei salti comanda il multiplexer del PC e, se il salto è preso, annulla l'istruzione appena prelevata con il segnale IF.Flush.

ID, solo l'istruzione in IF è stata prelevata per errore, e IF.Flush la trasforma in una `nop`. Con i numeri della sezione precedente i cicli persi scendono da 0,375 a $0{,}15 \times 0{,}5 + 0{,}05 = 0{,}125$ per istruzione.

Anticipare la decisione, però, ha un prezzo.

- *Il confronto in RISC-V non è banale.* MIPS confronta due registri solo per uguaglianza, con `beq` e `bne`, e un confronto di uguaglianza si realizza con 32 porte XOR e un albero di OR, molto più veloce di una sottrazione. RISC-V ha anche `blt`, `bge`, `bltu` e `bgeu`, che richiedono un vero confronto di grandezza. E il confronto viene *dopo* la lettura dei registri, che con l'accorgimento del mezzo ciclo occupa già la seconda metà della fase ID: la fase si allunga, e con essa rischia di allungarsi il periodo di clock.
- *Nascono nuovi conflitti sui dati.* Il confronto usa i registri un ciclo prima dell'ALU, quindi una dipendenza che prima si risolveva con la propagazione ora può richiedere uno stallo. La tabella b.1 riassume i casi: servono percorsi di propagazione verso il confronto e nuove condizioni nell'unità di rilevamento.

Tabella b.1 Un salto deciso in ID che usa il risultato di un'istruzione precedente. La distanza è 1 se le due istruzioni sono consecutive, 2 se ce n'è una in mezzo, e così via.

Istruzione precedente	Distanza	Che cosa succede
aritmetica o logica	1	è in EX: il risultato non c'è ancora, uno stallo e poi propagazione da EX/MEM
aritmetica o logica	2	è in MEM: propagazione da EX/MEM
load	1	è in EX: due stalli, poi il dato arriva dal banco dei registri
load	2	è in MEM: uno stallo, poi il dato arriva dal banco dei registri
qualunque	3 o più	il banco dei registri ha già il valore

- *Anche `jalr` richiede un registro.* La sua destinazione è *rs1* + *imm*: il sommatore in ID deve poter ricevere `rs1` al posto del PC, con gli stessi problemi di propagazione del confronto. C'è anche un caso sottile: se il registro è il link appena scritto da una `jal`, come in una funzione che ritorna subito, il valore da propagare da EX/MEM non è il risultato dell'ALU ma $PC + 4$. Per questo i progetti reali scelgono già in EX il valore che l'istruzione scriverà, e propagano quello.

Molti processori didattici, come la versione migliorata di Patterson e Hennessy, decidono in ID. Altri, come quello del libro di Harris e Harris, decidono in EX, con una penalità di due cicli e un periodo di clock meno vincolante. Non c'è una scelta giusta in assoluto: è un compromesso fra cicli persi e durata del ciclo, e il nostro processore didattico, da qui in avanti, decide in ID.

b.4 Ritardare il salto

Resta un ciclo di penalità, occupato dall'istruzione che segue il salto. I progettisti di MIPS e di SPARC ebbero un'idea radicale: modificare la definizione dell'ISA stabilendo che quell'istruzione viene *sempre* eseguita, che il salto sia preso o no. Il salto diventa **ritardato**: avviene dopo l'istruzione successiva, che occupa il cosiddetto *delay slot*. Spetta al compilatore riempire lo slot con un'istruzione utile, scelta in uno di questi modi:

1. un'istruzione *precedente* al salto e indipendente da esso, spostata dopo: è utile in ogni caso, ed è la scelta migliore;
2. un'istruzione della *destinazione*, copiata nello slot: è utile se il salto viene preso, e deve essere innocua se non lo è;
3. un'istruzione del *percorso sequenziale*: è utile se il salto non viene preso, e

deve essere innocua se lo è.

Se non trova nulla, il compilatore inserisce una `nop`.

Il salto ritardato funziona bene quando la penalità è di un ciclo. Ma l'ISA vive più a lungo delle sue realizzazioni: quando le pipeline si sono allungate e la penalità è salita a dieci o venti cicli, lo slot unico non nascondeva più quasi nulla, ma tutti i processori compatibili dovevano continuare a rispettarlo. RISC-V ha imparato la lezione: la specifica stabilisce espressamente che le sue istruzioni di salto *non* hanno delay slot visibili all'architettura.

b.5 Indovinare: la previsione statica

La nostra pipeline, in realtà, formula già un'ipotesi: preleva le istruzioni successive come se il salto non venisse preso. Se l'ipotesi è giusta, non perde nulla. Questa è una **previsione** *statica*: «statica» perché non cambia durante l'esecuzione. Le previsioni statiche possibili sono poche.

- *Sempre non preso*, la nostra: non richiede hardware aggiuntivo, e sbaglia ogni volta che il salto viene preso.
- *Sempre preso*: richiede di conoscere la destinazione già in IF, e vedremo fra poco come.
- *Indietro preso, avanti non preso*: un salto all'indietro (immediato negativo) è quasi sempre la chiusura di un ciclo, e viene preso a ogni iterazione tranne l'ultima; un salto in avanti spesso salta un caso raro. Il segno dell'immediato è il bit 31 dell'istruzione, disponibile appena l'istruzione è stata prelevata.

La specifica di RISC-V consiglia proprio questa terza regola: il software deve supporre che i salti all'indietro siano previsti presi e quelli in avanti non presi, almeno la prima volta che si incontrano. Di conseguenza il compilatore organizza il codice in modo che il percorso più frequente sia quello sequenziale. Confrontiamo due modi di scrivere un ciclo:

```
ciclo:  bge  t0, t1, fuori
        # corpo del ciclo
        addi t0, t0, 1
        j    ciclo
fuori:
```

```
        bge  t0, t1, fuori
ciclo:  # corpo del ciclo
        addi t0, t0, 1
        blt  t0, t1, ciclo
fuori:
```

A sinistra il controllo è in testa, come in un `while`: il salto in avanti `bge` è previsto non preso e sbaglia solo all'uscita, ma a ogni iterazione si paga anche la `j` all'indietro, che salta sempre. A destra il controllo è in fondo,

come in un `do … while`, con un test iniziale per il caso in cui il ciclo non va eseguito nemmeno una volta: a ogni iterazione c'è un solo salto, all'indietro, previsto preso. I compilatori trasformano i cicli nella seconda forma proprio per questo.

b.6 Imparare dal passato: la previsione dinamica

Una previsione statica tratta tutti i salti allo stesso modo. Ma ogni salto ha un comportamento caratteristico: quello che chiude un ciclo viene quasi sempre preso, quello che controlla un errore quasi mai. La **previsione dinamica** osserva il comportamento di ogni salto durante l'esecuzione e prevede che continui a comportarsi così.

La tabella della storia dei salti

Lo strumento più semplice è una piccola memoria, la *tabella della storia dei salti* (in inglese *branch history table*, BHT), consultata in IF con alcuni bit bassi del PC. Ogni riga memorizza l'esito più recente del salto che si trova a quell'indirizzo. Se la riga indica «preso», la fase IF preleva la destinazione; altrimenti l'istruzione successiva. Quando il salto viene finalmente deciso, se la previsione era sbagliata si annullano le istruzioni prelevate e si corregge la riga.

Due osservazioni. Poiché si usano solo alcuni bit del PC, due salti diversi possono condividere la stessa riga e interferire fra loro; con qualche migliaio di righe accade di rado. E la tabella non deve essere sempre corretta: una previsione sbagliata costa qualche ciclo, ma non produce mai un risultato errato, perché il salto viene comunque controllato.

Un bit o due

Con un bit per riga la tabella ricorda soltanto l'ultimo esito. Prendiamo un ciclo di dieci iterazioni, eseguito molte volte dentro un ciclo più esterno, che si chiude con un salto all'indietro. Il salto viene preso nove volte e non preso una, all'uscita. Il predittore a un bit sbaglia due volte per ogni esecuzione del ciclo: all'uscita, quando prevede «preso» come la volta prima, e alla prima iterazione dell'esecuzione successiva, quando prevede «non preso» perché l'ultima volta il ciclo era uscito. Otto previsioni giuste su dieci, l'80%.

Il secondo errore è evitabile: un singolo esito anomalo ha modificato la previsione. Con due bit per riga si può richiedere che la previsione cambi solo dopo *due* errori consecutivi. I due bit formano un contatore che sale con i salti presi e scende con quelli non presi, senza mai uscire dall'intervallo da

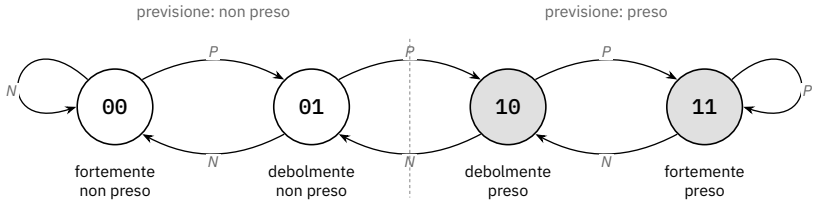


Figura b.3 Il predittore a due bit. P è un salto preso, N un salto non preso. Negli stati grigi la previsione è «preso». Per passare da una previsione all'altra partendo da uno stato «forte» servono due errori di fila.

00_2 a 11_2; la previsione è «preso» per 10_2 e 11_2. È una macchina a stati con quattro stati, gli unici della parte III (figura b.3).

Rifacciamo il conto sul ciclo di dieci iterazioni. A regime il contatore è in 11_2. All'uscita il salto non viene preso: errore, e il contatore scende a 10_2, che prevede ancora «preso». Alla prima iterazione dell'esecuzione successiva il salto viene preso, come previsto, e il contatore torna a 11_2. Un solo errore su dieci: il 90%.

Sapere dove saltare

La tabella indica *se* saltare, ma non *dove*: la destinazione si conosce solo in ID, quando l'istruzione è stata decodificata e l'immediato sommato al PC. Per prelevare la destinazione già nel ciclo successivo, la fase IF ha bisogno di un'altra memoria, il **buffer delle destinazioni** (*branch target buffer*, BTB). È una piccola tabella, consultata con il PC come la BHT, che a ogni istruzione di salto già incontrata associa la destinazione calcolata l'ultima volta. Se il PC si trova nel BTB e la previsione è «preso», il PC successivo è la destinazione, selezionata da un ulteriore ingresso del multiplexer del PC; se la previsione era giusta il salto non costa nulla.

Nei processori reali i predittori combinano la storia di ogni salto con quella degli ultimi salti eseguiti, perché l'esito di un salto dipende spesso da quello dei precedenti, e sbagliano pochi salti su cento. È necessario: con pipeline da quindici o venti fasi, ogni errore costa altrettanti cicli.

1977 Due bit per indovinare

Il contatore a due bit fu introdotto nel 1977 da Tom McWilliams e Curt Widdoes per S-1, un supercalcolatore progettato per il laboratorio nazionale di Lawrence Livermore, e reinventato in modo indipendente nel 1979 da Jim Smith alla Control Data. L'articolo che Smith pubblicò nel 1981, *A Study of Branch Prediction Strategies*, confrontava sistematicamente molte strategie, statiche e dinamiche, misurandole su programmi reali, ed è diventato uno dei testi più citati della materia. Per anni il contatore a due bit è stato il predittore dinamico per eccellenza: il Pentium originale di Intel ne usava una variante, con un difetto di realizzazione che lo rendeva meno preciso del previsto. I predittori moderni sono molto più sofisticati, ma quasi tutti hanno ancora, da qualche parte, piccoli contatori saturanti come quello della figura b.3.

b.7 Le eccezioni

C'è un altro modo in cui il flusso di un programma può cambiare direzione: quando si verifica un evento che il programma non può o non deve gestire da solo. RISC-V distingue due casi.

- Un'**eccezione** è causata da un'istruzione: è *sincrona*, perché eseguendo di nuovo lo stesso programma con gli stessi dati, nelle stesse condizioni, si ripresenta alla stessa istruzione. Esempi: un'istruzione illegale, una load a un indirizzo non allineato o non accessibile, una `ecall`.
- Un'**interruzione** (*interrupt*) è causata da un evento esterno al programma, come un timer che scade o un dispositivo che ha terminato un'operazione: è *asincrona*, perché può arrivare in qualunque momento, mentre si esegue qualunque istruzione.

In entrambi i casi il processore deve interrompere il programma, eseguire il codice di un *gestore* del sistema operativo e, se possibile, riprendere il programma da dove l'aveva lasciato. Rispetto ai testi che seguono MIPS, conviene notare che in RISC-V l'overflow aritmetico *non* genera eccezioni (stato 0x4, § 4.6), e nemmeno la divisione per zero.

Le operazioni dell'hardware

Quando un'istruzione provoca un'eccezione, il processore, a livello macchina, esegue in un solo passo le operazioni seguenti:

1. scrive nel registro `mepc` (*machine exception PC*) l'indirizzo dell'istruzione che ha provocato l'eccezione;
2. scrive nel registro `mcause` un numero che codifica la causa dell'evento (tabella b.2); il bit più significativo vale 1 per le interruzioni e 0 per le eccezioni;

Tabella b.2 Alcune cause di eccezione in RISC-V, con il codice scritto in `mcause` e la fase della nostra pipeline in cui vengono rilevate.

Codice	Causa	Fase
0	destinazione di un salto non allineata	ID
1, 12	errore di accesso o di pagina nel prelievo dell'istruzione	IF
2	istruzione illegale	ID
3	`ebreak`, punto di interruzione per il debugger	ID
4, 6	load o store a un indirizzo non allineato	MEM
5, 7, 13, 15	errore di accesso o di pagina in una load o in una store	MEM
8, 11	`ecall` dal livello utente o dal livello macchina	ID

3. scrive in `mtval` un'informazione utile al gestore, come l'indirizzo che ha causato l'errore;
4. disattiva le interruzioni e passa al livello di privilegio *machine*;
5. carica nel PC l'indirizzo del gestore, contenuto nel registro `mtvec` (*machine trap vector*).

Questi registri non fanno parte dei 32 registri interi: sono *registri di controllo e di stato* (CSR), che si leggono e si scrivono con istruzioni apposite dell'estensione Zicsr, come `csrrw` e `csrrs`, e con le loro pseudoistruzioni `csrw` e `csrr`. Il gestore legge `mcause`, svolge le operazioni necessarie e torna al programma con l'istruzione `mret`, che carica nel PC il valore di `mepc` e ripristina il livello di privilegio e le interruzioni di prima.

Un dettaglio pratico: per una `ecall`, `mepc` contiene l'indirizzo della `ecall` stessa. Se il gestore tornasse con `mret` senza modificarlo, eseguirebbe di nuovo la `ecall`, all'infinito. Per questo, prima di tornare, il gestore somma 4 a `mepc`. Per un'istruzione che ha trovato la pagina di memoria mancante, invece, è proprio quello che si vuole: il gestore carica la pagina, e l'istruzione viene rieseguita, questa volta con successo (stato 0xe).

Eccezioni nella pipeline

Nella pipeline le eccezioni sono conflitti sul controllo, e si gestiscono con lo stesso meccanismo dei salti: al multiplexer del PC si aggiunge un ingresso con il contenuto di `mtvec`, e le istruzioni che non devono completarsi vengono annullate. La difficoltà è decidere *quali*.

Il gestore, e prima ancora il programmatore, si aspetta un'eccezione **precisa**: tutte le istruzioni precedenti a quella che l'ha provocata sono state completate, e né lei né quelle successive hanno cambiato lo stato. Solo così

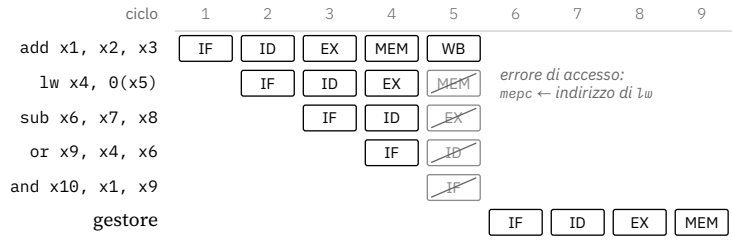


Figura b.4 Un'eccezione precisa. Al ciclo 5 la `lw` scopre in MEM un errore di accesso: la `add`, più vecchia, termina; la `lw` e le istruzioni più giovani vengono annullate, e al ciclo 6 il PC vale `mtvec`.

`mepc` ha senso, e il programma può ripartire da lì come se nulla fosse successo. La figura b.4 mostra una load che scopre in MEM un errore di accesso: `add`, che la precede, completa la sua fase WB; la load e le tre istruzioni che la seguono vengono annullate, e al ciclo successivo IF preleva la prima istruzione del gestore.

Il problema si complica perché fasi diverse scoprono eccezioni diverse (tabella b.2), e nello stesso ciclo più istruzioni possono avere un'eccezione. Immaginiamo che nello stesso ciclo l'istruzione in MEM trovi un errore di pagina, quella in ID sia illegale e quella in IF non si possa prelevare. Quale va gestita? Quella *più vecchia*, cioè la più avanzata nella pipeline, perché in un'esecuzione sequenziale sarebbe la prima a fallire: le altre, a rigore, non sarebbero nemmeno cominciate. Se dopo il gestore il programma riprende, le istruzioni più giovani verranno prelevate di nuovo, e le loro eccezioni, se ci sono ancora, si ripresenteranno al momento giusto.

Per ottenere questo ordine, la soluzione più usata è non gestire l'eccezione appena viene scoperta, ma annotarla in un campo del registro di pipeline, che viaggia con l'istruzione come i segnali di controllo. Solo in un punto preciso il processore verifica se l'istruzione ha un'eccezione annotata: nella nostra pipeline, la fase MEM, dove si scoprono anche gli errori negli accessi alla memoria (e dove una store che ha un'eccezione non deve scrivere). Le istruzioni vi transitano una alla volta e nell'ordine del programma, quindi la prima eccezione gestita è sempre quella dell'istruzione più vecchia. Le interruzioni, che non appartengono a nessuna istruzione, si agganciano a un'istruzione scelta dal processore, che funge da punto di ripresa.

> **Il costo di un'eccezione** Un salto mal previsto costa qualche ciclo. Un'eccezione costa molto di più: oltre ad annullare la pipeline, il gestore deve salvare in memoria i registri che userà, individuare la causa, svolgere il proprio compito

e ripristinare tutto. Una `ecall` che chiede al sistema operativo di scrivere un carattere può costare centinaia o migliaia di istruzioni. Per questo le librerie raccolgono molti caratteri in un buffer e li consegnano al sistema operativo con una sola chiamata.

Condizione di transizione

uscita verso **`0xc`**

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Perché nella pipeline dello stato 0x9 un salto preso costa tre cicli? ↺ § b.1

2 Con il 20% di salti, di cui tre quarti presi, quanto vale il CPI se ogni salto preso costa tre cicli? E se ne costa uno? ↺ § b.1

3 Come si annulla un'istruzione che è già entrata nella pipeline? Perché in RISC-V non basta azzerare IF/ID? ↺ § b.2

4 Che cosa serve per decidere i salti in ID, e quali nuovi problemi crea? ↺ § b.3

5 `lw t0, 0(a0)` è seguita subito da `beq t0, zero, fine`. Con la decisione in ID, quanti stalli servono? ↺ § b.3

6 Che cos'è un salto ritardato, e perché RISC-V non lo prevede? ↺ § b.4

7 Un ciclo di 4 iterazioni viene eseguito molte volte. Quante previsioni giuste fa, per ogni esecuzione, un predittore a un bit? E uno a due bit? ↺ § b.6

8 A che cosa serve il BTB, se c'è già la BHT? ↺ § b.6

9 Che cosa contengono `mepc`, `mcause` e `mtvec`? Perché il gestore di una `ecall` somma 4 a `mepc`? ↺ § b.7

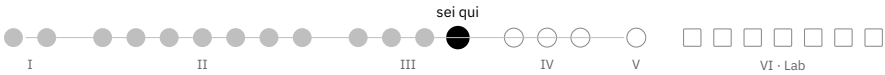
10 Che cos'è un'eccezione precisa? Se nello stesso ciclo due istruzioni hanno un'eccezione, quale viene gestita per prima? ↺ § b.7

STATO

0xC

1	1	0	0
b3	b2	b1	b0

Oltre la pipeline: più istruzioni per ciclo



→ INGRESSI

- La pipeline e l'accelerazione (stato 0x9, § 9.2).
- Conflitti sui dati, propagazione e riordino del codice (stato 0xa).
- Previsione dei salti ed eccezioni precise (stato 0xb).

USCITE →

- Spiegare perché la corsa alla frequenza si è fermata.
- Distinguere emissione multipla statica (VLIW) e dinamica (superscalare).
- Pianificare l'ordine delle istruzioni di un ciclo per un processore a doppia emissione, anche srotolandolo.
- Riconoscere le dipendenze WAR e WAW e spiegare come la ridenominazione dei registri le elimini.
- Descrivere l'esecuzione fuori ordine (*out-of-order*), il ritiro in ordine e il multithreading, con l'esempio di un processore reale.

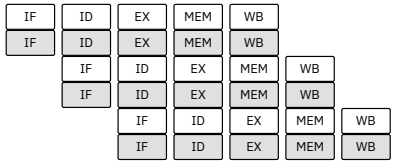


Tabella c.1 L'evoluzione dei processori con pipeline, secondo i dati di Patterson e Hennessy. L'anno del Pentium Pro, talvolta riportato come 1997, è stato corretto: il processore uscì nel novembre 1995.

Processore	Anno	Clock	Fasi	Emissione	Core	Potenza
Intel 486	1989	25 MHz	5	1	1	5 W
Intel Pentium	1993	66 MHz	5	2	1	10 W
Intel Pentium Pro	1995	200 MHz	10	3	1	29 W
Pentium 4 Willamette	2001	2000 MHz	22	3	1	75 W
Pentium 4 Prescott	2004	3600 MHz	31	3	1	103 W
Intel Core	2006	2930 MHz	14	4	2	75 W
Sun UltraSPARC III	2003	1950 MHz	14	4	1	90 W
Sun T1 (Niagara)	2005	1200 MHz	6	1	8	70 W

La pipeline ci ha portati a un'istruzione per ciclo, con un ciclo più corto. Per andare oltre ci sono due strade: accorciare ancora il ciclo, o completare più di un'istruzione per ciclo. L'industria ha percorso la prima finché ha potuto, poi ha dovuto imboccare la seconda. Questo stato racconta entrambe, fino a un processore reale, e si chiude con una curiosità: il processore più semplice che si possa immaginare.

c.1 La corsa alla frequenza

Suddividere l'istruzione in fasi più numerose e più brevi accorcia il ciclo: dieci fasi da 100 ps invece di cinque da 200. Per vent'anni i produttori hanno fatto proprio questo, aiutati dai transistor sempre più piccoli e veloci. La tabella c.1 mostra dove si è arrivati.

In quindici anni il clock è salito di oltre cento volte e le fasi sono passate da 5 a 31. Poi, di colpo, la tendenza si inverte: il clock scende, le fasi diminuiscono, compaiono più core. Due fattori hanno fermato la corsa.

- *Le pipeline profonde costano.* Con 31 fasi un salto previsto male spreca circa trenta cicli di lavoro, e i conflitti sui dati richiedono stalli più lunghi, perché fra il momento in cui un valore serve e quello in cui è pronto ci sono più fasi. Inoltre ogni registro di pipeline aggiunge un ritardo fisso: quando le fasi diventano molto corte, quel ritardo diventa una parte importante del ciclo.
- *La potenza.* La potenza dissipata da un circuito digitale quando commuta è circa

$$P \approx \alpha C V^2 f,$$

dove C è la capacità dei transistor, V la tensione di alimentazione, f la

frequenza e α la frazione di circuito che commuta. Per decenni, rimpicciolendo i transistor, si è potuta abbassare la tensione e compensare l'aumento di frequenza. Verso il 2005 la tensione non è più potuta scendere molto, e ogni aumento di frequenza si è tradotto in calore. Un processore da 100 W concentra quella potenza in un paio di centimetri quadrati: per unità di superficie, più di una piastra elettrica da cucina. Raffreddarlo con l'aria diventa difficile e costoso.

Questo limite ha un nome: il **muro della potenza**. Il Niagara della tabella mostra la strada presa dopo: un clock più basso, una pipeline corta e semplice, e otto core. Ai processori con più core sarà dedicato lo stato 0xf; qui ci occupiamo dell'altra strada, che opera all'interno di un singolo core: eseguire più istruzioni nello stesso ciclo.

2004 Il processore da 7 GHz che non nacque mai

Il successore del Pentium 4 Prescott si chiamava Tejas. Doveva superare i 7 GHz con una pipeline di 40 o 50 fasi, e Intel parlava apertamente di arrivare a 10 GHz entro il 2011. Il 7 maggio 2004 Intel annunciò che Tejas e il suo fratello per i server, Jayhawk, erano cancellati. Le cause furono il calore e il consumo: per salire ancora di frequenza i processori avrebbero richiesto sistemi di raffreddamento fuori dalla portata di un normale calcolatore. Intel ripiegò sul Pentium M, un discendente del vecchio Pentium Pro pensato per i portatili, con una pipeline più corta e un'attenzione maniacale ai consumi. Da lì nacquero i processori Core, con due core per chip. Da allora la frequenza dei processori per calcolatori personali è cresciuta pochissimo, e l'aumento delle prestazioni è venuto da altre soluzioni.

---

c.2 Più istruzioni per ciclo

Un processore che preleva, decodifica ed esegue più istruzioni nello stesso ciclo si dice a **emissione multipla** (in inglese *multiple issue*); il numero massimo di istruzioni per ciclo è la sua *larghezza di emissione*. Con due istruzioni per ciclo il CPI ideale scende a 0,5. Per i processori a emissione multipla si preferisce spesso il numero inverso, le istruzioni per ciclo o **IPC**: un processore a quattro vie ha un IPC massimo di 4.

Leggere più istruzioni non è difficile: basta prelevare 64 o 128 bit alla volta dalla memoria istruzioni. Il difficile è *eseguirle* insieme senza cambiare il risultato. Tutti i problemi della pipeline si aggravano: due istruzioni prelevate insieme possono dipendere l'una dall'altra, e se la prima è un salto preso la seconda non andava nemmeno prelevata. Occorre decidere quali istruzioni possono essere avviate insieme: può farlo il compilatore, prima dell'esecuzione, oppure l'hardware, durante: sono le due grandi famiglie dei

processori a emissione multipla.

c.3 Emissione statica: il compilatore decide

Nell'emissione multipla **statica** il compilatore raggruppa le istruzioni in *pacchetti* che il processore esegue in parallelo senza controlli. Ogni pacchetto è di fatto una sola istruzione molto lunga, da cui il nome *VLIW*, da *very long instruction word*. Prendiamo un processore VLIW a due vie derivato dalla nostra pipeline: ogni pacchetto da 64 bit contiene un'istruzione aritmetica, logica o di salto e un'istruzione di accesso alla memoria. Per eseguirle insieme servono:

- un banco dei registri con quattro porte di lettura e due di scrittura, perché le due istruzioni leggono e scrivono insieme;
- un sommatore in più, che calcola l'indirizzo della load o della store mentre l'ALU opera sull'altra istruzione;
- regole precise per il compilatore: le due istruzioni di un pacchetto devono essere indipendenti, e il risultato di una load si può usare solo due pacchetti dopo. Se il compilatore non trova nulla da collocare in una delle due metà, inserisce una `nop`.

Un esempio

Questo ciclo somma la costante in `s2` a ogni elemento di un vettore, partendo dall'ultimo; `s1` punta all'elemento corrente, e il ciclo finisce quando `s1` arriva a zero.

```
ciclo:  lw   t0, 0(s1)         # t0 = elemento
        add  t0, t0, s2        # somma la costante
        sw   t0, 0(s1)         # rimette l'elemento a posto
        addi s1, s1, -4        # passa all'elemento precedente
        bne  s1, zero, ciclo
```

La tabella c.2 mostra il miglior raggruppamento possibile. L'`add` deve attendere la load per due pacchetti; la `addi` può essere anticipata al secondo pacchetto, ma allora la `sw`, che viene dopo, trova `s1` già diminuito di 4 e deve usare lo spiazzamento 4 invece di 0. Il compilatore è in grado di eseguire anche queste correzioni. Risultato: cinque istruzioni in quattro cicli, un IPC di 1,25, lontano dal 2 teorico.

Srotolare i cicli

Il problema è che in un'iterazione ci sono poche istruzioni, e quasi tutte dipendono dalla precedente. La soluzione è lo **srotolamento dei cicli** (*loop*

Tabella c.2 Il ciclo raggruppato in pacchetti da due istruzioni.

	Aritmetica o salto	Memoria	Ciclo
`ciclo:`	`nop`	`lw t0, 0(s1)`	1
	`addi s1, s1, -4`	`nop`	2
	`add t0, t0, s2`	`nop`	3
	`bne s1, zero, ciclo`	`sw t0, 4(s1)`	4

Tabella c.3 Il ciclo srotolato quattro volte: 14 istruzioni in 8 cicli.

	Aritmetica o salto	Memoria	Ciclo
`ciclo:`	`addi s1, s1, -16`	`lw t0, 0(s1)`	1
	`nop`	`lw t1, 12(s1)`	2
	`add t0, t0, s2`	`lw t2, 8(s1)`	3
	`add t1, t1, s2`	`lw t3, 4(s1)`	4
	`add t2, t2, s2`	`sw t0, 16(s1)`	5
	`add t3, t3, s2`	`sw t1, 12(s1)`	6
	`nop`	`sw t2, 8(s1)`	7
	`bne s1, zero, ciclo`	`sw t3, 4(s1)`	8

unrolling): si copia il corpo del ciclo più volte, per esempio quattro, e si fa avanzare il puntatore di quattro elementi per iterazione. Le quattro copie operano su elementi diversi e sono indipendenti fra loro, purché usino registri diversi: il compilatore usa `t0`, `t1`, `t2` e `t3` al posto del solo `t0`. Si eliminano anche tre `addi` e tre `bne` su quattro.

Con lo srotolamento (tabella c.3) quattro elementi richiedono otto cicli invece di sedici: 14 istruzioni in 8 cicli, un IPC di 1,75, un CPI di circa 0,57. Si noti la prima riga: la prima load usa ancora il vecchio valore di `s1`, perché nel pacchetto le letture avvengono prima della scrittura della `addi`; tutti gli spiazzamenti successivi tengono conto della diminuzione di 16. Il prezzo è un codice più lungo e più registri occupati, e la necessità di gestire a parte i vettori la cui lunghezza non è un multiplo di quattro.

Lo srotolamento serve anche senza emissione multipla: riduce il numero di salti, e quindi le occasioni di sbagliare una previsione. I compilatori sanno farlo da soli: GCC, per esempio, srotola i cicli con l'opzione `-funroll-loops`, e con `-O3` srotola completamente i cicli brevi e ne trasforma altri in operazioni vettoriali, che operano su più elementi alla volta. In uno dei laboratori ne misureremo l'effetto.

Pregi e limiti del VLIW

Il VLIW mantiene l'hardware semplice, e funziona benissimo quando il lavoro è regolare e prevedibile: elaborazione dei segnali, grafica, calcolo su grandi vettori, dove la stessa operazione si ripete su moltissimi dati indipendenti. Per questo i processori di segnale digitale e molti acceleratori lo usano ancora. Nei programmi generici funziona meno bene: il compilatore non sa quanto durerà una load (può trovare il dato subito o attendere la memoria centrale per centinaia di cicli), non sa quali salti verranno presi, e un programma compilato per un VLIW a quattro vie va ricompilato per uno a sei. Intel e HP hanno tentato questa strada con l'Itanium, presentato nel 2001: un'architettura ambiziosa che non riuscì mai a imporsi fuori da una nicchia di server, e che Intel ha smesso di produrre nel 2021.

c.4 Emissione dinamica: l'hardware decide

Nei processori **superscalari** è l'hardware a decidere, durante l'esecuzione, quali istruzioni possono partire insieme. Il programma è quello di sempre, e lo stesso codice viene eseguito, più o meno velocemente, su processori di larghezza diversa. Un superscalare semplice esamina due istruzioni consecutive e le emette insieme se sono indipendenti, altrimenti emette solo la prima. I superscalari moderni vanno molto oltre, ed eseguono le istruzioni *fuori ordine* (*out-of-order*).

Esecuzione fuori ordine

Consideriamo questo frammento:

```
        lw   t0, 0(a0)        # dato lontano: la memoria risponde tardi
        add  t1, t0, t2       # deve aspettare la load
        sub  t3, t4, t5       # indipendente da tutto
        and  t6, t4, t2       # indipendente da tutto
```

In una pipeline con esecuzione in ordine, se la load deve attendere a lungo la memoria, l'`add` si ferma e con essa tutte le istruzioni successive, anche `sub` e `and`, che non hanno bisogno di nulla. Un processore con esecuzione **fuori ordine** le esegue subito, mentre `add` attende. Il processore si organizza in tre parti.

1. *Uno stadio frontale in ordine* (*front-end*). Le istruzioni vengono prelevate, decodificate e preparate nell'ordine del programma, con l'aiuto della previsione dei salti, e depositate in code di attesa: le *stazioni di prenotazione* (*reservation stations*). Possono essere *distribuite*, una davanti a ogni gruppo

di unità di esecuzione, oppure riunite in un'unica *stazione di prenotazione unificata* (*unified reservation station*), detta anche coda di scheduling.

2. *Un motore di esecuzione fuori ordine.* Un'istruzione lascia la coda e viene inviata a un'unità di esecuzione appena i suoi operandi sono pronti, a prescindere dalla sua posizione nel programma. I risultati vengono inoltrati direttamente alle istruzioni che li attendono.
3. *Una fase di ritiro in ordine.* I risultati non vengono scritti subito nello stato visibile al programmatore. Restano in un *buffer di riordino*, e vengono resi definitivi, cioè *ritirati*, nell'ordine originale.

Il ritiro in ordine è ciò che preserva la semantica di von Neumann. Se una load provoca un'eccezione, le istruzioni successive eventualmente già eseguite non sono ancora state ritirate, e si possono scartare: l'eccezione resta precisa. Lo stesso vale per i salti: le istruzioni prelevate dopo un salto previsto vengono eseguite in modo **speculativo**, e se la previsione era sbagliata si scartano prima del ritiro. Il processore può così procedere di decine o centinaia di istruzioni in avanti, scommettendo sulle previsioni.

Nuovi conflitti: WAR e WAW

L'esecuzione fuori ordine trasforma in conflitti due tipi di dipendenza che nella pipeline con esecuzione in ordine erano innocui (stato 0xa, § a.1). Guardiamo queste tre istruzioni:

```
mul  x3, x1, x5        # 1: lenta
addi x4, x3, 1         # 2: legge x3 (RAW con la 1)
addi x3, x5, 1         # 3: scrive x3
```

- Fra 1 e 2 c'è la solita dipendenza *RAW*, lettura dopo scrittura: la 2 deve attendere il risultato della 1.
- Fra 2 e 3 c'è un'**antidipendenza**, o dipendenza *WAR*, scrittura dopo lettura: la 3 non deve scrivere x3 prima che la 2 l'abbia letto. Ma la 3 è pronta subito, e un processore con esecuzione fuori ordine tenderebbe a eseguirla prima.
- Fra 1 e 3 c'è una **dipendenza di uscita**, o dipendenza *WAW*, scrittura dopo scrittura: alla fine x3 deve contenere il valore della 3, non quello della 1, che è lenta e potrebbe finire dopo.

Le dipendenze RAW sono *vere*: il valore si trasmette effettivamente da un'istruzione all'altra. Le WAR e le WAW, invece, sono dipendenze di *nome*: la 3 non usa nulla della 1 e della 2, scrive soltanto, per coincidenza, nello stesso registro, perché i registri sono soltanto 32 e il compilatore li riusa. Sono, in fondo, conflitti strutturali sui registri, e come tutti i conflitti strutturali si

risolvono aggiungendo risorse.

La ridenominazione dei registri

Non è possibile aggiungere registri all'ISA: i loro numeri hanno 5 bit nella codifica, e cambiare l'ISA significherebbe ricompilare tutto il software esistente. Nulla impedisce, però, di aggiungerne *all'interno* del processore. Con la **ridenominazione dei registri** (*register renaming*) il processore possiede molti più registri fisici dei 32 architetturali, e una tabella che indica, momento per momento, in quale registro fisico si trova il valore attuale di ogni registro architetturale. Ogni istruzione che scrive un registro riceve un registro fisico nuovo, libero:

```
mul  p10, p1, p5       # x3 -> p10
addi p11, p10, 1       # x4 -> p11, legge x3 = p10
addi p12, p5, 1        # x3 -> p12
```

Ora la 3 scrive `p12`, e può farlo in qualsiasi momento: la 2 legge `p10`, e alla fine la tabella indica che `x3` è `p12`. Le dipendenze WAR e WAW sono sparite; resta solo quella vera, fra `p10` e la 2. Il registro fisico che conteneva il valore precedente di `x3` torna libero quando viene ritirata l'istruzione successiva che scrive `x3`, qui la `mul`: da quel momento nessuna istruzione può più averne bisogno. Allo stesso modo `p10` si libera quando viene ritirata la 3. La stessa idea, applicata dal compilatore con i registri `t0`–`t3`, rendeva possibile lo srotolamento della tabella c.3.

1967 L'idea di Tomasulo

Nel 1967 Robert Tomasulo, ingegnere IBM, pubblicò il progetto dell'unità in virgola mobile del System/360 Model 91. Le istruzioni attendevano i loro operandi in *stazioni di prenotazione* distribuite, tre davanti al sommatore e due davanti al moltiplicatore, e venivano avviate appena li ricevevano; un bus comune annunciava ogni risultato a tutte le stazioni in attesa. Le stazioni fungevano anche da registri nascosti, e così il Model 91 eliminava i conflitti WAR e WAW: era, in sostanza, la ridenominazione dei registri. Mancava però il ritiro in ordine, e quando un'istruzione provocava un'eccezione il processore non sempre era in grado di individuare quale: le sue interruzioni erano *imprecise*. L'algoritmo restò quasi inutilizzato fuori da IBM per vent'anni, finché negli anni Novanta, completato con il buffer di riordino e unito alla previsione dei salti, divenne il cuore di quasi tutti i processori ad alte prestazioni.

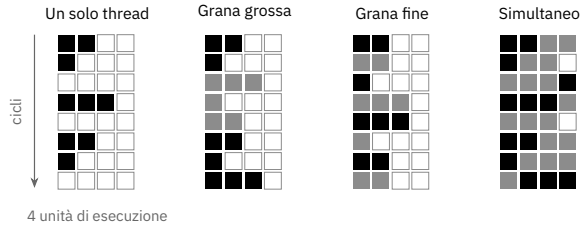


Figura c.1 Come un processore superscalare a quattro unità usa i suoi cicli. Ogni riga è un ciclo, ogni casella un'unità: in nero le istruzioni di un thread, in grigio quelle di un secondo thread, in bianco le unità inutilizzate.

c.5 Il multithreading

Anche il miglior processore con esecuzione fuori ordine lascia spesso unità inattive: un programma tipico non ha abbastanza istruzioni indipendenti per riempire quattro o sei unità a ogni ciclo, e quando una load attende la memoria centrale per centinaia di cicli, il lavoro indipendente finisce presto. L'idea del **multithreading** è occupare i cicli inutilizzati con le istruzioni di un altro flusso di esecuzione, un altro *thread*.

Un thread è un flusso di istruzioni con il suo PC e i suoi registri. I thread di uno stesso programma condividono la memoria; i processi di programmi diversi no. Normalmente è il sistema operativo a passare da un thread all'altro, salvando in memoria tutto lo stato del processore: un'operazione costosa, che non si può eseguire a ogni stallo. Un processore con multithreading hardware mantiene invece al proprio interno lo stato di più thread, e passa dall'uno all'altro senza costo. Le varianti sono tre (figura c.1).

- *A grana grossa*: si cambia thread solo quando quello attivo si blocca a lungo, per esempio su un accesso alla memoria centrale.
- *A grana fine*: si alterna un thread diverso a ogni ciclo, così gli stalli di un thread sono coperti dagli altri.
- *Simultaneo* (SMT): in uno stesso ciclo le unità di un processore superscalare eseguono istruzioni di thread diversi. È la variante dei processori moderni, che Intel chiama *Hyper-Threading*.

Il multithreading non accelera il singolo thread, anzi può rallentarlo un po', perché le risorse si dividono. Aumenta però il lavoro complessivo svolto dal processore, con poco hardware in più: un secondo PC, un secondo insieme di registri architetturali e poche altre strutture.

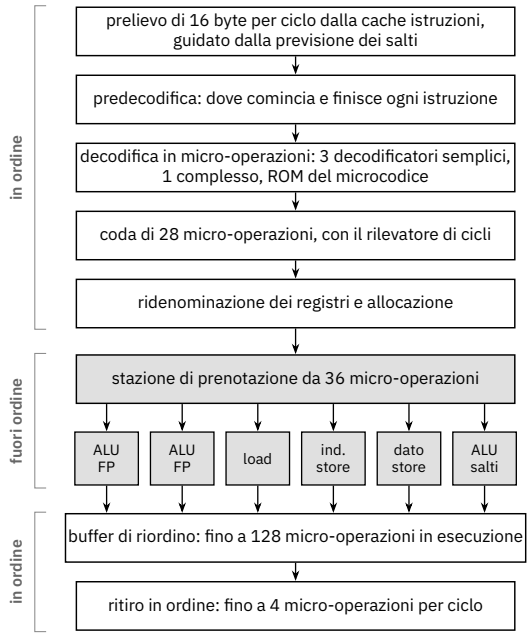


Figura c.2 Il percorso delle istruzioni in un core Nehalem, molto semplificato. Le istruzioni x86 diventano micro-operazioni simili a istruzioni RISC; la parte in grigio le esegue fuori ordine attraverso sei porte, e il buffer di riordino le ritira nell'ordine del programma.

c.6 Un processore vero: Intel Nehalem

Mettiamo insieme tutto con un esempio reale: Nehalem, la microarchitettura Intel del 2008 dei primi Core i7. All'esterno esegue l'ISA x86-64, un'ISA CISC con istruzioni di lunghezza variabile, da 1 a 15 byte, eredità di decenni di compatibilità. All'interno è un processore RISC superscalare con esecuzione fuori ordine. La figura c.2 ne mostra il percorso, e conviene ritrovarvi le idee di questa parte.

- *Lo stadio frontale in ordine* (*front-end*). A ogni ciclo vengono prelevati 16 byte dalla cache delle istruzioni. Poiché le istruzioni x86 hanno lunghezza variabile, una fase apposita deve prima determinare dove comincia ognuna: è il prezzo della CISC, che RISC-V, con le sue istruzioni da 32 bit, non paga. Quattro decodificatori, tre per le istruzioni semplici e uno per quelle complesse, traducono le istruzioni in *micro-operazioni*, istruzioni semplici in stile RISC; le istruzioni più complesse sono tradotte da una ROM di microcodice, come nel controllo microprogrammato dello stato 0x8. Le micro-operazioni sono accodate in una coda da 28 posti; se un

piccolo ciclo vi è contenuto interamente, un *rilevatore di cicli* lo fa ripartire direttamente dalla coda, spegnendo le fasi precedenti e risparmiando energia. L'ultima fase eseguita in ordine è la ridenominazione, che assegna i registri fisici: x86-64 ha solo 16 registri interi architetturali, e senza ridenominazione le dipendenze di nome sarebbero ovunque.

- *Il motore di esecuzione fuori ordine.* Le micro-operazioni attendono in una stazione di prenotazione unificata da 36 posti, e fino a sei per ciclo vengono inviate a sei porte di esecuzione: tre per i calcoli, tre per la memoria. Fino a 128 micro-operazioni possono trovarsi contemporaneamente in esecuzione nella pipeline (*in-flight*).
- *La fase di ritiro in ordine.* Il buffer di riordino ritira fino a quattro micro-operazioni per ciclo nell'ordine del programma.

La pipeline completa è lunga circa 16 fasi: tanti sono i cicli che si perdono, nel caso migliore, quando un salto viene previsto male. Le cache di primo livello sono due, una per le istruzioni e una per i dati, come nel modello di Harvard del nostro datapath; le vedremo nello stato 0xd.

Ogni core di Nehalem esegue due thread in modo simultaneo, e il sistema operativo vede due processori logici per core. Le risorse si dividono in tre modi: lo stato dei registri architetturali è *replicato*, uno per thread; il buffer di riordino e i buffer di load e store sono *divisi a metà*; la stazione di prenotazione unificata e le cache sono *condivise* e assegnate dinamicamente al thread che ne ha bisogno. Le unità di esecuzione non dispongono nemmeno dell'informazione sul thread da cui proviene la micro-operazione che stanno eseguendo.

> **Indovinare ha un prezzo** L'esecuzione speculativa sembrava invisibile: le istruzioni eseguite per errore vengono scartate e non cambiano lo stato. Nel gennaio 2018 due gruppi di ricercatori, uno di Google Project Zero e uno guidato da Paul Kocher, mostrarono che non è del tutto vero. Le istruzioni scartate lasciano tracce indirette, per esempio dati caricati nella cache, e misurando i tempi di accesso un programma può dedurre informazioni che non dovrebbe vedere. L'attacco, chiamato *Spectre*, riguardava processori Intel, AMD e ARM, cioè quasi tutti i processori che speculano. Le contromisure, in parte software e in parte nei processori successivi, costano un po' di prestazioni: un promemoria del fatto che ogni accorgimento dell'architettura ha conseguenze anche fuori da essa.

c.7 L'altro estremo: una sola istruzione

Abbiamo visto processori sempre più complessi. Chiudiamo la parte III andando nella direzione opposta: qual è l'ISA più semplice possibile che

sappia ancora calcolare tutto ciò che è calcolabile? Le load e le store non sono indispensabili: servono solo perché esistono i registri, e si può lavorare direttamente in memoria. I salti condizionati, invece, sono indispensabili: senza la possibilità di scegliere in base ai dati, un calcolatore eseguirebbe sempre la stessa sequenza.

Una risposta è un **calcolatore a un'istruzione** (*one instruction set computer*, OISC). Una delle varianti più note ha come unica istruzione *sottrai e salta se negativo*, con tre indirizzi di memoria:

$$\texttt{sbn}\ A, B, C: \quad \mathrm{Mem}[A] \leftarrow \mathrm{Mem}[A] - \mathrm{Mem}[B]; \quad \text{se } \mathrm{Mem}[A] < 0, \text{ salta a } C.$$

Poiché l'istruzione è una sola, non serve un opcode: ogni istruzione è formata solo dai tre indirizzi. Con indirizzi da 32 bit, un'istruzione occupa 96 bit, e il PC avanza di 12 byte alla volta. L'hardware è minuscolo: una memoria, un PC, un sommatore per $PC + 12$, un sottrattore e un multiplexer che seleziona $PC + 12$ o C esaminando il bit del segno.

Tutta la complessità si trasferisce al software. Per azzerare una variabile la si sottrae a se stessa; per copiare A in B, usando una cella di appoggio T, servono quattro istruzioni, tutte con C uguale all'indirizzo dell'istruzione successiva, così che si prosegua in ogni caso:

```
i1:     sbn  T, T, i2          # T = 0
i2:     sbn  T, A, i3          # T = -A
i3:     sbn  B, B, i4          # B = 0
i4:     sbn  B, T, i5          # B = 0 - (-A) = A
i5:     ...                    # si prosegue qui
```

Un salto incondizionato richiede un risultato sicuramente negativo, per esempio sottraendo una cella che vale 1 da una cella appena azzerata. Si può scrivere qualunque programma, ma ogni operazione richiede molte istruzioni. È l'estremo opposto del CISC, e mostra bene il compromesso che sta dietro ogni ISA: un'istruzione in meno nell'hardware è un po' di lavoro in più nel software, e viceversa. RISC-V sta, ragionevolmente, nel mezzo.

Condizione di transizione uscita verso **`0xd`**

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Perché dopo il 2004 la frequenza dei processori ha smesso di crescere? ↺ § c.1

2 Che differenza c'è fra un processore VLIW e uno superscalare? Chi decide quali istruzioni partono insieme? ↺ § c.2

3 Nella tabella c.2, perché la `sw` usa lo spiazzamento 4? ↺ § c.3

4 Quanto vale l'IPC del ciclo srotolato della tabella c.3? Perché servono quattro registri diversi? ↺ § c.3

5 Perché un processore con esecuzione fuori ordine ritira le istruzioni in ordine? ↺ § c.4

6 In `add x1, x2, x3`; `sub x4, x1, x5`; `or x1, x6, x7`, trova le dipendenze RAW, WAR e WAW. ↺ § c.4

7 Come elimina la ridenominazione le dipendenze WAR e WAW? Perché non elimina le RAW? ↺ § c.4

8 Che differenza c'è fra multithreading a grana fine e simultaneo? ↺ § c.5

9 In Nehalem, quali parti operano in ordine e quali fuori ordine? Perché la ridenominazione viene eseguita in ordine? ↺ § c.6

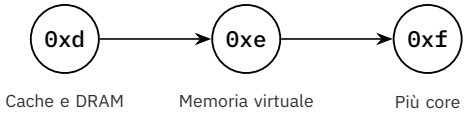
10 Con l'istruzione `sbn`, come si somma A a B? ↺ § c.7

PARTE

IV

La memoria

Il processore della parte III legge e scrive una memoria ideale, che risponde in un ciclo. Ora la guardiamo com'è davvero: lenta, enorme, condivisa da molti programmi e da più core. Vedremo come le cache la fanno sembrare veloce, come la memoria virtuale la divide fra i processi e li protegge l'uno dall'altro, e come più core si mettono d'accordo sui dati che condividono.

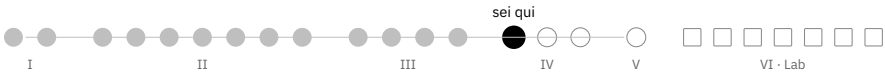


STATO

0xd

1	1	0	1
b3	b2	b1	b0

La cache e la memoria principale



→ INGRESSI

- La fase MEM e il muro della memoria (stato 0x6, § 6.4).
- Gli stalli della pipeline e il CPI (stati 0x9 e 0xa).
- Lo XOR e i numeri binari (stati 0x1 e 0x5).

USCITE →

- Spiegare il principio di località e perché la memoria è organizzata a livelli.
- Scomporre un indirizzo in tag, indice e offset, e simulare a mano una cache diretta o associativa.
- Calcolare il numero effettivo di bit occupati da una cache.
- Distinguere le politiche di scrittura e classificare i miss.
- Calcolare il CPI di un processore con uno o due livelli di cache.
- Spiegare come la DRAM trasferisce un blocco e calcolarne la banda.
- Costruire un codice di Hamming e correggere un bit sbagliato.

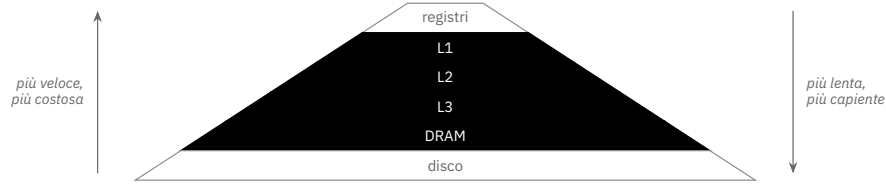


Tabella d.1 I livelli della gerarchia di memoria in un calcolatore di oggi. I valori sono indicativi: un ciclo è un periodo di clock del processore, circa 0,3 ns a 3 GHz.

Livello	Tecnologia	Dimensione	Tempo di accesso	Chi lo gestisce
registri	flip-flop	centinaia di byte	meno di un ciclo	compilatore, programmatore
cache L1	SRAM	32–64 KiB	1–4 cicli	hardware
cache L2	SRAM	256 KiB–2 MiB	circa 10 cicli	hardware
cache L3	SRAM	8–64 MiB	30–50 cicli	hardware
memoria principale	DRAM	8–64 GiB	50–70 ns	hardware e sistema operativo
memoria di massa	SSD, disco	terabyte	da decine di μs a ms	sistema operativo

Nelle parti II e III abbiamo dato per scontato che la memoria risponda in un ciclo, 200 picosecondi. Nello stato 0x6 avevamo già ammesso che è una finzione: la memoria principale risponde in decine di nanosecondi, cioè centinaia di cicli. Se ogni `lw` e ogni prelievo di istruzione costassero tanto, tutto il lavoro svolto per la pipeline sarebbe vanificato. Questo stato racconta come si rende credibile la finzione: con piccole memorie veloci, le cache, che anticipano quali dati serviranno. Poi scende un livello, dentro la DRAM, e si chiude con una domanda che diventa inevitabile quando i bit sono miliardi: che cosa succede se uno di essi cambia spontaneamente?

d.1 Il divario e la gerarchia

Vorremmo una memoria grande, veloce ed economica. Le tecnologie disponibili offrono al massimo due di queste proprietà: la SRAM è veloce ma costosa e poco densa, la DRAM è densa ed economica ma lenta, i dischi sono capientissimi e lentissimi. E il divario fra processore e memoria cresce da decenni. Le prestazioni dei processori sono raddoppiate, per molti anni, ogni anno e mezzo o due; il tempo di accesso della DRAM migliora di pochi punti percentuali l'anno, e per dimezzarlo servono circa dieci anni.

La soluzione è la stessa che usiamo tutti i giorni con i libri: quelli che stiamo leggendo sono sulla scrivania, gli altri sugli scaffali, quelli che non apriamo da anni in cantina. La memoria di un calcolatore è organizzata in una **gerarchia** di livelli: ogni livello è più piccolo, più veloce e più costoso per byte di quello sottostante, e contiene una copia dei dati del livello sottostante che servono in questo momento (tabella d.1). Se l'accorgimento funziona, il processore vede una memoria grande quanto l'ultimo livello e veloce quasi quanto il primo.

I livelli comunicano soltanto con quelli adiacenti: i dati passano dalla

DRAM alla cache L3, da questa alla L2, poi alla L1 e infine ai registri. Fra i registri e la L1 i dati sono trasferiti dal programma, con load e store; fra le cache e la DRAM dall'hardware, in modo trasparente per il programma; fra la DRAM e il disco dal sistema operativo, come vedremo nello stato 0xe.

d.2 La località

Perché una piccola memoria dovrebbe contenere proprio i dati che servono? Perché i programmi non accedono alla memoria a caso. Guardiamo un ciclo che somma gli elementi di un vettore:

```
        li   t0, 0            # somma = 0
        li   t1, 0            # i = 0
ciclo:  slli t2, t1, 2        # t2 = 4*i
        add  t2, a0, t2       # indirizzo di v[i]
        lw   t3, 0(t2)        # v[i]: indirizzi consecutivi
        add  t0, t0, t3
        addi t1, t1, 1
        blt  t1, a1, ciclo    # le stesse sei istruzioni, n volte
```

Le sei istruzioni del ciclo vengono prelevate a ogni iterazione: se l'indirizzo A è stato usato da poco, è probabile che venga usato di nuovo presto. È la **località temporale**. La `lw` legge `v[0]`, poi `v[1]`, `v[2]`..., e le istruzioni stesse sono memorizzate una dopo l'altra: se è stato usato l'indirizzo A, è probabile che presto servano $A + 4$, $A + 8$ e i vicini. È la **località spaziale**.

Una gerarchia sfrutta la località in due modi. Per quella temporale, mantiene vicino al processore i dati usati di recente. Per quella spaziale, non trasferisce i byte uno alla volta ma a gruppi contigui, detti **blocchi** o *linee*: quando serve `v[0]`, arrivano anche `v[1]`, `v[2]` e `v[3]`, che serviranno subito dopo. Un programma che saltasse a caso per la memoria non avrebbe località, e nessuna cache lo aiuterebbe.

Fissiamo il vocabolario, che è quasi sempre inglese. Quando il dato cercato è nella cache abbiamo un **hit** (successo); quando non c'è, un **miss** (fallimento), e il blocco va prelevato dal livello inferiore. La frazione di accessi che trovano il dato è il *tasso di hit* (*hit rate*); il resto è il **tasso di miss** (*miss rate*), cioè 1 − tasso di hit. Il *tempo di hit* è il tempo per accedere alla cache e determinare se il dato è presente; la **penalità di miss** (*miss penalty*) è il tempo per prelevare il blocco dal livello inferiore, copiarlo nella cache e consegnare il dato. Il tempo di hit è di qualche ciclo, la penalità di decine o centinaia: per questo anche pochi miss pesano molto.

La cache è gestita da un circuito apposito, il **controllore della cache**, ed è *trasparente*: il programma usa gli indirizzi di sempre e non ha alcuna informa-

zione sulla sua esistenza. Ne subisce però gli effetti, e chi scrive programmi veloci ne tiene conto.

> **Righe o colonne.** In C una matrice è memorizzata per righe: in memoria, l'elemento `a[i][j+1]` segue `a[i][j]`. Due cicli annidati che la scorrono riga per riga leggono indirizzi consecutivi; gli stessi cicli con gli indici scambiati saltano ogni volta di un'intera riga, e ogni accesso può cadere in un blocco diverso. Il numero di istruzioni è identico, ma per matrici grandi la seconda versione può essere molte volte più lenta.

d.3 La cache a mappatura diretta

Una cache deve risolvere due problemi: dove collocare un blocco, e come ritrovarlo. La soluzione più semplice è dare a ogni blocco *un solo* posto possibile. Numeriamo i blocchi della memoria e le righe della cache; il blocco numero b è collocato nella riga $b \bmod N$, dove N è il numero di righe. È la **cache a mappatura diretta** (*direct mapped*).

Se il blocco è lungo B byte e N è una potenza di due, i conti si riducono a una semplice suddivisione dei bit dell'indirizzo:

$$b = \left\lfloor \frac{\text{indirizzo}}{B} \right\rfloor, \qquad \text{riga} = b \bmod N, \qquad \text{tag} = \left\lfloor \frac{b}{N} \right\rfloor.$$

I $\log_2 B$ bit meno significativi dell'indirizzo sono l'**offset**, la posizione del byte dentro il blocco; i $\log_2 N$ bit successivi sono l'**indice**, il numero della riga; i bit rimanenti, i più significativi, sono il **tag**. Molti blocchi diversi sono associati alla stessa riga: il tag, memorizzato nella riga insieme ai dati, indica quale di essi vi si trova in questo momento. A ogni riga si aggiunge infine un **bit di validità** V: quando il processore si accende la cache contiene valori casuali, e V vale 0 finché nella riga non viene caricato un blocco vero.

Una traccia a mano

Prendiamo una cache di esempio, volutamente minuscola, con indirizzi da 8 bit, 4 righe e blocchi da 8 byte, cioè due parole. L'offset ha 3 bit, l'indice 2, il tag 3. Il programma esegue otto `lw` (tabella d.2). Per ogni indirizzo separiamo i tre campi, andiamo alla riga indicata dall'indice e confrontiamo il tag.

La prima lettura trova la riga 1 vuota: miss, e il controllore carica l'intero blocco da `0x48` a `0x4f`. La seconda legge `0x4c`, che appartiene allo stesso blocco: hit, grazie alla località spaziale. La terza occupa la riga 2. La quarta, `0xc8`, ha indice 1 ma tag diverso: miss, e il blocco di `0x48` viene *sostituito*. Da qui in poi i due blocchi si contendono la riga 1 e si sostituiscono a vicenda

Tabella d.2 Otto letture in una cache diretta da 4 righe di 8 byte. Nella colonna binaria i campi sono separati: tag, indice, offset. M[48–4f] indica il blocco che contiene i byte da `0x48` a `0x4f`.

	Indirizzo	Riga	Tag	Esito	Contenuto della riga dopo
1	`0x48` = `010 01 000`	1	`010`	miss	M[48–4f]
2	`0x4c` = `010 01 100`	1	`010`	**hit**	M[48–4f]
3	`0x10` = `000 10 000`	2	`000`	miss	M[10–17]
4	`0xc8` = `110 01 000`	1	`110`	miss	M[c8–cf]
5	`0x48` = `010 01 000`	1	`010`	miss	M[48–4f]
6	`0x14` = `000 10 100`	2	`000`	**hit**	M[10–17]
7	`0xcc` = `110 01 100`	1	`110`	miss	M[c8–cf]
8	`0x4c` = `010 01 100`	1	`010`	miss	M[48–4f]

a ogni accesso, mentre le righe 0 e 3 restano vuote. Risultato: 2 hit su 8. Ritroveremo questo esempio nella § d.5, con una cache organizzata meglio.

L'hardware

La figura d.1 mostra una cache vera, con indirizzi da 32 bit, 1024 righe e blocchi di 8 parole, cioè 32 byte. L'offset ha 5 bit, divisi in 3 bit che selezionano la parola e 2 che selezionano il byte all'interno della parola; l'indice ha 10 bit, il tag $32 - 10 - 5 = 17$.

L'indice seleziona una riga, come il numero di registro seleziona un registro nel banco dello stato 0x3. Il tag della riga è inviato a un comparatore insieme al tag dell'indirizzo; se sono uguali e V vale 1, il segnale *hit* assume il valore 1. Intanto un multiplexer, comandato dai 3 bit della parola, seleziona una delle otto parole del blocco. Tutto avviene in parallelo: la lettura dei dati non attende il confronto, e se poi il confronto fallisce il dato viene semplicemente ignorato.

Che cosa succede in caso di miss? Nel nostro processore a pipeline la risposta è la più semplice possibile: la pipeline si ferma. È uno stallo come quelli dello stato 0xa, solo molto più lungo. Il controllore chiede il blocco al livello inferiore, lo scrive nella riga insieme al nuovo tag, pone V a 1, e l'accesso viene ripetuto, questa volta con un hit. Vale sia per la cache dei dati, durante la fase MEM, sia per quella delle istruzioni, durante la fase IF.

Perché l'indice occupa i bit centrali

Nulla vieterebbe di usare come indice i bit più significativi e come tag quelli centrali: ogni blocco avrebbe comunque un posto e un nome unici. Consideriamo però un vettore di 1000 parole. I suoi indirizzi hanno tutti gli

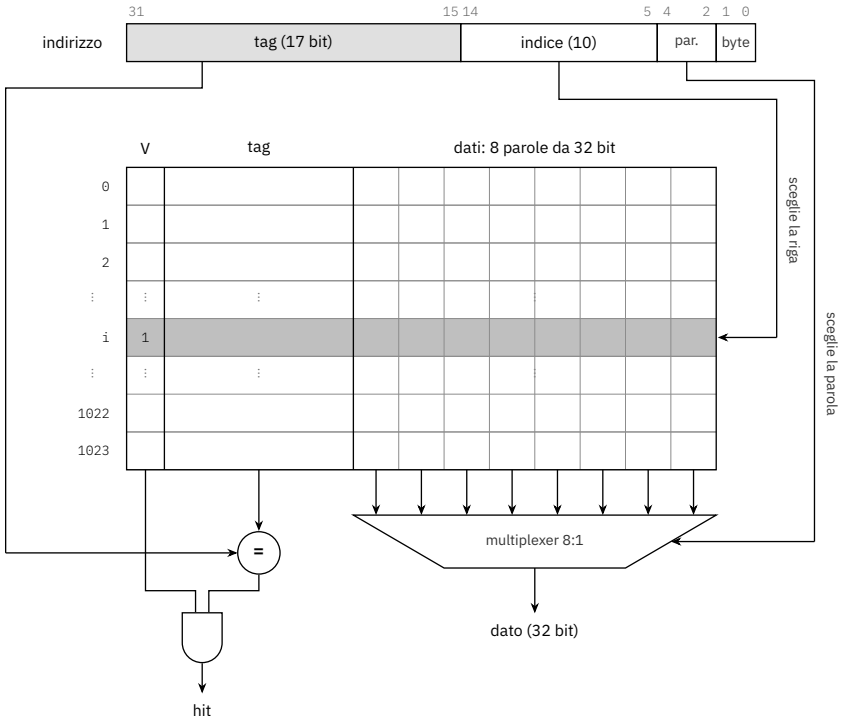


Figura d.1 Una cache a mappatura diretta da 32 KiB: 1024 righe, ciascuna con il bit V, un tag da 17 bit e un blocco di 8 parole. L'indice seleziona la riga, il comparatore indica se il tag è quello corretto, e il multiplexer seleziona la parola. I 2 bit del byte non servono per leggere una parola allineata.

stessi bit alti, e con l'indice preso da quei bit finirebbero *tutti nella stessa riga*, sostituendosi a vicenda. Prendendo l'indice dai bit appena sopra l'offset, invece, blocchi consecutivi sono collocati in righe consecutive: il vettore si distribuisce su tutta la cache, e la località spaziale opera a nostro favore.

d.4 Quanto è grande una cache

Una cache occupa più bit di quanti dati contiene, perché ogni riga contiene, oltre ai dati, il tag e il bit di validità. Cominciamo da un piccolo sistema con indirizzi da 16 bit, che indirizzano 2^{16} byte, cioè 64 KiB. Le parole sono da 2 byte e i blocchi da 8 parole, cioè 16 byte; la cache ha 16 righe. L'indirizzo si divide così:

- 1 bit seleziona il byte all'interno della parola, 3 bit la parola all'interno del blocco: l'offset ha 4 bit;

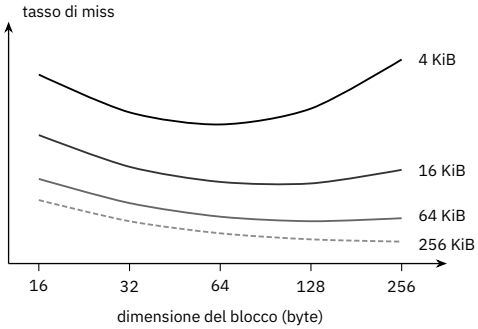


Figura d.2 Andamento tipico del tasso di miss al crescere del blocco, per cache di varie dimensioni. Le curve sono qualitative. Nelle cache grandi blocchi più grandi aiutano quasi sempre; in quelle piccole, oltre un certo punto, peggiorano.

- 4 bit di indice selezionano una delle $2^4 = 16$ righe;
- i $16 - 4 - 4 = 8$ bit rimanenti formano il tag.

I dati occupano $16 \times 16 = 256$ byte, cioè 2048 bit. Ogni riga aggiunge 8 bit di tag e 1 bit di validità, in tutto $16 \times 9 = 144$ bit: la cache contiene $2048 + 144 = 2192$ bit. I suoi dati sono $256/65\,536 \approx 0{,}39\%$ della memoria indirizzabile.

Ora la cache della figura d.1: 1024 righe con 32 byte di dati, 17 bit di tag e un bit V. Ogni riga occupa $256+17+1 = 274$ bit, e la cache $1024\times274 = 280\,576$ bit, 34,25 KiB per 32 KiB di dati: il 7% in più. Rispetto ai 4 GiB indirizzabili, 32 KiB sono $2^{15}/2^{32} = 1/131\,072$: lo 0,00076%. Eppure, come vedremo, una cache così trova il dato in più del 95% dei casi. È la misura di quanto sia potente la località.

!

Frazioni e percentuali $1/131\,072$ vale circa 0,0000076. Per esprimerlo come percentuale, il valore va moltiplicato per cento: 0,00076%. Si trova a volte scritto 0,0000076%, che è cento volte più piccolo. Quando si confrontano grandezze così diverse, conviene ragionare in potenze di due: 2^{15} contro 2^{32}, un rapporto di 2^{17}.

Quanto deve essere grande il blocco

Blocchi più grandi sfruttano meglio la località spaziale: un miss porta con sé più dati vicini, e i miss diminuiscono. Fino a un certo punto (figura d.2). Con la capacità fissata, blocchi più grandi significano meno righe, e quindi più blocchi che si contendono lo stesso posto; inoltre ogni miss deve trasferire più byte, e la penalità cresce. In una cache piccola, oltre una certa dimensione il tasso di miss torna a salire.

Alla penalità si può rimediare in parte. Con il *riavvio anticipato* (*early*

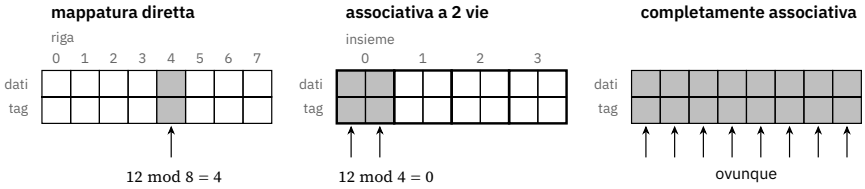


Figura d.3 Dove può essere collocato il blocco 12 in una cache da 8 blocchi. In grigio i posti possibili; le frecce indicano i tag da confrontare per ritrovarlo: uno, due oppure tutti e otto.

restart) il processore riparte appena arriva la parola che attendeva, senza attendere il resto del blocco; con la *parola critica per prima* (*critical word first*) la memoria invia per prima proprio quella parola. Oggi quasi tutte le cache usano blocchi da 64 byte.

d.5 Più libertà: le cache associative

Nella traccia della tabella d.2 due blocchi si contendevano la riga 1 mentre due righe restavano vuote. Il rimedio è lasciare al blocco più di un posto possibile. All'estremo opposto della mappatura diretta c'è la **cache completamente associativa**: un blocco può essere collocato in qualunque riga. Non c'è più indice, l'indirizzo è costituito soltanto da tag e offset, e per ritrovare un blocco bisogna confrontare il tag con *tutte* le righe, in parallelo: un comparatore per riga. Con migliaia di righe è troppo costoso in area ed energia; si usa solo per memorie piccolissime, come il TLB che vedremo nello stato 0xe.

Il compromesso è la **cache associativa a insiemi** (*set associative*). Le righe sono raggruppate in *insiemi* di n righe, una per ciascuna delle n *vie* (*ways*): una cache a n vie è come n cache dirette affiancate. L'indice sceglie l'insieme, e dentro l'insieme il blocco può occupare la riga di una qualunque delle n vie. Per cercarlo bastano n comparatori. La mappatura diretta è il caso $n = 1$; la completamente associativa è il caso di un solo insieme con tutte le righe. La figura d.3 mostra dove può essere collocato il blocco numero 12 in una cache da 8 blocchi organizzata nei tre modi.

Con la stessa capacità, raddoppiare le vie dimezza il numero di insiemi: l'indice perde un bit e il tag ne guadagna uno. Per la cache da 32 KiB con blocchi da 32 byte, cioè 1024 blocchi:

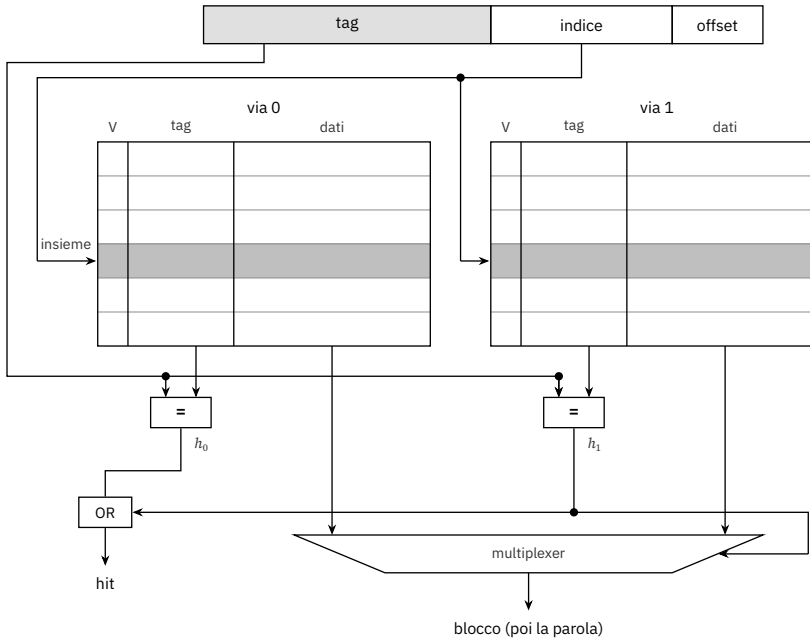


Figura d.4 Una cache associativa a due vie. L'indice seleziona un insieme, cioè la stessa riga in entrambe le vie; i due comparatori operano in parallelo (ognuno controlla anche il bit V della sua riga). Al più uno dei due esiti h_0, h_1 vale 1: l'OR fornisce il segnale di hit, e h_1 seleziona il blocco in uscita.

Vie	1	2	4	8	1024
insiemi	1024	512	256	128	1
bit di indice	10	9	8	7	0
bit di tag	17	18	19	20	27
comparatori	1	2	4	8	1024

La figura d.4 mostra l'hardware di una cache a due vie. L'indice seleziona la stessa riga in entrambe le vie; i due tag vengono confrontati in parallelo con quello dell'indirizzo; al più uno dei due confronti ha successo, e il suo esito comanda il multiplexer che seleziona il blocco corretto. Il prezzo è un comparatore per ogni via e un multiplexer sul cammino del dato: il tempo di hit cresce un poco.

La traccia, di nuovo

Rifacciamo la traccia della tabella d.2 con una cache della stessa capacità, 32 byte, ma a due vie: due insiemi di due righe da 8 byte. L'offset resta di 3 bit, l'indice ha un solo bit (il bit 3) e il tag i 4 bit più alti. Ora `0x48` e `0xc8` sono

associati entrambi all'insieme 1, ma vi trovano posto insieme, uno per via; `0x10` è associato all'insieme 0. I miss sono solo i primi accessi ai tre blocchi (letture 1, 3 e 4); tutte le altre letture sono hit: 5 hit su 8, contro i 2 della cache diretta. Anche una cache completamente associativa da 4 righe otterrebbe 5 hit: per questa traccia due vie bastano.

Quale blocco sostituire

Quando un blocco deve entrare in un insieme pieno, uno dei blocchi presenti va sostituito. Nella mappatura diretta non c'è scelta; con più vie serve una **politica di sostituzione**. La più naturale è **LRU** (*least recently used*): viene sostituito il blocco non usato da più tempo, il meno probabile da riusare per la località temporale. Con due vie basta un bit per insieme, che indica quale via è stata usata per ultima. Con quattro, otto o sedici vie tenere l'ordine esatto di tutti gli accessi costa troppo, e si usano approssimazioni. Una semplice è **NRU** (*not recently used*): ogni riga ha un *bit di riferimento*, posto a 1 a ogni accesso e azzerato periodicamente per tutte le righe; al momento di sostituire si sceglie una riga con il bit a 0, cioè non usata di recente, anche se non necessariamente la più vecchia. Alcune cache, infine, sostituiscono un blocco scelto a caso: con molte vie, il risultato è di poco peggiore di LRU.

d.6 Le scritture

Le letture non pongono problemi: la cache contiene una copia, e una copia si può leggere quante volte si vuole. Una `sw`, invece, modifica il dato, e la copia in cache e l'originale in memoria rischiano di non coincidere più. Ci sono due politiche.

Con la **scrittura immediata** (*write-through*) ogni scrittura aggiorna sia la cache sia la memoria. Le due copie coincidono sempre, ma ogni `sw` costerebbe un accesso alla memoria. Con il 10% di store e 100 cicli per scrivere in memoria, un processore con CPI 1 passerebbe a $1 + 0{,}1 \times 100 = 11$. Per questo si aggiunge un **buffer di scrittura** (*write buffer*): la store deposita il dato nel buffer e il processore prosegue, mentre il buffer si svuota verso la memoria autonomamente. Il processore si ferma solo se il buffer è pieno.

Con la **scrittura differita** (*write-back*) la store aggiorna solo la cache. La memoria verrà aggiornata più tardi, quando il blocco sarà sostituito. Per sapere se va ricopiato, ogni riga ha un **bit dirty**: vale 0 quando il blocco arriva dalla memoria e passa a 1 alla prima scrittura. Al momento della sostituzione, se il bit vale 1 il blocco viene prima scritto in memoria e solo dopo rimpiazzato; se vale 0 basta sovrascriverlo. Un dato scritto cento volte in un ciclo costa una sola scrittura in memoria, alla fine.

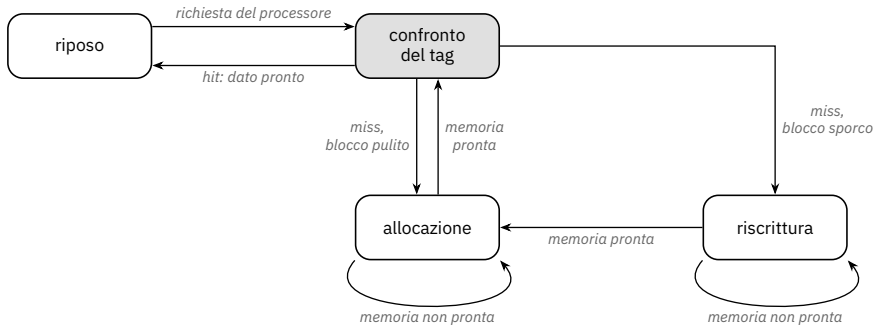


Figura d.5 Il controllore di una cache write-back con allocazione in scrittura. Se il blocco da sostituire è sporco, va prima ricopiato in memoria; poi arriva il nuovo blocco, e il confronto viene ripetuto.

Resta il caso di una store che genera un miss. Con l'**allocazione in scrittura** (*write allocate*) il blocco viene prima caricato nella cache e poi modificato, come se fosse stato letto. Senza (*no-write allocate*) la scrittura va direttamente in memoria e la cache non viene modificata. Le coppie tipiche sono write-back con allocazione, che scommette su altre scritture vicine, e write-through senza allocazione.

Il controllore come macchina a stati

Il controllore di una cache write-back è una macchina a stati finiti, della stessa famiglia del controllo multiciclo dello stato 0x8 (figura d.5). A riposo attende una richiesta del processore. Quando arriva, confronta il tag: se è un hit consegna il dato, o lo scrive e pone a 1 il bit dirty, e torna a riposo. Se è un miss esamina il blocco da sostituire: se è sporco passa per lo stato di riscrittura, finché la memoria non ha ricevuto il blocco; poi passa allo stato di allocazione, finché il nuovo blocco non è arrivato; infine torna a confrontare il tag, che ora produce un hit.

d.7 Perché si sbaglia: le tre C

I miss non sono tutti uguali, e conoscerne la causa aiuta a scegliere il rimedio. Si classificano in tre famiglie, le **tre C** dall'iniziale dei nomi inglesi (tabella d.3).

- **Obbligati** (*compulsory*, o a freddo): il primo accesso a un blocco è sempre un miss, perché il blocco non è mai stato nella cache.
- **Di capacità** (*capacity*): il programma usa più blocchi di quanti la cache ne possa contenere, e alcuni vengono sostituiti e poi ricaricati.

Tabella d.3 Le tre C dei miss: cause, rimedi ed effetti collaterali dei rimedi.

Miss	Causa	Rimedio	Effetto collaterale
obbligato	primo accesso al blocco	blocchi più grandi, prelievo anticipato	penalità maggiore, blocchi caricati inutilmente
di capacità	la cache è troppo piccola	cache più grande	tempo di hit maggiore
di conflitto	troppi blocchi nello stesso insieme	più vie	tempo di hit ed energia maggiori

- **Di conflitto** (*conflict*): ci sarebbe posto, ma troppi blocchi sono associati allo stesso insieme. Sono i miss che una cache completamente associativa della stessa capacità non avrebbe.

Nella traccia della tabella d.2 le letture 1, 3 e 4 sono miss obbligati, le letture 5, 7 e 8 miss di conflitto: la cache a due vie li ha eliminati tutti e tre.

> **Indovinare prima.** Il *prelievo anticipato* (*prefetching*) affronta proprio i miss obbligati: il controllore carica un blocco prima che venga chiesto. L'Intel Nehalem, che abbiamo incontrato nello stato 0xc, ha due meccanismi per la cache dati: uno rileva letture in ordine crescente e carica il blocco successivo; l'altro memorizza, per ogni load, la distanza fra un indirizzo e il successivo, e se è regolare carica in anticipo il blocco che servirà.

d.8 Quanto costano i miss

Una prima misura è il **tempo medio di accesso** alla memoria (AMAT, *average memory access time*):

$$\text{AMAT} = \text{tempo di hit} + \text{tasso di miss} \times \text{penalità di miss}.$$

Con un tempo di hit di 1 ciclo, il 5% di miss e una penalità di 100 cicli, l'AMAT è $1 + 0{,}05 \times 100 = 6$ cicli: sei volte il tempo di hit, anche se il 95% degli accessi va a buon fine.

Per il processore conta di più il CPI. Ogni istruzione accede alla cache delle istruzioni una volta, e alla cache dei dati se è una load o una store. I cicli di stallo per istruzione si sommano al CPI ideale, quello che avremmo con una cache perfetta:

$$\text{CPI} = \text{CPI}_{\text{ideale}} + \underbrace{m_I \times P}_{\text{istruzioni}} + \underbrace{f_{LS} \times m_D \times P}_{\text{dati}},$$

dove m_I e m_D sono i tassi di miss delle due cache, f_{LS} la frazione di load e store e P la penalità.

Facciamo i conti con numeri tipici degli esempi didattici: CPI ideale 2, penalità di 100 cicli, 36% di load e store, tasso di miss del 2% per le istruzioni e del 4% per i dati.

- Stalli per le istruzioni: $0{,}02 \times 100 = 2$ cicli.
- Stalli per i dati: $0{,}36 \times 0{,}04 \times 100 = 1{,}44$ cicli.
- CPI: $2 + 2 + 1{,}44 = 5{,}44$.

Con una cache perfetta il processore sarebbe $5{,}44/2 = 2{,}72$ volte più veloce. E se raddoppiassimo la frequenza del processore lasciando la memoria com'è? La penalità, misurata in cicli, raddoppierebbe a 200: il CPI diventa $2+4+2{,}88 = 8{,}88$, e il guadagno è solo $2 \times 5{,}44/8{,}88 \approx 1{,}23$. Ecco il muro della memoria in numeri: un processore più veloce trascorre soltanto più cicli in attesa.

> **Non fermarsi per un miss.** Il nostro processore si ferma a ogni miss. Un processore con esecuzione fuori ordine, come quelli dello stato 0xc, può continuare a eseguire le istruzioni che non dipendono dal dato mancante, e la cache deve allora gestire altri accessi mentre il miss è in corso: si dice *non bloccante*. Ogni core di Nehalem può gestire fino a dieci miss della cache dati alla volta.

d.9 Cache a più livelli

Una sola cache deve scegliere fra essere piccola e veloce o grande e capace di trattenere più dati. Con due o tre livelli non serve scegliere. La cache di primo livello (L1) è piccola e collocata accanto al core, ed è divisa in una cache per le istruzioni e una per i dati: è la parte «Harvard» del processore descritta nello stato 0x0. Il suo obiettivo è il tempo di hit, che deve rientrare in uno o due cicli di pipeline. La L2 e la L3 sono più grandi e più lente, contengono sia istruzioni sia dati, e il loro obiettivo è ridurre i miss che arrivano fino alla DRAM.

Riprendiamo l'esempio, aggiungendo una cache L2. Un miss in L1 che trova il dato in L2 costa 25 cicli; un accesso che manca anche in L2 costa altri 100 cicli per andare in memoria. Il tasso di miss *globale* della L2 è dello 0,5%: solo mezzo accesso su cento, fra tutti quelli del programma, arriva in memoria. Le istruzioni accedono alla memoria $1 + 0{,}36 = 1{,}36$ volte ciascuna, in media.

- Miss in L1 delle istruzioni: $0{,}02 \times 25 = 0{,}5$ cicli.
- Miss in L1 dei dati: $0{,}36 \times 0{,}04 \times 25 = 0{,}36$ cicli.
- Miss anche in L2: $1{,}36 \times 0{,}005 \times 100 = 0{,}68$ cicli.
- CPI: $2 + 0{,}5 + 0{,}36 + 0{,}68 = 3{,}54$.

Da 5,44 a 3,54: con la L2 il processore è 1,54 volte più veloce. Curiosamente, se

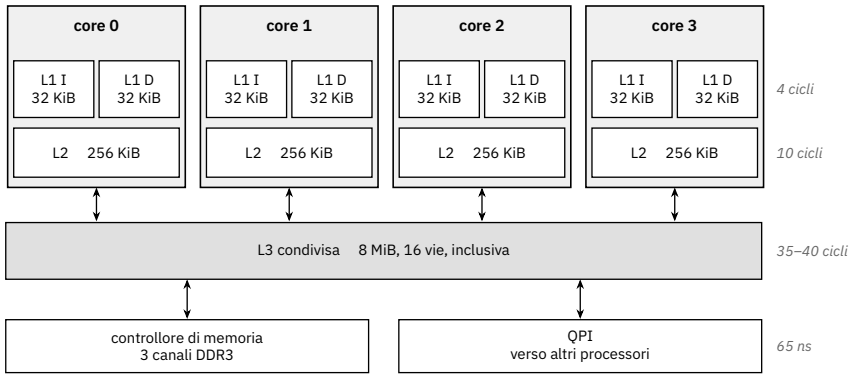


Figura d.6 La gerarchia di memoria dell'Intel Nehalem. A destra i tempi di accesso approssimativi; la DRAM locale risponde in circa 65 ns, cioè qualche centinaio di cicli.

misuriamo la L2 solo sugli accessi che le arrivano, il suo tasso di miss risulta elevato: i miss in L1 sono $0{,}02 + 0{,}36 \times 0{,}04 = 0{,}0344$ per istruzione, quelli in L2 $1{,}36 \times 0{,}005 = 0{,}0068$, cioè circa uno su cinque. Un *tasso locale* del 20% sembra disastroso, ma conta quello globale: la L2 riceve solo i casi difficili, quelli che la L1 non ha potuto soddisfare.

Un processore vero

La figura d.6 mostra la gerarchia dell'Intel Nehalem, con quattro core su un chip. Tutte le cache usano blocchi da 64 byte. Ogni core ha le sue cache L1, una per le istruzioni (32 KiB, 4 vie) e una per i dati (32 KiB, 8 vie), che rispondono in 4 cicli, e una L2 privata da 256 KiB a 8 vie, write-back, che risponde in circa 10 cicli. La L3 da 8 MiB a 16 vie è condivisa dai quattro core e risponde in 35–40 cicli o più. La L3 è **inclusiva**: ogni blocco presente in una L1 o in una L2 è presente anche in L3. Vedremo nello stato 0xf perché questa scelta sia utile quando i core sono più di uno. Il controllore della memoria risiede sullo stesso chip, e gestisce tre canali DDR3.

Il concorrente AMD della stessa generazione (l'Opteron detto *Barcelona*) faceva scelte diverse: L1 più grandi, da 64 KiB ciascuna, ma solo a due vie, con un LRU esatto da un bit; L2 da 512 KiB a 16 vie; L3 condivisa da 2 MiB a 32 vie. Le due architetture eseguono lo stesso codice x86: la gerarchia di memoria è uno dei terreni su cui i produttori si fanno concorrenza senza toccare l'ISA.

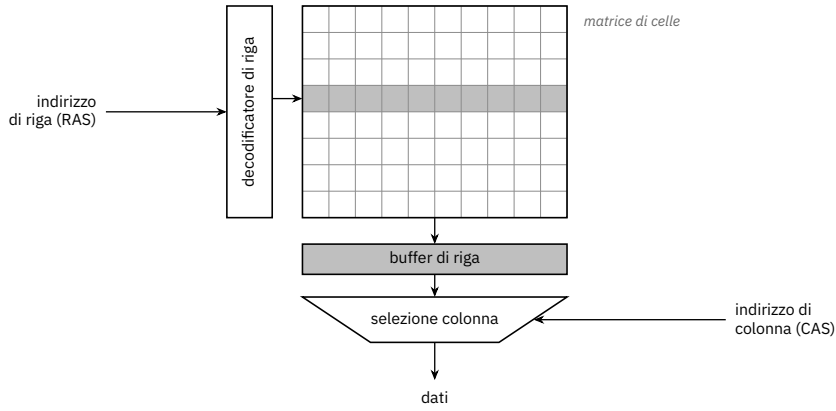


Figura d.7 L'organizzazione di un chip DRAM. Il RAS copia un'intera riga nel buffer di riga; il CAS seleziona le colonne da consegnare. Letture successive dalla stessa riga sono molto più veloci.

1965 Una memoria schiava

Nell'aprile del 1965 Maurice Wilkes, l'inventore della microprogrammazione (stato 0x8), pubblicò un articolo di due pagine che proponeva una piccola memoria veloce, la *slave memory*, «memoria schiava», che tenesse copia delle parole usate di recente dalla grande memoria principale. Tre anni dopo, nel gennaio 1968, IBM annunciò il System/360 Model 85, il primo calcolatore in commercio con una memoria del genere, da 16 o 32 KiB. Gli ingegneri la chiamavano *high-speed buffer*; il direttore della rivista IBM che doveva pubblicare i loro articoli trovò il nome troppo consumato e propose *cache*, dal francese *cacher*, nascondere. Da allora il nome, e l'idea, non sono più stati abbandonati.

---

d.10 Dentro la DRAM

Sotto le cache c'è la memoria principale, realizzata con DRAM. Abbiamo visto nello stato 0x6 che una cella DRAM è un condensatore con un transistor. Le celle sono disposte in una matrice di righe e colonne (figura d.7), e l'indirizzo viene diviso in un *indirizzo di riga* e un *indirizzo di colonna*, inviati uno dopo l'altro sugli stessi piedini del chip, per risparmiarne la metà. Il segnale **RAS** (*row access strobe*) indica che sui piedini è presente l'indirizzo di riga: l'intera riga, migliaia di bit, viene letta e copiata in un *buffer di riga*. Il segnale CAS (*column access strobe*) trasmette poi l'indirizzo di colonna, che seleziona i bit da consegnare. Finché si legge dalla stessa riga, basta cambiare la colonna, e l'accesso è molto più rapido; per passare a un'altra riga bisogna prima riscrivere quella aperta e preparare le linee.

Tabella d.4 Le generazioni di SDRAM. Le velocità sono in milioni di trasferimenti al secondo (MT/s) e sono quelle tipiche di ogni generazione; il prefetch indica quanti dati per piedino vengono letti dalla matrice a ogni accesso interno.

Tipo	Velocità (MT/s)	Prefetch	Tensione	Novità
SDR	66–133	1n	3,3 V	un dato per ciclo di clock
DDR	200–400	2n	2,5 V	due dati per ciclo
DDR2	400–1066	4n	1,8 V	bus esterno più veloce
DDR3	800–2133	8n	1,5 V	consumi ridotti, self-refresh automatico
DDR4	1600–3200	8n	1,2 V	gruppi di banchi indipendenti
DDR5	4800–8800	16n	1,1 V	due sottocanali per modulo

Il condensatore perde carica, e ogni riga va letta e riscritta periodicamente: è il **refresh**, che deve interessare tutte le righe entro una finestra fissata (64 ms nelle DDR4, 32 ms nelle DDR5) e occupa la memoria per una piccola frazione del tempo, nell'ordine dell'1–2%.

Sincrone, a doppia velocità

Le DRAM di oggi sono **SDRAM**: *sincrone*, cioè comandate da un clock come il resto del sistema, e capaci di consegnare una sequenza di dati consecutivi, un *burst*, con un solo comando. Le DDR (*double data rate*) trasferiscono un dato sia sul fronte di salita sia su quello di discesa del clock. Generazione dopo generazione la matrice interna è rimasta lenta; è cresciuto il numero di bit letti in parallelo dalla matrice a ogni accesso interno (il *prefetch*) e la velocità con cui vengono poi serializzati sui piedini (tabella d.4).

La banda

Un miss in cache deve trasferire un blocco intero, e la penalità dipende da come la memoria è organizzata. Usiamo i tempi dell'esempio classico, misurati in cicli del bus di memoria, più lento del processore: un ciclo per inviare l'indirizzo, 15 cicli per un accesso alla DRAM, un ciclo per trasferire una parola sul bus. Il blocco è di 4 parole, 16 byte (figura d.8).

(a) Una parola alla volta, ognuna con un accesso completo: $1 + 4 \times 15 + 4 \times 1 = 65$ cicli per 16 byte, circa 0,25 byte per ciclo.

(b) Con un burst dalla stessa riga, solo la prima parola paga l'accesso completo; le altre richiedono solo la selezione della colonna, per esempio 5 cicli, e ogni trasferimento si sovrappone all'accesso alla colonna successiva: $1 + 15 + 3 \times 5 + 1 = 32$ cicli, 0,5 byte per ciclo.

(c) Con quattro **banchi** indipendenti, una parola per banco (si dice memoria *interfogliata*, in inglese *interleaved*), i quattro accessi avvengono in paralle-

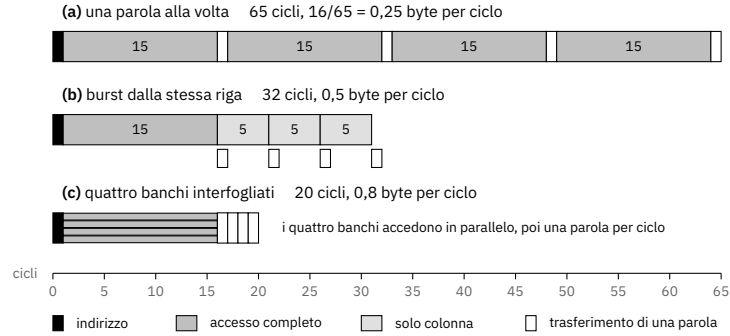


Figura d.8 Tre modi di trasferire un blocco di 4 parole. In (b) i trasferimenti, sulla riga sottile, si sovrappongono agli accessi alle colonne successive.

lo e poi le parole escono una per ciclo: $1 + 15 + 4 \times 1 = 20$ cicli, 0,8 byte per ciclo.

Le memorie reali combinano entrambe le idee: burst dentro ogni chip, più banchi e più canali indipendenti. I tre canali DDR3 di Nehalem, ciascuno largo 8 byte e capace di 1333 MT/s, offrono in teoria $3 \times 8 \times 1{,}333 \approx 32$ GB/s.

d.11 Quando un bit si inverte

Un modulo di memoria contiene decine di miliardi di condensatori, ognuno con una carica minuscola. Una fluttuazione di tensione, un disturbo elettromagnetico, una particella alfa emessa da tracce radioattive nei materiali del contenitore o un raggio cosmico possono invertire il valore di un bit: uno 0 diventa 1 o viceversa. Sono *errori transitori* (*soft errors*): la cella non è guasta, ma il dato sì. Nella trasmissione dei dati, quando si rileva un errore si chiede di ripetere l'invio. Per un dato in memoria non esiste una sorgente a cui richiederlo: se la cella ha cambiato valore, l'originale non esiste più. Occorre rilevare l'errore e correggerlo autonomamente.

Parità e distanza

Il rimedio più semplice è il **bit di parità**: si aggiunge alla parola un bit scelto in modo che il numero totale di 1 sia pari. Se in lettura il numero di 1 è dispari, un bit è cambiato. La parità rileva ogni errore singolo, ma non indica *quale* bit è sbagliato, e quindi non permette di correggerlo; due errori, poi, si compensano e passano inosservati.

Per ottenere di più occorre il concetto di distanza. La **distanza di Hamming** fra due sequenze di bit della stessa lunghezza è il numero di posizioni in cui

differiscono: si calcola con lo XOR bit a bit, contando gli 1. Fra `10010101` e `01010001` è 3. Un *codice* è l'insieme delle sequenze considerate lecite, e la sua distanza d è la minima distanza fra due sequenze lecite. Se tutte le sequenze sono lecite, $d = 1$: un errore trasforma un dato valido in un altro dato valido, e non viene rilevato. Con la parità, metà delle sequenze sono illecite e $d = 2$. In generale, un codice con distanza d:

- rileva fino a $d-1$ errori, perché servono almeno d bit cambiati per passare da una sequenza lecita a un'altra;
- corregge fino a $\lfloor (d-1)/2 \rfloor$ errori, perché la sequenza ricevuta resta più vicina all'originale che a qualunque altra sequenza lecita;
- più in generale, corregge fino a t errori e ne rileva fino a $s \geq t$ se $d \geq t+s+1$: per esempio corregge t errori e ne rileva $t+1$ se $d \geq 2t+2$.

Con $d = 5$, per esempio, si possono correggere 2 errori, oppure correggerne 1 e rilevarne fino a 3, oppure solo rilevarne fino a 4.

Il codice di Hamming

Per correggere un errore singolo (SEC, *single error correction*) serve $d = 3$. Aggiungiamo agli M bit di dato K bit di controllo, in tutto $M+K$ bit. In lettura i bit di controllo ricalcolati devono indicare se c'è un errore e, se c'è, in quale delle $M+K$ posizioni: in tutto $M+K+1$ casi diversi, che i K bit devono poter distinguere. Quindi

$$2^K \geq M + K + 1.$$

Il K minimo cresce lentamente: con $M = 1$ serve $K = 2$; con $M = 4$, $K = 3$; con $M = 8$, $K = 4$; con $M = 16$, $K = 5$; con $M = 32$, $K = 6$; con $M = 64$, $K = 7$. Il costo relativo K/M scende: proteggere una parola da 32 bit costa 6 bit, proteggere separatamente i suoi 4 byte ne costerebbe $4 \times 4 = 16$.

L'idea di Richard Hamming è disporre i bit in modo che il risultato dei controlli sia direttamente la posizione dell'errore. Si numerano le posizioni da 1. I bit di controllo occupano le posizioni che sono potenze di due (1, 2, 4, 8...), i dati in tutte le altre. Il bit di controllo in posizione 2^i è la parità di tutte le posizioni il cui numero, scritto in binario, ha un 1 nel bit i (tabella d.5). Così ogni posizione è controllata da una combinazione diversa di bit di controllo, quella data dal suo numero in binario: la posizione $7 = 4+2+1$ è controllata da C_4, C_2 e C_1, la posizione $5 = 4+1$ da C_4 e C_1.

In scrittura, con i bit numerati $y_1 \ldots y_7$:

$$C_1 = D_1 \oplus D_2 \oplus D_4, \qquad C_2 = D_1 \oplus D_3 \oplus D_4, \qquad C_4 = D_2 \oplus D_3 \oplus D_4.$$

Tabella d.5 Il codice di Hamming con 4 bit di dati e 3 di controllo. Ogni bit di controllo è la parità delle posizioni segnate sulla sua riga; le colonne sono le posizioni da 1 a 7.

Posizione	1	2	3	4	5	6	7
In binario	001	010	011	100	101	110	111
Contenuto	C_1	C_2	D_1	C_4	D_2	D_3	D_4
C_1	●		●		●		●
C_2		●	●			●	●
C_4				●	●	●	●

In lettura si ricalcolano le parità includendo i bit di controllo letti, e si ottengono i tre bit della **sindrome**:

$$E_1 = y_1 \oplus y_3 \oplus y_5 \oplus y_7, \quad E_2 = y_2 \oplus y_3 \oplus y_6 \oplus y_7, \quad E_4 = y_4 \oplus y_5 \oplus y_6 \oplus y_7.$$

Se $E_4E_2E_1 = 000$ non ci sono errori; altrimenti il numero $E_4E_2E_1$, letto in binario, è la posizione del bit sbagliato, ed è sufficiente invertirlo.

Proviamo con il dato $D_1D_2D_3D_4 = 1001$. I controlli valgono $C_1 = 1 \oplus 0 \oplus 1 = 0$, $C_2 = 1 \oplus 0 \oplus 1 = 0$, $C_4 = 0 \oplus 0 \oplus 1 = 1$, e in memoria si scrive $y_1 \dots y_7 = 0011001$. Supponiamo che il bit 3 si inverta: leggiamo 0001001. La sindrome vale

$$E_1 = 0 \oplus 0 \oplus 0 \oplus 1 = 1, \quad E_2 = 0 \oplus 0 \oplus 0 \oplus 1 = 1, \quad E_4 = 1 \oplus 0 \oplus 0 \oplus 1 = 0,$$

cioè $011 = 3$: il bit sbagliato è il terzo. Il controllore lo inverte, rimuove i bit di controllo e consegna 1001. Tutto questo avviene nel controllore della memoria, con alberi di porte XOR, ed è trasparente per il processore come la cache.

Due errori: SECDED

Che cosa succede se si invertono due bit? Riprendiamo 0011001 e supponiamo che cambino i bit 3 e 4: leggiamo 0000001. La sindrome vale $111 = 7$, e il controllore, che presuppone un errore singolo, inverte il bit 7: ora i bit sbagliati sono tre. Il codice di Hamming da solo non distingue un errore doppio da uno singolo.

Il rimedio costa un bit: si aggiunge un *bit di parità globale* P calcolato su tutti gli altri. In lettura si ricalcola anche questa parità, ottenendo Q, che vale 0 se la parità è verificata e 1 altrimenti. Un errore singolo cambia sempre la parità globale, un errore doppio mai. Le combinazioni di sindrome e parità si leggono così:

E	Q	**Significato**
$= 0$	0	nessun errore
$= 0$	1	errore singolo nel bit di parità globale: i dati sono corretti
$\neq 0$	1	errore singolo nella posizione E: si corregge
$\neq 0$	0	errore doppio: non si corregge, si segnala un'eccezione

È il codice **SECDED** (*single error correction, double error detection*). Nell'esempio, con due errori, $E = 7$ e $Q = 0$: il controllore non modifica alcun bit e segnala il problema. Tre errori contemporanei potrebbero sfuggire alla rilevazione, ma sono abbastanza rari da risultare trascurabili. Le memorie dei server usano moduli **ECC** (*error correcting code*) con 72 bit invece di 64: 64 di dati, 7 di Hamming e uno di parità globale. Serve dunque un chip in più ogni otto.

!

Una condizione necessaria non basta Per correggere due errori (DEC, *double error correction*) serve $d = 5$, e i bit di controllo devono distinguere anche le $(M + K)(M + K - 1)/2$ coppie di posizioni sbagliate:

$$2^K \geq 1 + (M + K) + \frac{(M + K)(M + K - 1)}{2}.$$

Con $M = 4$ la disuguaglianza è soddisfatta già da $K = 6$. Ma è una condizione *necessaria*, non sufficiente: stabilisce che con meno di 6 bit la correzione è impossibile, non che con 6 sia possibile. Si dimostra che nessun codice di 10 bit con distanza 5 contiene 16 sequenze lecite: per 4 bit di dati ne servono almeno 7 di controllo. In ogni caso, il costo è tale che nelle memorie si preferisce SECDED.

1950 **Un fine settimana alla Bell**

Alla fine degli anni Quaranta Richard Hamming lavorava ai Bell Labs e usava, nei fine settimana, un calcolatore a relè, il Model V, che leggeva i programmi da nastro perforato. La macchina rilevava gli errori grazie a codici di controllo, ma quando nessun operatore era presente si limitava ad abbandonare il lavoro e passare al successivo. Trovare il lunedì mattina il proprio calcolo interrotto lo esasperava: «se la macchina sa accorgersi di un errore, perché non può trovare dov'è e correggerlo?». Il risultato fu l'articolo *Error detecting and error correcting codes*, pubblicato nel 1950, che introdusse anche la distanza che porta il suo nome. Il codice SECDED entrò nelle memorie dei calcolatori già nel 1961, con l'IBM Stretch incontrato nello stato 0x9.

Condizione di transizione uscita verso `0xe`

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Che cosa sono la località temporale e quella spaziale? Quale delle due giustifica i blocchi di più parole? ↺ § d.2

2 Una cache diretta ha 16 righe da 16 byte e gli indirizzi sono di 16 bit. In quale riga è collocato l'indirizzo `0x1234`, e con quale tag? ↺ § d.3

3 Quanti bit occupa in tutto una cache diretta con 256 righe da 16 byte, se gli indirizzi sono di 32 bit? ↺ § d.4

4 Perché l'indice si ricava dai bit centrali dell'indirizzo e non da quelli alti? ↺ § d.3

5 Una cache da 64 KiB ha blocchi da 64 byte ed è associativa a 4 vie. Quanti insiemi ha? Quanti bit di indice e di tag, con indirizzi da 32 bit? Quanti comparatori? ↺ § d.5

6 In una cache write-back, a che cosa serve il bit dirty? Come si comporta una store che genera un miss, con l'allocazione in scrittura? ↺ § d.6

7 Classifica con le tre C i miss di una cache diretta che legge più volte, alternandoli, due blocchi con lo stesso indice. ↺ § d.7

8 CPI ideale 1, 30% di load e store, tassi di miss dell'1% per le istruzioni e del 5% per i dati, penalità di 50 cicli. Quanto vale il CPI? ↺ § d.8

9 Con gli stessi tempi della figura d.8, quanti cicli servono per un blocco di 8 parole con 8 banchi interfogliati? Quanti byte per ciclo? ↺ § d.10

10 Codifica il dato 1010 con il codice di Hamming a 7 bit. Se poi si inverte il bit 6, quanto vale la sindrome? ↺ § d.11

11 Con SECDED, che cosa significa una sindrome diversa da zero con la parità globale verificata? ↺ § d.11

STATO

0xe

1	1	1	0
b3	b2	b1	b0

La memoria virtuale

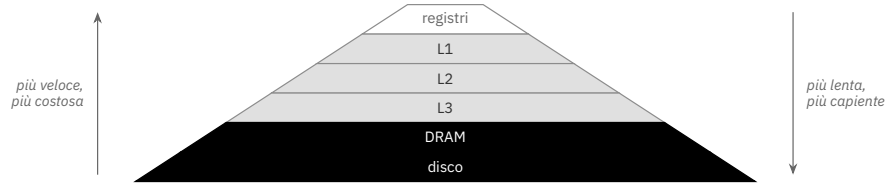


→ INGRESSI

- La cache: tag, indice, associatività, write-back (stato 0xd).
- Le eccezioni precise e i registri `mepc`, `mcause`, `mtval` (stato 0xb, § b.7).
- I livelli di privilegio e la `ecall` (stato 0x1, § 1.11).

USCITE →

- Distinguere indirizzi virtuali e fisici, e spiegare perché servono.
- Tradurre a mano un indirizzo con una tabella delle pagine, anche a due livelli come in RISC-V.
- Descrivere la sequenza di eventi di un page fault e il componente che lo gestisce.
- Spiegare a che cosa serve il TLB e come lavora insieme alla cache.
- Spiegare come la memoria virtuale protegge i processi l'uno dall'altro.



Finora ogni nostro programma disponeva dell'intera macchina: iniziava da un indirizzo fisso, `0x00400000` in RARS, collocava lo stack in alto e usava tutti gli indirizzi necessari. Su un calcolatore vero, mentre leggi queste righe, ci sono centinaia di programmi in esecuzione, e ciascuno opera nello stesso modo. Questo stato descrive il meccanismo che lo rende possibile: la memo-

ria principale diventa a sua volta una cache, il disco il livello sottostante, e fra il programma e la memoria si interpone una tabella di traduzione. Le idee sono quelle dello stato 0xd; cambiano le proporzioni, e con loro le scelte.

e.1 Tanti programmi, una memoria

Un **programma** è un file sul disco: codice e dati iniziali, statici. Un **processo** è un programma caricato in memoria e in esecuzione, con i suoi registri, il suo stack, i suoi dati che cambiano. Lo stesso programma può dare vita a più processi, come due finestre dello stesso editor.

Un sistema moderno esegue molti processi a turno: è la **multiprogrammazione**. Il sistema operativo assegna il processore a un processo per qualche millisecondo, poi lo sospende e passa a un altro. Per il processore il cambio di contesto costa poco: basta salvare in memoria il PC e i registri del processo uscente e caricare quelli del processo entrante. Per la memoria, invece, il costo sarebbe elevato. Se a ogni cambio si dovesse copiare sul disco tutta la memoria del processo uscente e ricaricare quella dell'entrante, il calcolatore impiegherebbe tutto il tempo a trasferire dati. Tutti i processi devono quindi *risiedere insieme* in memoria, anche se uno solo per core viene eseguito in ogni istante.

Da qui nasce il problema. Ogni programma è stato compilato come se avesse tutta la memoria per sé, a partire dagli stessi indirizzi. Chiamiamo **indirizzi virtuali** quelli che il programma genera, con le sue `lw`, `sw` e con il PC, e **indirizzi fisici** quelli delle celle vere della memoria. L'insieme degli indirizzi virtuali di un processo è il suo *spazio di indirizzamento virtuale*. Serve un meccanismo che traduca gli uni negli altri, diverso per ogni processo, e che impedisca a un processo di leggere o scrivere la memoria degli altri.

e.2 Partizioni e registro base

La prima soluzione storica divide la memoria fisica in **partizioni** fisse, per esempio in due, quattro o otto parti, e assegna una partizione a ogni processo. Il processore contiene un **registro base**, caricato dal sistema operativo a ogni cambio di contesto con l'indirizzo iniziale della partizione del processo in esecuzione. A ogni accesso l'hardware somma il registro base all'indirizzo virtuale (figura e.1). Un secondo registro, il *limite*, contiene la dimensione della partizione: un indirizzo virtuale che la supera provoca un'eccezione, e il processo non può accedere al di fuori della propria partizione.

Nel caso più semplice, con la memoria divisa in due metà, i processi usano indirizzi virtuali da 31 bit, e il bit più alto dell'indirizzo fisico, determinato

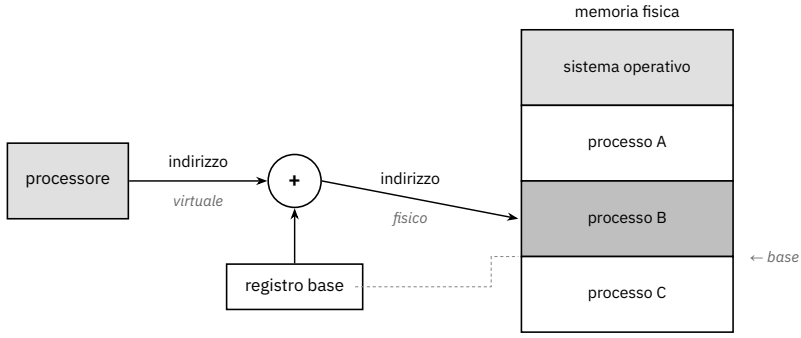


Figura e.1 Partizioni fisse e registro base. Il processo B è in esecuzione: il registro base contiene l'indirizzo iniziale della sua partizione, e ogni indirizzo virtuale viene rilocato in quella partizione.

dal registro base, seleziona la metà. Ma con centinaia di processi le partizioni fisse non sono adeguate: dividere la memoria in centinaia di parti uguali lascerebbe a ciascun processo troppo poco spazio, e ogni processo deve comunque risiedere interamente in memoria. Si può suddividere il programma in più frammenti, ognuno con il suo indirizzo base, e caricarli dove c'è posto; ma resta il limite di fondo: la somma dei programmi non può superare la memoria fisica.

e.3 Le pagine

La soluzione definitiva viene da un'analogia con la cache. Nella cache conservavamo solo i blocchi necessari, e prelevavamo gli altri dal livello inferiore quando mancavano. Applichiamo lo stesso principio un livello più in basso: lo spazio virtuale di ogni processo risiede, per intero, sul disco; la memoria principale ne contiene solo le parti in uso, e funge da cache per il disco.

Lo spazio virtuale e la memoria fisica si dividono in **pagine** della stessa dimensione, fissata dal sistema operativo e di solito di 4 KiB (a volte 8 o 16). Le pagine della memoria fisica si chiamano anche pagine fisiche (in inglese *frames*). Un indirizzo virtuale si divide in due parti: i bit bassi sono l'*offset* dentro la pagina, 12 per pagine da 4 KiB; i bit alti sono il **numero di pagina virtuale** (VPN, *virtual page number*). La traduzione sostituisce il VPN con il *numero di pagina fisica* (PPN, *physical page number*) e lascia l'offset com'è: dentro una pagina, i byte restano nello stesso ordine.

Dove è registrato quale pagina fisica corrisponde a una pagina virtuale? In una **tabella delle pagine** (*page table*), una per ogni processo, che il sistema operativo mantiene in memoria (figura e.2). La tabella è un vettore indicizza-

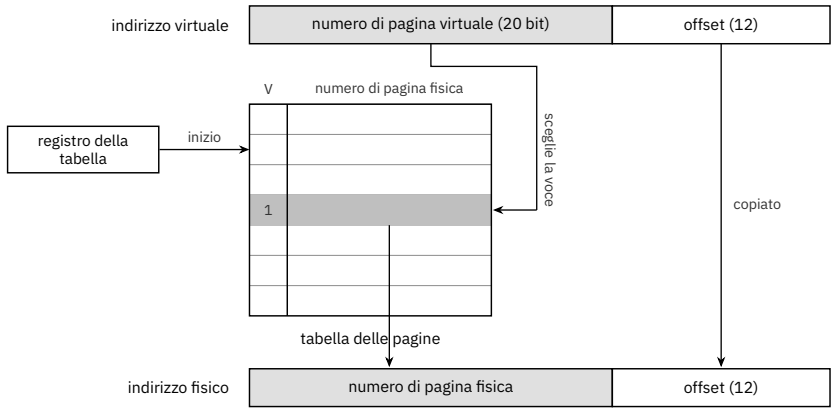


Figura e.2 La traduzione con una tabella delle pagine. Il numero di pagina virtuale seleziona una voce della tabella, che contiene il numero di pagina fisica; l'offset resta invariato.

to dal VPN; ogni voce contiene un bit di validità e, se la pagina è in memoria, il numero della pagina fisica. Se la pagina non è in memoria il bit di validità vale 0, e il sistema operativo sa dove trovarla sul disco. Un registro del processore, il *registro della tabella delle pagine*, contiene l'indirizzo fisico della tabella del processo in esecuzione; cambiare processo significa, per la memoria, cambiare il valore di questo registro.

Le pagine di un processo possono occupare pagine fisiche qualsiasi, non contigue, e anche non tutte in memoria. Due processi possono usare gli stessi indirizzi virtuali senza interferire, perché le loro tabelle li traducono in pagine fisiche diverse. E la somma dei loro spazi virtuali può superare di molto la memoria fisica, perché solo le pagine in uso devono essere presenti in memoria.

Una cache con proporzioni diverse

La memoria virtuale è una cache in cui la memoria principale funge da cache e il disco da memoria (tabella e.1). Le proporzioni, però, modificano ogni scelta. Un miss in cache costa centinaia di cicli; un **page fault**, cioè un accesso a una pagina che non è in memoria, costa una lettura dal disco: decine di microsecondi per un SSD, millisecondi per un disco magnetico, cioè da decine di migliaia a milioni di cicli. Con penalità così, ogni scelta di progetto mira a ridurre i fallimenti, anche a costo di complicare il resto.

- Le pagine sono grandi, perché il disco impiega molto ad avviare un trasferimento e poco a proseguirlo.

Tabella e.1 La cache e la memoria virtuale a confronto.

	Cache	Memoria virtuale
unità	blocco, 32–128 byte	pagina, 4 KiB o più
fallimento	miss	page fault
penalità	decine o centinaia di cicli	da decine di migliaia a milioni di cicli
posizionamento	diretto o associativo a insiemi	completamente associativo, tramite la tabella
scritture	write-through o write-back	sempre write-back
sostituzione	in hardware, LRU approssimato	in software, con il bit di accesso
chi lo gestisce	hardware	sistema operativo, aiutato dall'hardware

- Il posizionamento è *completamente associativo*: una pagina virtuale può essere collocata in qualunque pagina fisica, e i miss di conflitto spariscono. Non serve confrontare tutti i tag, perché la tabella indica direttamente dove si trova la pagina.
- Le scritture sono sempre *write-back*: scrivere sul disco a ogni `sw` sarebbe impensabile. Ogni pagina ha un bit dirty.
- La sostituzione è decisa dal software, cioè dal sistema operativo, che può permettersi algoritmi raffinati: qualche microsecondo per scegliere bene è trascurabile rispetto a millisecondi di disco.

Resta un problema di dimensioni. Con indirizzi virtuali da 32 bit e pagine da 4 KiB il VPN ha 20 bit: la tabella ha 2^{20} voci, più di un milione, e con voci da 4 byte occupa 4 MiB, per ogni processo, anche per un programma minuscolo. Vediamo come RISC-V risolve questo problema.

e.4 Le pagine in RISC-V: Sv32

Per i processori a 32 bit RISC-V definisce lo schema **Sv32**. Le pagine sono da 4 KiB, e il VPN da 20 bit si divide in due metà da 10 bit, VPN[1] e VPN[0] (figura e.3). La tabella delle pagine è un *albero a due livelli*. La tabella di primo livello ha $2^{10} = 1024$ voci da 4 byte, e occupa esattamente una pagina; VPN[1] seleziona una sua voce, che punta a una tabella di secondo livello, anch'essa di una pagina; VPN[0] seleziona in questa la voce che contiene il numero di pagina fisica. Una tabella di secondo livello descrive 1024 pagine, cioè 4 MiB di spazio virtuale; se un processo non usa nessun indirizzo di quei 4 MiB, la tabella non serve, e la voce di primo livello resta non valida. Un programma piccolo, con codice e dati in basso e lo stack in alto, richiede

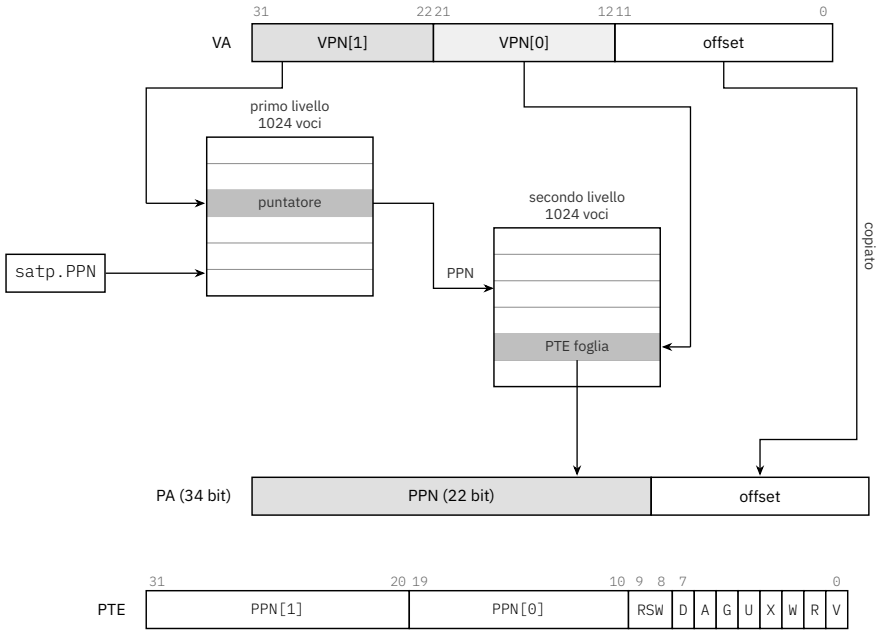


Figura e.3 La traduzione in Sv32. Il registro `satp` indica la tabella di primo livello; VPN[1] seleziona una voce che punta alla tabella di secondo livello, e VPN[0] seleziona in questa la voce foglia, con il numero di pagina fisica. In basso il formato di una voce (PTE).

soltanto la tabella di primo livello e due o tre di secondo: 12 o 16 KiB invece di 4 MiB.

La voce della tabella

Ogni voce, o PTE (*page table entry*), è una parola da 32 bit. I 22 bit alti sono un numero di pagina fisica: con 22 bit di PPN e 12 di offset, l'indirizzo fisico ha 34 bit, e un processore a 32 bit può indirizzare fino a 16 GiB di memoria fisica. I 10 bit bassi sono di controllo:

- **V** (*valid*): la voce è valida. Se vale 0, tutti gli altri bit sono a disposizione del sistema operativo, che può usarli per registrare, per esempio, dove si trova la pagina sul disco.
- **R**, **W**, **X**: la pagina si può leggere, scrivere, eseguire. Se valgono tutti e tre 0, la voce non descrive una pagina ma *punta* alla tabella del livello successivo.
- **U** (*user*): la pagina è accessibile dal livello utente. Le pagine del sistema operativo hanno U a 0.

- **G** (*global*): la traduzione è la stessa in tutti i processi, come per le pagine del sistema operativo.
- **A** (*accessed*) e **D** (*dirty*): la pagina è stata usata, o scritta, dall'ultima volta che il bit è stato azzerato. Sono il bit di riferimento e il bit dirty di cui parlavamo per la memoria virtuale (§ e.3). È l'hardware a porli a 1 durante la traduzione, oppure, se il processore adotta la variante più semplice, solleva un page fault e lascia che sia il sistema operativo ad aggiornarli; solo il software li azzera.
- **RSW**: due bit riservati al sistema operativo.

Il registro **satp** (*supervisor address translation and protection*) è il registro della tabella delle pagine. Il bit più alto, MODE, seleziona fra Sv32 e nessuna traduzione (*Bare*, il modo in cui abbiamo lavorato finora); 9 bit contengono un identificatore del processo, l'ASID, su cui torneremo; i 22 bit bassi sono il numero di pagina fisica della tabella di primo livello. È scrivibile solo dai livelli supervisor e machine: un programma utente non può cambiare la propria tabella.

Un esempio

Traduciamo l'indirizzo virtuale `0x00403abc`, con `satp`.PPN pari a `0x80000`. In binario

$$\underbrace{0000000001}_{\text{VPN[1]=1}}\ \underbrace{0000000011}_{\text{VPN[0]=3}}\ \underbrace{101010111100}_{\text{offset=0xabc}}$$

1. La tabella di primo livello inizia all'indirizzo fisico `0x80000` × 4096, cioè `0x80000000`. La voce numero 1 (VPN[1]) si trova quattro byte più avanti, a `0x80000004`. Supponiamo che contenga `0x20000401`: PPN `0x80001`, V = 1, R = W = X = 0. È un puntatore.
2. La tabella di secondo livello inizia quindi a `0x80001000`. La voce numero 3 (VPN[0]) si trova a `0x8000100c`. Supponiamo che contenga `0x048d14d7`: PPN `0x12345` e bit di controllo `0xd7`, cioè D, A, U, W, R e V. È una foglia: una pagina utente leggibile e scrivibile.
3. L'indirizzo fisico è il PPN seguito dall'offset: `0x12345abc`.

Si osservi che anche una voce di primo livello può essere una foglia, se ha R, W o X diversi da zero. In quel caso descrive direttamente una *megapagina* da 4 MiB, utile per grandi zone di memoria come il codice del sistema operativo. Nei processori a 64 bit gli schemi Sv39, Sv48 e Sv57 usano lo stesso principio con tre, quattro o cinque livelli.

e.5 Il page fault

Se durante la traduzione l'hardware trova una voce con V = 0, o una pagina che non consente l'accesso richiesto (una `sw` su una pagina senza W, un prelievo da una pagina senza X, un accesso utente a una pagina con U = 0), solleva un'eccezione di **page fault** (lo stesso accade, nella variante più semplice, se il bit A o il bit D devono essere posti a 1): codice 12 per il prelievo di un'istruzione, 13 per una load, 15 per una store (tabella b.2). Quando c'è un sistema operativo queste eccezioni vengono di solito consegnate direttamente al livello supervisor, che ha registri gemelli di quelli dello stato 0xb: `sepc`, `scause`, `stval`, e l'istruzione di ritorno `sret`. In `stval` il processore scrive l'indirizzo virtuale che ha causato il problema. Il sistema operativo:

1. controlla che l'indirizzo appartenga davvero al processo; in caso contrario, il processo contiene un errore e viene terminato (su Linux è il famoso *segmentation fault*);
2. cerca una pagina fisica libera; se non ce ne sono, ne sceglie una da liberare, preferendo quelle con il bit A a 0, e se ha D a 1 la riscrive sul disco;
3. avvia la lettura della pagina mancante dal disco e, durante l'attesa, esegue un altro processo;
4. quando la pagina è arrivata aggiorna la voce della tabella, con il nuovo PPN e V = 1;
5. torna al processo con `sret`, che riesegue l'istruzione interrotta: questa volta la traduzione riesce.

Il punto 5 è possibile solo perché l'eccezione è *precisa*, come abbiamo preteso nello stato 0xb: le istruzioni precedenti sono state completate, quella che ha causato il fault e le successive no, e il processo può ripartire come se l'eccezione non fosse mai avvenuta.

Molti sistemi caricano le pagine solo quando servono: all'avvio di un programma nessuna pagina è in memoria, e i primi istanti sono una raffica di page fault. È la *paginazione su richiesta*. Il meccanismo funziona finché i processi in esecuzione hanno in memoria le pagine che stanno usando. Se la memoria non basta, ogni processo, per caricare una sua pagina, ne espelle una necessaria a un altro, che a sua volta provocherà un page fault: il sistema trascorre quasi tutto il tempo in attesa del disco. È il **thrashing**, e il rimedio è eseguire meno processi insieme o aggiungere memoria.

1962 La memoria a un solo livello

Nel dicembre 1962 l'Università di Manchester e la Ferranti inaugurarono l'Atlas, progettato dal gruppo di Tom Kilburn. La sua memoria principale a nuclei magnetici, 16 384 parole, era divisa in 32 pagine da 512 parole; un tamburo magnetico ne conteneva circa centomila. Per ogni pagina della memoria a nuclei un registro indicava quale pagina del programma vi si trovasse, e per tradurre un indirizzo l'hardware confrontava il numero di pagina con tutti e 32 i registri insieme: una piccola memoria completamente associativa. Se nessuno corrispondeva, un'interruzione chiamava il sistema operativo, che portava la pagina dal tamburo, scegliendo quale sostituire mediante un programma che analizzava il comportamento dei cicli. Il programmatore vedeva un'unica memoria, grande quanto il tamburo: Kilburn la chiamò *one-level store*, memoria a un solo livello. Era la prima memoria virtuale.

e.6 Il TLB

C'è un problema di velocità. La tabella delle pagine risiede in memoria, e con Sv32 tradurre un indirizzo richiede due letture, una per livello. Ogni `lw` farebbe tre accessi alla memoria invece di uno, e lo stesso varrebbe per ogni prelievo di istruzione. Ancora una volta la soluzione è una cache, questa volta di traduzioni: il **TLB** (*translation lookaside buffer*).

Il TLB è una piccola cache dentro il processore, da qualche decina a qualche centinaio di voci, spesso completamente associativa. Ogni voce associa un numero di pagina virtuale, che funge da tag, a un numero di pagina fisica, e contiene una copia dei bit di controllo della PTE: V, R, W, X, U, G, A, D, oltre all'ASID del processo a cui appartiene. Non contiene né dati né istruzioni, solo corrispondenze fra pagine. Grazie alla località, una pagina da 4 KiB viene usata per molti accessi di seguito, e quasi tutte le traduzioni si trovano nel TLB.

Quando la traduzione non è nel TLB (figura e.4) bisogna leggere la tabella delle pagine e copiare la voce nel TLB. Quale componente esegue questa operazione? Dipende dall'architettura.

- Nei testi classici l'esempio è il MIPS, in cui il miss del TLB è un'eccezione gestita dal software. Una breve procedura del sistema operativo, con un indirizzo di ingresso riservato a questa eccezione, legge in un registro speciale (*Context*) l'indirizzo della voce necessaria, la carica con una `lw`, la scrive nel TLB con un'istruzione privilegiata (`tlbwr`) e torna al programma (`eret`). In tutto, da una decina a qualche decina di cicli.
- In RISC-V il formato della tabella è fissato dall'architettura proprio perché la visita possa essere eseguita da un circuito, il *page table walker*: legge le

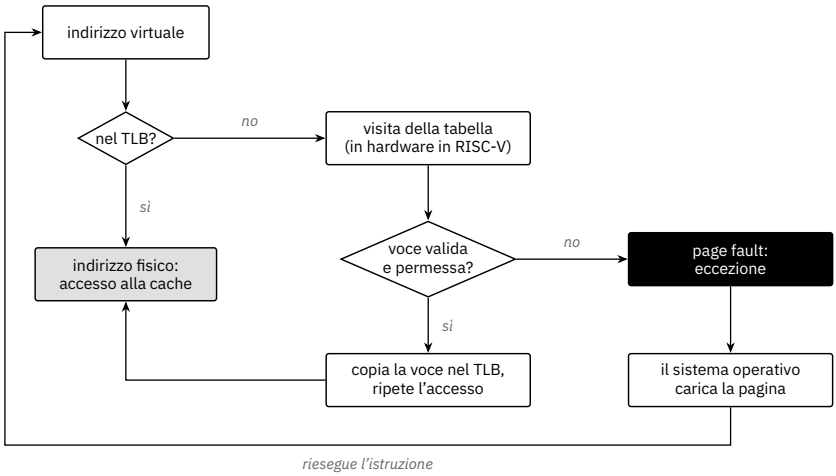


Figura e.4 Le operazioni eseguite a ogni accesso alla memoria. Il percorso a sinistra, il più frequente, è tutto in hardware, e la traduzione richiede al più un ciclo; la visita della tabella ne richiede qualche decina; il page fault milioni.

due voci, controlla i permessi, aggiorna il TLB e ripete l'accesso, senza eccezioni e senza software. La specifica nota che il riempimento software del TLB è un collo di bottiglia nei processori veloci; un'implementazione può comunque adottarlo, con un gestore al livello machine.

In entrambi i casi il sistema operativo entra in gioco solo se la pagina è effettivamente assente o l'accesso non è permesso: allora la visita si conclude con un page fault. Anche le letture delle voci passano per le cache, e le voci usate spesso si trovano lì.

Un compito, tuttavia, resta a carico del software. Il TLB è una copia della tabella, e se il sistema operativo modifica una voce, per esempio togliendo una pagina a un processo, la copia nel TLB diventa obsoleta. L'istruzione privilegiata `sfence.vma` svuota il TLB, tutto o solo le voci di un indirizzo o di un processo, e va eseguita dopo ogni modifica della tabella. Per non doverlo svuotare a ogni cambio di processo, le voci del TLB sono etichettate con l'ASID del processo, scritto in `satp`: le traduzioni di processi diversi possono convivere, e valgono solo quelle con l'ASID corrente. Le voci con il bit G valgono per tutti i processi.

Nehalem, per fare un esempio reale, ha due livelli di TLB, come per le cache: per i dati 64 voci a 4 vie, per le istruzioni 64 voci per thread, e un secondo livello comune da 512 voci. Un miss nel primo livello che trova la traduzione nel secondo costa 7 cicli.

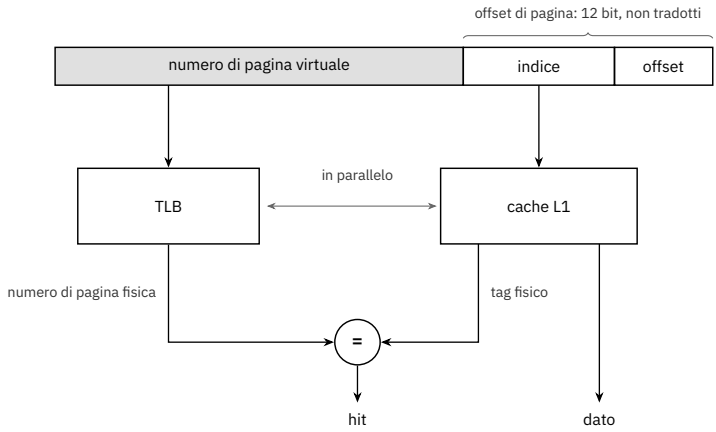


Figura e.5 Una cache indicizzata virtualmente ed etichettata fisicamente. L'indice deriva dall'offset di pagina, che non cambia nella traduzione: cache e TLB operano in parallelo, e alla fine si confronta il tag della cache con il numero di pagina fisica.

e.7 Il TLB e la cache insieme

A ogni accesso servono il TLB e la cache. In che ordine? Se la cache usa indirizzi fisici, bisogna prima tradurre e poi cercare nella cache: due passi in serie, e il tempo di hit cresce. Se la cache usa indirizzi virtuali, si evita la traduzione, ma sorgono due problemi. Lo stesso indirizzo virtuale, in due processi diversi, indica dati diversi: la cache potrebbe consegnare a un processo i dati di un altro, a meno di svuotarla a ogni cambio di processo. E la stessa pagina fisica, condivisa da due processi, può avere due indirizzi virtuali diversi, ed essere presente due volte nella cache: si chiama *alias*, e se un processo scrive in una copia l'altra diventa obsoleta.

La soluzione usata nelle cache L1 è un compromesso elegante: la cache **VIPT**, *indicizzata virtualmente, etichettata fisicamente* (figura e.5). L'offset dentro la pagina non cambia nella traduzione. Se l'indice e l'offset del blocco sono interamente contenuti nei 12 bit dell'offset di pagina, la cache può cominciare a leggere la riga mentre il TLB traduce il numero di pagina; quando la traduzione è disponibile, il numero di pagina fisica si confronta con il tag letto dalla cache. Le due ricerche avvengono in parallelo, e il tag fisico elimina ogni equivoco.

C'è un vincolo. Con pagine da 4 KiB, indice e offset del blocco insieme non possono superare 12 bit: ogni via della cache può contenere al massimo 4 KiB. Per avere una L1 più grande si aggiungono vie, non righe. La cache dati di Nehalem ha 32 KiB a 8 vie: 4 KiB per via, esattamente una pagina. Non è un

Tabella e.2 Le combinazioni di TLB, tabella delle pagine e cache.

TLB	Tabella	Cache	Possibile? Quando?
hit	hit	hit	sì: il caso migliore e più frequente
hit	hit	miss	sì: la pagina è in memoria, il blocco non è in cache
miss	hit	hit	sì: la traduzione è uscita dal TLB, il dato è ancora in cache
miss	hit	miss	sì: si visita la tabella, poi si va in memoria
miss	miss	miss	sì: è il page fault
hit	miss	–	no: una traduzione nel TLB implica la pagina in memoria
miss	miss	hit	no: un dato in cache implica la pagina in memoria

caso. (La cache delle istruzioni, 32 KiB a 4 vie, non rispetta la regola: ma le istruzioni non vengono scritte, e due copie dello stesso blocco non possono mai essere diverse.) La specifica RISC-V va oltre: richiede che, comunque sia realizzata la cache, gli alias non diventino mai visibili ai programmi.

Le combinazioni possibili

Un accesso può trovare o no la traduzione nel TLB, la pagina in memoria, il dato nella cache. Non tutte le combinazioni sono possibili (tabella e.2): un dato non può essere nella cache, e una traduzione non può essere nel TLB, se la pagina non è in memoria, perché quando il sistema operativo rimuove una pagina dalla memoria la rimuove anche dal TLB e dalla cache.

e.8 Protezione e condivisione

La memoria virtuale non serve soltanto a mantenere più processi in memoria: li isola anche l'uno dall'altro. Un processo può generare solo indirizzi virtuali, e la sua tabella li traduce solo nelle sue pagine fisiche: la memoria degli altri processi, per esso, non ha nemmeno un nome. Le tabelle stanno in pagine con U = 0, a cui il livello utente non può accedere, e `satp` è scrivibile solo dai livelli supervisor e machine. Nessun programma utente può quindi cambiare le proprie traduzioni.

I bit R, W e X affinano la protezione dentro un processo. Le pagine del codice sono leggibili ed eseguibili ma non scrivibili, e un bug non può modificare il programma; le pagine dei dati e dello stack sono leggibili e scrivibili ma non eseguibili, e un attacco che riesca a scrivere istruzioni in un vettore non può farle eseguire. Le pagine del sistema operativo hanno U = 0: di solito

sono mappate in ogni processo, per rendere rapide le chiamate di sistema, ma il programma utente non le può leggere.

La condivisione è altrettanto semplice: basta che le tabelle di due processi abbiano voci che puntano alla stessa pagina fisica. Così le librerie usate da tutti i programmi, come la libreria del C, sono presenti in memoria in un'unica copia, in pagine di sola lettura condivise da tutti i processi.

e.9 Il viaggio di una `lw`

Combiniamo i contenuti degli stati 0xd e 0xe seguendo una `lw` su un processore come Nehalem.

1. Nella fase EX la `lw` calcola un indirizzo virtuale; in MEM i bit dell'offset di pagina sono inviati subito alla cache L1 dei dati, che legge le otto vie dell'insieme; intanto il numero di pagina è inviato al TLB. Con un hit in entrambi, il dato arriva in 4 cicli.
2. Se il blocco non è in L1, la richiesta, ormai con l'indirizzo fisico, è inoltrata alla L2 (circa 10 cicli), poi alla L3 (35–40), poi alla DRAM (qualche centinaio di cicli, compresi il RAS e il CAS dello stato 0xd).
3. Se la traduzione non è nel TLB, l'hardware visita la tabella delle pagine, le cui voci a loro volta passano per le cache.
4. Se la pagina non è in memoria, page fault: il sistema operativo sospende il processo, legge la pagina dal disco ed esegue altri processi per qualche milione di cicli, poi riprende la `lw` da capo.

Dal primo caso all'ultimo ci sono sei ordini di grandezza. Eppure in media una `lw` costa poco più di un hit in L1, perché la località fa sì che il primo caso sia di gran lunga il più frequente. Tutta la gerarchia è una scommessa sulla località, e il comportamento dei programmi la rende vincente quasi sempre.

1968 L'insieme di lavoro

Negli anni Sessanta i primi sistemi a memoria virtuale avevano un comportamento misterioso: aggiungendo un processo, a volte le prestazioni crollavano all'improvviso, e il calcolatore sembrava fermo pur essendo pienamente occupato. Nel 1968 Peter Denning, dottorando al MIT dove si costruiva il sistema operativo Multics, pubblicò il *modello dell'insieme di lavoro* (*working set*): le pagine usate da un processo in un intervallo di tempo recente. Se la somma degli insiemi di lavoro dei processi attivi supera la memoria, i processi si sottraggono le pagine a vicenda e il sistema crolla nel thrashing. Lo stesso anno Denning ne spiegò le cause e il rimedio: sospendere qualche processo finché gli altri non hanno di nuovo spazio.

Condizione di transizione uscita verso `0xf`

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Perché tutti i processi devono risiedere insieme in memoria, anche se ne viene eseguito uno solo per core? ↺ § e.1

2 Quali limiti hanno le partizioni fisse con registro base? ↺ § e.2

3 Con indirizzi virtuali da 32 bit e pagine da 4 KiB, quanti bit hanno offset e VPN? Quanto occuperebbe una tabella a un solo livello, con voci da 4 byte? ↺ § e.3

4 Perché la memoria virtuale è completamente associativa e sempre write-back? ↺ § e.3

5 In Sv32, quanto valgono VPN[1], VPN[0] e offset dell'indirizzo `0x00801ffc`? ↺ § e.4

6 Che cosa indica una PTE con V = 1 e R = W = X = 0? ↺ § e.4

7 Perché il page fault deve essere un'eccezione precisa? ↺ § e.5

8 Senza TLB, quanti accessi alla memoria esegue una `lw` in Sv32? ↺ § e.6

9 Con pagine da 4 KiB, quanto può essere grande una cache VIPT a mappatura diretta? E una a 8 vie? ↺ § e.7

10 Perché un dato non può essere in cache se la sua pagina non è in memoria? ↺ § e.7

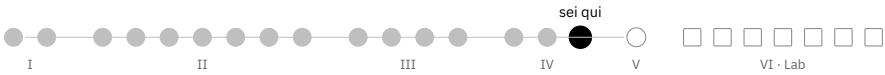
11 Come può il sistema operativo condividere una libreria fra due processi senza che uno dei due la modifichi? ↺ § e.8

STATO

0xf

1	1	1	1
b3	b2	b1	b0

Più core: coerenza e sincronizzazione

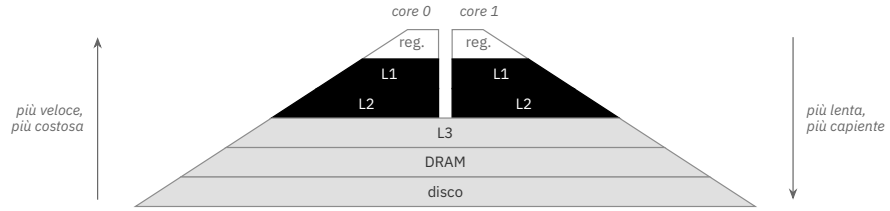


→ INGRESSI

- La fine della corsa alla frequenza e il multithreading (stato 0xc).
- Cache write-back, bit dirty, cache a più livelli e il controllore come macchina a stati (stato 0xd).
- Le estensioni di RISC-V, fra cui la A (stato 0x1, § 1.12).

USCITE →

- Stimare con la legge di Amdahl il guadagno di un programma su più core.
- Spiegare perché le cache private rendono incoerente la memoria, e distinguere coerenza e consistenza.
- Seguire una sequenza di accessi con il protocollo MESI.
- Riconoscere la falsa condivisione.
- Scrivere un aggiornamento atomico e un lucchetto con le istruzioni atomiche di RISC-V.



Nello stato 0xc abbiamo visto la corsa alla frequenza fermarsi contro il muro dei consumi, intorno al 2004. Da allora i transistor in più che ogni

generazione mette a disposizione sono stati spesi soprattutto in un altro modo: più processori completi, i **core**, sullo stesso chip. Due core non rendono più veloce un programma scritto per uno, ma ne possono eseguire due insieme, oppure eseguire insieme due parti dello stesso programma. Questo stato quantifica il guadagno ottenibile e descrive i due problemi che sorgono quando più core condividono la memoria: mantenere coerenti le loro cache e coordinare i programmi senza che interferiscano fra loro.

f.1 Più processori sullo stesso chip

Un processore multicore è un *multiprocessore a memoria condivisa*: tutti i core vedono la stessa memoria, con gli stessi indirizzi fisici. Ogni core ha le sue cache di primo livello, e spesso anche la L2, mentre l'ultimo livello e la DRAM sono comuni, come in Nehalem (figura d.6). Se più chip sono collegati fra loro, ognuno con la sua DRAM, la memoria resta unica ma il tempo di accesso dipende da dove si trova il dato: in un sistema con due Nehalem un accesso alla DRAM del proprio chip richiede circa 65 ns, uno alla DRAM dell'altro chip circa 105. Si parla allora di accesso *non uniforme* alla memoria (NUMA).

Più core si possono sfruttare in due modi. Il primo è la multiprogrammazione dello stato 0xe: processi indipendenti, ognuno con le sue pagine, eseguiti davvero insieme invece che a turno. Il secondo è un programma *parallelo*, diviso in più **thread**: flussi di esecuzione che appartengono allo stesso processo e quindi condividono il codice e la memoria, ma hanno ciascuno il proprio PC, i propri registri e il proprio stack. Passare da un thread a un altro dello stesso processo costa meno che cambiare processo, perché la tabella delle pagine resta la stessa. Sono i thread che il multithreading dello stato 0xc esegue sullo stesso core; su più core vengono eseguiti in parallelo, e ogni core può a sua volta eseguire più thread.

f.2 La legge di Amdahl

Con N core, un programma diventa N volte più veloce? Quasi mai. Supponiamo che una frazione f del tempo di esecuzione si possa dividere fra i core, e che il resto, $1-f$, debba essere eseguito da un solo core: leggere i dati, combinare i risultati, le parti intrinsecamente sequenziali. Con N core la parte parallela si riduce di un fattore N, l'altra no. L'accelerazione è

$$\text{accelerazione} = \frac{1}{(1-f)+f/N}.$$

È la **legge di Amdahl**. Se il 90% del tempo è parallelizzabile, con 8 core l'accelerazione è $1/(0{,}1+0{,}1125) \approx 4{,}7$; con 100 core è $1/(0{,}1+0{,}009) \approx 9{,}2$. E

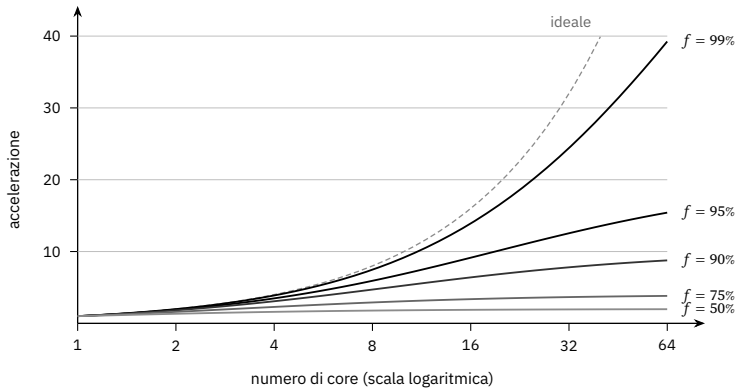


Figura f.1 La legge di Amdahl: l'accelerazione al crescere dei core, per diverse frazioni parallelizzabili f del tempo. Con $f = 90\%$, 64 core producono un'accelerazione inferiore a 9.

con infiniti core non supera mai $1/(1-f)$, cioè 10: il 10% sequenziale da solo impiega già un decimo del tempo originale (figura f.1).

La legge si può applicare anche in senso inverso. Supponiamo di voler rendere un programma 90 volte più veloce su 100 core: quale frazione può restare sequenziale? Da $90 = 1/((1-f)+f/100)$ si ricava $f \approx 0{,}9989$: solo lo 0,1% del tempo può essere sequenziale. Per questo scrivere programmi paralleli efficienti è difficile, e i guadagni maggiori si ottengono con problemi grandi, in cui la parte parallela cresce con i dati mentre quella sequenziale resta più o meno la stessa: è l'osservazione con cui John Gustafson, nel 1988, rilesse la legge di Amdahl in chiave ottimista.

1967 Una legge senza formula

Nell'aprile del 1967, a una conferenza ad Atlantic City, Gene Amdahl, che abbiamo incontrato nello stato 0x0 fra i progettisti del System/360, presentò un articolo di tre pagine per difendere i calcolatori a un solo processore veloce contro chi proponeva di collegarne molti in parallelo. Osservava che nei programmi reali circa il 40% delle istruzioni era dedicato a lavori di servizio sui dati, difficili da dividere, e stimava che, anche affidando quel lavoro a un processore a parte, il guadagno non sarebbe andato oltre le cinque o sette volte. L'articolo non contiene nemmeno un'equazione: la formula che porta il suo nome è stata scritta dopo, da altri, riassumendo il suo ragionamento.

Tabella f.1 Due cache write-back senza alcun protocollo: dopo il passo 3 il core 1 e la memoria vedono un valore obsoleto di `X`.

Passo	Evento	Cache del core 0	Cache del core 1	Memoria
0		–	–	10
1	il core 0 legge `X`	10	–	10
2	il core 1 legge `X`	10	10	10
3	il core 0 scrive 20	20	10	10
4	il core 1 legge `X`	20	**10**	10

f.3 Il problema della coerenza

Le cache private funzionano in modo ottimale finché ogni core opera su dati propri. I problemi sorgono con i dati condivisi. Supponiamo che la variabile `X` valga 10 in memoria, e che le cache siano write-back (tabella f.1). Il core 0 legge `X` e ne carica una copia nella propria cache; il core 1 esegue la stessa operazione. Poi il core 0 scrive 20 in `X`: la scrittura resta nella sua cache. Se il core 1 rilegge `X` trova nella sua cache il vecchio 10, e anche una lettura diretta della memoria restituirebbe il valore vecchio. Con cache write-through la memoria sarebbe aggiornata, ma la copia nella cache del core 1 resterebbe comunque obsoleta.

Il problema si presenta anche senza programmi paralleli. Il sistema operativo sospende un processo che era in esecuzione sul core 0, e quando lo riprende lo assegna al core 1: i dati scritti dal processo sono ancora nella cache del core 0, e il core 1 deve vederli.

Una memoria si dice **coerente** se, per ogni singola locazione:

1. un core che scrive `X` e poi lo rilegge, senza scritture di altri nel frattempo, legge il valore che ha scritto;
2. una scrittura di un core diventa prima o poi visibile alle letture degli altri core;
3. le scritture sulla stessa locazione sono *serializzate*: tutti i core le vedono nello stesso ordine, e nessuno può vedere 20 e poi 10 se l'ordine è stato 10 e poi 20.

La **consistenza** è un'altra cosa. Riguarda locazioni *diverse*: se un core scrive prima `A` e poi `B`, un altro core che vede il nuovo `B` vedrà anche il nuovo `A`? In altre parole: *quando* una scrittura diventa visibile, e in che ordine rispetto alle altre. La coerenza garantisce che tutti i core concordino sul valore di ogni variabile; la consistenza, che si chiama anche *modello di memoria*, stabilisce in quale ordine le modifiche diventano visibili. Torneremo sulla consistenza nella § f.7.

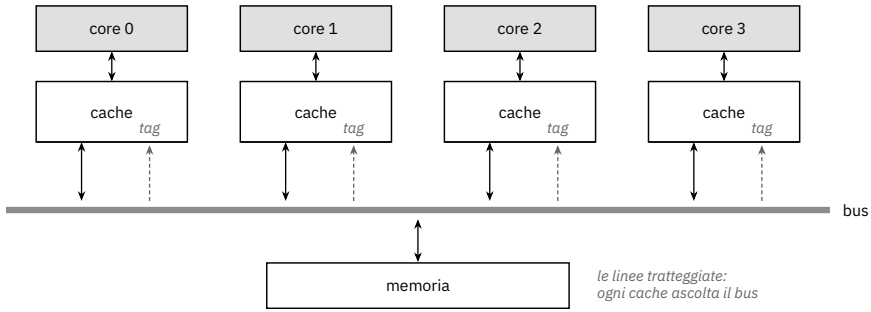


Figura f.2 Coerenza a snooping: tutte le richieste passano su un mezzo comune, e ogni controllore confronta gli indirizzi che vede passare con i propri tag. Spesso i tag sono duplicati, perché l'osservazione del bus non rallenti gli accessi del core.

f.4 Osservare il bus

Un protocollo di coerenza deve rispondere a due domande: come si determina quali cache hanno una copia di un blocco, e come si gestiscono le copie quando un core scrive.

Alla prima domanda ci sono due risposte. Nei protocolli a **snooping** (*to snoop*, curiosare) le cache sono collegate a un mezzo comune, un bus, su cui ogni richiesta viene trasmessa a tutti; ogni controllore osserva tutte le richieste degli altri e verifica nei propri tag se possiede il blocco (figura f.2). È semplice ed efficace con pochi core, ma ogni richiesta coinvolge tutte le cache, e il traffico cresce con il numero dei core. Nei protocolli a *directory* una tabella, la directory, registra per ogni blocco quali cache ne hanno una copia, e le richieste vengono inviate solo a quelle. Costa memoria, ma scala molto meglio.

Alla seconda domanda, come comportarsi quando un core scrive, ci sono di nuovo due risposte. Con l'*aggiornamento* (*write update*) il core che scrive invia il nuovo valore sul bus, e tutte le copie vengono aggiornate. Con l'**invalidazione** (*write invalidate*) il core che scrive invia sul bus solo l'indirizzo, con un segnale di invalidazione: gli altri controllori marcano come non valida la loro copia. Se poi un altro core deve leggere il dato, subisce un miss e riceve il valore nuovo. L'invalidazione è la scelta di quasi tutti i processori: se un core scrive più volte lo stesso blocco, invalida le altre copie una volta sola, mentre l'aggiornamento trasmetterebbe ogni scrittura.

f.5 Il protocollo MESI

Nello stato 0xd il controllore di una cache write-back teneva per ogni riga due bit, V e dirty. Con più core serve un'informazione in più: se la copia è l'unica o se ne esistono altre in altre cache. Il protocollo più diffuso, **MESI**, proposto nel 1984 all'Università dell'Illinois, assegna a ogni riga uno di quattro stati, che danno il nome al protocollo:

- **M** (*modified*): la riga è valida solo in questa cache ed è stata modificata; la memoria contiene un valore obsoleto.
- **E** (*exclusive*): la riga è valida solo in questa cache e coincide con la memoria.
- **S** (*shared*): la riga coincide con la memoria e potrebbe essere anche in altre cache.
- **I** (*invalid*): la riga non contiene un dato valido.

Ogni riga è governata da una macchina a stati che reagisce a due tipi di eventi: le letture e scritture del proprio core, e le richieste degli altri core viste sul bus (figura f.3).

Seguiamo le transizioni più importanti.

- *Lettura con miss*. Il controllore richiede il blocco sul bus. Se nessun'altra cache lo ha, il blocco arriva dalla memoria e la riga passa in E; se altre cache lo hanno, passa in S, e le eventuali copie in E passano anch'esse in S. Se un'altra cache lo ha in M, è quella cache a fornire il blocco, che nel frattempo viene riscritto in memoria, e anche la sua copia passa in S.
- *Scrittura*. In M la riga non cambia stato. In E la riga passa in M *in silenzio*, senza traffico sul bus: è il vantaggio dello stato E, perché i dati privati di un thread, letti e poi scritti, non generano traffico verso le altre cache. In S il controllore deve prima invalidare le altre copie con una richiesta sul bus, poi passa in M. In I chiede il blocco con l'intenzione di scriverlo: le altre copie vengono invalidate, e la riga arriva direttamente in M.
- *Richieste degli altri*. Una scrittura altrui porta in I qualunque copia; se la copia era in M, deve prima essere fornita al core che scrive. Una lettura altrui porta in S le copie in E e in M, e quelle in M vengono fornite e riscritte in memoria.

Rifacciamo con MESI la sequenza della tabella f.1 (tabella f.2). Al passo 3 il core 0 ha la riga in S, e prima di scrivere invalida la copia del core 1. Al passo 4 il core 1 subisce un miss; sul bus il core 0 riconosce l'indirizzo di una sua riga in M, fornisce il valore 20 e aggiorna la memoria, ed entrambe le copie passano in S.

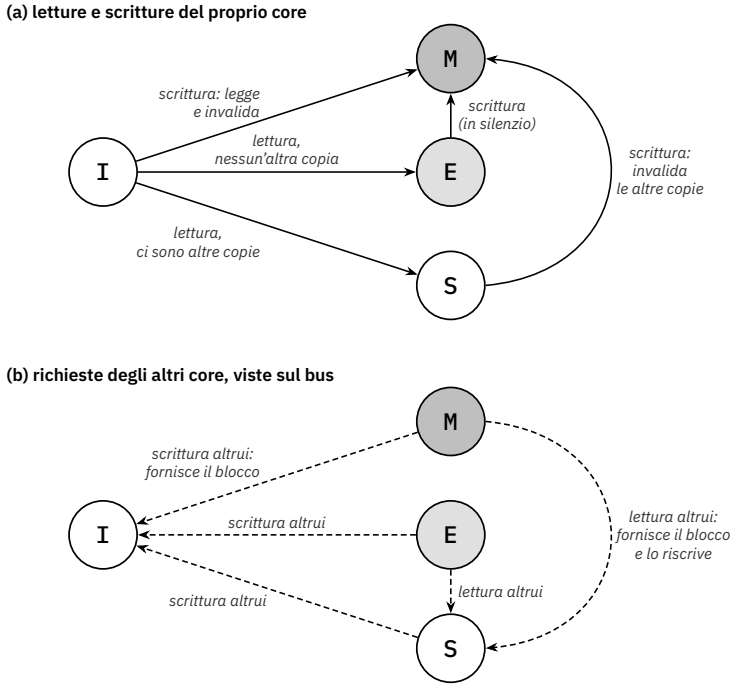


Figura f.3 Il protocollo MESI, diviso in due parti. In (a) le transizioni causate dal proprio core, in (b) quelle causate dalle richieste degli altri, osservate sul bus. Le letture in M, E e S e le scritture in M non cambiano stato e non generano traffico. Una riga in M che viene sostituita va riscritta in memoria; in E o in S si può semplicemente scartare.

f.6 La falsa condivisione

La coerenza opera a livello di blocco, non di variabile. Due variabili diverse che si trovano nello stesso blocco da 64 byte sono, per il protocollo, la stessa cosa. Consideriamo due thread che incrementano ciascuno il proprio contatore, milioni di volte:

```
struct { long a; long b; } conta;     // a e b: stesso blocco
// thread 0:  for (...) conta.a++;
// thread 1:  for (...) conta.b++;
```

I due thread non condividono nessun dato, eppure ogni scrittura del thread 0 invalida il blocco nella cache del thread 1, e viceversa: il blocco viene trasferito da un core all'altro a ogni incremento, e ogni volta è un miss. Il programma parallelo può risultare più lento di quello sequenziale. È la **falsa condivisione** (*false sharing*): i miss sono di coerenza, una quarta C che si aggiunge alle tre

Tabella f.2 La sequenza della tabella f.1 con il protocollo MESI. Fra parentesi lo stato delle righe.

Passo	Evento	Sul bus	Core 0	Core 1	Memoria
1	il core 0 legge x	lettura	10 (E)	– (I)	10
2	il core 1 legge x	lettura	10 (S)	10 (S)	10
3	il core 0 scrive 20	invalidazione	20 (M)	– (I)	10
4	il core 1 legge x	lettura, fornita dal core 0	20 (S)	20 (S)	20

dello stato 0xd, ma la condivisione è solo apparente. Il rimedio è collocare i due contatori in blocchi diversi, allineando ciascuno a 64 byte o separandoli con byte di riempimento.

f.7 Sincronizzazione

La coerenza garantisce che ogni lettura e ogni scrittura, prese una per una, vedano valori sensati. Non garantisce che una *sequenza* di accessi avvenga senza interferenze. Due thread incrementano lo stesso contatore, partendo da 0, con il codice più immediato:

```
        lw   t0, 0(a0)        # legge il contatore
        addi t0, t0, 1
        sw   t0, 0(a0)        # lo riscrive
```

Se i due thread leggono entrambi 0 prima che uno dei due abbia scritto, entrambi scrivono 1: un incremento è andato perso, e il risultato dipende da come si intrecciano le istruzioni. È una **corsa critica** (*race condition*): un errore che si manifesta solo in alcune esecuzioni, ed è fra i più difficili da individuare.

Istruzioni atomiche

Serve un'operazione che legga, modifichi e riscriva un dato in un unico passo *atomico*, cioè indivisibile: nessun altro core può accedere al dato fra la lettura e la scrittura. È il compito dell'estensione A di RISC-V, che ne offre di due tipi.

Le **AMO** (*atomic memory operations*) eseguono l'intera operazione in un'unica istruzione. `amoadd.w t0, t1, (a0)` legge la parola all'indirizzo `a0`, la copia in `t0`, e scrive in memoria la somma con `t1`, il tutto atomicamente. Il contatore diventa:

```
        li       t1, 1
        amoadd.w zero, t1, (a0)   # contatore += 1, in un colpo solo
```

Un modo semplice di realizzare un'AMO: il controllore trasferisce il blocco nella propria cache in stato M e lo trattiene finché l'operazione non è completata, e il protocollo di coerenza garantisce il resto. Con `amoswap.w`, che scambia un registro con una parola di memoria, si costruisce un **lucchetto** (*lock*), che protegge una *sezione critica*: una porzione di codice che un solo thread alla volta deve eseguire. Ecco l'esempio proposto dalla specifica, con il lucchetto all'indirizzo in `a0` (0 libero, 1 occupato):

```
        li           t0, 1
attesa: lw           t1, 0(a0)       # il lucchetto è occupato?
        bnez         t1, attesa      # se sì, riprova
        amoswap.w.aq t1, t0, (a0)    # prova a prenderlo: scrive 1
        bnez         t1, attesa      # era già preso: riprova
        # ... sezione critica ...
        amoswap.w.rl zero, zero, (a0)  # libera il lucchetto: scrive 0
```

La `amoswap` scrive 1 e restituisce il valore precedente: se era 0, il lucchetto era libero e ora è acquisito; se era 1, un altro thread lo deteneva già, e il tentativo viene ripetuto. La `lw` prima della `amoswap` sembra superflua, ma non lo è: mentre il lucchetto è occupato, i thread in attesa leggono la propria copia in stato S, senza traffico sul bus; provano la `amoswap`, che è una scrittura e invalida le copie altrui, solo quando il lucchetto sembra libero.

Il secondo tipo è la coppia `lr.w` (*load reserved*) e `sc.w` (*store conditional*). La `lr.w` legge una parola e *prenota* l'indirizzo; la `sc.w` scrive solo se la prenotazione è ancora valida, cioè se nessun altro ha scritto nel frattempo, e scrive in un registro 0 se l'operazione è riuscita e un valore diverso da zero in caso contrario. Fra le due si può inserire un breve calcolo, con poche istruzioni intere, senza altri accessi alla memoria né salti all'indietro: se un altro core ha scritto nel frattempo, la sequenza viene ripetuta. Anche in questo caso è il protocollo di coerenza a svolgere il lavoro: la prenotazione viene cancellata quando il blocco viene invalidato da una scrittura altrui. Questo è lo scambio condizionato (*compare and swap*) della specifica: scrive `a2` all'indirizzo `a0` solo se lì c'è ancora il valore atteso `a1`.

```
cas:    lr.w  t0, (a0)        # legge e prenota
        bne   t0, a1, fallito # non è il valore atteso
        sc.w  t0, a2, (a0)    # prova a scrivere
        bnez  t0, cas         # prenotazione persa: riprova
        li    a0, 0           # riuscito
        ret
fallito: li   a0, 1
        ret
```

L'ordine delle scritture

Resta la consistenza. Per ottenere prestazioni elevate, i processori non rendono sempre visibili gli accessi alla memoria nell'ordine del programma. Una store finisce in un buffer e raggiunge la cache più tardi, mentre le load successive, dirette ad altri indirizzi, la sorpassano; i processori con esecuzione fuori ordine dello stato 0xc riordinano anche le load. In alcune architetture perfino due store possono diventare visibili in ordine inverso, per esempio se la prima genera un miss in cache e la seconda un hit. Gli x86 come Nehalem rendono visibili le store sempre nell'ordine del programma; RISC-V non lo garantisce. Per un solo thread non cambia nulla, perché il processore controlla che una load veda le store precedenti allo stesso indirizzo. Consideriamo però un thread che prepara un dato e poi imposta una bandierina, e a un altro che attende la bandierina per leggere il dato. Se la scrittura della bandierina diventa visibile prima di quella del dato, il secondo thread legge un dato obsoleto.

Il modello di memoria di RISC-V, chiamato RVWMO (*weak memory ordering*), è appunto debole: permette questi riordini, e chiede al programmatore di indicare esplicitamente quando l'ordine è rilevante. L'istruzione **fence** impone che gli accessi precedenti siano visibili prima di quelli successivi:

```
# thread 0: produce
    sw    t1, 0(a0)  # dato
    fence w, w
    li    t2, 1
    sw    t2, 0(a1)  # bandierina
```

```
# thread 1: consuma
attesa: lw    t2, 0(a1)
        beqz  t2, attesa
        fence r, r
        lw    t3, 0(a0)  # dato
```

I suffissi `.aq` (*acquire*) e `.rl` (*release*) delle istruzioni atomiche, che abbiamo usato nel lucchetto, ottengono lo stesso effetto in modo mirato: nessun accesso della sezione critica può essere anticipato prima della `amoswap.w.aq` che acquisisce il lucchetto, né ritardato dopo la `amoswap.w.rl` che lo rilascia. In pratica queste istruzioni sono generate dalle librerie e dai compilatori; ma sapere che esistono spiega perché i programmi paralleli scritti senza le primitive di sincronizzazione falliscono in modi apparentemente inspiegabili.

2001 **Due core in un chip**

Il primo processore per calcolatori di uso generale con due core completi sullo stesso pezzo di silicio fu l'IBM POWER4, del 2001, destinato ai server: 174 milioni di transistor, frequenze fra 1,1 e 1,3 GHz, una cache L2 da circa 1,4 MiB condivisa dai due core. Per anni i processori a più core rimasero confinati nelle macchine costose. Poi, intorno al 2005, i limiti di consumo che abbiamo visto nello stato 0xc spinsero anche Intel e AMD a integrare due core nei processori per personal computer, e il numero non ha più smesso di crescere.

f.8 La coerenza in Nehalem

Chiudiamo con il processore che ci accompagna dallo stato 0xc. Nehalem usa una variante di MESI con un quinto stato, **MESIF**. Lo stato F (*forward*) è un caso particolare di S: quando un blocco è condiviso da più cache, una sola lo mantiene in F, ed è quella cache a rispondere alle richieste di lettura, invece di rispondere tutte o di coinvolgere la memoria. Quando un'altra cache legge il blocco, lo stato F passa a quella cache, l'ultima arrivata.

Il protocollo è a snooping, con un aiuto che lo avvicina a una directory: la L3 inclusiva dello stato 0xd. Poiché ogni blocco presente nelle L1 e nelle L2 è anche in L3, ogni blocco della L3 ha quattro bit, uno per core, che indicano quali core potrebbero averne una copia. Se il bit di un core è a 0, quel core sicuramente non ha il blocco, e non è necessario interrogarlo. E se il blocco non è affatto in L3, non è in nessuna cache del chip. Le richieste che arrivano dall'altro processore, attraverso il collegamento QPI, possono così essere risolte dalla L3 senza interrogare le cache dei core. Quando un core deve scrivere un blocco in stato S invia una *richiesta di proprietà* (*request for ownership*) che invalida le altre copie, esattamente come nella figura f.3.

La parte IV si chiude qui. Abbiamo visto la memoria trasformarsi da scatola ideale che risponde in un ciclo a gerarchia di cache, DRAM, disco, tabelle di traduzione e protocolli di coerenza. Manca un ultimo pezzo per avere un calcolatore completo: il modo in cui il processore comunica con il mondo esterno. È lo stato 0x10, il primo il cui numero non è rappresentabile in quattro bit.

Condizione di transizione uscita verso `0x10`

Se sai rispondere a tutte le domande, passa allo stato successivo. Se no, la freccia ↺ ti dice dove tornare.

1 Che cosa condividono e che cosa no due thread dello stesso processo? ↺ § f.1

2 Un programma è parallelo per l'80%. Quale accelerazione si ottiene con 4 core? E con infiniti core? ↺ § f.2

3 Perché due cache write-through, senza protocollo, non bastano a mantenere coerente la memoria? ↺ § f.3

4 Che differenza c'è fra coerenza e consistenza? ↺ § f.3

5 Che differenza c'è fra snooping e directory? E fra aggiornamento e invalidazione? ↺ § f.4

6 Il core 0 ha un blocco in E e lo scrive; poi il core 1 lo legge. Quali stati attraversano le due copie, e che cosa transita sul bus? ↺ § f.5

7 A che cosa serve lo stato E? ↺ § f.5

8 Che cos'è la falsa condivisione, e come si evita? ↺ § f.6

9 Perché `lw`, `addi`, `sw` su un contatore condiviso possono perdere incrementi anche con cache coerenti? Quale istruzione risolve il problema? ↺ § f.7

10 Nel lucchetto, a che cosa serve la `lw` prima della `amoswap`? ↺ § f.7

11 Come riduce il traffico di snooping la L3 inclusiva di Nehalem? ↺ § f.8

PARTE

V

Il mondo esterno

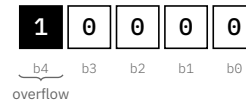
Un processore che non riceve dati e non mostra risultati non serve a nulla. L'ultimo stato collega il calcolatore al mondo: i dispositivi, i collegamenti che li uniscono al processore, e i tre modi di comandarli, dal controllo ciclico alle interruzioni al DMA.

0x10

I/O

STATO

0x10



Ingresso e uscita: il processore e il mondo

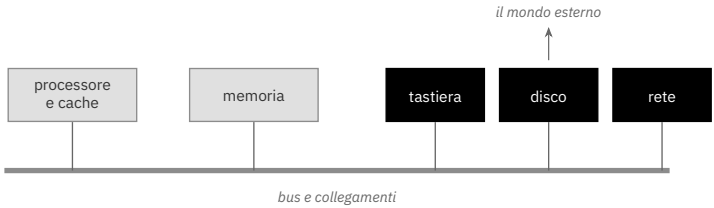


→ INGRESSI

- Gli organi di von Neumann e il bus di sistema (stato 0x0).
- Le eccezioni, `mtvec`, `mepc`, `mcause` e i registri CSR (stato 0xb, § b.7).
- Cache, coerenza e memoria virtuale (parte IV).

USCITE →

- Confrontare i dispositivi per banda e latenza.
- Spiegare come si organizzano i collegamenti di un calcolatore, e perché oggi sono seriali e punto a punto.
- Seguire un protocollo di handshake su un bus asincrono.
- Comandare un dispositivo con load e store, a controllo ciclico o con le interruzioni di RISC-V.
- Calcolare il costo di ciascuna tecnica per il processore, e spiegare perché esiste il DMA.
- Riconoscere i problemi che il DMA crea con cache e memoria virtuale.



Il processore che abbiamo costruito è in grado di calcolare, saltare, leggere e scrivere la memoria. Ma un calcolatore che non riceve dati e non mostra risultati è inutile: serve un modo per comunicare con la tastiera, lo schermo, il disco, la rete. Sono gli ultimi due organi di von Neumann, l'ingresso e l'uscita, e il motivo per cui questo stato ha un bit in più degli altri: è il punto in cui le informazioni escono dal processore. Vedremo come si collegano i dispositivi, come il processore li comanda con le istruzioni che già conosce, e come evita di sprecare tempo in attesa delle loro risposte.

10.1 Un mondo di dispositivi

Processore e memoria sono componenti ben definiti; i dispositivi di ingresso e uscita, o **periferiche**, sono invece un insieme vastissimo ed eterogeneo. Alcuni interagiscono con una persona, come tastiera, mouse e schermo; altri con altre macchine, come la scheda di rete; altri ancora conservano dati, come i dischi. E le loro velocità differiscono di molti ordini di grandezza (tabella 10.1). Per questo non studieremo i dispositivi uno per uno, ma il problema che hanno in comune: come collegarli al processore e alla memoria, e come trasferire i dati.

Due misure descrivono le prestazioni di un sistema di ingresso e uscita. La **banda** (*bandwidth* o *throughput*) è la quantità di dati trasferiti per unità di tempo: è determinante quando si copia un file grande. La **latenza** è il tempo

Tabella 10.1 Alcuni dispositivi e la loro velocità di trasferimento. I valori sono ordini di grandezza.

Dispositivo	Ruolo	Partner	Velocità
tastiera	ingresso	persona	100 bit/s
mouse	ingresso	persona	qualche kbit/s
audio di qualità CD	uscita	persona	1,4 Mbit/s
disco magnetico	memorizzazione	macchina	1–2 Gbit/s
schermo 4K a 60 immagini al secondo	uscita	persona	12 Gbit/s
rete Ethernet	ingresso e uscita	macchina	1–100 Gbit/s
SSD su PCI Express	memorizzazione	macchina	50 Gbit/s

fra una richiesta e la sua conclusione: è determinante quando si attende una risposta, come tra il clic del mouse e il movimento del puntatore, o in un gioco in rete. Le due grandezze non sono necessariamente correlate. Un furgone carico di dischi, come si dice spesso, ha una banda enorme e una latenza di ore.

10.2 Collegare i dispositivi

Nello stato 0x0 abbiamo visto il *bus*, un insieme di fili condivisi su cui transitano dati, indirizzi e segnali di controllo. Ogni uso del bus è una **transazione**: un'operazione completa, che comprende l'indirizzo, il dato e i segnali di controllo, e che va portata a termine per intero. Le transazioni fondamentali sono due: la lettura, che trasferisce un dato dal dispositivo o dalla memoria, e la scrittura, che lo trasferisce verso di essi.

Progettare un bus veloce è più difficile di quanto sembri.

- Più fili trasportano più bit alla volta, ma i segnali su fili diversi non arrivano tutti nello stesso istante, ed è necessario attendere il più lento.
- Fili più lunghi comportano più tempo per la propagazione, e segnali più disturbati.
- Ogni componente collegato allo stesso filo aggiunge un carico elettrico e rallenta tutti: le prestazioni migliori si ottengono con collegamenti *punto a punto*, fra due soli componenti.

Sincroni e asincroni

In un **bus sincrono** tutti i componenti condividono un clock, e il protocollo è scandito dai suoi fronti: l'indirizzo al ciclo 1, il dato al ciclo 5. È semplice e veloce, ma ha due limiti: il componente più lento impone la frequenza a tutti, e su fili lunghi il clock stesso arriva in istanti diversi ai vari componenti (si chiama *clock skew*). I bus sincroni sono quindi corti e veloci, come quello fra processore e memoria.

Un **bus asincrono** non ha clock. I due interlocutori si coordinano con un **handshake**, una stretta di mano: ogni passo viene confermato dall'altro prima di procedere. Funziona con dispositivi di qualunque velocità, anche su distanze maggiori, al prezzo di un protocollo più complicato. La figura 10.1 mostra un dispositivo che legge una parola dalla memoria, con tre segnali di controllo, *ReadReq* (richiesta di lettura), *Ack* (conferma) e *DataRdy* (dato pronto), e le linee dei dati, usate prima per l'indirizzo e poi per il dato.

Il dispositivo inizia ponendo l'indirizzo sulle linee dei dati e attivando ReadReq. Poi:

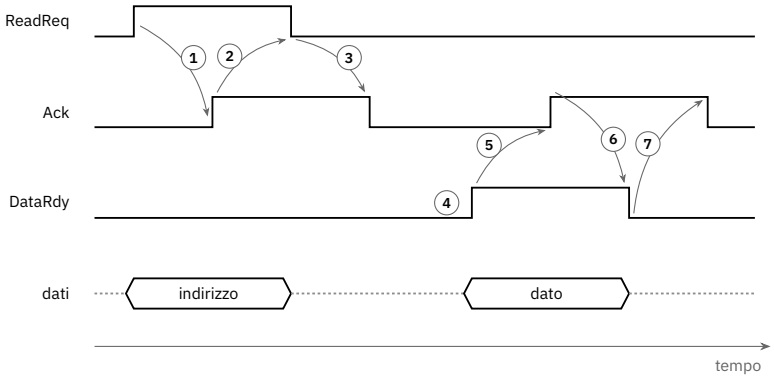


Figura 10.1 Un dispositivo legge una parola dalla memoria su un bus asincrono. Le frecce grigie collegano ogni evento a quello che lo provoca; i numeri sono i passi elencati nel testo. Le linee tratteggiate indicano che nessun componente sta pilotando le linee dei dati.

1. la memoria rileva ReadReq, legge l'indirizzo e attiva Ack;
2. il dispositivo rileva Ack, disattiva ReadReq e rilascia le linee dei dati;
3. la memoria rileva la disattivazione di ReadReq e disattiva Ack;
4. quando il dato è pronto, la memoria lo pone sulle linee e attiva DataRdy;
5. il dispositivo rileva DataRdy, legge il dato e attiva Ack;
6. la memoria rileva Ack, disattiva DataRdy e rilascia le linee;
7. il dispositivo rileva la disattivazione di DataRdy e disattiva Ack: la transazione è conclusa.

Nessun passo dipende da un tempo prefissato: se la memoria è lenta, il passo 4 avviene più tardi, e tutto il resto si adatta. Il prezzo è che ogni informazione richiede più scambi di andata e ritorno sui fili.

Da paralleli a seriali

Per decenni i collegamenti veloci sono stati *paralleli*: molti fili affiancati, un bit per filo. L'interfaccia dei dischi Parallel ATA, per esempio, aveva 16 fili di dati e raggiungeva i 133 MB/s. Al crescere della frequenza, però, mantenere allineati i segnali di tanti fili diventa impossibile. La soluzione che si è imposta è opposta: pochi fili, anzi una sola coppia per direzione, su cui i bit transitano uno dopo l'altro a frequenze altissime, con il clock ricavato dai dati stessi. Serial ATA trasmette su una coppia a 6 Gbit/s, cioè circa 600 MB/s; PCI Express, il collegamento principale dei calcolatori di oggi, usa *corsie* (*lanes*) seriali indipendenti e punto a punto, da una a sedici per dispositivo, e con 16 corsie della quarta generazione supera i 30 GB/s per direzione. Anche USB, il bus

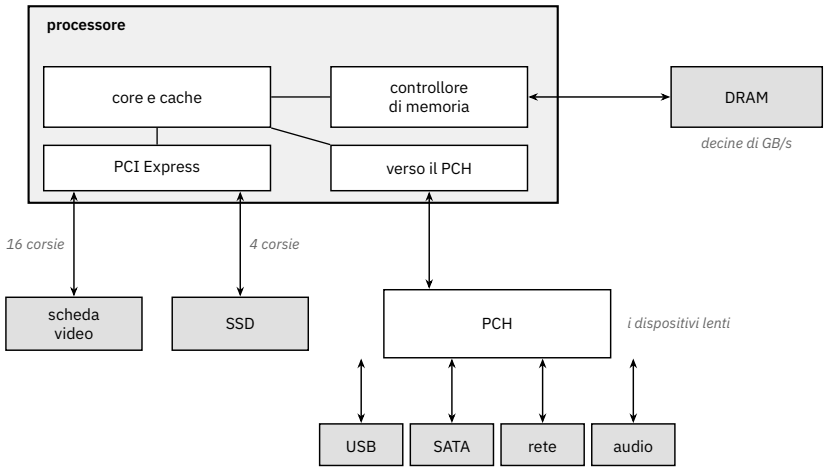


Figura 10.2 L'organizzazione di un calcolatore di oggi. Memoria e collegamenti PCI Express più veloci partono direttamente dal processore; i dispositivi più lenti transitano attraverso un chip separato, il PCH.

universale per i dispositivi esterni, è seriale. Paradossalmente, trasmettere i bit in sequenza è più veloce che trasmetterli in parallelo.

10.3 L'architettura di un calcolatore moderno

Collegare tutti i componenti a un unico bus significherebbe vincolarli alla velocità del dispositivo più lento. I calcolatori usano quindi una gerarchia di collegamenti, dai più veloci, vicini al processore, ai più lenti. Per anni l'organizzazione classica è stata quella dello stato 0x0: un *bus del processore* (*front side bus*) verso un primo chip, il *Northbridge*, che gestiva la memoria e la scheda video, e da questo un collegamento verso il *Southbridge*, che raccoglieva i dispositivi lenti.

Oggi il Northbridge è stato integrato nel processore (figura 10.2). Nehalem ha integrato sul chip il controllore della memoria, come abbiamo visto nello stato 0xd; dal 2009 Intel ha spostato nel processore anche le corsie PCI Express più veloci, per la scheda video e, oggi, anche per gli SSD. Resta un solo chip esterno, che Intel chiama PCH (*platform controller hub*): erede del Southbridge, raccoglie USB, SATA, rete e audio, ed è collegato al processore da un collegamento dedicato.

Alcuni dispositivi sono a loro volta processori specializzati. Le *schede video* (GPU) contengono migliaia di unità di calcolo semplici, e oltre alla grafica eseguono calcoli scientifici e d'intelligenza artificiale molto più rapidamente del

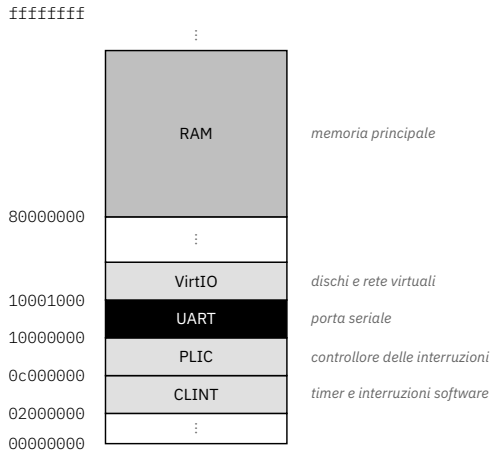


Figura 10.3 Una parte della mappa degli indirizzi fisici della macchina `virt` di QEMU, non in scala. La memoria comincia a `0x80000000`; più in basso ci sono i registri dei dispositivi.

processore. I *processori di rete* elaborano i pacchetti, smistandoli o filtrando attacchi, senza impegnare il processore principale.

10.4 Comunicare con un dispositivo

Un dispositivo si presenta al processore come un piccolo insieme di **registri del dispositivo**: un registro di *controllo*, in cui il processore scrive i comandi; un registro di *stato*, da cui ricava se il dispositivo è pronto o ha incontrato un errore; uno o più registri *dati*. In che modo il processore vi accede?

Una possibilità sono istruzioni apposite: gli x86 hanno `in` e `out`, che leggono e scrivono uno spazio di indirizzi separato, riservato ai dispositivi. RISC-V, come quasi tutte le architetture RISC, non ha istruzioni di questo tipo e usa l'**I/O mappato in memoria** (*memory-mapped I/O*): alcuni indirizzi fisici non corrispondono a celle di memoria ma ai registri dei dispositivi, e sono sufficienti le normali load e store. È quello che anticipavamo nello stato 0x6: un indirizzo può corrispondere a un dispositivo, e allora leggerlo può avere effetti collaterali. La figura 10.3 mostra la mappa degli indirizzi della macchina `virt` dell'emulatore QEMU, molto usata per sperimentare con RISC-V.

La porta seriale della macchina `virt` è un circuito molto diffuso, il 16550, con i registri a partire da `0x10000000`. Una scrittura nel registro all'indirizzo base trasmette un carattere; il registro a distanza 5 è il registro di stato, e

il suo bit 5 vale 1 quando la porta è pronta ad accettare un nuovo carattere da trasmettere. Per stampare il carattere in `a0` sono sufficienti le istruzioni seguenti:

```
        li    t0, 0x10000000    # indirizzo della porta seriale
attesa: lbu   t1, 5(t0)         # legge il registro di stato
        andi  t1, t1, 0x20      # bit 5: pronta a trasmettere?
        beqz  t1, attesa        # no: riprova
        sb    a0, 0(t0)         # sì: scrive il carattere
```

Tre accorgimenti rendono possibile questo meccanismo. Primo, gli indirizzi dei dispositivi non devono essere gestiti dalla cache: il registro di stato cambia autonomamente, e una copia in cache sarebbe sempre obsoleta. In RISC-V ogni zona della memoria fisica ha degli attributi, fissati dalla piattaforma, che indicano se è memoria normale o un dispositivo, e se può essere memorizzata in cache. Secondo, l'ordine è rilevante: l'istruzione `fence` dello stato 0xf ha anche dei bit per gli accessi ai dispositivi, `i` e `o`, per imporre che una scrittura a un dispositivo avvenga, per esempio, dopo quelle in memoria che la preparano. Terzo, la protezione: le pagine con i registri dei dispositivi non vengono mappate nello spazio dei processi utente, e solo il sistema operativo può usarle. I programmi chiedono l'I/O al sistema operativo, con la `ecall` dello stato 0x1, e il sistema operativo funge da arbitro fra i processi che richiedono la stessa stampante.

10.5 Aspettare o essere avvisati

Il ciclo dell'esempio interroga il dispositivo finché non è pronto: si chiama **controllo ciclico** (*polling*). È semplice, ma il processore trascorre il tempo in attesa. Quanto costa? Supponiamo un processore a 1 GHz e un'interrogazione da 400 cicli, contando anche la chiamata alla procedura del sistema operativo.

- Un mouse interrogato 30 volte al secondo costa $30 \times 400 = 12\,000$ cicli al secondo: lo 0,0012% del processore. Il costo è trascurabile.
- Un disco di vecchia generazione, che trasferisce 4 MB/s a blocchi di 16 byte, va interrogato $4 \times 10^6/16 = 250\,000$ volte al secondo: 10^8 cicli, il 10% del processore, solo per chiedere «hai finito?».

L'alternativa è la notifica da parte del dispositivo: quando richiede attenzione, il dispositivo invia un'**interruzione** (*interrupt*), che, come abbiamo visto nello stato 0xb, sospende il programma in corso e fa intervenire il sistema operativo. Il processore non spreca più tempo con il mouse fermo. Ma ogni interruzione ha un costo fisso (salvare i registri, eseguire il gestore, ripristinarli), che possiamo stimare in 500 cicli: per il disco, che richiede attenzione

250 000 volte al secondo, servono $1{,}25 \times 10^8$ cicli, il 12,5%. Le interruzioni risolvono il problema dei dispositivi che comunicano di rado, non quello dei dispositivi che comunicano frequentemente. Per questi ultimi servirà il DMA (§ 10.6).

Le interruzioni in RISC-V

Le interruzioni arrivano in qualunque momento, anche molte insieme, e l'hardware deve poterle filtrare e ordinare. RISC-V svolge questo compito con tre registri di controllo, che si leggono e si scrivono con le istruzioni CSR dello stato 0xb:

- `mip` (*interrupt pending*) ha un bit per ogni sorgente di interruzione, che passa a 1 quando la sorgente richiede attenzione: il timer (bit 7), le interruzioni software inviate da un altro core (bit 3), i dispositivi esterni (bit 11);
- `mie` (*interrupt enable*) ha un bit per ogni sorgente, che indica se quella sorgente è abilitata: è la *maschera* delle interruzioni;
- nel registro `mstatus`, il bit MIE (bit 3) abilita o disabilita tutte le interruzioni insieme, mentre il processore è al livello machine; quando esegue un programma utente, le interruzioni destinate al livello machine sono sempre ammesse.

Un'interruzione viene accettata se il suo bit vale 1 sia in `mip` sia in `mie`, cioè se l'AND dei due non è zero, e se MIE vale 1. Se ne arrivano più d'una contemporaneamente, ha la precedenza quella esterna, poi quella software, poi il timer. Quando ne accetta una, l'hardware esegue le stesse operazioni previste per le eccezioni: scrive in `mepc` l'indirizzo dell'istruzione interrotta, in `mcause` il codice della causa con il bit più alto a 1, copia MIE in un altro bit di `mstatus`, MPIE, e azzera MIE, in modo che il gestore non venga interrotto a sua volta. Infine salta al gestore, all'indirizzo in `mtvec`. Nel modo *vettorizzato* (bit 0 di `mtvec` a 1) le interruzioni provocano invece un salto a BASE + 4 × causa, dove BASE è `mtvec` con i due bit bassi azzerati, e ogni causa ha la sua procedura; le eccezioni sono comunque dirette a BASE. Alla fine, `mret` ricopia MPIE in MIE e torna all'istruzione indicata da `mepc`. A differenza della `ecall`, qui il gestore non deve sommare 4: l'istruzione interrotta non era ancora stata eseguita.

Prima di tutto questo, il programma di avvio predispone l'ambiente:

```
        la    t0, gestore
        csrw  mtvec, t0          # indirizzo del gestore
        li    t0, 0x800          # bit 11: dispositivi esterni
        csrs  mie, t0            # abilita la sorgente
        csrsi mstatus, 0x8       # bit 3, MIE: abilitazione generale
```

I dispositivi esterni sono molti e le linee in `mip` poche: fra i dispositivi e il processore c'è un *controllore delle interruzioni*, che nelle piattaforme RISC-V si chiama PLIC (*platform-level interrupt controller*). Il PLIC raccoglie le richieste di tutti i dispositivi, assegna a ciascuno una priorità e presenta al processore una sola interruzione esterna; il gestore legge un suo registro per sapere quale dispositivo l'ha causata, e vi scrive per segnalare il completamento. Durante l'attesa, il processore può eseguire `wfi` (*wait for interrupt*) e sospendersi, con un consumo ridotto, fino alla prossima interruzione.

1954 Un calcolatore su un camion

Le prime forme di interruzione comparvero all'inizio degli anni Cinquanta. Il primo calcolatore che le usò per l'ingresso e l'uscita, al posto del controllo ciclico, è considerato il DYSEAC, costruito dal National Bureau of Standards per il Signal Corps dell'esercito americano ed entrato in funzione nell'aprile del 1954. Era anche uno dei primi calcolatori mobili, forse il primo: pesava una ventina di tonnellate ed era montato su un camion. Nel 1957 il TX-2 del MIT introdusse le interruzioni con più livelli di priorità, l'idea che oggi ritroviamo nel PLIC.

10.6 Il DMA

Un disco o una scheda di rete non trasferiscono un byte alla volta, ma blocchi da migliaia di byte. Se fosse il processore a copiarli, con load e store, impiegherebbe il proprio tempo in semplici operazioni di copia. L'**accesso diretto alla memoria** (DMA, *direct memory access*) affida il lavoro a un circuito apposito, il *controllore DMA*, capace di leggere e scrivere la memoria autonomamente (figura 10.4).

1. Il processore scrive nei registri del controllore l'indirizzo di memoria, il numero di byte e la direzione del trasferimento, e torna a eseguire altri programmi.
2. Il controllore trasferisce i dati fra il dispositivo e la memoria, usando il bus quando il processore non lo impegna, o sottraendogli qualche ciclo.
3. Finito il trasferimento, il controllore invia un'interruzione.

Rifacciamo il conto del disco da 4 MB/s, con blocchi da 4 KiB: circa 1000 trasferimenti al secondo. Se programmare il controllore costa 1000 cicli e

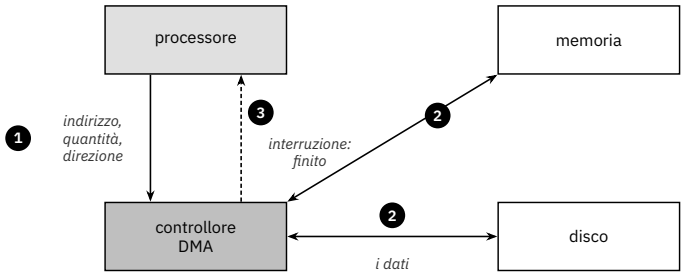


Figura 10.4 Un trasferimento con il DMA. (1) Il processore programma il controllore; (2) il controllore trasferisce i dati fra disco e memoria, mentre il processore esegue altre attività; (3) alla fine un'interruzione avvisa il processore.

l'interruzione finale 500, il processore spende $1000 \times 1500 = 1{,}5 \times 10^6$ cicli al secondo, lo 0,15%, invece del 10–12,5%. Un calcolatore ha molti controllori DMA; nei sistemi di oggi sono spesso i dispositivi stessi, come le schede di rete e gli SSD, a leggere e scrivere la memoria attraverso PCI Express.

Il DMA e la cache

Il DMA porta con sé un problema che conosciamo: la coerenza. Il controllore legge e scrive la memoria, ma il processore accede alla cache.

- Se il DMA scrive in memoria un blocco appena letto dal disco, e il processore aveva in cache una copia di quegli indirizzi, la copia è obsoleta.
- Se il processore ha scritto in una cache write-back i dati da inviare al disco, la memoria non è ancora aggiornata, e il DMA legge i dati obsoleti.

Le soluzioni ricordano quelle dello stato 0xf. La prima consiste nel far transitare il DMA attraverso la cache, ma questa scelta sottrarrebbe al processore proprio la risorsa che usa di più. La seconda è trattare il DMA come un core in più: le sue scritture invalidano le copie nelle cache, e le sue letture trovano il dato nella cache se lì è più recente. Si dice che l'I/O è *coerente*, ed è il caso dei processori per personal computer e server. La terza, comune nei sistemi più piccoli, lascia il compito al software: prima di un'uscita il sistema operativo riscrive in memoria i blocchi modificati (dirty) interessati (o usa per quei dati la scrittura immediata), e prima di un ingresso, e di nuovo dopo, invalida le copie in cache. RISC-V ha istruzioni apposite nell'estensione Zicbom: `cbo.clean`, `cbo.flush` e `cbo.inval`.

Il DMA e la memoria virtuale

Occorre considerare anche la memoria virtuale. Un programma richiede di leggere un file in un suo vettore, indicato da un indirizzo virtuale; il control-

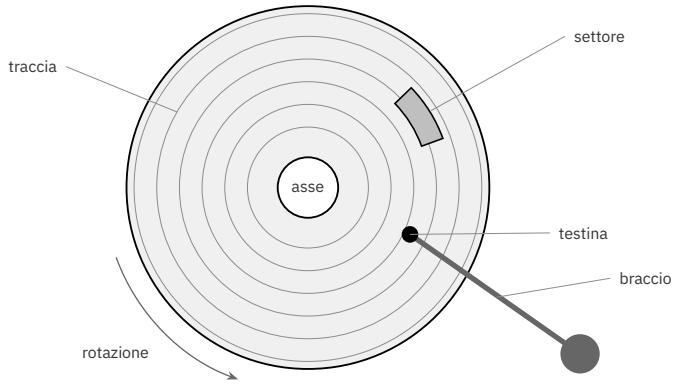


Figura 10.5 Un piatto di un disco magnetico visto dall'alto. Per leggere un settore il braccio posiziona la testina sulla traccia corretta, poi si attende che la rotazione conduca il settore sotto la testina.

lore DMA, però, opera con indirizzi fisici. Il sistema operativo deve tradurre l'indirizzo prima di programmare il controllore, e c'è una complicazione: pagine consecutive nello spazio virtuale non sono consecutive in memoria fisica. Un trasferimento che attraversa il confine di una pagina va quindi suddiviso in più segmenti, uno per pagina; molti controllori accettano direttamente un elenco di segmenti. Inoltre, durante il trasferimento, le pagine interessate non devono essere spostate su disco: il sistema operativo le *blocca* in memoria. L'alternativa è dare al DMA la propria unità di traduzione, con il suo TLB: è la IOMMU dei sistemi moderni, che traduce gli indirizzi dei dispositivi e impedisce a un dispositivo di scrivere in zone non autorizzate.

10.7 Il disco, da vicino

Nello stato 0xe abbiamo affermato che un page fault costa milioni di cicli. Vediamo da dove deriva questo valore. Un disco magnetico contiene piatti che ruotano a velocità costante, ricoperti di materiale magnetico. I dati sono registrati su cerchi concentrici, le *tracce*, divise in *settori*, tipicamente da 4096 byte. Una testina per faccia, montata su un braccio mobile, legge e scrive (figura 10.5).

Il tempo di accesso a un settore è la somma di quattro termini. Eseguiamo il calcolo per un disco a 7200 giri al minuto.

- *Posizionamento* (*seek*): il braccio raggiunge la traccia. I produttori dichiarano tempi medi di qualche millisecondo; assumiamo 5 ms.
- *Latenza di rotazione*: in media occorre attendere mezzo giro. A 7200 giri al minuto un giro dura $60/7200$ s $\approx 8{,}33$ ms, e mezzo giro circa 4,17 ms.

- *Trasferimento*: 4 KiB a 200 MB/s richiedono circa 0,02 ms.
- *Controllore*: il circuito del disco aggiunge qualche decimo di millisecondo, assumiamo 0,2 ms.

In tutto circa 9,4 ms: per un processore a 3 GHz, quasi 30 milioni di cicli. Quasi tutto il tempo è meccanico, e il trasferimento vero e proprio è una frazione minima: per questo le pagine sono grandi, e per questo il sistema operativo cerca di leggere settori vicini. Le memorie a stato solido (SSD), realizzate con memoria flash senza parti in movimento, rispondono in qualche decina di microsecondi: cento volte più rapide, ma pur sempre centinaia di migliaia di cicli.

1956 Cinquanta piatti da sessanta centimetri

Nel giugno 1956 IBM consegnò il primo disco magnetico in commercio, l'IBM 350, parte del sistema 305 RAMAC, a un'azienda cartaria di San Francisco. Aveva cinquanta piatti da 24 pollici, circa 61 centimetri, che giravano a 1200 giri al minuto, conteneva cinque milioni di caratteri da 6 bit, circa 3,75 MB, e con il suo compressore pesava più di una tonnellata. Il tempo medio di accesso era inferiore al secondo: un'enormità oggi, ma allora una rivoluzione, perché in un archivio di milioni di caratteri si poteva leggere un dato qualsiasi senza scorrere un nastro dall'inizio.

10.8 Il cerchio si chiude

Con l'ingresso e l'uscita il calcolatore è completo: i cinque organi di von Neumann dello stato 0x0 sono tutti al loro posto, e sappiamo come funziona ciascuno. Il processore preleva e decodifica le istruzioni, le esegue in una pipeline, predice i salti; la memoria è una gerarchia di cache, DRAM e disco, tradotta e protetta dalla memoria virtuale; i dispositivi comunicano con il processore attraverso indirizzi di memoria, interruzioni e DMA. Resta la parte più coinvolgente: lavorare direttamente sulla macchina. È il compito dei laboratori, che cominciano preparando gli strumenti.

Condizione di transizione uscita verso **L0**

Se sai rispondere a tutte le domande, passa al laboratorio Lo. Se no, la freccia ↺ ti dice dove tornare.

1 Fai un esempio di dispositivo in cui conta soprattutto la banda, e uno in cui conta soprattutto la latenza. ↺ § 10.1

2 Perché i collegamenti veloci di oggi sono seriali e punto a punto? ↺ § 10.2

3 Che differenza c'è fra un bus sincrono e uno asincrono? Nel protocollo della figura 10.1, perché il passo 3 attende il passo 2? ↺ § 10.2

4 Che cosa è rimasto, oggi, del Southbridge? ↺ § 10.3

5 In che modo una `sb` raggiunge un dispositivo invece della memoria? Perché i registri dei dispositivi non devono essere memorizzati in cache? ↺ § 10.4

6 Un dispositivo va interrogato 1000 volte al secondo e ogni interrogazione costa 400 cicli. Che frazione di un processore a 2 GHz consuma il controllo ciclico? ↺ § 10.5

7 Quando viene accettata un'interruzione in RISC-V? Che cosa accade al bit MIE durante l'esecuzione del gestore? ↺ § 10.5

8 Perché dopo un'interruzione il gestore non somma 4 a `mepc`, mentre dopo una `ecall` sì? ↺ § 10.5

9 Perché il DMA riduce il costo dei trasferimenti grandi più delle interruzioni? ↺ § 10.6

10 Che problemi crea il DMA con la cache e con la memoria virtuale? ↺ § 10.6

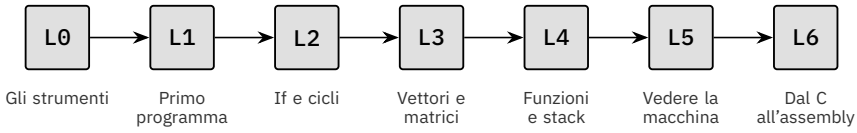
11 Quanto vale la latenza di rotazione media di un disco a 10 000 giri al minuto? ↺ § 10.7

PARTE

VI

Laboratorio

Un linguaggio si impara parlandolo. In questa parte si scrivono, si eseguono e si osservano programmi assembly nel simulatore QtRvSim: prima si prepara il banco di lavoro, poi si procede dal primo programma fino al codice prodotto da un compilatore C.



LABORATORIO

L0



Gli strumenti



→ INGRESSI

- Macchine virtuali e livelli di un sistema: ISA e ABI (stato 0x0, § 0.5).
- Pseudo-istruzioni, `ecall` e direttive dell'assemblatore (stato 0x1, § 1.8, § 1.11 e § 1.13).

USCITE →

- Installare QtRvSim e orientarsi nella sua finestra.
- Eseguire un programma tutto insieme o un'istruzione alla volta, e leggerne l'uscita.
- Conoscere le regole dell'assemblatore di QtRvSim.
- Conoscere le funzionalità di RARS, Ripes, GCC con QEMU e Compiler Explorer, e perché lo stesso programma a volte funziona in uno e non in un altro.
- Scrivere programmi che funzionano in tutti.

Gli stati hanno costruito il processore sulla carta. Nei laboratori lo vedremo in funzione: scriveremo programmi, li eseguiremo un'istruzione alla volta, osserveremo i registri cambiare e le istruzioni attraversare la pipeline. Non occorre un processore RISC-V reale: è sufficiente un **simulatore**, un programma che ne riproduce il funzionamento sul calcolatore che hai già. Questo capitolo prepara il banco di lavoro.

Lo strumento dei laboratori è **QtRvSim**, e tutti gli esercizi sono pensati per questo simulatore. Ma non è l'unico: ce ne sono altri, ognuno con i suoi punti di forza, e prima o poi ti capiterà di usarli, o di trovare in rete un programma

scritto per uno di essi. Conviene conoscerne le funzionalità, e soprattutto perché un programma che funziona in uno a volte non funziona in un altro.

L0.1 Simulare, emulare, compilare

Nello stato 0x0 abbiamo visto che una macchina virtuale è un programma che imita un calcolatore. I programmi che permettono di eseguire codice RISC-V su un normale PC, con un processore x86 o Arm, si dividono in tre famiglie, in base a ciò che intendono mostrare.

Simulatori dell'ISA
: Eseguono un'istruzione alla volta e mostrano solo ciò che l'ISA rende visibile: i registri, la memoria, il PC. Il modo in cui il circuito realizza le istruzioni non è rappresentato. È il caso di RARS.

Simulatori della microarchitettura
: Imitano anche il circuito, componente per componente, come lo abbiamo disegnato negli stati: si vedono il datapath, i segnali di controllo, la pipeline, la cache. Sono lenti, ma per i programmi di un corso sono pienamente adeguati. Sono QtRvSim e Ripes.

Emulatori
: Hanno il solo scopo di eseguire il programma, il più rapidamente possibile. Un **emulatore** come QEMU traduce blocchi di istruzioni RISC-V in istruzioni della macchina che lo ospita, e poi le esegue direttamente: è molto più veloce di un simulatore, ma non mostra nulla dell'organizzazione interna del processore.

A queste si aggiunge la **catena di compilazione** (*toolchain*), che non esegue i programmi ma li produce (figura L0.1). Il **compilatore** traduce il C in assembly; l'**assemblatore** traduce l'assembly in codice macchina, un file *oggetto* per ogni sorgente; il **linker** unisce i file oggetto, assegna a ogni etichetta il suo indirizzo definitivo e scrive l'**eseguibile**. Su Linux l'eseguibile è un file in formato **ELF** (*Executable and Linkable Format*), che contiene il codice, i dati iniziali e il **punto d'ingresso**: l'indirizzo della prima istruzione da eseguire. I simulatori didattici, invece, hanno un assemblatore integrato che scrive il codice direttamente nella loro memoria simulata, senza file intermedi.

Un aspetto accomuna tutti questi strumenti: eseguono la stessa ISA. Una `add` produce lo stesso risultato in QtRvSim, in RARS e in QEMU, perché l'ISA è un contratto (§ 0.5). Tutto il resto, però, è una scelta di chi ha scritto lo strumento: a quale indirizzo viene caricato il programma, quali servizi offre la `ecall`, quali direttive accetta l'assemblatore, quali estensioni sono presenti, che cosa si può vedere. In altre parole, l'*ambiente* in cui viene eseguito il

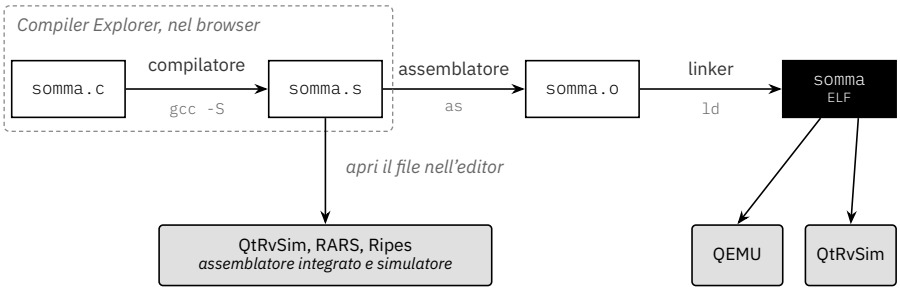


Figura L0.1 Le strade che portano un programma all'esecuzione. In alto, la catena di compilazione produce un eseguibile ELF, che QEMU esegue e QtRvSim è in grado di caricare. In basso, i simulatori didattici assemblano il sorgente autonomamente, direttamente nella memoria simulata; anche Ripes è in grado di caricare un eseguibile ELF.

programma, e con esso l'ABI. Quasi tutte le differenze di comportamento hanno origine qui, e il resto del capitolo è dedicato a conoscerle.

L0.2 QtRvSim

QtRvSim è nato all'Università tecnica ceca di Praga, per i corsi di architettura dei calcolatori di quell'ateneo. Il suo predecessore, QtMips, era un simulatore MIPS nato nel 2017 da una tesi di laurea; fra il 2020 e il 2021 è stato riscritto per RISC-V, e da allora è cresciuto con il lavoro di altre tesi e dei docenti del corso. È *software libero*: il codice è pubblico e chiunque può studiarlo, modificarlo e ridistribuirlo.

Rispetto agli altri strumenti ha un pregio raro: mostra il processore come lo abbiamo studiato. Il datapath sullo schermo è quello della parte II, con i multiplexer, l'unità di controllo e i nomi dei segnali; può diventare la pipeline a cinque fasi della parte III, con o senza l'unità che rileva i conflitti e propaga i dati; ha cache di primo e di secondo livello configurabili, un predittore dei salti, i registri CSR e le interruzioni dello stato 0xb, la memoria virtuale, e alcune periferiche: un terminale collegato a una porta seriale, una fila di LED, tre manopole e un piccolo schermo. Esegue le istruzioni di RV32IMA con i registri CSR (e la variante a 64 bit); non ha la virgola mobile né le istruzioni compresse.

Installarlo

QtRvSim funziona su Windows, macOS e Linux, e c'è perfino una versione che funziona nel browser, senza installare nulla (tabella L0.1). Tutti i file si scaricano dalla pagina delle versioni del progetto:

Tabella L0.1 Come installare QtRvSim. Al posto di `<v>` c'è la versione, per esempio `v0.9.9`.

Sistema	Come si installa
Browser	Apri `comparch.edu.cvut.cz/qtrvsim/app`. Non serve installare nulla, ma la versione è ancora sperimentale: va bene per una prova, meno per i laboratori.
Windows	Scarica `qtrvsim-mingw32-<v>.zip`, estrailo in una cartella e avvia `qtrvsim_gui.exe`.
macOS	Scarica `qtrvsim-macos-<v>.dmg` e trascina l'applicazione in *Applicazioni*. Al primo avvio il sistema la blocca, perché viene da uno sviluppatore che non conosce: apri *Impostazioni di Sistema → Privacy e sicurezza* e premi *Apri comunque*.

Ubuntu	Aggiungi l'archivio del progetto e installa il pacchetto:
	`sudo add-apt-repository ppa:qtrvsimteam/ppa`
	`sudo apt-get update`
	`sudo apt-get install qtrvsim`

Altri Linux	Con Flatpak: `flatpak install flathub cz.cvut.edu.comparch.qtrvsim`. Ci sono anche pacchetti per Debian, Fedora e openSUSE (sul servizio Open Build Service), per Arch (nell'AUR) e per Nix.

`github.com/cvut/qtrvsim/releases`

Al momento della stampa la versione più recente è la 0.9.9: i nomi dei menu e i comandi di questo libro si riferiscono a quella. Le versioni successive potrebbero cambiare qualche dettaglio, non la sostanza.

Su Linux il programma si avvia con il comando `qtrvsim_gui`. Accanto c'è `qtrvsim_cli`, una versione senza finestre che esegue un programma da riga di comando e stampa lo stato finale: non serve per i laboratori, ma è comoda per verificare automaticamente molti programmi.

L0.3 Una panoramica della finestra

All'avvio QtRvSim chiede quale macchina simulare. La finestra di configurazione ha a sinistra un elenco di pagine; la prima, *Presets and ELF File*, offre quattro configurazioni pronte, i *preset*, che corrispondono alle macchine studiate negli stati (tabella L0.2). Le altre pagine permettono di configurare manualmente ogni parametro: il processore (*Core ISA and Hazards*), il predittore dei salti, la memoria virtuale, i tempi di accesso alla memoria, le tre cache e l'emulazione del sistema operativo. In basso ci sono tre pulsanti: *Example* carica un programma di esempio, *Start empty* apre il simulatore senza programma, *Load machine* carica un eseguibile ELF scelto con *Browse*. La stessa finestra si riapre in ogni momento con *File → New simulation* (`Ctrl+N`), per cambiare macchina.

Tabella L0.2 I preset di QtRvSim e gli stati in cui si studia la macchina corrispondente.

Preset	Che cosa simula
No pipeline no cache	Il processore a ciclo singolo completato nello stato 0x7.
No pipeline with cache	Lo stesso, con le cache di primo livello dello stato 0xd.
Pipelined without hazard unit and without cache	La pipeline a cinque fasi dello stato 0x9, senza stalli né propagazione: i conflitti sui dati producono risultati sbagliati, come nello stato 0xa.
Pipelined with hazard unit and cache	La pipeline con l'unità di rilevamento dei conflitti e la propagazione dello stato 0xa, più le cache.
Custom	Qualsiasi combinazione, scelta nelle altre pagine.

Dopo la scelta compare la finestra principale (figura L0.2). Le sue parti si chiamano *pannelli*. All'inizio ce ne sono solo due, il programma e il datapath; gli altri si aprono dal menu *Windows*, e si possono spostare, affiancare e staccare trascinandoli per il titolo. *Windows* → *Reset Windows* ripristina la disposizione iniziale.

La barra degli strumenti

Contiene i comandi che userai di continuo: *Compile Source* (`Ctrl+E`), quasi in fondo alla barra, assembla il sorgente aperto nell'editor e lo carica in memoria; *Run* (`Ctrl+R`) esegue il programma, *Step* (`Ctrl+T`) un passo alla volta, *Pause* lo sospende, *Reload* lo riporta all'inizio. I pulsanti da *1x* a *25x* fissano quante istruzioni al secondo esegue *Run*, per osservare il funzionamento del datapath; *Unlimited* e *Max* eseguono il programma alla massima velocità. Su macOS, al posto di `Ctrl` si usa `Cmd`.

Program

A sinistra, le istruzioni del programma come sono in memoria, già tradotte: le pseudo-istruzioni sono già state espanse. I registri sono indicati con il numero (`x10`); *Machine* → *Mnemonics Registers* mostra i nomi convenzionali (`a0`). Con la pipeline, il menu in cima seleziona quale fase seguire: *Follow fetch* evidenzia l'istruzione appena prelevata.

Core

Al centro, il datapath, con i valori sui fili e i segnali di controllo. Con la pipeline, sopra ogni fase compare l'istruzione che la occupa. In alto a destra, i contatori dei cicli (*Cycles*) e degli stalli (*Stalls*).

Registers e Memory

Si aprono con *Windows* → *Registers* (`Ctrl+D`) e *Windows* → *Memory* (`Ctrl+M`): i 32 registri, ciascuno con il numero e il nome convenzionale, e il PC; la memoria, a partire dall'indirizzo che scegli.

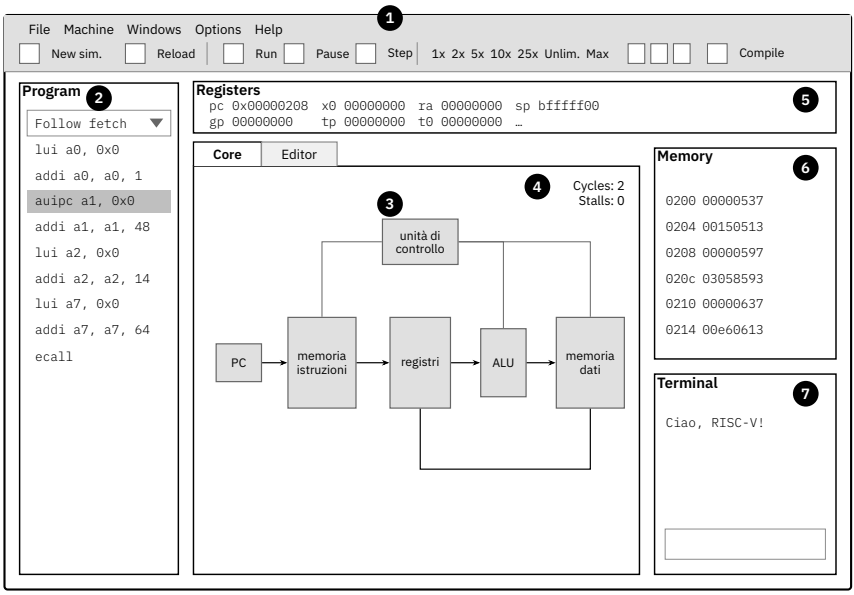


Figura L0.2 La finestra di QtRvSim, schematizzata, con i pannelli più utili aperti. (1) Menu e barra degli strumenti. (2) Il programma in memoria, con l'istruzione corrente evidenziata. (3) Il datapath animato, nella scheda *Core*; accanto, la scheda *Editor* con i sorgenti aperti. (4) I contatori dei cicli e degli stalli. (5) I registri. (6) La memoria. (7) Il terminale. Gli ultimi tre si aprono dal menu *Windows*.

Gli altri pannelli

Dal menu *Windows*: *Terminal*, su cui i programmi scrivono la loro uscita; *Program Cache*, *Data Cache* e *L2 Cache*, con il contenuto delle cache e le statistiche di hit e miss; i registri di controllo (*Control and Status Registers*); le periferiche (*Peripherals* e *LCD display*); *Messages*, con gli errori dell'assemblatore.

L0.4 La prova del banco

Prima di cominciare un lavoro, un elettricista prova gli strumenti su un circuito che conosce. Procediamo allo stesso modo con un programma che stampa un messaggio e termina, usando le due chiamate di sistema dello stato 0x1: la 64, che scrive dei byte (in `a0` il file, dove 1 è lo schermo; in `a1` l'indirizzo dei byte; in `a2` il loro numero), e la 93, che termina.

```
        .globl _start
        .text
_start: li   a0, 1              # 1 = uscita standard (lo schermo)
        la   a1, msg            # indirizzo del primo carattere
        li   a2, 14             # quanti byte scrivere
        li   a7, 64             # servizio 64: write
        ecall
        li   a0, 0              # codice di uscita 0
        li   a7, 93             # servizio 93: exit
        ecall

        .data
msg:    .asciz "Ciao, RISC-V!\n"
```

Il messaggio è lungo 14 byte: 13 caratteri visibili più l'a capo, che nel sorgente si scrive `\n`. La direttiva `.asciz` aggiunge in fondo un byte nullo, che qui non viene scritto.

1. Avvia QtRvSim, scegli il preset *No pipeline no cache* e premi *Start empty*.
2. Apri un editor con *File* → *New source* (`Ctrl+F`), scrivi il programma e salvalo con `Ctrl+S`; oppure apri un file già scritto con *File* → *Open source*.
3. Premi *Compile Source* (`Ctrl+E`). Se c'è un errore, il pannello *Messages* indica la riga; se non ce ne sono, il programma compare nel pannello *Program*.
4. Apri il terminale da *Windows* → *Terminal*, scegli la velocità *Unlimited* (all'avvio è *1x*, e il messaggio comparirebbe dopo qualche secondo) e premi *Run*.

Nel terminale compare `Ciao, RISC-V!`. Il simulatore si ferma subito dopo la scrittura: per impostazione predefinita QtRvSim sospende l'esecuzione a ogni chiamata di sistema, così da consentire di esaminare l'effetto della chiamata. Premi di nuovo *Run*: la `ecall` numero 93 ferma la simulazione. A questo punto interrompi anche tu l'esecuzione: se premessi *Run* ancora una volta, il processore proseguirebbe oltre la fine del codice e proverebbe a eseguire i byte del messaggio, fermandosi con un errore. Se le pause dopo ogni chiamata di sistema ti disturbano, si possono disattivare nella pagina *System Emulation and IRQ* della configurazione, deselezionando *Stop on known system call*.

Ora la parte più istruttiva. Apri il pannello dei registri (`Ctrl+D`), premi *Reload* e poi più volte *Step*, osservando il PC e i registri. La prima istruzione è all'indirizzo `0x00000200`, e la prima `li` non è una sola istruzione, ma due (qui con i nomi convenzionali dei registri):

```
0x00000200   lui   a0, 0x0
0x00000204   addi  a0, a0, 1
```

La spiegazione, insieme ad altre regole dell'assemblatore, è nella prossima sezione. Se tutto ha funzionato, il banco è pronto.

> **Direttive per QtRvSim.** Una riga come `#pragma qtrvsim show terminal` all'inizio del sorgente fa aprire automaticamente a QtRvSim il terminale (allo stesso modo, `show registers` e `show memory` aprono i rispettivi pannelli). Gli altri strumenti la interpretano come un commento, perché comincia con `#`, e la ignorano.

L0.5 Le regole dell'assemblatore

L'assemblatore integrato di QtRvSim è volutamente semplice: legge il sorgente una riga alla volta e scrive ogni istruzione in memoria, una dopo l'altra, a partire da `0x00000200`, l'indirizzo da cui il processore simulato comincia a eseguire. Da questa scelta discendono quasi tutte le sue particolarità.

Prima il codice, poi i dati

QtRvSim accetta le direttive `.text` e `.data`, ma le ignora: non esistono una zona del codice e una dei dati, c'è solo «il punto in cui scrivere il prossimo elemento». Un programma che comincia con i dati, come si usa in RARS,

```
        .data
msg:    .asciz "dati prima\n"
        .text
_start: li   a0, 1
        ...
```

si ferma subito con un errore:

```
Instruction with following encoding is not supported: 69746164
```

Il processore ha prelevato da `0x00000200` il primo dato e ha tentato di eseguirlo. I byte `0x64`, `0x61`, `0x74`, `0x69`, letti come parola little-endian, danno `0x69746164`: sono le lettere `d`, `a`, `t`, `i`. È l'idea del programma memorizzato (stato 0x2) vista dal lato sbagliato: per la memoria istruzioni e dati sono tutti numeri, e il processore esegue qualunque cosa il PC indichi.

La regola è semplice: *in QtRvSim il codice viene prima dei dati*. Se vuoi i dati a un indirizzo preciso, comodo da ritrovare nel pannello della memoria, usa la direttiva `.org`, che sposta il punto di scrittura. Gli esempi che accompagnano QtRvSim collocano i dati a `0x00000400`:

Tabella L0.3 Le direttive più comuni nei quattro strumenti. «Ignorata» significa che l'assemblatore la accetta ma non produce alcun effetto.

Direttiva	QtRvSim	RARS	Ripes	GNU as
`.text`, `.data`	ignorate	sì; dati solo dopo `.data`	sì	sì
`.globl`	ignorata	sì	ignorata	sì
`.word`, `.byte`	sì	sì	sì	sì
`.half`	no	sì	sì	sì
`.asciz`	sì	sì	sì	sì
`.ascii`	sì	sì	no	sì
`.string`	no	sì	sì	sì
`.space`	sì	sì	no	sì
`.align`	no	sì	sì	sì
`.equ`	sì	ignorata (usa `.eqv`)	sì	sì
`.org`	sì	ignorata	no	sì
`.float`, `.double`	no	sì	no	sì

```
        ...
        ecall                   # fine del codice
        .org 0x400              # i dati da 0x400 in poi
valore: .word 10, 20, 0
```

Attenzione però: se il codice supera i 512 byte che separano `0x200` da `0x400`, cioè 128 istruzioni, i dati si sovrappongono al codice.

Le direttive

QtRvSim riconosce poche direttive: `.word` e `.byte` per i numeri, `.ascii` e `.asciz` per i testi (la seconda aggiunge lo zero finale), `.space` o `.skip` per riservare dei byte a zero, `.equ` o `.set` per dare un nome a una costante, `.org` per spostare il punto di scrittura. Non ci sono `.half` per le mezze parole, `.string`, che altrove è un sinonimo di `.asciz`, né `.align`. Gli altri strumenti hanno scelto insiemi diversi, e la tabella L0.3 li mette a confronto.

!

Etichette fuori posto. Prima di ogni `.word` e di ogni istruzione, QtRvSim allinea l'indirizzo al primo multiplo di 4, riempiendo con zeri. L'etichetta, però, riceve l'indirizzo di prima. Dopo `msg: .asciz "abc!"`, che occupa 5 byte a partire da `0x218`, la riga `b: .word 7` colloca il 7 a `0x220`, ma l'etichetta `b` vale `0x21d`. Una `lw` da `b` legge allora i tre byte di riempimento e il primo byte della parola, e QtRvSim non segnala nulla: il registro riceve `0x07000000`. Il rimedio è semplice, e vale per tutti gli strumenti: nei dati, colloca prima le parole e poi i byte e le stringhe.

Una pseudo-istruzione, due istruzioni

Nella prova del banco, `li a0, 1` è diventata `lui` più `addi`. La tabella 1.6 dello stato 0x1 diceva che per una costante piccola è sufficiente una `addi`: è la traduzione adottata da RARS e dall'assemblatore GNU. QtRvSim, invece, traduce sempre `li` in due istruzioni, qualunque sia il valore. Il programma funziona ugualmente, ma è più lungo: se in un laboratorio conti istruzioni o cicli, considera quelle del pannello *Program*, non quelle del sorgente. E se vuoi davvero una sola istruzione, scrivila: `addi a0, zero, 1`.

Le chiamate di sistema

Le `ecall` di QtRvSim riproducono un sottoinsieme delle chiamate di Linux: 93 termina, 64 scrive, 63 legge, e ci sono pochi altri servizi per aprire e chiudere file (tabella B.5 nell'appendice B). Non c'è un servizio che stampa un numero, come l'1 di RARS: per QtRvSim una `ecall` con `a7 = 1` è un servizio sconosciuto, e il programma si ferma. Per stampare il valore di un registro bisogna prima convertirlo in una sequenza di caratteri, cifra per cifra, ed è un ottimo esercizio di divisioni e cicli. Nel frattempo, il pannello dei registri mostra tutte le informazioni necessarie.

Stack e dispositivi

All'avvio, QtRvSim pone lo stack pointer `sp` a `0xbfffff00`; gli altri registri valgono zero, `gp` compreso. In fondo allo spazio degli indirizzi ci sono le periferiche, che si comandano con load e store come nello stato 0x10: la porta seriale collegata al terminale ha i registri a partire da `0xffffc000`, i LED e le manopole da `0xffffc100`, lo schermo da `0xffe00000`. La porta seriale non è il 16550 della macchina `virt` di QEMU visto nello stato 0x10: è un circuito più semplice, con un registro di stato e uno dati per la trasmissione e altrettanti per la ricezione. Il principio, però, è lo stesso: si attende che il bit «pronto» valga 1, poi si scrive il carattere.

L0.6 Gli altri strumenti

RARS

RARS (*RISC-V Assembler and Runtime Simulator*) è un ambiente completo, scritto in Java: un editor, un assemblatore, un simulatore dell'ISA e una serie di strumenti aggiuntivi. Deriva da MARS, un simulatore MIPS scritto da due docenti americani, Pete Sanderson e Kenneth Vollmar, per accompagnare il libro di Patterson e Hennessy, e ne ha ereditato l'aspetto e le convenzioni. Non mostra nulla dell'organizzazione interna del processore, ma è molto

comodo per scrivere e correggere programmi: si può eseguire un'istruzione alla volta e anche tornare indietro, e i servizi della `ecall` sono numerosi, dalla stampa di un intero (`1`) o di un carattere (`11`) alla lettura di un numero dalla tastiera (`5`).

Installarlo. Serve Java, dalla versione 8 in poi: se non è installato, è adatta qualsiasi distribuzione gratuita di OpenJDK. Poi si scarica il file `rars1_6.jar` dalla pagina `github.com/TheThirdOne/rars/releases`, e lo si avvia con un doppio clic o dal terminale:

```
java -jar rars1_6.jar
```

La 1.6, del 2023, è l'ultima versione stabile al momento della stampa.

Che cosa cambia. I dati devono sempre seguire un `.data`: RARS non accetta `.word` o `.asciz` nella zona del codice. Il codice comincia a `0x00400000` e i dati a `0x10010000`, gli indirizzi che hai visto negli esperimenti degli stati 0x2 e 0x6. RARS ha la virgola mobile, F e D, ma non le istruzioni atomiche, e dei livelli di privilegio implementa solo quello utente: non dispone di `mret` né dei CSR del livello macchina. Per questo alcuni esperimenti degli stati usano RARS e non QtRvSim: la somma 0,1 + 0,2 dello stato 0x5 richiede la virgola mobile, e il programma che legge le proprie istruzioni dello stato 0x2 stampa i numeri in esadecimale con il servizio `34`.

Fra gli strumenti aggiuntivi, nel menu *Tools*, ci sono un simulatore di cache (*Data Cache Simulator*), uno per il predittore dei salti (*BHT Simulator*), un contatore delle istruzioni eseguite, uno schermo a pixel (*Bitmap Display*) e una tastiera con terminale comandati con load e store.

Ripes

Ripes, scritto da Morten Borup Petersen e presentato nel 2021 a un convegno sull'insegnamento dell'architettura dei calcolatori, è anch'esso un simulatore della microarchitettura. Si installa come QtRvSim, dalla pagina delle versioni del progetto:

`github.com/mortbopet/Ripes/releases`

Su Linux è un unico file AppImage, da rendere eseguibile con `chmod +x` e avviare; su Windows e su macOS un archivio da estrarre (su Windows serve la libreria di sistema di Visual C++, che di solito è già presente). Anche Ripes ha una versione sperimentale che funziona nel browser, a `ripes.dk`. Al momento della stampa l'ultima versione è la 2.2.6.

Che cosa cambia. I processori fra cui scegliere sono sei: quello a ciclo singolo, quattro varianti della pipeline a cinque fasi (senza propagazione né rilevamento dei conflitti, con uno solo dei due, con entrambi) e una pipeline a sei fasi a doppia emissione, come quelle dello stato 0xc. Tutti tranne l'ultimo si possono visualizzare in due schemi, uno semplificato e uno esteso con tutti i segnali di controllo. Ripes ha due funzioni che QtRvSim non ha: il comando *Reverse*, che torna indietro di un ciclo, e il diagramma della pipeline (*Show pipeline diagram*, nella documentazione *stage table*), come quelli dello stato 0x9, che mostra in quale fase si trovava ogni istruzione a ogni ciclo. Il simulatore di cache traccia anche l'andamento del tasso di hit durante l'esecuzione.

Per contro, Ripes supporta solo le estensioni M e C: non dispone di CSR, di istruzioni atomiche e nemmeno di `ebreak` o `fence`. Il codice comincia a `0x00000000` e i dati a `0x10000000`. Fra le direttive manca `.ascii` (si usa `.asciz`); le `ecall` sono in gran parte quelle di RARS, comprese le tre di Linux che conosciamo.

GCC e QEMU

Gli strumenti dei professionisti sono un compilatore completo e un emulatore. GCC, il compilatore del progetto GNU, esiste in una versione che produce codice RISC-V; QEMU emula molti processori, RISC-V compreso. Su Linux, o su Windows con WSL, il sottosistema che consente di eseguire Linux all'interno di Windows, si installano con un comando:

```
sudo apt install gcc-riscv64-unknown-elf qemu-user
```

Su macOS si usa Homebrew:

```
brew install riscv64-elf-gcc
```

In questo caso i comandi cominciano con `riscv64-elf-` invece che con `riscv64-unknown-elf-`. Il nome contiene `riscv64`, ma lo stesso compilatore produce anche codice a 32 bit. Ecco come assemblare la prova del banco ed eseguirla:

```
riscv64-unknown-elf-gcc -march=rv32im -mabi=ilp32 \
        -nostdlib -static -o ciao ciao.s
qemu-riscv32 ./ciao
```

`-march=rv32im` richiede istruzioni RV32I con l'estensione M; `-mabi=ilp32` stabilisce che `int`, `long` e i puntatori sono a 32 bit; `-nostdlib` rinuncia alla libreria del C, che a un programma assembly non serve; `-static` produce un

eseguibile autosufficiente. Sono le opzioni da usare anche per gli eseguibili destinati a QtRvSim, che li carica con *Browse* e *Load machine* nella finestra di configurazione.

Che cosa cambia. Il linker cerca l'etichetta `_start` come punto d'ingresso: se manca, emette un avviso e usa come punto d'ingresso l'inizio del codice. Per questo la prova del banco dichiara `_start` con `.globl`, anche se per i simulatori non è necessario. Il codice non inizia all'inizio di una pagina: nella prova del banco la prima istruzione è a `0x00010094`, perché il file viene caricato a partire da `0x00010000` e i primi byte sono occupati dalle intestazioni ELF. Soprattutto, `qemu-riscv32` esegue il programma come un vero processo Linux: le `ecall` sono quelle del sistema operativo che lo ospita, la velocità è prossima a quella di un processore reale, e non è visibile nulla dell'organizzazione interna. Questo modo di QEMU, che emula un solo programma e non un calcolatore intero, non esiste su macOS né su Windows (dove si usa WSL); su macOS il compilatore funziona, e l'eseguibile si carica in QtRvSim.

Fra gli strumenti di GCC ce n'è uno da ricordare: il disassemblatore, che legge un eseguibile e ne stampa le istruzioni, con gli indirizzi e la codifica. È il modo più rapido per verificare in quali istruzioni sono state tradotte le pseudo-istruzioni.

```
riscv64-unknown-elf-objdump -d ciao
```

Compiler Explorer

Compiler Explorer, all'indirizzo `godbolt.org`, mette a disposizione dal browser centinaia di compilatori, eseguiti sui suoi server. A sinistra si scrive un programma in C, a destra compare l'assembly generato, con i colori che associano ogni riga del C alle istruzioni che ne sono nate. Per RISC-V si sceglie un compilatore come *RISC-V (32-bits) gcc* o *RISC-V rv32gc clang*, e nel campo delle opzioni si scrivono `-march=rv32im -mabi=ilp32`, per ottenere solo istruzioni supportate da QtRvSim, e il livello di ottimizzazione. Con `-O2`, la funzione

```
int somma(int a, int b) {
    return a + b;
}
```

diventa due sole istruzioni, `add a0, a0, a1` e `ret`: gli argomenti arrivano in `a0` e `a1`, il risultato viene restituito in `a0`, come prescrive l'ABI dello stato 0x1. Senza ottimizzazioni, con `-O0`, le istruzioni diventano una dozzina: il compilatore salva gli argomenti sullo stack e li ricarica, senza sfruttare i valori

già presenti nei registri. Perché lo faccia, e che cosa si guadagna ottimizzando, sarà il tema dell'ultimo laboratorio.

2012 Un sito per chiudere una discussione

Nel 2012 Matt Godbolt, un programmatore che lavorava per una società di trading, discuteva con un collega: in C++, scorrere un vettore con un indice è più veloce o più lento che scorrerlo con un iteratore? Invece di misurare, i due confrontarono l'assembly prodotto dal compilatore. L'idea gli piacque, e in una settimana di tempo libero Godbolt ne fece una pagina web che compilava il codice e ne mostrava l'assembly; a maggio ottenne il permesso di pubblicarla. La chiamò *GCC Explorer*; nel 2014 diventò *Compiler Explorer*, ma l'indirizzo è rimasto quello con il suo cognome. Oggi è uno degli strumenti più usati da chi scrive compilatori, e da chi li vuole capire.

> **E ancora.** Gli strumenti per RISC-V sono molti di più. Venus, usato nel corso CS 61C di Berkeley, e CPUlator, che simula anche ARM e MIPS, funzionano nel browser. Spike è il simulatore di riferimento: chi progetta un processore RISC-V confronta il proprio circuito con Spike, istruzione per istruzione, per verificare di aver rispettato la specifica. Il nome ricorda il chiodo d'oro, il *golden spike*, piantato nel 1869 per celebrare il completamento della prima ferrovia transcontinentale americana.

L0.7 Un programma per tutti

La tabella L0.4 riassume che cosa offre ogni strumento, la tabella L0.5 dove colloca il programma. Messe insieme, spiegano quasi tutte le differenze che incontrerai.

Il risultato pratico è un piccolo insieme di regole. Un programma che le rispetta, come la prova del banco, funziona senza modifiche in QtRvSim, in RARS, in Ripes e con GCC e QEMU: lo abbiamo verificato in tutti e quattro.

- Il codice viene per primo, dopo un `.text`; i dati vengono dopo, preceduti da `.data`.
- La prima istruzione ha l'etichetta `_start`, dichiarata con `.globl`.
- Per i dati sono sufficienti `.word`, `.byte` e `.asciz`, e le parole vengono prima dei byte e delle stringhe.
- Le uniche `ecall` sono la 64 per scrivere, la 63 per leggere e la 93 per terminare.
- Le istruzioni sono quelle di RV32I, tranne `ebreak` e `fence`, che Ripes non supporta, e quelle dell'estensione M.
- Il programma non dipende dagli indirizzi: le etichette si raggiungono con

Tabella L0.4 Gli strumenti a confronto.

Strumento	Dove gira	Che cosa mostra	Estensioni	Servizi della `ecall`
QtRvSim	Windows, macOS, Linux, browser	datapath, pipeline, cache, CSR, periferiche	IMA, CSR	Linux (63, 64, 93…)
RARS	ovunque ci sia Java	registri e memoria; strumenti per cache e salti	IMFD	propri (1, 4, 5, 10, 11…) e Linux
Ripes	Windows, macOS, Linux, browser	datapath, diagramma della pipeline, cache	IMC	di RARS (1, 4, 10, 11…) e Linux
GCC e QEMU	GCC ovunque, QEMU su Linux (Windows con WSL)	nulla: solo l'uscita del programma	tutte	Linux, completi
Compiler Explorer	browser	l'assembly prodotto dal compilatore	tutte	—

Tabella L0.5 Dove ogni strumento colloca il programma: l'indirizzo della prima istruzione, quello dei dati, il valore iniziale di `sp` e i registri dei dispositivi.

Strumento	Codice	Dati	`sp`	Dispositivi
QtRvSim	`0x00000200`	dopo il codice	`0xbfffff00`	da `0xffe00000`
RARS	`0x00400000`	`0x10010000`	`0x7fffeffc`	da `0xffff0000`
Ripes	`0x00000000`	`0x10000000`	`0x7ffffff0`	da `0xf0000000`
GCC e QEMU	poco dopo `0x00010000`	dopo il codice	scelto da QEMU	nessuno

`la`, mai scrivendo l'indirizzo a mano.

Nei laboratori useremo QtRvSim, e potremo concederci le sue comodità, come `.org` e le periferiche. Ma quando un programma trovato in rete, o scritto per un altro strumento, si rifiuta di funzionare, queste regole indicano dove cercare la causa.

1990 **MIPS al contrario**

L'11 maggio 1990 James Larus, professore all'Università del Wisconsin, annunciò su Usenet un programma che aveva scritto per il suo corso di compilatori: un simulatore che eseguiva l'assembly dei processori MIPS R2000 e R3000. Lo chiamò SPIM, cioè MIPS letto al contrario, e avvertò che era «abbastanza lento»: i programmi venivano eseguiti a un venticinquesimo della velocità di una vera workstation MIPS. SPIM accompagnò per molte edizioni il libro di Patterson e Hennessy, che gli dedicava un'appendice scritta dallo stesso Larus. MARS ne riprese le convenzioni, RARS le ereditò da MARS, e gli indirizzi `0x00400000` e `0x10010000` che RARS usa ancora oggi vengono da lì. Anche QtRvSim gli rende omaggio: la sua porta seriale compare due volte nello spazio degli indirizzi, e la seconda, a `0xffff0000`, è dove la cercavano i programmi scritti per SPIM e MARS.

Condizione di transizione uscita verso **L1**

Se sai rispondere a tutte le domande, passa al laboratorio L1. Se no, la freccia ↺ ti dice dove tornare.

1 Che differenza c'è fra un simulatore come QtRvSim e un emulatore come QEMU? Quando conviene l'uno e quando l'altro? ↺ § L0.1

2 Perché una `add` produce lo stesso risultato in tutti gli strumenti, e una `ecall` no? ↺ § L0.1

3 Quale preset di QtRvSim sceglieresti per vedere un conflitto sui dati produrre un risultato sbagliato? ↺ § L0.3

4 Perché il programma della prova del banco si ferma dopo la `ecall` numero 64, anche se non è finito? ↺ § L0.4

5 Un programma comincia con `.data` e una stringa, e solo dopo ha `.text` e il codice. Perché funziona in RARS e non in QtRvSim? Come lo correggi? ↺ § L0.5

6 Quanti byte occupa `li t0, 5` in QtRvSim? E in RARS? ↺ § L0.5

7 Un programma usa `li a7, 1` ed `ecall` per stampare un numero. In quali strumenti funziona, e come si comporta QtRvSim? ↺ § L0.5

8 A che cosa servono le opzioni `-march=rv32im` e `-mabi=ilp32` di GCC? ↺ § L0.6

9 Perché, con GCC, la prima istruzione della prova del banco è a `0x00010094` e non a `0x00010000`? ↺ § L0.6

10 Quali regole deve rispettare un programma per funzionare in QtRvSim, RARS, Ripes e QEMU? ↺ § L0.7

LABORATORIO

L1



Primo programma



→ INGRESSI

- Registri, load e store, pseudo-istruzioni, `ecall` e il primo programma (stato 0x1, § 1.13).
- QtRvSim installato e la prova del banco riuscita (laboratorio L0, § L0.4).
- Preset: *No pipeline no cache*.

USCITE →

- Eseguire un programma un'istruzione alla volta, prevedendo prima che cosa cambierà.
- Leggere registri e memoria nei pannelli di QtRvSim, byte per byte.
- Disegnare la mappa della memoria di un programma: codice, dati, heap, stack e dispositivi.
- Leggere e scrivere testo con le chiamate di sistema.
- Riconoscere gli errori più comuni dai loro sintomi.

Il primo programma dello stato 0x1 legge due numeri dalla memoria, li somma e scrive il risultato. Sulla carta l'abbiamo seguito riga per riga; ora lo eseguiamo effettivamente, un'istruzione alla volta, e osserviamo che cosa succede ai registri e alla memoria. È un esercizio più utile di quanto sembri: chi impara a *prevedere* l'effetto di ogni istruzione prima di premere *Step* ha già imparato metà dell'assembly.

L1.1 Il programma dello stato 0x1

Apri QtRvSim con il preset *No pipeline no cache*, crea un sorgente con *File* → *New source* e scrivi il programma:

```
        .text
        .globl main
main:
        la   t0, valore      # t0 <- indirizzo dell'etichetta valore
        lw   t1, 0(t0)       # t1 <- 10
        lw   t2, 4(t0)       # t2 <- 20
        add  t3, t1, t2      # t3 <- 30
        sw   t3, 8(t0)       # terza parola <- 30
        li   a7, 93          # servizio 93: exit
        li   a0, 0           # codice di uscita 0
        ecall

        .data
valore: .word 10, 20, 0
```

Salvalo, per esempio come `primo.s`, e premi *Compile Source*. Apri anche i registri (`Ctrl+D`) e la memoria (`Ctrl+M`), e attiva *Machine* → *Mnemonics Registers*, così il pannello *Program* usa i nomi `t0`, `a7` invece dei numeri.

Il pannello *Program* mostra il programma come appare al processore: undici istruzioni, da `0x00000200` a `0x00000228`, e non otto come nel sorgente. Le tre pseudo-istruzioni sono diventate due istruzioni ciascuna: `la` è diventata `auipc` e `addi`, come previsto nello stato 0x1; le due `li`, per la scelta di QtRvSim vista nel laboratorio L0, sono diventate `lui` e `addi`. Subito dopo l'ultima istruzione, all'indirizzo `0x0000022c`, cominciano i dati: QtRvSim colloca tutto in sequenza, e la direttiva `.data` non modifica il punto di scrittura.

L1.2 Un'istruzione alla volta

Ora premi *Step* una volta per istruzione. Prima di ogni pressione, prova a prevedere che cosa cambierà; poi verifica nel pannello dei registri, dove il registro appena scritto diventa rosso e quelli appena letti diventano blu. La tabella L1.1 riporta i valori che devi osservare.

Tre momenti meritano attenzione.

- **La coppia `auipc`–`addi`.** La prima istruzione scrive in `t0` il proprio indirizzo, `0x200`; la seconda aggiunge 44, cioè `0x2c`, la distanza fra quell'istruzione e l'etichetta `valore`. L'assemblatore non scrive nel codice l'indirizzo dei dati, ma la loro *distanza* dal PC: per questo il programma funzionerebbe anche se lo collocassimo interamente altrove.

Tabella L1.1 Il primo programma eseguito un passo alla volta in QtRvSim. I valori sono in esadecimale.

PC	Istruzione	Effetto	Che cosa osservare
200	`auipc t0, 0x0`	`t0` = 200	PC più 0×2^{12}
204	`addi t0, t0, 44`	`t0` = 22c	ora `t0` punta a `valore`
208	`lw t1, 0(t0)`	`t1` = a	la parola a 22c
20c	`lw t2, 4(t0)`	`t2` = 14	la parola a 230
210	`add t3, t1, t2`	`t3` = 1e	nessun accesso alla memoria
214	`sw t3, 8(t0)`	M[234] = 1e	cambia la memoria, non i registri
218	`lui a7, 0x0`	`a7` = 0	prima metà di `li a7, 93`
21c	`addi a7, a7, 93`	`a7` = 5d	93 in esadecimale
220	`lui a0, 0x0`	`a0` = 0	
224	`addi a0, a0, 0`	`a0` = 0	
228	`ecall`	fine	il simulatore si ferma

- **Le due `lw`.** Usano lo stesso registro base, con spiazzamenti 0 e 4, e `t0` non cambia: l'indirizzo calcolato vale solo per quell'istruzione.
- **La `sw`.** È l'unica istruzione che non cambia nessun registro. Per osservarne l'effetto, scrivi `22c` nella casella in fondo al pannello della memoria: la terza parola passa da `0` a `1e`.

Nel pannello della memoria c'è anche un menu che seleziona la dimensione delle celle. Con *Byte*, a partire da `0x22c` si leggono `0a 00 00 00 14 00 00 00 1e 00 00 00`: ogni parola comincia dal suo byte meno significativo. È l'ordine little-endian dello stato 0x1, osservato direttamente.

> **Tornare indietro.** QtRvSim non consente di tornare indietro di un passo. Se hai perso il filo dell'esecuzione, *Reload* riporta all'inizio il PC e i registri, e puoi ricominciare; la memoria, però, resta com'è. Se il programma ha cambiato i suoi dati, per ripartire da quelli iniziali ricompila con *Compile Source*. Per fermarti a un punto preciso senza premere *Step* cento volte, fai clic nella colonna *BP* del pannello *Program*, accanto all'istruzione: è un *breakpoint*, e *Run* si fermerà lì.

L1.3 La mappa della memoria

Quando un programma viene caricato, la memoria si divide in zone con compiti diversi (figura L1.1).

Codice

Le istruzioni, che in QtRvSim cominciano a `0x00000200`. In presenza di un sistema operativo reale questa zona è di sola lettura: un programma

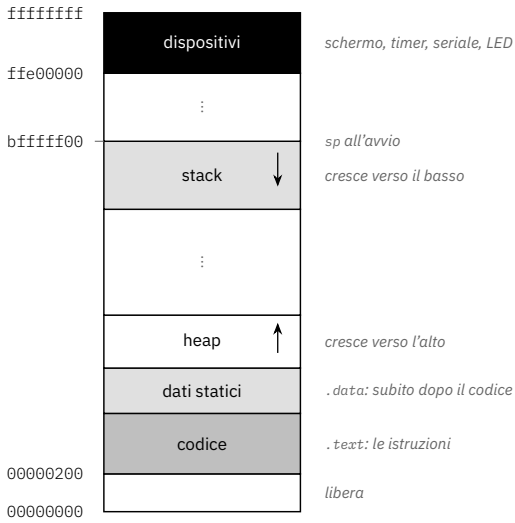


Figura L1.1 La mappa della memoria di un programma in QtRvSim, non in scala. Il codice e i dati occupano poche centinaia di byte in basso; lo stack parte da `0xbfffff00` e scende.

non può modificare le proprie istruzioni.

Dati statici
: Le variabili che esistono per tutta la durata del programma, con il loro valore iniziale: quelle dichiarate con `.word`, `.asciz` e simili. In QtRvSim si trovano subito dopo il codice, o all'indirizzo indicato da `.org`.

Heap
: La memoria richiesta durante l'esecuzione, quella che in C si ottiene con `malloc`. Comincia dopo i dati statici e cresce verso gli indirizzi alti, man mano che il programma la richiede al sistema operativo.

Stack (pila)
: La memoria delle funzioni: indirizzi di ritorno, registri salvati, variabili locali. Comincia in alto, a `0xbfffff00`, dove punta `sp` all'avvio, e cresce verso il *basso*. Partendo dai due estremi, stack e heap si dividono lo spazio libero intermedio, senza dover decidere in anticipo quanto assegnarne a ciascuno. Gli dedicheremo il laboratorio L4.

Dispositivi
: In cima allo spazio degli indirizzi, i registri delle periferiche, che si comandano con load e store come nello stato 0x10.

Gli altri strumenti adottano scelte diverse (tabella L0.5 nel laboratorio L0):

RARS, per esempio, colloca i dati statici a `0x10010000`. L'ABI prevede anche un registro dedicato ai dati statici, `gp` (*global pointer*): se punta in mezzo ai dati, `gp` più uno spiazzamento di 12 bit raggiunge 4 KiB di variabili con una sola istruzione, senza `auipc`. Il linker, quando può, lo usa così, come vedremo nel laboratorio L6; in QtRvSim `gp` vale zero, e i dati si raggiungono con `la`.

Per collocare la zona dei dati in un punto preciso, aggiungi `.org 0x400` prima dell'etichetta `valore`, ricompila e riesegui: ora `addi` somma un numero diverso, perché la distanza fra il codice e i dati è cambiata, e la terza parola compare a `0x408`.

L1.4 Comunicare con il terminale

Un programma che non comunica è difficile da controllare. Le chiamate di sistema 64 e 63 scrivono e leggono byte: nella 64, che conosciamo dalla prova del banco, `a0` indica dove scrivere (1 è lo schermo), `a1` l'indirizzo dei byte e `a2` il loro numero; la 63 è simmetrica, con `a0` = 0 per la tastiera, `a1` l'indirizzo in cui depositare i byte letti e `a2` il massimo da leggere. Al ritorno, `a0` contiene il numero di byte effettivamente letti.

Il programma seguente legge una riga e la ripete:

```
        .globl _start
        .text
_start: li   a0, 0            # 0 = ingresso standard (la tastiera)
        la   a1, buf          # dove mettere i byte letti
        li   a2, 32           # al massimo 32 byte
        li   a7, 63           # servizio 63: read
        ecall                 # a0 = byte letti davvero
        mv   a2, a0           # ne scrive tanti quanti ne ha letti
        li   a0, 1
        la   a1, buf
        li   a7, 64           # servizio 64: write
        ecall
        li   a0, 0
        li   a7, 93
        ecall

        .data
buf:    .space 32             # 32 byte a zero
```

In QtRvSim la tastiera è la casella *Input* in fondo al terminale. Il simulatore non si sospende in attesa: quando il programma esegue la `read`, preleva i caratteri presenti nella casella in quel momento, e al termine aggiunge un a capo. Quindi scrivi il testo *prima* di premere *Run*, senza premere Invio (Invio aggiungerebbe un secondo a capo). Se scrivi `Mobius`, la `read` deposita in `buf` i

7 byte `Mobius\n` e restituisce 7, e la `write` li ripete nel terminale.

!

La lunghezza giusta. La chiamata 64 scrive esattamente `a2` byte, né uno di più né uno di meno: ignora il concetto di stringa e di zero finale. Con un valore troppo piccolo il messaggio risulta troncato; con uno troppo grande vengono scritti anche i byte che seguono in memoria, spesso zeri invisibili o pezzi di altre stringhe. Nel prossimo laboratorio impareremo a calcolare la lunghezza con un ciclo.

L1.5 Riconoscere gli errori

Gli errori dei principianti sono sempre gli stessi, e conviene riconoscerli dal sintomo.

L'assemblatore rifiuta il programma

Il pannello *Messages* indica file, riga e motivo. I motivi più frequenti sono una virgola mancante, un'etichetta scritta in due modi diversi, una direttiva che QtRvSim non riconosce come `.string`, o un immediato troppo grande: `lw t1, 0x1000(zero)` produce l'errore `argument range exceed`, perché lo spiazzamento di una load ha 12 bit con segno e raggiunge al massimo 2047. Anche la forma `lw t1, valore`, che RARS accetta, in QtRvSim non è supportata: prima `la`, poi `lw`.

Il programma si ferma con «not supported: a»

Manca la `ecall` finale. Il processore ha superato l'ultima istruzione e ha tentato di eseguire i dati: `0x0000000a` è il numero 10, e non corrisponde ad alcuna istruzione valida. Ogni programma deve terminare esplicitamente.

Un valore letto è assurdo

Spesso l'indirizzo non è allineato: per esempio una `lw` a `0x402` invece che a `0x400`. RARS segnalerebbe un'eccezione, come nello stato 0x6; QtRvSim invece esegue l'accesso e restituisce quattro byte a cavallo di due parole. Il risultato è sbagliato, ma nessun messaggio lo segnala: controlla l'indirizzo nel registro base.

Un registro non cambia

Forse l'istruzione scrive in `zero`, che resta sempre 0, o in un registro diverso da quello previsto: `t1` è `x6`, `s1` è `x9`. Con i nomi convenzionali attivati, il pannello *Program* elimina ogni ambiguità.

L1.6 Esercizi

Per ogni esercizio, prima di eseguire, scrivi su un foglio quali valori ti aspetti nei registri e in memoria; poi controlla con *Step*. Le soluzioni sono nell'ap-

pendice C.

Esercizio L1.1 Tre numeri

Tre parole consecutive in memoria contengono 7, −3 e 12. Scrivi un programma che le sommi e scriva il risultato nella quarta parola. Quanto vale la quarta parola alla fine, in esadecimale?

Esercizio L1.2 Scambio

Due parole `x` e `y` valgono 111 e 222. Scambiale: alla fine `x` deve valere 222 e `y` 111. Quanti registri servono?

Esercizio L1.3 Un'espressione

Con `a = 6`, `b = 4`, `c = 9`, `d = 2` in memoria, calcola $(a + b) \times (c - d)$ e scrivi il risultato dopo i quattro dati. La moltiplicazione è l'istruzione `mul` dell'estensione M.

Esercizio L1.4 Little-endian

Scrivi la parola `0x11223344` in memoria con `sw`; rileggi i suoi quattro byte con quattro `lbu` in `t2`, `t3`, `t4`, `t5`; poi cambia con `sb` solo il byte più significativo in `0xaa` e rileggi la parola intera. Quali valori ti aspetti in ogni registro? Controlla anche con il pannello della memoria in modalità *Byte*.

Esercizio L1.5 Saluto

Scrivi un programma che chieda «Come ti chiami? », legga il nome dal terminale e risponda «Ciao, » seguito dal nome e da un punto esclamativo. Attenzione all'a capo che QtRvSim aggiunge alla fine del testo letto: non deve comparire fra il nome e il punto esclamativo.

Esercizio L1.6 Senza `la`

Riscrivi il programma della § L1.1 collocando i dati a `0x400` con `.org` e raggiungendoli senza `la`, con `lw t1, 0x400(zero)`. Perché funziona? Perché lo stesso accorgimento non funzionerebbe in RARS, dove i dati sono a `0x10010000`?

Condizione di transizione uscita verso **L2**

Se sai rispondere a tutte le domande, passa al laboratorio L2. Se no, la freccia ↺ ti dice dove tornare.

1. A quale indirizzo QtRvSim carica la prima istruzione? E dove colloca i dati, se non c'è un `.org`? ↺ § L1.1
2. In che cosa si traduce `la t0, valore`, e perché la seconda istruzione somma proprio 44? ↺ § L1.2
3. Dopo `sw t3, 8(t0)`, quali sono i byte agli indirizzi da `0x234` a `0x237`? ↺ § L1.2
4. In quale direzione crescono lo stack e l'heap? Perché partono da estremi opposti? ↺ § L1.3
5. L'utente scrive `ciao` nella casella *Input* e il programma esegue una `read` di al massimo 32 byte. Che cosa contiene la memoria, e quale valore viene restituito in `a0`? ↺ § L1.4
6. Un programma i cui dati cominciano con `.word 20` si ferma con «Instruction with following encoding is not supported: 14». Che cosa è successo? ↺ § L1.5

LABORATORIO

L2



If e cicli



→ INGRESSI

- I salti condizionati e le pseudo-istruzioni di salto (stato 0x1, § 1.6 e § 1.8).
- Il laboratorio L1: eseguire un passo alla volta, leggere e scrivere sul terminale.
- Preset: *No pipeline no cache*.

USCITE →

- Tradurre in assembly `if`, `if-else`, `while`, `do-while` e `for`.
- Scorrere una stringa carattere per carattere.
- Stampare un numero intero, cifra per cifra.
- Leggere un numero scritto dall'utente.
- Misurare con il contatore dei cicli di clock quanto costa ogni iterazione di un ciclo.

Un processore esegue le istruzioni una dopo l'altra, e l'unico modo per modificare il flusso di esecuzione è un salto. Con i salti condizionati si costruiscono tutte le strutture di controllo del C: la scelta fra due rami, la ripetizione, l'uscita da un ciclo. In questo laboratorio impariamo a tradurle con alcuni schemi fissi, che torneranno in ogni programma successivo, e scriviamo due frammenti di codice che riutilizzeremo spesso: uno che stampa un numero e uno che lo legge.

L2.1 Scegliere

Consideriamo un `if-else` del C, con `a` in `t0`, `b` in `t1` e `max` in `t2`:

```
if (a < b)
    max = b;
else
    max = a;
```

In assembly si traduce così:

```
        bge  t0, t1, altrimenti   # se NON (a < b), salta al ramo else
        mv   t2, t1               # ramo then: max = b
        j    fine                 # salta oltre il ramo else
altrimenti:
        mv   t2, t0               # ramo else: max = a
fine:
```

Lo schema ha una regola che all'inizio sorprende: il salto verifica la condizione *opposta* a quella del C. Il motivo è che il ramo *then* viene subito dopo il salto, e ci si arriva «cadendo» quando il salto non viene preso; il salto serve a scavalcarlo quando la condizione è falsa. Le coppie da ricordare sono semplici: `<` diventa `bge`, `>=` diventa `blt`, `==` diventa `bne`, `!=` diventa `beq`; `>` diventa la pseudo-istruzione `ble`, `<=` la pseudo-istruzione `bgt`. Il `j fine` alla fine del ramo *then* impedisce di cadere anche nel ramo *else*; senza `else`, non serve.

Il programma seguente usa una catena di `if–else` per stabilire se il numero in memoria è negativo, nullo o positivo. Ogni ramo prepara in `a1` e `a2` il messaggio e la sua lunghezza, e tutti convergono all'etichetta `stampa`.

```
        .globl _start
        .text
_start: la   t0, n
        lw   t1, 0(t0)          # t1 = n
        bltz t1, negativo       # se n < 0
        beqz t1, nullo          # altrimenti, se n == 0
        la   a1, msg_pos        # altrimenti: positivo
        li   a2, 9
        j    stampa
negativo:
        la   a1, msg_neg
        li   a2, 9
        j    stampa
nullo:  la   a1, msg_zero
        li   a2, 5
stampa: li   a0, 1              # i tre rami si ritrovano qui
        li   a7, 64
        ecall
        li   a0, 0
        li   a7, 93
        ecall

        .data
n:      .word -7
msg_neg:  .asciz "negativo\n"
msg_zero: .asciz "zero\n"
msg_pos:  .asciz "positivo\n"
```

Prova a cambiare il numero in 0 e in 15, e segui con *Step* quale ramo viene eseguito. Nota il nome dell'etichetta `nullo`: `zero` è il nome di un registro, e RARS rifiuta un'etichetta con quel nome. QtRvSim la accetta, ma è meglio evitare la confusione.

L2.2 Ripetere

Un ciclo è un salto all'indietro. La forma più diretta è quella del `while`: un controllo all'inizio, che esce quando la condizione è falsa, e un salto incondizionato alla fine, che torna al controllo. Ecco la somma dei numeri da 1 a 10:

```
        .globl _start
        .text
_start: li   t0, 1              # i = 1
        li   t1, 0              # s = 0
        li   t2, 10             # n = 10
ciclo:  bgt  t0, t2, fine       # while (i <= n) {
        add  t1, t1, t0         #     s = s + i;
        addi t0, t0, 1          #     i = i + 1;
        j    ciclo              # }
fine:   mv   a0, t1             # codice di uscita: s
        li   a7, 93
        ecall
```

Il `for` del C è lo stesso schema: `for (i = 1; i <= n; i++)` si traduce ponendo `i = 1` prima del ciclo e `i++` in fondo al corpo. Il programma termina con la somma come codice di uscita; prima della `ecall` la trovi anche in `t1`, cioè 55, `0x37`.

Ogni iterazione esegue quattro istruzioni, e due sono salti: il controllo in cima e il `j` in fondo. Si ottiene un risultato migliore spostando il controllo in fondo al ciclo, come in un `do–while`:

```
        .globl _start
        .text
_start: li   t0, 1              # i = 1
        li   t1, 0              # s = 0
        li   t2, 10             # n = 10
        bgt  t0, t2, fine       # se non va eseguito nemmeno una volta
ciclo:  add  t1, t1, t0         # do {  s = s + i;
        addi t0, t0, 1          #       i = i + 1;
        ble  t0, t2, ciclo      # } while (i <= n);
fine:   mv   a0, t1
        li   a7, 93
        ecall
```

Ora ogni iterazione ha un solo salto, e tre istruzioni invece di quattro. Il controllo prima del ciclo serve solo se il ciclo potrebbe non essere eseguito nemmeno una volta: con `n` minore di 1, il `for` del C non esegue mai il corpo, mentre un `do–while` lo esegue sempre almeno una volta.

Quanto si guadagna? È QtRvSim stesso a indicarlo. Con il processore a ciclo singolo ogni istruzione dura un ciclo, e il contatore *Cycles* in alto a destra della scheda *Core* conta le istruzioni eseguite. Esegui i due programmi fino alla fine: il primo raggiunge 51, il secondo 41. I dieci cicli di differenza sono i dieci `j` risparmiati. In un ciclo eseguito un milione di volte, il risparmio sarebbe di un milione di istruzioni. Per questo i compilatori traducono quasi

sempre i cicli nella seconda forma, come vedremo nel laboratorio L6.

> **Contare a mano.** Nel primo programma ci sono 6 istruzioni prima del ciclo (le tre `li` di QtRvSim valgono due istruzioni ciascuna), $10 \times 4 = 40$ nelle dieci iterazioni, 1 per l'ultimo controllo, che esce, e 4 per terminare (`mv`, la `li` doppia e la `ecall`): 51. Rifare il conto è il modo migliore per verificare di aver capito il flusso del programma.

L2.3 Caratteri e stringhe

Per il processore un carattere è un numero di un byte, il suo codice ASCII: `A` vale 65, `a` 97, la cifra `0` vale 48, l'a capo 10, lo spazio 32. Una stringa è una sequenza di byte consecutivi, terminata da un byte a zero, quello che aggiunge `.asciz`. RARS e l'assemblatore GNU accettano anche la forma `li t0, 'a'`; QtRvSim no, e richiede il valore numerico. Per leggibilità, riportalo sempre in un commento, o assegnagli un nome con `.equ`: dopo `.equ ACAPO, 10` si può scrivere `li t0, ACAPO`.

Scorrere una stringa significa leggere un byte alla volta finché non si trova lo zero. Ecco come si calcola la lunghezza di una stringa e la si stampa, senza dover contare i caratteri a mano come nel laboratorio L1:

```
        .globl _start
        .text
_start: la   s0, stringa          # indirizzo del primo carattere
        li   t0, 0                # contatore dei caratteri
ciclo:  add  t1, s0, t0           # indirizzo del carattere numero t0
        lbu  t2, 0(t1)            # il carattere, senza segno
        beqz t2, fine             # trovato lo zero finale: esce
        addi t0, t0, 1
        j    ciclo
fine:   li   a0, 1
        mv   a1, s0
        mv   a2, t0               # la lunghezza appena calcolata
        li   a7, 64
        ecall
        li   a0, 0
        li   a7, 93
        ecall

        .data
stringa: .asciz "Architettura dei calcolatori\n"
```

`s0` punta sempre al primo carattere e `t0` funge da contatore; ogni iterazione calcola l'indirizzo $s0 + t0$ e legge un byte. Quando il byte letto è zero, `t0`

contiene proprio il numero di caratteri che lo precedono, cioè la lunghezza, ed è pronto per `a2`.

> **`lb` o `lbu`?** Per i caratteri ASCII, che vanno da 0 a 127, le due istruzioni producono lo stesso risultato. La differenza compare con i byte da 128 a 255, per esempio le lettere accentate in alcune codifiche: `lb` estende il segno e restituisce un numero negativo, `lbu` no. Per confrontare caratteri, `lbu` è la scelta più sicura.

L2.4 Stampare un numero

QtRvSim non ha un servizio che stampa un numero: la `write` scrive byte, e il numero 1234 va prima trasformato nei quattro caratteri `1`, `2`, `3`, `4`, cioè nei byte 49, 50, 51, 52. Il metodo è quello del calcolo manuale: il resto della divisione per 10 è l'ultima cifra, il quoziente è il numero senza l'ultima cifra.

$$1234 = 123 \times 10 + 4, \quad 123 = 12 \times 10 + 3, \quad 12 = 1 \times 10 + 2, \quad 1 = 0 \times 10 + 1.$$

Le cifre si ottengono da destra a sinistra, quindi conviene riempire un *buffer*, una zona di memoria di appoggio, partendo dalla fine, e alla fine stampare la porzione riempita. A ogni cifra si somma 48, il codice del carattere `0`.

```
        .globl _start
        .text
_start: la   t0, numero
        lw   t1, 0(t0)          # il numero da stampare
        la   t2, fine_buf       # il buffer si riempie da destra
        li   t5, 10             # 10 = codice ASCII dell'a capo
        addi t2, t2, -1
        sb   t5, 0(t2)          # l'ultimo carattere: a capo
        li   t3, 10             # la base
        li   t4, 0              # 1 se il numero è negativo
        bgez t1, cifre
        li   t4, 1
        neg  t1, t1             # valore assoluto
cifre:  remu t5, t1, t3         # ultima cifra (0-9)
        divu t1, t1, t3         # toglie l'ultima cifra
        addi t5, t5, 48         # 48 = codice ASCII di '0'
        addi t2, t2, -1
        sb   t5, 0(t2)
        bnez t1, cifre          # almeno una cifra, anche per lo zero
        beqz t4, scrivi
        li   t5, 45             # 45 = codice ASCII di '-'
        addi t2, t2, -1
        sb   t5, 0(t2)
scrivi: li   a0, 1
```

```
        mv   a1, t2              # dal primo carattere
        la   a2, fine_buf
        sub  a2, a2, t2          # lunghezza = fine - inizio
        li   a7, 64
        ecall
        li   a0, 0
        li   a7, 93
        ecall

        .data
numero: .word -1234
buf:    .space 12                # 10 cifre, il segno e l'a capo
fine_buf:
```

Tre dettagli meritano attenzione.

- **Il ciclo è un `do–while`**, con il controllo in fondo: così anche lo zero produce una cifra, `0`, invece di una stringa vuota.
- **Il segno** si gestisce a parte: si memorizza in `t4` se il numero era negativo, si lavora sul valore assoluto e alla fine si aggiunge un `-` davanti.
- **La divisione è senza segno**, `divu` e `remu`, anche se il numero ha un segno. Il motivo è il numero più negativo, -2^{31}: il suo opposto, 2^{31}, non esiste in complemento a 2, e `neg` restituisce di nuovo `0x80000000`. Letto senza segno, però, `0x80000000` vale proprio 2 147 483 648, e le cifre risultano corrette.

Prova il programma con 0, 7, −5, 1000 e −2147483648. La lunghezza da stampare è la differenza fra la fine del buffer e il punto raggiunto da `t2`: un'altra lunghezza calcolata invece che contata.

!

Fermarsi a ogni `ecall`. Un programma che stampa dentro un ciclo esegue molte `write`, e QtRvSim, con le impostazioni predefinite, si ferma dopo ognuna. Per i programmi di questo laboratorio conviene disattivare *Stop on known system call* nella pagina *System Emulation and IRQ* della configurazione (*File → New simulation*).

L2.5 Esercizi

Esercizio L2.1 Il massimo di tre

Tre parole in memoria valgono −4, 17 e 9. Scrivi nella quarta il massimo, usando solo confronti e salti.

Esercizio L2.2 Contare le vocali

Conta le vocali minuscole della frase «il processore esegue una istruzione

alla volta» e termina con il risultato come codice di uscita. Quante istruzioni di salto servono per ogni carattere, nel caso peggiore?

Esercizio L2.3 Tutto maiuscolo

Trasforma in maiuscolo, direttamente in memoria, le lettere minuscole della stringa «risc-v, 32 registri!» seguita da un a capo, poi stampala. Le altre lettere e i simboli devono restare invariati. Suggerimento: `a` e `A` distano 32.

Esercizio L2.4 Fattoriale

Calcola il fattoriale di un numero n in memoria con un ciclo e `mul`. Prova con $n = 12$ e con $n = 13$: il secondo risultato è sbagliato. Perché? Il valore che ottieni ha qualche relazione con quello corretto, 6 227 020 800?

Esercizio L2.5 Il doppio

Leggi un numero dal terminale, con il segno meno se c'è, e stampa il suo doppio. Per trasformare il testo in un numero si esegue il procedimento inverso della stampa: per ogni cifra, $n = n \times 10 +$ cifra. La conversione si arresta al primo carattere che non è una cifra, per esempio l'a capo.

Esercizio L2.6 Un esercizio d'esame

Traduci in assembly questo frammento di C, con `n` e `passi` in memoria, e scopri quanti passi servono partendo da `n = 27`.

```
int n = 27, passi = 0;
while (n != 1) {
    if (n % 2 == 0)
        n = n / 2;
    else
        n = 3 * n + 1;
    passi++;
}
```

Si può risolvere senza moltiplicazioni né divisioni: pensa a quali istruzioni calcolano n mod 2, $n/2$ e $3n$ per un n positivo.

Esercizio L2.7 Numeri primi

Stampa, uno per riga, i numeri primi minori di 50. Un numero n è primo se nessun d fra 2 e $\sqrt{n}$ lo divide; per non calcolare la radice, controlla $d \times d \leq n$. Il ciclo interno cerca un divisore, quello esterno prova i candidati, e la stampa è quella della § L2.4. Quanti cicli di clock servono, in tutto?

Condizione di transizione uscita verso **L3**

Se sai rispondere a tutte le domande, passa al laboratorio L3. Se no, la freccia ↺ ti dice dove tornare.

1 Perché il salto di un `if` verifica la condizione opposta a quella scritta in C? ↺ § L2.1

2 Quali sono i due modi di tradurre un `while`? Quante istruzioni costa ogni iterazione nei programmi della § L2.2? ↺ § L2.2

3 Quando `lb` e `lbu` producono risultati diversi? ↺ § L2.3

4 Come si scrive in QtRvSim `li t0, 'A'`? ↺ § L2.3

5 Perché le cifre di un numero si producono da destra a sinistra, e che cosa comporta per il buffer? ↺ § L2.4

6 Che cosa succederebbe al numero -2^{31} se la stampa usasse `div` e `rem` invece di `divu` e `remu`? ↺ § L2.4

LABORATORIO

L3



Vettori e matrici



→ INGRESSI

- Load e store con registro base e spiazzamento, allineamento (stato 0x1, § 1.4).
- Il laboratorio L2: cicli e contatore dei cicli.
- Preset: *No pipeline no cache*.

USCITE →

- Raggiungere l'elemento i di un vettore con un indice o con un puntatore, e stabilire quale delle due forme è meno costosa.
- Disporre una matrice in memoria riga per riga e calcolare l'indirizzo di un elemento.
- Scrivere la stessa operazione in due modi e confrontarne il costo con QtRvSim.

Il C ci ha abituati a scrivere `v[i]` e `m[y][x]` senza pensarci. Il processore non gestisce né vettori né matrici: opera solo su indirizzi, e una memoria che è una lunga fila di byte. In questo laboratorio impariamo a trasformare gli indici in indirizzi, e scopriamo che il modo di farlo può rendere un programma molte volte più lento, o più veloce.

L3.1 Indice o puntatore

Un vettore di interi è una sequenza di parole consecutive. Se il primo elemento è all'indirizzo v, l'elemento i è all'indirizzo $v + 4i$, perché ogni parola occupa 4 byte. Ci sono due modi di scorrerlo.

Il primo riproduce lo schema del C: si mantiene l'indice `i` in un registro e a ogni iterazione si calcola l'indirizzo. Moltiplicare per 4 è uno scorrimento

a sinistra di due posizioni, `slli`.

```
        .globl _start
        .text
_start: la   s0, v              # indirizzo del primo elemento
        li   s1, 10             # numero di elementi
        li   t0, 0              # i
        li   a0, 0              # somma
ciclo:  slli t1, t0, 2          # i * 4: lo spiazzamento in byte
        add  t1, s0, t1         # &v[i] = v + i * 4
        lw   t2, 0(t1)          # v[i]
        add  a0, a0, t2
        addi t0, t0, 1          # i++
        blt  t0, s1, ciclo
        li   a7, 93             # somma come codice di uscita
        ecall

        .data
v:      .word 3, 1, 4, 1, 5, 9, 2, 6, 5, 3
```

Il secondo usa un **puntatore**: un registro che contiene direttamente l'indirizzo dell'elemento corrente, e che a ogni iterazione avanza di 4. Al posto del contatore si usa l'indirizzo di *fine*, quello subito dopo l'ultimo elemento: il ciclo termina quando il puntatore lo raggiunge.

```
        .globl _start
        .text
_start: la   t0, v              # p = &v[0]
        addi t1, t0, 40         # fine = &v[10], subito dopo l'ultimo
        li   a0, 0              # somma
ciclo:  lw   t2, 0(t0)          # *p
        add  a0, a0, t2
        addi t0, t0, 4          # p++: la parola successiva
        bne  t0, t1, ciclo      # finché p != fine
        li   a7, 93
        ecall

        .data
v:      .word 3, 1, 4, 1, 5, 9, 2, 6, 5, 3
```

Le due versioni calcolano la stessa somma, 39, ma il contatore dei cicli di QtRvSim indica 71 per la prima e 48 per la seconda. Il ciclo con l'indice esegue sei istruzioni per elemento, due delle quali solo per calcolare l'indirizzo; quello con il puntatore ne esegue quattro. Il puntatore è meno leggibile, perché non indica *quale* elemento si sta elaborando, ma è meno costoso: è la forma che preferiscono i compilatori quando ottimizzano.

!

Nessun controllo. Il processore non conosce i limiti di un vettore. Se un ciclo esegue un'iterazione di troppo, legge la parola che segue in memoria, qualunque sia il suo contenuto, e non si verifica alcun effetto visibile: il risultato è semplicemente sbagliato. Se poi *scrive*, distrugge i dati vicini. In C accade lo stesso, ed è all'origine di moltissimi errori e problemi di sicurezza. Controlla sempre la prima e l'ultima iterazione.

L3.2 Il prodotto scalare

Il prodotto scalare di due vettori A e B di n elementi è $A_0B_0+A_1B_1+\cdots+A_{n-1}B_{n-1}$. In C si scrive con un ciclo e due indici; in assembly conviene usare due puntatori, uno per vettore, che avanzano insieme.

```
        .globl _start
        .text
_start: la   t0, va              # puntatore al vettore A
        la   t1, vb              # puntatore al vettore B
        li   t2, 5               # elementi rimasti
        li   a0, 0               # accumulatore
ciclo:  lw   t3, 0(t0)           # A[i]
        lw   t4, 0(t1)           # B[i]
        mul  t5, t3, t4          # prodotto parziale, in un temporaneo
        add  a0, a0, t5          # e poi nell'accumulatore
        addi t0, t0, 4
        addi t1, t1, 4
        addi t2, t2, -1
        bnez t2, ciclo
        li   a7, 93              # A . B come codice di uscita
        ecall

        .data
va:     .word 1, 2, 3, 4, 5
vb:     .word 6, 7, 8, 9, 10
```

Il prodotto di ogni coppia viene scritto prima in un registro temporaneo, `t5`, e solo dopo nell'accumulatore `a0`: RISC-V non ha un'istruzione che moltiplichi e sommi in un'unica operazione, e un'architettura load/store non può nemmeno moltiplicare direttamente due parole in memoria. Il risultato è 130.

L3.3 Matrici

La memoria ha una sola dimensione, la matrice due. Per memorizzarla la si «appiattisce»: si dispongono le righe una dopo l'altra, la prima, poi la seconda,

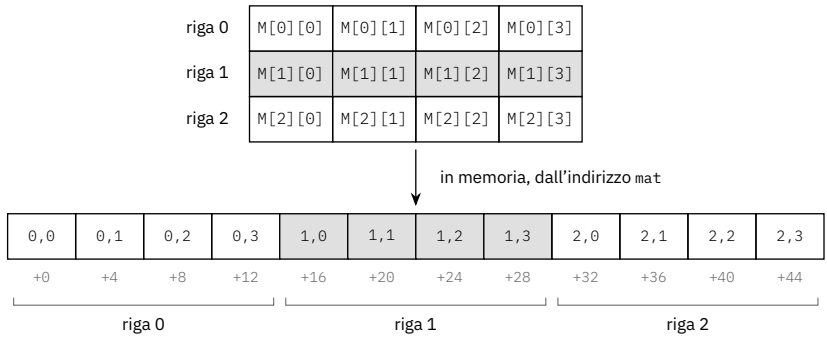


Figura L3.1 Una matrice di 3 righe e 4 colonne e la sua disposizione in memoria, riga dopo riga. La riga 1 (in grigio) comincia 16 byte dopo l'inizio: una riga intera, 4×4 byte.

e così via (figura L3.1). È l'ordine per righe, quello del C.

Per raggiungere l'elemento della riga y e della colonna x in una matrice di parole con `LATO` colonne si saltano y righe intere, ciascuna di `LATO` elementi, e poi x elementi:

$$\text{indirizzo di } M[y][x] = \texttt{mat} + (y \times \texttt{LATO} + x) \times 4.$$

In assembly, con x in `a0`, y in `a1` e `LATO` in `a2`:

```
        mul  t0, a1, a2      # y * LATO: salta le righe precedenti
        add  t0, t0, a0      # + x: la colonna
        slli t0, t0, 2       # * 4: da elementi a byte
        add  t0, s0, t0      # + l'indirizzo della matrice, in s0
        lw   t1, 0(t0)       # M[y][x]
```

Ne seguono due fatti utili. Gli elementi vicini *sulla stessa riga* sono vicini anche in memoria, a 4 byte l'uno dall'altro; gli elementi vicini *sulla stessa colonna* distano una riga intera, $4 \times$ `LATO` byte. Scorrere una matrice per righe significa quindi leggere la memoria in ordine; scorrerla per colonne significa accedere a indirizzi distanti fra loro. Per il risultato non cambia nulla; per la cache, come vedremo nel laboratorio L5, cambia moltissimo.

L3.4 Le diagonali: versione lenta e versione veloce

Vogliamo calcolare la somma degli elementi sulle due diagonali di una matrice quadrata: la principale, dove $x = y$, e l'altra, dove $x + y + 1 =$ `LATO`. Se `LATO` è dispari, le due diagonali si incrociano nell'elemento centrale, che va contato una volta sola.

La soluzione più diretta scorre tutta la matrice con due cicli annidati e, per ogni elemento, verifica se si trova su una diagonale. È corretta per

costruzione, e il centro viene contato una volta sola perché il controllo è un *oppure*:

```
        .globl _start
        .text
_start: la   s0, mat
        li   s1, 5              # LATO
        li   a0, 0              # somma
        li   t0, 0              # y, la riga
righe:  li   t1, 0              # x, la colonna
colonne:
        beq  t0, t1, prendi     # x == y: diagonale principale
        add  t2, t0, t1
        addi t2, t2, 1
        bne  t2, s1, avanti     # x + y + 1 != LATO: nemmeno l'altra
prendi: mul  t2, t0, s1         # y * LATO
        add  t2, t2, t1         # + x
        slli t2, t2, 2          # * 4 byte
        add  t2, s0, t2         # &M[y][x]
        lw   t3, 0(t2)
        add  a0, a0, t3
avanti: addi t1, t1, 1
        blt  t1, s1, colonne
        addi t0, t0, 1
        blt  t0, s1, righe
        li   a7, 93             # somma delle diagonali
        ecall

        .data
mat:    .word  1,  2,  3,  4,  5
        .word  6,  7,  8,  9, 10
        .word 11, 12, 13, 14, 15
        .word 16, 17, 18, 19, 20
        .word 21, 22, 23, 24, 25
```

Questa soluzione, però, è inefficiente: per una matrice 5×5 visita 25 elementi per sommarne 9. Conviene invece saltare direttamente da un elemento della diagonale al successivo. Sulla diagonale principale l'elemento dopo $M[y][y]$ è $M[y+1][y+1]$: una riga sotto e una colonna a destra, cioè $(\texttt{LATO} + 1) \times 4$ byte dopo. Sull'altra diagonale si scende di una riga e si arretra di una colonna: $(\texttt{LATO} - 1) \times 4$ byte. Due puntatori svolgono tutto il lavoro in un unico ciclo; alla fine, se `LATO` è dispari, si sottrae l'elemento centrale, che è stato sommato due volte.

```
        .globl _start
        .text
```

```
_start: la   s0, mat
        li   s1, 5              # LATO
        mv   t0, s0             # p1 parte da M[0][0]
        addi t1, s1, -1
        slli t1, t1, 2
        add  t1, s0, t1         # p2 parte da M[0][LATO-1]
        addi t2, s1, 1
        slli t2, t2, 2          # passo di p1: (LATO + 1) * 4 byte
        addi t3, s1, -1
        slli t3, t3, 2          # passo di p2: (LATO - 1) * 4 byte
        mv   t4, s1             # righe rimaste
        li   a0, 0
ciclo:  lw   t5, 0(t0)
        add  a0, a0, t5
        lw   t5, 0(t1)
        add  a0, a0, t5
        add  t0, t0, t2         # una riga giù e una colonna a destra
        add  t1, t1, t3         # una riga giù e una colonna a sinistra
        addi t4, t4, -1
        bnez t4, ciclo
        andi t5, s1, 1          # LATO dispari? il centro c'è due volte
        beqz t5, fine
        srli t5, s1, 1          # c = LATO / 2
        mul  t6, t5, s1
        add  t6, t6, t5         # c * LATO + c
        slli t6, t6, 2
        add  t6, s0, t6         # &M[c][c]
        lw   t5, 0(t6)
        sub  a0, a0, t5
fine:   li   a7, 93
        ecall

        .data
mat:    .word  1,  2,  3,  4,  5
        .word  6,  7,  8,  9, 10
        .word 11, 12, 13, 14, 15
        .word 16, 17, 18, 19, 20
        .word 21, 22, 23, 24, 25
```

Entrambe le versioni restituiscono 117. La tabella L3.1 riporta i cicli misurati con QtRvSim per tre dimensioni della matrice: la versione lenta cresce con il *quadrato* di `LATO`, quella veloce in proporzione a `LATO`. Per una matrice 1000×1000 la differenza sarebbe di centinaia di volte.

Tabella L3.1 Cicli per la somma delle diagonali, sul processore a ciclo singolo di QtRvSim.

LATO	Versione lenta	Versione veloce
5	220	67
8	499	84
9	608	99

L3.5 Esercizi

Esercizio L3.1 Il massimo e la sua posizione

Trova il valore massimo del vettore `12, -7, 31, 8, 31, 0, 25, -40, 19, 30` e la sua posizione, contando da 0, e scrivili in due parole in memoria. Se il massimo compare due volte, quale posizione restituisce il tuo programma?

Esercizio L3.2 Al contrario

Inverti sul posto il vettore `1, 2, …, 10`, senza usare un secondo vettore. Suggerimento: due puntatori, uno che parte dal primo elemento e avanza, l'altro dall'ultimo e arretra.

Esercizio L3.3 Un esercizio d'esame

Traduci in assembly, con un puntatore, questo frammento di C:

```
int v[8] = {5, 8, 8, 3, 9, 12, 1, 4};
int conta = 0;
for (int i = 1; i < 8; i++)
    if (v[i] > v[i-1])
        conta++;
```

Si può leggere ogni elemento una volta sola?

Esercizio L3.4 Somme delle righe

Data una matrice di 3 righe e 4 colonne, scrivi in un vettore di tre parole la somma di ciascuna riga. Hai bisogno della formula dell'indirizzo?

Esercizio L3.5 La trasposta

Trasponi sul posto una matrice 4×4: l'elemento $M[i][j]$ va scambiato con $M[j][i]$. Attenzione a non scambiare ogni coppia due volte, perché la seconda volta ripristinerebbe la matrice originale.

Esercizio L3.6 Matrice per vettore

Calcola $w = M \times v$, con M di dimensione 3×3 e v di 3 elementi: ogni w_i è il

prodotto scalare della riga i di M per v.

Esercizio L3.7 Misurare

Adatta entrambe le versioni della somma delle diagonali a una matrice 8×8 con i numeri da 1 a 64, e misura i cicli. Ricontrolla i valori della tabella L3.1: quante istruzioni costa ogni elemento visitato dalla versione lenta?

Condizione di transizione uscita verso **L4**

Se sai rispondere a tutte le domande, passa al laboratorio L4. Se no, la freccia ↺ ti dice dove tornare.

1. Quale indirizzo ha l'elemento $v[7]$ di un vettore di parole che comincia a `0x400`? ↺ § L3.1
2. Perché il ciclo con il puntatore esegue meno istruzioni di quello con l'indice? ↺ § L3.1
3. Che cosa succede se un ciclo su un vettore di 10 elementi esegue 11 iterazioni? ↺ § L3.1
4. Perché nel prodotto scalare il prodotto passa da un registro temporaneo prima di essere sommato? ↺ § L3.2
5. A quanti byte di distanza sono $M[2][3]$ e $M[3][3]$ in una matrice di parole con 10 colonne? E $M[2][3]$ e $M[2][4]$? ↺ § L3.3
6. Perché la versione veloce delle diagonali deve sottrarre l'elemento centrale, e quella lenta no? ↺ § L3.4

LABORATORIO

L4



Funzioni e stack



→ INGRESSI

- `jal`, `jalr` e i nomi convenzionali dei registri (stato 0x1, § 1.3 e § 1.6).
- I laboratori L2 e L3: cicli, stampa di un numero, vettori.
- Preset: *No pipeline no cache*.

USCITE →

- Chiamare una funzione e tornare al chiamante.
- Rispettare le convenzioni sui registri: chi salva che cosa.
- Costruire e smontare un record di attivazione nello stack.
- Scrivere funzioni ricorsive e seguirne lo stack in QtRvSim.
- Leggere e scrivere un algoritmo ricorsivo completo, il merge sort.

Finora ogni programma era un blocco unico. I programmi veri sono fatti di funzioni, che si chiamano l'una con l'altra, a volte anche se stesse. Per il processore una funzione non ha nulla di particolare: è una porzione di codice a cui si salta e da cui si ritorna. La difficoltà risiede in tutto il resto: dove collocare gli argomenti, dove lasciare il risultato, come evitare che la funzione chiamata distrugga i registri del chiamante. A questo servono le convenzioni dell'ABI, e lo stack.

L4.1 Chiamare e tornare

Una chiamata richiede due operazioni: saltare all'inizio della funzione e memorizzare l'indirizzo di ritorno. L'istruzione `jal` le esegue entrambe: salta all'etichetta e scrive in `ra` (*return address*) l'indirizzo dell'istruzione successiva,

cioè PC + 4. Alla fine la funzione esegue `ret`, una pseudo-istruzione per `jalr zero, 0(ra)`, che salta all'indirizzo contenuto in `ra`. Gli argomenti sono passati in `a0`–`a7`, il risultato è restituito in `a0` (e in `a1`, se serve una seconda parola).

```
        .globl _start
        .text
_start: li   a0, -4
        li   a1, 17
        jal  massimo            # a0 = massimo(-4, 17)
        li   a1, 9
        jal  massimo            # a0 = massimo(a0, 9)
        li   a7, 93             # il massimo, come codice di uscita
        ecall

# massimo(a0, a1): restituisce in a0 il maggiore dei due
massimo:
        bge  a0, a1, fatto
        mv   a0, a1
fatto:  ret
```

Esegui il programma un passo alla volta e osserva `ra`. La prima `jal`, all'indirizzo `0x210`, salta a `massimo`, `0x22c`, e scrive in `ra` `0x214`; la seconda, a `0x21c`, scrive `0x220`. La stessa `ret` torna ogni volta nel punto corretto, perché legge l'indirizzo che la `jal` ha appena scritto. Il risultato finale è 17.

La funzione `massimo` è una **funzione foglia**: non ne chiama altre. Per questo può evitare di usare lo stack. Se chiamasse un'altra funzione con `jal`, `ra` verrebbe sovrascritto, e la sua `ret` perderebbe l'indirizzo di ritorno.

L4.2 Chi salva che cosa

Una funzione usa dei registri per svolgere i propri calcoli; ma anche il chiamante li sta usando. Chi deve garantire la conservazione dei valori? L'ABI di RISC-V divide i registri in due gruppi, e la colonna «chi lo preserva» della tabella 1.2 nello stato 0x1 indica proprio questo.

Li preserva chi chiama: `t0`–`t6`, `a0`–`a7`, `ra`
: La funzione chiamata può modificarli liberamente. Se il chiamante ha in un registro `t` un valore di cui ha bisogno dopo la chiamata, spetta al chiamante salvarlo prima, o meglio conservarlo altrove.

Li preserva chi è chiamato: `s0`–`s11`, `sp`
: La funzione chiamata deve restituirli inalterati. Se vuole usarli, salva il loro valore all'inizio e lo ripristina alla fine. Il chiamante può quindi contare sul fatto che, dopo una chiamata, i suoi registri `s` sono intatti.

Vediamo la regola in azione. Il codice di stampa del laboratorio L2 diventa una funzione, `stampa_intero`, che riceve il numero in `a0` e usa solo registri `t` e `a`; il programma principale la chiama in un ciclo per stampare i numeri da 1 a 5.

```
        .globl _start
        .text
_start: li   s0, 1              # il contatore sta in un registro s:
        li   s1, 5              # stampa_intero promette di non toccarlo
ciclo:  mv   a0, s0
        jal  stampa_intero
        addi s0, s0, 1
        ble  s0, s1, ciclo
        li   a0, 0
        li   a7, 93
        ecall

# stampa_intero(a0): stampa il numero in a0 e va a capo.
# Usa solo registri t e a, quindi non salva nulla sulla pila.
stampa_intero:
        la   t0, fine_buf       # il buffer si riempie da destra
        li   t3, 10
        addi t0, t0, -1
        sb   t3, 0(t0)          # l'a capo
        li   t1, 10
        li   t2, 0              # 1 se il numero è negativo
        bgez a0, si_cifre
        li   t2, 1
        neg  a0, a0
si_cifre:
        remu t3, a0, t1
        divu a0, a0, t1
        addi t3, t3, 48         # '0'
        addi t0, t0, -1
        sb   t3, 0(t0)
        bnez a0, si_cifre
        beqz t2, si_scrivi
        li   t3, 45             # '-'
        addi t0, t0, -1
        sb   t3, 0(t0)
si_scrivi:
        li   a0, 1
        mv   a1, t0
        la   a2, fine_buf
        sub  a2, a2, t0
        li   a7, 64
        ecall
```

```
        ret

        .data
buf:    .space 12
fine_buf:
```

Il contatore si trova in `s0`, il limite in `s1`: per convenzione, la funzione non li modifica. Ecco invece che cosa succede se il programma principale usa `t0` e `t1`:

```
_start: li   t0, 1              # SBAGLIATO: il contatore in t0
        li   t1, 5
ciclo:  mv   a0, t0
        jal  stampa_intero
        addi t0, t0, 1
        ble  t0, t1, ciclo
```

Il programma stampa `1` e termina. `stampa_intero` usa `t0` come puntatore nel buffer, e al ritorno `t0` contiene un indirizzo, intorno a `0x2ca`: sommato `1`, è molto più grande di `5`, e il ciclo termina. L'errore non è nella funzione ma nel chiamante, che ha fatto affidamento su un registro `t`.

L4.3 Lo stack e il record di attivazione

Una funzione che ne chiama un'altra deve salvare `ra`; una che usa registri `s` deve salvarne il valore. Dove? Non in posizioni fisse della memoria, perché la stessa funzione può essere attiva più volte contemporaneamente, come vedremo con la ricorsione. I valori si salvano nello **stack** (pila), la zona in cima alla memoria a cui punta `sp`.

Lo stack cresce verso il basso. Per riservare spazio si sottrae da `sp`; per liberarlo si somma. Le due operazioni elementari, che spesso si chiamano *push* e *pop*, sono:

```
        addi sp, sp, -4      # push: fa posto a una parola
        sw   ra, 0(sp)       #       e ci scrive ra
        ...
        lw   ra, 0(sp)       # pop:  rilegge ra
        addi sp, sp, 4       #       e libera il posto
```

In pratica non si procede una parola alla volta: all'inizio della funzione, nel **prologo**, si riserva tutto lo spazio necessario con una sola sottrazione, e si salvano i registri a spiazzamenti diversi; alla fine, nell'**epilogo**, si rileggono e si restituisce lo spazio. Lo spazio riservato da una chiamata si chiama **record di attivazione** (*stack frame*). Contiene, quando servono, l'indirizzo di ritorno,

i registri `s` salvati, le variabili locali che non trovano posto nei registri e gli argomenti oltre l'ottavo.

```
        .globl _start
        .text
_start: li   a0, 4
        jal  somma_quadrati       # a0 = 1 + 4 + 9 + 16
        li   a7, 93
        ecall

# somma_quadrati(a0 = n): restituisce 1*1 + 2*2 + ... + n*n
somma_quadrati:
        addi sp, sp, -16          # prologo: un record di 16 byte
        sw   ra, 12(sp)           # chiamerà quadrato: va salvato ra
        sw   s0, 8(sp)            # userà s0 e s1: vanno salvati
        sw   s1, 4(sp)
        mv   s0, a0               # n, che deve sopravvivere alle chiamate
        li   s1, 0                # la somma
sq_ciclo:
        beqz s0, sq_fine
        mv   a0, s0
        jal  quadrato             # a0 = n * n
        add  s1, s1, a0
        addi s0, s0, -1
        j    sq_ciclo
sq_fine:
        mv   a0, s1               # il risultato
        lw   s1, 4(sp)            # epilogo: tutto come prima
        lw   s0, 8(sp)
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret

# quadrato(a0): restituisce a0 * a0. Foglia: non salva nulla
quadrato:
        mul  a0, a0, a0
        ret
```

La funzione `somma_quadrati` chiama `quadrato`, quindi salva `ra`; mantiene `n` e la somma in `s0` e `s1`, che sopravvivono alle chiamate, quindi salva anche quelli. Il record è di 16 byte anche se ne servirebbero 12: l'ABI di RISC-V chiede che `sp` sia sempre un multiplo di 16, per semplificare il lavoro dei compilatori e la gestione dei dati che richiedono un allineamento maggiore. Il risultato è $1 + 4 + 9 + 16 = 30$.

Per osservare il record in QtRvSim, fermati con *Step* subito dopo il prologo e scrivi `bffffef0` nella casella del pannello della memoria: troverai `0x20c`,

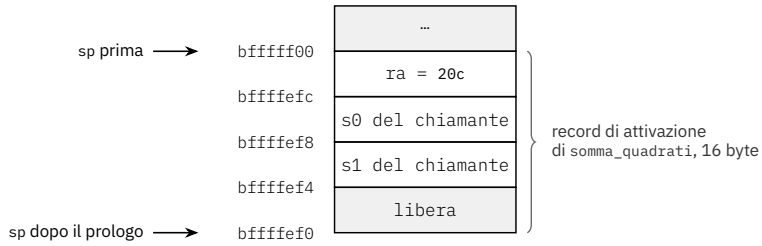


Figura L4.1 Lo stack durante `somma_quadrati`. Ogni riga è una parola; gli indirizzi crescono verso l'alto, lo stack verso il basso.

l'indirizzo di ritorno nel programma principale, a `0xbffffefc`, e i due zeri che `s0` e `s1` contenevano all'inizio (figura L4.1).

> **Il frame pointer.** Gli spiazzamenti del record sono relativi a `sp`, e funzionano finché `sp` rimane invariato durante la funzione. Se la funzione deve riservare altro spazio durante l'esecuzione, per esempio per un vettore di dimensione variabile, `sp` cambia valore e gli spiazzamenti cambiano. Per questo l'ABI prevede un secondo puntatore, il **frame pointer** `fp`, che è un altro nome di `s0`: si fissa all'inizio del record e non viene più modificato. Il codice di un compilatore senza ottimizzazioni lo usa sempre: lo vedremo nel laboratorio L6, con `-O0`.

L4.4 La ricorsione

Una funzione ricorsiva chiama se stessa. Per il processore non c'è nessuna differenza con una chiamata qualsiasi: ogni chiamata ha il suo record nello stack, con il suo `ra` e i suoi valori salvati, e nessuna interferisce con le altre. Il fattoriale è l'esempio più semplice: $n! = n \times (n-1)!$, e $1! = 0! = 1$.

```
        .globl _start
        .text
_start: li   a0, 5
        jal  fatt                # a0 = 5!
        li   a7, 93
        ecall

# fatt(a0 = n): restituisce n!, ricorsivamente
fatt:   addi sp, sp, -16
        sw   ra, 12(sp)
        sw   a0, 8(sp)           # n serve dopo la chiamata
        li   t0, 1
        bgt  a0, t0, fatt_ric    # n > 1: caso ricorsivo
        li   a0, 1               # caso base: 0! = 1! = 1
```

```
        j    fatt_esci
fatt_ric:
        addi a0, a0, -1
        jal  fatt                # a0 = (n - 1)!
        lw   t0, 8(sp)           # ricarica n
        mul  a0, t0, a0          # n * (n - 1)!
fatt_esci:
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret
```

Qui `n` non viene conservato in un registro `s`: si salva nel record insieme a `ra`, e dopo la chiamata si rilegge in un temporaneo. Segui con *Step* la chiamata `fatt(5)`: `sp` scende di 16 a ogni livello, da `0xbffffef0` a `0xbffffeb0`, e in ogni record si trovano n e l'indirizzo di ritorno. Il primo vale `0x20c`, il punto del programma principale; tutti gli altri `0x244`, il punto dentro `fatt` subito dopo la chiamata ricorsiva. Poi lo stack risale, e ogni livello moltiplica il risultato per il proprio n: 120.

La successione di Fibonacci, $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ con $\text{fib}(0) = 0$ e $\text{fib}(1) = 1$, ha due chiamate ricorsive. Il risultato della prima deve sopravvivere alla seconda, e per questo viene conservato in `s1`, che la funzione salva.

```
        .globl _start
        .text
_start: li   a0, 10
        jal  fib                 # a0 = fib(10)
        li   a7, 93
        ecall

# fib(a0 = n): fib(0) = 0, fib(1) = 1, fib(n) = fib(n-1) + fib(n-2)
fib:    addi sp, sp, -16
        sw   ra, 12(sp)
        sw   a0, 8(sp)           # n, che serve per la seconda chiamata
        sw   s1, 4(sp)           # s1 conterrà fib(n-1): va salvato
        li   t0, 2
        blt  a0, t0, fib_esci    # n < 2: fib(n) = n, già in a0
        addi a0, a0, -1
        jal  fib                 # a0 = fib(n-1)
        mv   s1, a0
        lw   a0, 8(sp)           # ricarica n
        addi a0, a0, -2
        jal  fib                 # a0 = fib(n-2)
        add  a0, a0, s1          # fib(n-1) + fib(n-2)
fib_esci:
        lw   s1, 4(sp)           # a0 NON si ripristina: è il risultato
```

```
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret
```

fib(10) = 55, ma per calcolarlo servono 177 chiamate e 2569 cicli: ogni chiamata ne genera altre due, e gli stessi valori vengono ricalcolati molte volte. fib(15) = 610 costa già 28 611 cicli. Lo stack, invece, resta piccolo: non scende mai più di 10 record, 160 byte, perché in ogni momento è attiva una sola catena di chiamate, al massimo profonda n.

!

Non ripristinare il risultato. L'epilogo rilegge i registri salvati nel prologo, ma non tutti: `a0` è stato salvato solo per conservare n durante la prima chiamata, e ora contiene il risultato. Un epilogo che ripristinasse anche `a0` restituirebbe n invece di fib(n). È un errore frequente e subdolo, perché per $n = 0$, 1 e 5 il risultato sembra corretto per coincidenza.

L4.5 Il merge sort

Il merge sort ordina un vettore dividendolo a metà, ordinando ricorsivamente le due metà e poi fondendole: due sequenze già ordinate si fondono in una sola confrontando ogni volta i primi elementi rimasti e selezionando il minore. È un algoritmo *divide et impera*, ed è un ottimo esercizio di convenzioni, perché la funzione ricorsiva deve conservare molti valori attraverso tre chiamate.

```
        .globl _start
        .text
_start: la   a0, v
        la   a1, tmp
        li   a2, 0
        li   a3, 9
        jal  mergesort            # ordina v[0..9]
        la   s0, v                # poi stampa v, un numero per riga
        addi s1, s0, 40
stampa: lw   a0, 0(s0)
        jal  stampa_intero
        addi s0, s0, 4
        bne  s0, s1, stampa
        li   a0, 0
        li   a7, 93
        ecall

# mergesort(a0 = v, a1 = tmp, a2 = sinistra, a3 = destra):
# ordina v[sinistra..destra], usando tmp come appoggio
```

```
mergesort:
        bge  a2, a3, ms_fine      # zero o un elemento: già ordinato
        addi sp, sp, -32          # prologo
        sw   ra, 28(sp)
        sw   s0, 24(sp)           # v
        sw   s1, 20(sp)           # tmp
        sw   s2, 16(sp)           # sinistra
        sw   s3, 12(sp)           # destra
        sw   s4, 8(sp)            # centro
        mv   s0, a0
        mv   s1, a1
        mv   s2, a2
        mv   s3, a3
        add  s4, s2, s3
        srai s4, s4, 1            # centro = (sinistra + destra) / 2
        mv   a2, s2               # a0 e a1 contengono ancora v e tmp
        mv   a3, s4
        jal  mergesort            # ordina la metà sinistra
        mv   a0, s0
        mv   a1, s1
        addi a2, s4, 1
        mv   a3, s3
        jal  mergesort            # ordina la metà destra
        mv   a0, s0
        mv   a1, s1
        mv   a2, s2
        mv   a3, s4
        mv   a4, s3
        jal  merge                # fonde le due metà
        lw   s4, 8(sp)            # epilogo
        lw   s3, 12(sp)
        lw   s2, 16(sp)
        lw   s1, 20(sp)
        lw   s0, 24(sp)
        lw   ra, 28(sp)
        addi sp, sp, 32
ms_fine:
        ret

# merge(a0 = v, a1 = tmp, a2 = sinistra, a3 = centro, a4 = destra):
# fonde v[sinistra..centro] e v[centro+1..destra], già ordinati.
# Foglia: usa solo registri t e a.
merge:  slli t0, a2, 2
        add  t0, a0, t0           # i: puntatore nella metà sinistra
        slli t1, a3, 2
        add  t1, a0, t1           # fine della metà sinistra (compresa)
        addi t2, t1, 4            # j: puntatore nella metà destra
        slli t3, a4, 2
```

```
        add  t3, a0, t3          # fine della metà destra (compresa)
        slli t4, a2, 2
        add  t4, a1, t4          # k: puntatore in tmp
        mv   a5, t4              # inizio della zona usata in tmp
m_ciclo:
        bgtu t0, t1, m_destra    # metà sinistra finita
        bgtu t2, t3, m_sinistra  # metà destra finita
        lw   t5, 0(t0)
        lw   t6, 0(t2)
        bgt  t5, t6, m_prendi_j
        sw   t5, 0(t4)           # prende dalla sinistra
        addi t0, t0, 4
        addi t4, t4, 4
        j    m_ciclo
m_prendi_j:
        sw   t6, 0(t4)           # prende dalla destra
        addi t2, t2, 4
        addi t4, t4, 4
        j    m_ciclo
m_sinistra:                      # copia il resto della metà sinistra
        bgtu t0, t1, m_ricopia
        lw   t5, 0(t0)
        sw   t5, 0(t4)
        addi t0, t0, 4
        addi t4, t4, 4
        j    m_sinistra
m_destra:                        # copia il resto della metà destra
        bgtu t2, t3, m_ricopia
        lw   t5, 0(t2)
        sw   t5, 0(t4)
        addi t2, t2, 4
        addi t4, t4, 4
        j    m_destra
m_ricopia:                       # riporta il risultato da tmp in v
        slli t0, a2, 2
        add  t0, a0, t0
m_copia:
        bgeu a5, t4, m_fatto
        lw   t5, 0(a5)
        sw   t5, 0(t0)
        addi a5, a5, 4
        addi t0, t0, 4
        j    m_copia
m_fatto:
        ret

# stampa_intero(a0): stampa il numero in a0 e va a capo.
# Usa solo registri t e a, quindi non salva nulla sulla pila.
```

```
stampa_intero:
        la   t0, fine_buf         # il buffer si riempie da destra
        li   t3, 10
        addi t0, t0, -1
        sb   t3, 0(t0)            # l'a capo
        li   t1, 10
        li   t2, 0                # 1 se il numero è negativo
        bgez a0, si_cifre
        li   t2, 1
        neg  a0, a0
si_cifre:
        remu t3, a0, t1
        divu a0, a0, t1
        addi t3, t3, 48           # '0'
        addi t0, t0, -1
        sb   t3, 0(t0)
        bnez a0, si_cifre
        beqz t2, si_scrivi
        li   t3, 45               # '-'
        addi t0, t0, -1
        sb   t3, 0(t0)
si_scrivi:
        li   a0, 1
        mv   a1, t0
        la   a2, fine_buf
        sub  a2, a2, t0
        li   a7, 64
        ecall
        ret

        .data
v:      .word 38, 27, 43, 3, 9, 82, 10, -5, 27, 0
tmp:    .space 40
buf:    .space 12
fine_buf:
```

`mergesort` riceve il vettore, un vettore di appoggio della stessa dimensione e i due estremi. Se c'è al più un elemento ritorna immediatamente, senza nemmeno costruire il record. Altrimenti salva `ra` e i cinque registri `s` in cui conserva vettore, appoggio, estremi e centro, e chiama due volte se stessa e una volta `merge`. Il prologo è pesante, 32 byte e sei `sw`, ed è il prezzo delle chiamate: un motivo in più per cui i compilatori, quando possono, trasformano la ricorsione in un ciclo.

`merge` è una foglia, e usa solo registri `t` e `a`: tre puntatori, uno per metà e uno nel vettore di appoggio, più i due puntatori di fine. Quando una delle due metà è esaurita, copia gli elementi rimanenti dell'altra, poi riporta il

risultato nel vettore originale. Il programma stampa i dieci numeri ordinati, da -5 a 82, in 1348 cicli.

L4.6 Esercizi

Esercizio L4.1 Stampare una stringa

Scrivi una funzione foglia `lunghezza` che restituisca il numero di caratteri di una stringa terminata da zero, e una funzione `stampa_stringa` che la usi per stampare la stringa con la `write`. Quale delle due deve salvare `ra`, e perché?

Esercizio L4.2 Potenza

Scrivi una funzione ricorsiva `potenza(b, e)` che calcoli b^e con $b^0 = 1$ e $b^e = b \times b^{e-1}$. Quanti record ci sono nello stack, al massimo, per 3^5?

Esercizio L4.3 Somma ricorsiva

Scrivi una funzione ricorsiva che sommi gli n elementi di un vettore: la somma di un vettore vuoto è 0, quella di un vettore di n elementi è il primo elemento più la somma degli altri $n - 1$.

Esercizio L4.4 Trova l'errore

Questa versione di Fibonacci restituisce risultati sbagliati.

```
# fib(a0 = n), con un errore
fib:    addi sp, sp, -16
        sw   ra, 12(sp)
        sw   a0, 8(sp)
        sw   s1, 4(sp)
        li   t0, 2
        blt  a0, t0, fib_esci
        addi a0, a0, -1
        jal  fib
        mv   s1, a0
        lw   a0, 8(sp)
        addi a0, a0, -2
        jal  fib
        add  a0, a0, s1
fib_esci:
        lw   ra, 12(sp)
        lw   a0, 8(sp)
        lw   s1, 4(sp)
        addi sp, sp, 16
        ret
```

Che cosa restituisce per $n = 2$, 5 e 6, e perché? Correggila.

Esercizio L4.5 Ricerca binaria

Scrivi una funzione ricorsiva `cerca(v, sinistra, destra, x)` che cerchi x nel vettore ordinato `v` e ne restituisca la posizione, oppure -1. Si esamina l'elemento centrale: se è x, la ricerca termina; se è più grande, si cerca a sinistra; altrimenti a destra.

Esercizio L4.6 Euclide

Il massimo comun divisore si calcola con l'algoritmo di Euclide:

$$\mathrm{mcd}(a, 0) = a, \qquad \mathrm{mcd}(a, b) = \mathrm{mcd}(b, a \bmod b).$$

Scrivilo come funzione ricorsiva e calcola $\mathrm{mcd}(1071, 462)$. È davvero necessario salvare qualcosa nello stack?

Esercizio L4.7 Un esercizio d'esame

Traduci in assembly queste due funzioni, rispettando le convenzioni:

```
int pari(int x) {
    return (x & 1) == 0;
}

int conta_pari(int *v, int n) {
    int c = 0;
    for (int i = 0; i < n; i++)
        if (pari(v[i]))
            c++;
    return c;
}
```

Prova `conta_pari` sul vettore `3, 8, -2, 7, 10, 0, 13, -6`.

Condizione di transizione uscita verso **L5**

Se sai rispondere a tutte le domande, passa al laboratorio L5. Se no, la freccia ↺ ti dice dove tornare.

1. Che cosa scrive `jal` in `ra`, e perché la stessa `ret` torna ogni volta nel punto corretto? ↺ § L4.1
2. Perché una funzione foglia può fare a meno dello stack? ↺ § L4.1
3. Dopo una chiamata, su quali registri può contare il chiamante? Su quali no? ↺ § L4.2
4. Perché i registri salvati vanno nello stack e non in una zona fissa della memoria? ↺ § L4.3
5. Perché il record di `somma_quadrati` è di 16 byte se ne sarebbero sufficienti 12? ↺ § L4.3
6. Quanti record ci sono nello stack, al massimo, durante il calcolo di fib(10)? E quante chiamate in tutto? ↺ § L4.4

LABORATORIO

L5



Vedere la macchina



→ INGRESSI

- La pipeline, i conflitti sui dati, la propagazione e gli stalli (stati 0x9 e 0xa).
- I salti nella pipeline e i predittori (stato 0xb).
- La cache (stato 0xd) e l'I/O, con il controllo ciclico e le interruzioni (stato 0x10).
- I laboratori L2–L4.

USCITE →

- Osservare un conflitto sui dati che produce un risultato sbagliato, e i due rimedi dell'hardware.
- Misurare gli stalli ed eliminarli riordinando il codice.
- Misurare il costo dei salti e confrontare i predittori.
- Misurare il tasso di hit della cache e capire da che cosa dipende.
- Comandare i dispositivi di QtRvSim con il controllo ciclico e con le interruzioni.

Negli stati abbiamo studiato la pipeline, i predittori e la cache sulla carta, con diagrammi e conti fatti a mano. QtRvSim permette di osservarne il funzionamento, e soprattutto di modificarli: disattivare la propagazione, scegliere il predittore, dimensionare la cache. È il laboratorio in cui la teoria delle parti III, IV e V diventa un esperimento. Tutti i programmi sono brevi: ciò che interessa non è il risultato, ma *quanto costa* ottenerlo.

Per ogni esperimento, la configurazione si sceglie all'avvio, o in qualsiasi momento con *File → New simulation*. Conviene partire da un preset e poi passare alle pagine di dettaglio: appena si modifica un parametro, il preset diventa *Custom*.

Tabella L5.1 Il programma `distanze.s` con le tre configurazioni della pipeline di QtRvSim.

Unità dei conflitti	t1	t2	t3	t4	Cicli	Stalli
assente (*none*)	0	0	10	10	9	0
solo stalli (*stall*)	10	10	10	10	11	2
stalli e propagazione (*forward*)	10	10	10	10	9	0

L5.1 La pipeline senza protezioni

Scegli il preset *Pipelined without hazard unit and without cache*: la pipeline a cinque fasi dello stato 0x9, senza alcuna protezione contro i conflitti. Il programma seguente scrive `t0` e poi lo legge con quattro istruzioni, a distanza crescente.

```
        .globl _start
        .text
_start: addi t0, zero, 5          # scrive t0
        add  t1, t0, t0           # legge t0 subito dopo: distanza 1
        add  t2, t0, t0           # distanza 2
        add  t3, t0, t0           # distanza 3
        add  t4, t0, t0           # distanza 4
        ebreak                    # ferma il simulatore
```

Ogni `add` dovrebbe produrre 10. Esegui il programma un ciclo alla volta, osservando nella scheda *Core* le istruzioni che avanzano nelle fasi, e alla fine verifica i registri: `t1` e `t2` valgono 0, `t3` e `t4` valgono 10 (tabella L5.1). Le prime due `add` hanno letto `t0` nella fase ID, quando la `addi` non l'aveva ancora scritto. La terza, invece, legge il valore corretto: quando arriva in ID, la `addi` è in WB, e il banco dei registri scrive nella prima metà del ciclo e legge nella seconda. È esattamente il quadro dello stato 0xa: un conflitto a distanza 1 o 2 produce un valore obsoleto, da distanza 3 in poi non c'è più problema.

Ora apri la pagina *Core ISA and Hazards* della configurazione: *Hazard unit* offre due rimedi. *Stall when hazard is detected* arresta le istruzioni dipendenti finché il dato non è nel banco dei registri: i risultati diventano corretti, ma costano due cicli in più, i due *Stalls* del contatore. *Stall or forward when hazard is detected* aggiunge la propagazione: il risultato della `addi` passa direttamente dall'uscita dell'ALU all'ingresso dell'istruzione successiva, e non è necessario alcuno stallo.

L5.2 Stalli e riordino

La propagazione non risolve tutto. Una load dispone del dato solo alla fine di MEM, e se l'istruzione successiva lo usa in EX, nemmeno la propagazione può fornirlo in tempo: serve uno stallo. Il programma seguente somma due vettori elemento per elemento, e ogni iterazione usa subito il valore appena caricato.

```
        .globl _start
        .text
_start: la   t0, va
        la   t1, vb
        la   t5, vc
        li   t6, 8                # elementi
ciclo:  lw   t2, 0(t0)
        lw   t3, 0(t1)
        add  t4, t2, t3           # usa t3 appena caricato: stallo
        sw   t4, 0(t5)
        addi t0, t0, 4
        addi t1, t1, 4
        addi t5, t5, 4
        addi t6, t6, -1
        bnez t6, ciclo
        li   a0, 0
        li   a7, 93
        ecall

        .data
va:     .word 1, 2, 3, 4, 5, 6, 7, 8
vb:     .word 10, 20, 30, 40, 50, 60, 70, 80
vc:     .space 32
```

Con il preset *Pipelined with hazard unit and cache*, che ha la propagazione, il contatore indica 117 cicli e 8 stalli: uno per iterazione, tra la seconda `lw` e la `add`. Il rimedio è software: fra le due istruzioni si spostano due `addi` che non dipendono da esse, e che dovevano comunque essere eseguite.

```
ciclo:  lw   t2, 0(t0)
        lw   t3, 0(t1)
        addi t0, t0, 4            # spostate qui: separano la lw
        addi t1, t1, 4            # dall'istruzione che usa t3
        add  t4, t2, t3           # ora t3 è pronto
        sw   t4, 0(t5)
        addi t5, t5, 4
        addi t6, t6, -1
        bnez t6, ciclo
```

Le istruzioni sono le stesse, il risultato è lo stesso, ma gli stalli scendono a zero e i cicli a 109. Spostare le `addi` è lecito perché le load hanno già usato `t0` e `t1`: se le `addi` precedessero le load, gli spiazzamenti andrebbero corretti. È il lavoro svolto dal compilatore quando ottimizza, e che vedremo all'opera nel laboratorio L6.

Sul processore a ciclo singolo entrambe le versioni costano 85 cicli: gli stalli sono un problema della pipeline, non del programma. La pipeline, d'altra parte, è molto più veloce di quanto indichino questi numeri, perché il suo ciclo di clock è molto più corto: nell'esempio dello stato 0x9, un quarto di quello del ciclo singolo.

L5.3 Il prezzo dei salti

Il ciclo più semplice possibile, con un salto all'indietro:

```
_start: li   t0, 10          # contatore
        li   t1, 0
ciclo:  addi t1, t1, 1
        addi t0, t0, -1
        bnez t0, ciclo
        ebreak
```

Sul processore a ciclo singolo costa 35 cicli, uno per istruzione eseguita. Sulla pipeline con propagazione ne costa 65, pur senza nessuno stallo. La differenza deriva dai nove salti *presi*. QtRvSim usa la pipeline di base dello stato 0xb: il salto si decide in MEM, e intanto sono già entrate nella pipeline le tre istruzioni che seguono la `bnez` in memoria. Quando il salto risulta preso, vanno eliminate: tre cicli persi per salto, $9 \times 3 = 27$.

Per ridurre la penalità serve un predittore. Nella pagina *Branch Predictor* della configurazione attiva il riquadro *Branch Predictor* e scegli il tipo; dal menu *Windows*, il pannello *Branch Predictor (Info)* mostra quanti salti ha elaborato (*Jump/Branch count*), quanti ne ha previsti in modo errato (*Misprediction count*) e la precisione. Con *Always taken*, *Backward Taken Forward Not Taken* o *Smith 1 bit* il ciclo scende a 44 cicli, con due soli errori su dieci: il primo salto, quando il predittore non dispone ancora di alcuna informazione (il buffer delle destinazioni, BTB, non contiene la destinazione, oppure il contatore parte da «non preso»), e l'ultimo, quando il ciclo termina.

La differenza fra i predittori emerge con due cicli annidati:

```
_start: li   s0, 10          # ciclo esterno: 10 giri
esterno:
        li   s1, 4           # ciclo interno: 4 giri
interno:
        addi s1, s1, -1
        bnez s1, interno
        addi s0, s0, -1
        bnez s0, esterno
        ebreak
```

Con *Smith 1 bit* il predittore commette 22 errori su 50; con *Smith 2 bit* (stato iniziale *weakly not taken*) solo 13, e il programma passa da 192 a 165 cicli. Il motivo è quello dello stato 0xb: a ogni uscita dal ciclo interno il predittore a un bit inverte la previsione, e sbaglia di nuovo al rientro; quello a due bit si limita a indebolire la previsione, e al rientro prevede correttamente.

L5.4 La cache

Per la cache conviene il processore a ciclo singolo, così l'esperimento non è influenzato dagli effetti della pipeline: preset *No pipeline with cache*, poi, nella pagina *L1 Data Cache*, *Number of sets* 4, *Block size* 4 (parole per blocco), *Degree of associativity* 1 e *Replacement policy* LRU. È una cache a mappatura diretta di 4 blocchi da 16 byte, 64 byte in tutto: minuscola, ma anche il nostro programma è minuscolo. Il pannello *Data Cache* (`Ctrl+Shift+M`) mostra il contenuto di ogni blocco, con validità e tag, e le statistiche.

Il programma somma i 64 elementi di una matrice 8×8 riga per riga, come nel laboratorio L3:

```
# somma degli elementi di una matrice 8 x 8, riga per riga
        .globl _start
        .text
_start: la   s0, mat
        li   s1, 8               # LATO
        li   a0, 0               # somma
        li   t1, 0               # i, la riga
righe:  li   t2, 0               # j, la colonna
colonne:
        mul  t3, t1, s1          # i * LATO + j
        add  t3, t3, t2
        slli t3, t3, 2
        add  t3, s0, t3          # &M[i][j]
        lw   t4, 0(t3)
        add  a0, a0, t4
        addi t2, t2, 1           # il ciclo interno scorre la riga
```

```
        blt  t2, s1, colonne
        addi t1, t1, 1
        blt  t1, s1, righe
        ebreak

        .org 0x400
mat:    .word 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
        .word 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
        .word 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
        .word 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
```

La seconda versione, `matrice-colonne.s`, è identica tranne che per il calcolo dell'indirizzo, $j \times 8 + i$ invece di $i \times 8 + j$: così il ciclo interno percorre una colonna. Le due versioni eseguono le stesse istruzioni, 553 cicli, e producono la stessa somma; ma il comportamento della cache è opposto.

- **Per righe**: 48 hit e 16 miss, un tasso di hit del 75%. Ogni miss carica in cache un blocco di quattro parole consecutive, e le tre letture successive trovano le parole già presenti: è la località spaziale.
- **Per colonne**: 0 hit e 64 miss. Due elementi consecutivi della colonna distano una riga, 32 byte, cioè due blocchi. Quando il ciclo passa alla colonna successiva, che userebbe gli stessi blocchi, sono già stati sostituiti da altri.

Accanto al tasso di hit il pannello riporta *Memory stall cycles* e *Improved speed*. QtRvSim, infatti, non rallenta effettivamente il processore in caso di miss, perché renderebbe la simulazione molto più difficile da seguire: calcola a parte i cicli persi, in base ai tempi della pagina *Memory Timings*. Con i valori predefiniti, 10 cicli per ogni accesso alla memoria, anche la versione per righe ha un *Improved speed* del 91%: la cache sembra *rallentare* il programma. Il motivo è che un miss legge quattro parole, e senza accesso a burst ciascuna ha il costo pieno: 40 cicli in tutto. Attiva *Burst enable* con *Burst* a 2: ora le parole dopo la prima costano 2 cicli, come nella DRAM dello stato 0xd, e la versione per righe sale al 200%, mentre quella per colonne passa solo dal 24 al 59%. La cache conviene solo se un blocco costa poco più di una parola, e solo se il programma usa effettivamente le parole caricate con il blocco.

L'ultimo esperimento mostra i miss di conflitto. Il programma `conflitti.s` somma due vettori di 16 parole, `va` e `vb`, che in memoria distano esattamente 64 byte, quanto l'intera cache: ogni `va[i]` e il suo `vb[i]` sono associati allo stesso insieme.

```
        .globl _start
        .text
```

```
_start: la   t0, va
        la   t1, vb
        li   t2, 16              # elementi
        li   a0, 0
ciclo:  lw   t3, 0(t0)           # va[i]
        lw   t4, 0(t1)           # vb[i]: 64 byte più avanti
        add  a0, a0, t3
        add  a0, a0, t4
        addi t0, t0, 4
        addi t1, t1, 4
        addi t2, t2, -1
        bnez t2, ciclo
        ebreak

        .org 0x400
va:     .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
vb:     .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
```

Con la cache a mappatura diretta i due vettori si sostituiscono a vicenda a ogni lettura: 32 miss su 32 accessi. Con la stessa capacità divisa in 2 insiemi da 2 vie, i blocchi dei due vettori convivono, e il tasso di hit sale al 75%. Lo stesso accade raddoppiando la cache a mappatura diretta. È la classificazione dello stato 0xd: i miss del primo esperimento per colonne erano anche di capacità; questi, a parte gli 8 obbligati, sono tutti di conflitto.

L5.5 I dispositivi

QtRvSim simula alcune periferiche, che si comandano con load e store come nello stato 0x10. Le più utili sono la porta seriale, collegata al pannello *Terminal*, e il pannello *Peripherals*, che si apre dal menu *Windows*: mostra una parola in esadecimale, in decimale e in binario (sulla scheda di laboratorio a cui si ispira sarebbe una fila di 32 LED), due LED colorati e tre manopole.

La porta seriale ha i registri a partire da `0xffffc000`: a distanza 8 il registro di stato del trasmettitore, il cui bit 0 vale 1 quando la porta può accettare un carattere, e a distanza 12 il registro in cui scriverlo. Ecco il controllo ciclico dello stato 0x10, in versione QtRvSim:

```
        .equ SERIALE, 0xffffc000      # registri della porta seriale
        .equ TX_STATO, 0x08           # stato del trasmettitore
        .equ TX_DATO,  0x0c           # dato da trasmettere
_start: li   t0, SERIALE
        la   t1, msg
ciclo:  lbu  t2, 0(t1)                # prossimo carattere
        beqz t2, fine                 # zero: fine della stringa
attendi:
```

```
        lw   t3, TX_STATO(t0)
        andi t3, t3, 1              # bit 0: pronto
        beqz t3, attendi
        sw   t2, TX_DATO(t0)        # spedisce il carattere
        addi t1, t1, 1
        j    ciclo
fine:   ebreak
msg:    .asciz "Ciao dalla seriale!\n"
```

Le costanti definite con `.equ` rendono il codice leggibile: l'istruzione `lw t3, TX_STATO(t0)` si legge «il registro di stato», non «la parola a distanza 8». Il messaggio compare nel terminale, esattamente come con la chiamata di sistema 64; ma qui non c'è nessun sistema operativo, e il programma comunica direttamente con il dispositivo.

Il pannello delle periferiche ha i registri a partire da `0xffffc100`. La parola a distanza 4 viene mostrata in esadecimale, in decimale e in binario; quella a distanza `0x10` è il colore del primo LED, nel formato `0x00RRGGBB`; quella a distanza `0x24` contiene le posizioni delle tre manopole, un byte ciascuna: rossa nei bit 23–16, verde nei bit 15–8, blu nei bit 7–0. Questo programma colora il LED con le manopole e mostra la somma delle tre posizioni:

```
        .equ SPILED,   0xffffc100   # LED e manopole
        .equ LED_LINE, 0x04         # la parola mostrata dal pannello
        .equ LED_RGB1, 0x10         # il colore del primo LED
        .equ MANOPOLE, 0x24         # rosso, verde e blu, un byte l'uno
        .globl _start
        .text
_start: li   t0, SPILED
ciclo:  lw   t1, MANOPOLE(t0)
        sw   t1, LED_RGB1(t0)       # il LED prende il loro colore
        srli t2, t1, 16
        andi t2, t2, 255            # rosso: bit 23-16
        srli t3, t1, 8
        andi t3, t3, 255            # verde: bit 15-8
        andi t4, t1, 255            # blu: bit 7-0
        add  t2, t2, t3
        add  t2, t2, t4
        sw   t2, LED_LINE(t0)       # la somma dei tre valori
        j    ciclo                  # per sempre: si ferma con Pause
```

Avvialo con *Run* alla velocità *Unlimited* e ruota le manopole con il mouse: il LED cambia colore e il numero cambia con loro. Il programma non termina mai: lo si arresta con *Pause*.

Infine le interruzioni. Il ricevitore della porta seriale può richiedere un'interruzione quando arriva un carattere, cioè quando lo scrivi nella casella

Input del terminale. Il programma seguente prepara `mtvec` e attiva tre abilitazioni: l'interruzione nel registro di stato del ricevitore, la sorgente in `mie` e le interruzioni in `mstatus`; poi resta in un ciclo che attende un a capo. Tutto il lavoro è svolto dal gestore: legge il carattere, lo ritrasmette e lo conta.

```
        .equ SERIALE,  0xffffc000     # porta seriale
        .equ RX_STATO, 0x00           # stato del ricevitore
        .equ RX_DATO,  0x04           # carattere ricevuto
        .equ TX_DATO,  0x0c           # carattere da trasmettere
        .globl _start
        .text
_start: li   s0, 0                    # contatore, PRIMA di abilitare
        li   s1, 0                    # l'ultimo carattere ricevuto
        la   t0, gestore
        csrw mtvec, t0                # indirizzo del gestore
        li   t0, SERIALE
        li   t1, 2
        sw   t1, RX_STATO(t0)         # bit 1: interruzione alla ricezione
        li   t1, 0x10000
        csrrs zero, mie, t1           # bit 16 di mie: la porta seriale
        csrrsi zero, mstatus, 8       # bit 3 di mstatus: MIE
attesa: li   t1, 10                   # il programma non fa nulla...
        bne  s1, t1, attesa           # ...finché non arriva un a capo
        ebreak

gestore:                              # eseguito a ogni carattere ricevuto
        li   t2, SERIALE
        lw   s1, RX_DATO(t2)          # legge: la richiesta si spegne
        sw   s1, TX_DATO(t2)          # e lo rispedisce
        addi s0, s0, 1
        mret                          # torna dove era stato interrotto
```

Cinque dettagli pratici. QtRvSim non riconosce le pseudo-istruzioni `csrs` e `csrsi` dello stato 0x10: si scrivono per esteso, `csrrs zero, mie, t1` e `csrrsi zero, mstatus, 8`. E il bit attivato in `mie` è il 16, non l'11 dei dispositivi esterni dello stato 0x10: QtRvSim collega la porta seriale direttamente al processore, senza un PLIC, su una delle linee che la specifica lascia alla piattaforma. Con le impostazioni predefinite, il simulatore si ferma all'ingresso di ogni interruzione: per farlo proseguire autonomamente, disattiva *Stop on interrupt entry* nella pagina *System Emulation and IRQ*. E i contatori vanno azzerati *prima* di abilitare le interruzioni: se nella casella c'è già del testo, la prima interruzione arriva appena `mstatus` lo permette, e un azzeramento collocato dopo cancellerebbe il lavoro del gestore. Infine, il gestore usa `t2` senza salvarlo: in questo caso è ammesso, perché il programma principale non lo usa, ma un gestore reale salva ogni registro che modifica, perché l'interruzione può

arrivare fra due istruzioni qualunque.

L5.6 Esercizi

Esercizio L5.1 Prevedere gli stalli

Quanti stalli costa questa sequenza con la sola unità degli stalli, e quanti con la propagazione? Prevedili, poi controlla con QtRvSim.

```
        .globl _start
        .text
_start: la   t0, d
        lw   t1, 0(t0)
        add  t2, t1, t1
        addi t3, t2, 1
        lw   t4, 4(t0)
        sw   t4, 8(t0)
        ebreak
        .org 0x400
d:      .word 5, 7, 0
```

Esercizio L5.2 Riordinare il prodotto scalare

Nel ciclo del prodotto scalare del laboratorio L3 la `mul` usa il valore appena caricato dalla seconda `lw`. Misura gli stalli con la propagazione, poi riordina il ciclo per eliminarli.

Esercizio L5.3 Il conto dei salti

Porta a 100 le iterazioni di `salti.s`. Prevedi i cicli sul processore a ciclo singolo e sulla pipeline senza predittore, poi verifica.

Esercizio L5.4 Un bit o due

In `annidati.s` riduci il ciclo interno a 3 iterazioni. Quanti errori commette ciascun predittore per ogni iterazione del ciclo esterno? Il totale cambia rispetto alla versione con 4 iterazioni?

Esercizio L5.5 Blocchi più grandi

Ripeti la scansione per righe con blocchi di 1, 2, 4 e 8 parole, lasciando 4 insiemi a mappatura diretta. Prevedi il tasso di hit prima di misurarlo.

Esercizio L5.6 Otto blocchi, quattro cache

Con blocchi di 4 parole e 8 blocchi in tutto, prova la scansione per colonne su quattro cache: a mappatura diretta (8 insiemi), a 2 vie (4 insiemi), a 4 vie (2 insiemi) e completamente associativa (1 insieme da 8 vie). In quali compa-

iono degli hit? Perché? Qual è la cache a mappatura diretta più piccola che ne trova?

Esercizio L5.7 Un semaforo

Fai accendere il primo LED colorato di verde, poi di giallo, poi di rosso, e ricomincia, con una pausa fra un colore e l'altro. Una pausa si ottiene con un ciclo vuoto: prova con qualche centinaio di migliaia di iterazioni, alla velocità *Unlimited*, e regola il numero empiricamente.

Esercizio L5.8 Contare con le interruzioni

Modifica il programma delle interruzioni perché mostri sulla fila di LED quanti caratteri sono arrivati, e perché il programma principale sia un ciclo vuoto, `attesa: j attesa`.

Condizione di transizione uscita verso **L6**

Se sai rispondere a tutte le domande, passa al laboratorio L6. Se no, la freccia ↺ ti dice dove tornare.

1. Senza unità dei conflitti, perché un'istruzione a distanza 3 legge il valore corretto e una a distanza 2 no? ↺ § L5.1
2. Perché la propagazione non elimina lo stallo dopo una load? ↺ § L5.2
3. Quanto costa in QtRvSim un salto preso, senza predittore, e perché proprio quella cifra? ↺ § L5.3
4. Perché nei cicli annidati il predittore a due bit commette meno errori di quello a un bit? ↺ § L5.3
5. Perché la scansione per colonne trova 0 hit nella cache di 4 blocchi, e quella per righe il 75%? ↺ § L5.4
6. Perché con i tempi predefiniti l'*Improved speed* della scansione per righe è sotto il 100%? ↺ § L5.4
7. Perché i contatori vanno azzerati prima di abilitare le interruzioni? ↺ § L5.5

LABORATORIO

L6



Dal C all'assembly



→ INGRESSI

- I laboratori L2–L5: strutture di controllo, vettori, funzioni, pipeline.
- Il compilatore, il linker e GCC (laboratorio L0, § L0.1 e § L0.6).
- Lo srotolamento dei cicli (stato 0xc).

USCITE →

- Tradurre a mano una funzione C in assembly, con un metodo.
- Leggere l'assembly prodotto da un compilatore, con e senza ottimizzazioni.
- Misurare in QtRvSim l'effetto delle ottimizzazioni, compreso lo srotolamento dei cicli.
- Descrivere il compito del linker e il codice eseguito prima di `main`.
- Ricostruire il C da un pezzo di assembly.

Nei laboratori precedenti abbiamo scritto assembly a mano, partendo da un'idea. Ma quasi tutto l'assembly esistente è prodotto da un compilatore, partendo dal C o da un altro linguaggio. In questo laboratorio percorriamo le due strade: dal C all'assembly, prima a mano, come all'esame, e poi con il compilatore, misurando quanto si guadagna ottimizzando; e dall'assembly al C, ricostruendo il programma che l'ha generato.

L6.1 Tradurre a mano

Tradurre una funzione C non richiede ispirazione, ma un metodo. Consideriamo la seguente:

```c
int conta_maggiori(int *v, int n, int x) {
    int c = 0;
    for (int i = 0; i < n; i++)
        if (v[i] > x)
            c++;
    return c;
}
```

1. **Gli argomenti e il risultato.** L'ABI li assegna a registri fissi: `v` in `a0`, `n` in `a1`, `x` in `a2`; il risultato sarà restituito in `a0`.
2. **Le variabili locali.** Se la funzione non ne chiama altre, come qui, sono assegnate a registri temporanei: `c` in `t0`. Se ne chiamasse, le variabili che devono sopravvivere alle chiamate andrebbero assegnate a registri `s`, da salvare nel prologo (laboratorio L4).
3. **Gli indici.** Un ciclo che scorre un vettore diventa un ciclo su un puntatore, con l'indirizzo di fine $v + 4n$ al posto del confronto $i < n$ (laboratorio L3).
4. **Il controllo.** Ogni `if` e ogni `for` si traduce con gli schemi del laboratorio L2, con la condizione negata nel salto.
5. **Il ritorno.** Il risultato si scrive in `a0`, e si esegue `ret`.

Il risultato, con un programma principale che chiama la funzione su un vettore di prova:

```
        .globl _start
        .text
_start: la   a0, v
        li   a1, 10
        li   a2, 5
        jal  conta_maggiori       # quanti elementi sono maggiori di 5?
        li   a7, 93
        ecall

# int conta_maggiori(int *v, int n, int x)
#   v in a0, n in a1, x in a2; c in t0; il puntatore in a0, la fine in t1
conta_maggiori:
        li   t0, 0                # c = 0
        blez a1, cm_fine          # n <= 0: nessun giro
        slli t1, a1, 2
        add  t1, a0, t1           # fine = v + n (in byte: v + 4n)
cm_ciclo:
        beq  a0, t1, cm_fine      # for (i = 0; i < n; i++)
        lw   t2, 0(a0)            #     v[i]
        ble  t2, a2, cm_avanti    #     if (v[i] > x)
        addi t0, t0, 1            #         c++;
cm_avanti:
```

```
        addi a0, a0, 4
        j    cm_ciclo
cm_fine:
        mv   a0, t0               # return c;
        ret

        .data
v:      .word 3, 9, 5, 12, -1, 6, 5, 8, 0, 7
```

Nel vettore ci sono cinque elementi maggiori di 5. Il controllo iniziale su `n` non è un dettaglio: con *n* negativo il ciclo del C non esegue alcuna iterazione, mentre il puntatore, che parte da `v` e sale, non incontrerebbe mai l'indirizzo di fine, che si trova più in basso. Ogni commento riporta la riga di C da cui deriva l'istruzione: è una buona abitudine, all'esame e fuori, perché rende la traduzione verificabile riga per riga.

L6.2 Il lavoro del compilatore

Vediamo come GCC traduce la stessa funzione (nella versione 13; altre versioni, e Compiler Explorer, possono differire in qualche dettaglio). Si compila in assembly con l'opzione `-S`:

```
riscv64-unknown-elf-gcc -march=rv32im -mabi=ilp32 -O0 -S conta.c
```

Senza ottimizzazioni, con `-O0`, il compilatore traduce ogni riga separatamente, come farebbe un principiante scrupoloso: ogni variabile ha una posizione riservata nello stack, e ogni volta che serve viene riletta e riscritta. Ecco la parte centrale del ciclo:

```
.L4:    lw   a5,-24(s0)        # i
        slli a5,a5,2
        lw   a4,-36(s0)        # v
        add  a5,a4,a5
        lw   a5,0(a5)          # v[i]
        lw   a4,-44(s0)        # x
        bge  a4,a5,.L3         # if (v[i] > x)
        lw   a5,-20(s0)        # c
        addi a5,a5,1
        sw   a5,-20(s0)        # c++
.L3:    lw   a5,-24(s0)
        addi a5,a5,1
        sw   a5,-24(s0)        # i++
.L2:    lw   a4,-24(s0)
        lw   a5,-40(s0)        # n
        blt  a4,a5,.L4         # i < n
```

`s0` è il frame pointer: `i` si trova a `-24(s0)`, `c` a `-20(s0)`, gli argomenti un po' più in basso, dove il prologo li ha copiati. Tredici istruzioni per iterazione, otto delle quali sono accessi alla memoria; quando `c` aumenta diventano sedici, e dieci gli accessi. È lento, ma ha un pregio: in un debugger ogni variabile ha sempre il suo valore in memoria, e il codice segue il sorgente riga per riga.

Con `-O2` la funzione cambia radicalmente:

```
conta_maggiori:
        ble  a1,zero,.L5       # n <= 0: nessun giro
        slli a1,a1,2
        mv   a5,a0             # il puntatore
        add  a3,a0,a1          # la fine: v + 4n
        li   a0,0              # c
.L4:    lw   a4,0(a5)
        addi a5,a5,4
        slt  a4,a2,a4          # a4 = (x < v[i]) ? 1 : 0
        add  a0,a0,a4          # c += a4
        bne  a3,a5,.L4
        ret
.L5:    li   a0,0
        ret
```

Ci sono tutte le tecniche dei laboratori: il puntatore con l'indirizzo di fine, il ciclo con il controllo in fondo, preceduto da un controllo iniziale per il caso $n \leq 0$, nessun accesso allo stack. E c'è un'idea nuova: l'`if` non è più un salto. `slt` scrive `1` se `x < v[i]` e `0` altrimenti, e quel valore si somma a `c`: incrementare di uno o di zero equivale a incrementare o non incrementare. Un salto in meno è un'occasione in meno di sbagliare una previsione. Cinque istruzioni per iterazione invece di tredici o sedici.

L6.3 Misurare le ottimizzazioni

Per misurare, eseguiamo in QtRvSim il codice prodotto dal compilatore. Un programma C, però, non comincia da `main`: un componente software deve preparare l'ambiente, chiamare `main` e, al ritorno, terminare il programma con il suo risultato. Nei sistemi reali questo compito è svolto da una porzione di codice della libreria del C; qui ne scriviamo uno minimo.

```
# avvio.S: il punto d'ingresso dei programmi C
        .globl _start
        .text
_start:
        .option push
        .option norelax
        la   gp, __global_pointer$    # il registro gp dell'ABI
        .option pop
        call main                     # a0 = valore restituito da main
        li   a7, 93                   # exit(a0)
        ecall
```

Il registro `gp` va preparato perché l'ABI lo riserva alle variabili globali, come anticipato nel laboratorio L1: il linker, quando può, trasforma gli accessi alle variabili vicine a `gp` in una sola istruzione relativa a `gp`. Le direttive `.option` gli impediscono di «ottimizzare» proprio l'istruzione che lo carica, che userebbe `gp` prima che sia pronto. Il programma di prova riempie un vettore di 1000 elementi e lo somma, poi restituisce la somma modulo 256 come codice di uscita:

```
#define N 1000
int v[N];

int main(void) {
    for (int i = 0; i < N; i++)
        v[i] = i;
    int s = 0;
    for (int i = 0; i < N; i++)
        s += v[i];
    return s % 256;
}
```

Si compila un eseguibile, con l'avvio e senza libreria:

```
riscv64-unknown-elf-gcc -march=rv32im -mabi=ilp32 -O2 \
        -nostdlib -static -o somma avvio.S somma.c
```

e lo si carica in QtRvSim con *File → New simulation*, *Browse* e *Load machine*. Al termine, il contatore dei cicli indica il costo dell'esecuzione. La tabella L6.1 riporta i risultati per vari livelli di ottimizzazione, sul processore a ciclo singolo e sulla pipeline con propagazione, senza predittore e con il predittore a due bit.

La tabella evidenzia quattro fenomeni.

- **Da `-O0` a `-O1`** le istruzioni eseguite si riducono a meno di un terzo: spariscono le letture e scritture delle variabili nello stack.

Tabella L6.1 Cicli di QtRvSim per il programma `somma.c`, compilato con GCC 13 e opzioni diverse. Fra parentesi, gli stalli. La dimensione del codice comprende i 20 byte di `avvio.S`.

Opzioni	Ciclo singolo	Pipeline	Pipeline e predittore	Codice
`-O0`	28 029	40 045 (6001)	34 057	204 byte
`-O1`	8 020	15 023 (1000)	9 041	112 byte
`-O2`, `-O3`	8 020	14 023 (0)	8 041	112 byte
`-O2 -funroll-loops`	4 520	5 273 (0)	4 541	224 byte

- **Da `-O1` a `-O2`** le istruzioni sono le stesse, ma sulla pipeline spariscono i 1000 stalli: `-O2` riordina le istruzioni perché nessuna usi il risultato di una load appena caricata, come abbiamo fatto a mano nel laboratorio L5.
- **`-O3` non aggiunge nulla.** Le sue ottimizzazioni in più, come una vettorizzazione più aggressiva, qui non trovano nulla da migliorare: il nostro RV32IM non ha nemmeno istruzioni vettoriali.
- **Lo srotolamento** promesso nello stato 0xc quasi dimezza le istruzioni: il compilatore replica otto volte il corpo del ciclo, e ogni iterazione elabora otto elementi con un solo controllo e un solo salto. Il prezzo è un codice due volte più lungo. Sulla pipeline senza predittore il guadagno è ancora maggiore, da 14 023 a 5273 cicli: i salti presi, che costano tre cicli l'uno, passano da 2000 a 250.

Il predittore a due bit, da parte sua, annulla quasi completamente il costo dei salti: con `-O2`, 8041 cicli sulla pipeline contro gli 8020 del ciclo singolo. È la stessa conclusione dello stato 0xb, misurata invece che calcolata.

> **E su un calcolatore vero?** Sul proprio PC i tempi si misurano con il comando `time`, che riporta il tempo trascorso (*real*), quello speso nel programma (*user*) e quello speso nel sistema operativo per conto del programma (*sys*); per misurare una sola porzione di programma si usa una funzione come `gettimeofday`, chiamata prima e dopo. Le misure variano un po' da un'esecuzione all'altra, a causa delle cache, degli altri programmi e della frequenza variabile del processore: per questo si ripetono più volte. Il contatore di QtRvSim, invece, fornisce sempre lo stesso valore, e questo è il suo pregio.

L6.4 Il linker

Un programma reale è diviso in molti file. Ogni file viene assemblato separatamente, in un file oggetto, e le etichette definite in un altro file restano in sospeso. Consideriamo un programma principale che chiama `quadrato`, definita in un secondo file:

```
        .globl _start
        .text
_start: li   a0, 6
        call quadrato            # definita in un altro file
        li   a7, 93
        ecall
```

```
        .globl quadrato          # visibile agli altri file
        .text
quadrato:
        mul  a0, a0, a0
        ret
```

La direttiva `.globl` rende `quadrato` visibile agli altri file. Assemblando il primo file da solo, con `as`, e esaminandolo con `objdump -d -r`, si vede che l'assemblatore ha lasciato un campo vuoto:

```
   4:   00000097    auipc  ra,0x0
                    4: R_RISCV_CALL_PLT  quadrato
                    4: R_RISCV_RELAX     *ABS*
   8:   000080e7    jalr   ra
```

La `call` è diventata la coppia `auipc`–`jalr`, ma con spiazzamento zero, e accanto c'è una **rilocazione**: una nota per il linker, che indica «qui va la distanza da `quadrato`, quando saprai dov'è». La seconda nota, `R_RISCV_RELAX`, permette al linker di accorciare la coppia, quando è possibile: fra poco vedremo che cosa significa. Il comando `nm` elenca i simboli del file oggetto, e contrassegna `quadrato` con `U`, non definito. Il linker riunisce i file, assegna a ogni etichetta il suo indirizzo definitivo e completa i campi vuoti:

```
00010074 <_start>:
   10074:   00600513    li     a0,6
   10078:   00c000ef    jal    10084 <quadrato>
   1007c:   05d00893    li     a7,93
   10080:   00000073    ecall
00010084 <quadrato>:
   10084:   02a50533    mul    a0,a0,a0
   10088:   00008067    ret
```

Ed ecco all'opera la seconda nota: la coppia `auipc`–`jalr` è diventata una sola `jal`. Ora che conosce la distanza, 12 byte, il linker determina che è sufficiente lo spiazzamento di 20 bit di `jal`, e riduce il codice. Si chiama **rilassamento**, ed è lo stesso meccanismo che le direttive `.option norelax` di `avvio.S` disattivano.

Questo è il **collegamento statico**: tutto il codice confluisce nell'eseguibile prima dell'esecuzione. I sistemi operativi usano anche il **collegamento dinamico**: le funzioni delle librerie comuni, come `printf`, restano in file separati, condivisi da tutti i programmi, e vengono collegate durante l'esecuzione. Di solito in modo *pigro*: la prima chiamata transita da una piccola procedura di appoggio, che individua la funzione effettiva in memoria e aggiorna una tabella di indirizzi; le chiamate successive saltano direttamente alla funzione. Si risparmiano memoria e disco, e la correzione di una libreria si estende a tutti i programmi che la usano, senza ricompilarli.

L6.5 Dall'assembly al C

L'operazione inversa della compilazione, ricostruire il C a partire dall'assembly, si chiama *reverse engineering*, e nella sua forma più semplice è un ottimo esercizio di lettura. Serve a comprendere programmi di cui non si dispone del sorgente, ed è proprio ciò da cui gli offuscatori cercano di proteggere il codice. Proviamo con questo frammento:

```
        li   t6, 0
        li   t5, 0
L1:     li   t0, 400
        beq  t6, t0, FINE
        la   t1, VETTA
        add  t1, t1, t6
        lw   t2, 0(t1)
        la   t3, VETTB
        add  t3, t3, t6
        lw   t4, 0(t3)
        mul  t2, t2, t4
        add  t5, t5, t2
        addi t6, t6, 4
        j    L1
FINE:
```

1. **Dare un nome ai registri.** `t6` parte da 0 e cresce; `t5` parte da 0 e accumula: chiamiamoli `X` e `S`.
2. **Riconoscere il ciclo.** Un confronto in cima che esce, un salto in fondo che torna: è un `while (X != 400)`.
3. **Riconoscere gli accessi ai vettori.** `VETTA + X` seguito da una `lw` è un elemento di `VETTA`; siccome `X` avanza di 4, l'indice effettivo è $i = X/4$, e il 400 finale corrisponde a 100 elementi.
4. **Ricostruire il corpo.** I due elementi si moltiplicano e il prodotto si somma in `S`.

Il risultato è il prodotto scalare del laboratorio L3:

```
int S = 0;
for (int i = 0; i < 100; i++)
    S += VETTA[i] * VETTB[i];
```

Si noti che l'espressione «prodotto scalare» non compare da nessuna parte: il nome della funzione, quello delle variabili e i commenti si perdono nella compilazione, e vanno dedotti dal comportamento.

L6.6 Il C e il sistema operativo

Il nostro `avvio.S` è una versione minima di ciò che accade effettivamente quando un programma C viene avviato sotto un sistema operativo. Il sistema operativo carica l'eseguibile in memoria, trasformando il *programma*, un file sul disco, in un *processo* (stato 0xe); prepara lo stack e vi deposita gli argomenti della riga di comando; poi salta al punto d'ingresso. Il codice di avvio della libreria prepara `gp`, inizializza la libreria stessa e chiama `main(argc, argv)`: `argc`, il numero di parole sulla riga di comando, in `a0`, e `argv`, l'indirizzo di un vettore di puntatori alle parole, in `a1`. Quando `main` torna, il suo risultato diventa il codice di uscita, passato a `exit`: 0 indica successo, un altro valore un errore.

Anche le funzioni più familiari della libreria si traducono, in ultima analisi, in una `ecall`. `printf` converte i numeri in caratteri, come il nostro codice del laboratorio L2, e poi richiede al sistema operativo di scriverli con la chiamata 64. `malloc` gestisce l'heap e, quando lo spazio non basta, richiede al sistema operativo di estenderlo, con la chiamata `brk` (numero 214, che QtRvSim emula) o con `mmap`. Ogni operazione di un programma che vada oltre i registri e la memoria passa da lì: il sistema operativo è l'unico che comunica con i dispositivi, e la `ecall` è il suo punto di accesso.

1957 Il primo compilatore ottimizzante

Nell'aprile 1957 IBM cominciò a distribuire ai clienti del suo calcolatore 704 il compilatore FORTRAN, il primo compilatore ottimizzante. Il gruppo di John Backus ci lavorava dal 1954, contro lo scetticismo di chi era convinto che nessuna macchina potesse produrre codice efficiente quanto quello scritto a mano. Per questo il compilatore fu progettato fin dall'inizio per ottimizzare: analizzava i cicli e il flusso del programma, e ne ricavava come usare al meglio i tre registri indice del 704. I programmi che produceva erano quasi veloci quanto quelli scritti a mano, e gli scettici dovettero ricredersi: a metà degli anni Sessanta FORTRAN era usato nella maggior parte dei grandi centri di calcolo. La tabella del laboratorio conferma che Backus aveva ragione.

L6.7 Esercizi

Esercizio L6.1 Un esercizio d'esame

Traduci a mano, con il metodo della § L6.1:

```
int massimo(int *v, int n) {
    int m = v[0];
    for (int i = 1; i < n; i++)
        if (v[i] > m)
            m = v[i];
    return m;
}
```

Esercizio L6.2 Contare un carattere

Traduci a mano questa funzione, che conta quante volte il carattere `c` compare nella stringa `s`, e provala contando le `a` di «la casa sulla cascata»:

```
int conta(char *s, char c) {
    int n = 0;
    while (*s != '\0') {
        if (*s == c)
            n++;
        s++;
    }
    return n;
}
```

Esercizio L6.3 Una funzione ricorsiva

Traduci a mano, rispettando le convenzioni, e calcola la somma delle cifre di 98765:

```c
int somma_cifre(int n) {
    if (n < 10)
        return n;
    return n % 10 + somma_cifre(n / 10);
}
```

Esercizio L6.4 Che cosa calcola?

Ricostruisci il C di questa funzione, che riceve un argomento in `a0`. Che cosa restituisce per `a0 = 5`?

```
f:      li   t0, 0
        li   t1, 1
L:      bgt  t1, a0, E
        mul  t2, t1, t1
        add  t0, t0, t2
        addi t1, t1, 1
        j    L
E:      mv   a0, t0
        ret
```

Esercizio L6.5 E questa?

Ricostruisci il C di questa funzione ricorsiva. Che cosa restituisce per `a0 = 5`? Perché quando `a0` vale zero non è necessario scrivere esplicitamente il risultato?

```
g:      addi sp, sp, -16
        sw   ra, 12(sp)
        sw   s0, 8(sp)
        mv   s0, a0
        beqz a0, G1
        addi a0, a0, -1
        jal  g
        add  a0, a0, s0
G1:     lw   s0, 8(sp)
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret
```

Esercizio L6.6 Il compilatore al lavoro

Compila la funzione `massimo` dell'esercizio L6.1 con `-O2`, con GCC o con Compiler Explorer, e confrontala con la tua traduzione. Il compilatore usa un salto per l'`if`? Come tratta il caso $n = 1$?

Esercizio L6.7 Srotolare a mano

Riscrivi a mano il ciclo del prodotto scalare del laboratorio L3 srotolandolo di due: ogni iterazione tratta due elementi. Che cosa succede se il numero di elementi è dispari? Misura i cicli sulla pipeline senza predittore, prima e dopo.



Condizione di transizione uscita verso `0x0`

Se sai rispondere a tutte le domande, l'automa torna nello stato iniziale: la macchina è completa, e ora è tua. Se no, la freccia ↺ ti dice dove tornare.

1. Quali sono i passi per tradurre a mano una funzione C in assembly? ↺ § L6.1
2. Perché con `-O0` ogni variabile ha una posizione riservata nello stack? ↺ § L6.2
3. In che modo la versione `-O2` di `conta_maggiori` conta senza un salto per l'`if`? ↺ § L6.2
4. `-O1` e `-O2` eseguono le stesse istruzioni: perché la seconda è più veloce sulla pipeline? ↺ § L6.3
5. Perché lo srotolamento produce un guadagno maggiore sulla pipeline senza predittore? ↺ § L6.3
6. Che cos'è una rilocazione, e chi la risolve? ↺ § L6.4
7. Quali operazioni vengono eseguite prima di `main`, e dopo? ↺ § L6.6

APPENDICE

A

Numeri binari, esadecimali e in complemento a 2

Questa appendice raccoglie le nozioni sui numeri che il libro presuppone. Se le conosci già, è sufficiente un'occhiata alle tabelle; in caso contrario, leggila prima dello stato 0x1.

A.1 Basi e notazione posizionale

Nel numero 345 la cifra 3 vale trecento perché si trova nella terza posizione da destra: $345 = 3 \times 10^2 + 4 \times 10^1 + 5 \times 10^0$. È la **notazione posizionale**, e 10 è la sua *base*. I circuiti digitali distinguono solo due valori, e usano la base 2: le cifre sono 0 e 1, i **bit**, e ogni posizione vale il doppio di quella alla sua destra.

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 2 + 1 = 11.$$

Per passare da decimale a binario si divide ripetutamente per 2: i resti, letti dall'ultimo al primo, sono le cifre. Per esempio 37: $37 = 2{\times}18{+}1$, $18 = 2{\times}9{+}0$, $9 = 2 \times 4 + 1$, $4 = 2 \times 2 + 0$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$; quindi $37 = 100101_2$. In alternativa si sottraggono le potenze di due più grandi possibili: $37 = 32{+}4{+}1$.

Le potenze di due ricorrono ovunque, e conviene ricordarne alcune (tabella A.1). In particolare $2^{10} = 1024$, quasi mille: per questo 2^{20} è circa un milione e 2^{30} circa un miliardo.

Una nota sulle unità. Nel sistema internazionale k, M e G valgono mille, un milione e un miliardo. Per le memorie si usano invece potenze di due, e per evitare ambiguità esistono prefissi appositi: Ki (*kibi*, 2^{10}), Mi (2^{20}), Gi (2^{30}), Ti (2^{40}). Questo libro scrive KiB, MiB e GiB per le memorie e kB, MB e GB per le velocità e per i dischi, che i produttori misurano in potenze di dieci. Un disco da «1 TB» contiene 10^{12} byte, circa 0,91 TiB.

Tabella A.1 Potenze di due e prefissi binari.

n	2^n	n	2^n	n	2^n
0	1	6	64	12	4096
1	2	7	128	16	65 536
2	4	8	256	20	1 048 576 = 1 Mi
3	8	9	512	30	$\approx 1{,}07 \times 10^9$ = 1 Gi
4	16	10	1024 = 1 Ki	32	$\approx 4{,}29 \times 10^9$ = 4 Gi
5	32	11	2048	64	$\approx 1{,}8 \times 10^{19}$

Tabella A.2 Le sedici cifre esadecimali.

Hex	Bin	Dec	Hex	Bin	Dec	Hex	Bin	Dec	Hex	Bin	Dec
`0`	`0000`	0	`4`	`0100`	4	`8`	`1000`	8	`c`	`1100`	12
`1`	`0001`	1	`5`	`0101`	5	`9`	`1001`	9	`d`	`1101`	13
`2`	`0010`	2	`6`	`0110`	6	`a`	`1010`	10	`e`	`1110`	14
`3`	`0011`	3	`7`	`0111`	7	`b`	`1011`	11	`f`	`1111`	15

A.2 Esadecimale

Le sequenze di bit lunghe sono illeggibili:

```
00010000000000001000000000000000
```

risulta di difficile interpretazione. La base 16, l'**esadecimale**, risolve il problema, perché ogni cifra corrisponde esattamente a 4 bit (tabella A.2). Le cifre oltre il 9 si scrivono con le lettere da `a` a `f`, e i numeri si segnalano con il prefisso `0x`: il numero di prima è `0x10010000`. In questo libro le cifre esadecimali sono sempre minuscole, per non confondere B con 8 e D con 0.

Per convertire da binario a esadecimale si raggruppano i bit a quattro a quattro, partendo da destra; per l'inverso si sostituisce ogni cifra con i suoi 4 bit:

$$\texttt{0x3a7} = \texttt{0011 1010 0111}_2 = 3 \times 256 + 10 \times 16 + 7 = 935.$$

Una parola da 32 bit si scrive con 8 cifre esadecimali, un byte con 2.

A.3 Interi senza segno

Con n bit si rappresentano gli interi da 0 a $2^n - 1$: da 0 a 255 con un byte, da 0 a 4 294 967 295 con 32 bit. Si sommano in colonna come in decimale, con il

riporto che passa alla posizione successiva quando la somma di una colonna vale 2 o 3:

```
  0110 1001   (105)
+ 0101 1100   (92)
-------------------
  1100 0101   (197)
```

Se il risultato non è rappresentabile in n bit, cioè se c'è un riporto oltre il bit più alto, si ha un **overflow**: con un byte, $200 + 100 = 300$ diventa $300 - 256 = 44$. I circuiti scartano il riporto finale; spetta al programma rilevarlo, se necessario.

A.4 Complemento a 2

Per i numeri con segno quasi tutti i calcolatori usano il **complemento a 2**. L'idea è semplice: il bit più alto ha peso *negativo*. Con n bit,

$$b_{n-1}b_{n-2}\ldots b_0 \;\longrightarrow\; -b_{n-1}\times 2^{n-1} + \sum_{i=0}^{n-2} b_i \times 2^i.$$

Con 8 bit il bit più alto vale -128: `0000 0101` vale 5, `1111 1011` vale $-128+123 = -5$, `1000 0000` vale -128, `1111 1111` vale -1. I numeri rappresentabili vanno da -2^{n-1} a $2^{n-1}-1$: da -128 a 127 con un byte, da $-2\,147\,483\,648$ a $2\,147\,483\,647$ con 32 bit. Il bit più alto funge da segno: vale 1 per i negativi.

Tre proprietà lo rendono comodo per l'hardware.

- **L'opposto** si ottiene invertendo tutti i bit e sommando 1. Infatti x più x con i bit invertiti produce una parola con tutti i bit a 1, cioè -1; quindi $-x = \overline{x}+1$. Per esempio $5 =$ `0000 0101`, invertito `1111 1010`, più uno: `1111 1011` $= -5$.
- **Somma e sottrazione** usano lo stesso circuito degli interi senza segno: $a - b = a + \overline{b} + 1$. È il principio sfruttato dall'ALU dello stato 0x5.
- **L'estensione del segno**: per allungare un numero a più bit si ricopia il bit più alto. -5 su 8 bit è `0xfb`, su 32 bit `0xfffffffb`; 5 su 8 bit è `0x05`, su 32 bit `0x00000005`. È ciò che eseguono `lb` e `lh`, e il generatore degli immediati dello stato 0x3.

Anche in questo caso si può superare l'intervallo rappresentabile. Nella somma con segno c'è overflow quando i due addendi hanno lo stesso segno e il risultato ha il segno opposto: con 8 bit, $100 + 50$ produce `1001 0110`, cioè -106. Sommando numeri di segno diverso l'overflow è impossibile. Lo stato 0x4 mostra come lo si rileva.

La stessa parola di bit ha significati diversi a seconda di come la si legge: `0xffffffff` vale 4 294 967 295 senza segno e -1 in complemento a 2. Per que-

sto RISC-V ha due versioni dei confronti: `slt` e `blt` confrontano con segno, `sltu` e `bltu` senza segno, e per loro -1 è il numero più grande di tutti.

A.5 Scorrimenti

Far scorrere i bit di k posizioni a sinistra, riempiendo con zeri, moltiplica per 2^k: `slli t0, t0, 2` moltiplica per 4, ed è il modo abituale di trasformare un indice di parola in uno spiazzamento in byte. Farli scorrere a destra divide per 2^k, ma bisogna decidere che cosa entra a sinistra. Lo scorrimento *logico* (`srl`) fa entrare zeri, ed è adatto ai numeri senza segno. Quello *aritmetico* (`sra`) ricopia il bit di segno, ed è adatto ai numeri in complemento a 2: -8 fatto scorrere a destra di un bit diventa -4. Attenzione però agli arrotondamenti: lo scorrimento aritmetico arrotonda verso meno infinito, e -7 fatto scorrere a destra di un bit produce -4, mentre la divisione intera `div` produce -3.

A.6 Operazioni bit a bit e maschere

AND, OR e XOR operano bit per bit, e servono a manipolare singoli bit con una **maschera**, una costante che indica quali bit modificare.

- AND con una maschera *estrae* o *azzera* dei bit: `andi t1, t1, 0x20` lascia solo il bit 5, come nel controllo della porta seriale dello stato 0x10.
- OR *pone a 1* dei bit: `ori t0, t0, 0x8` imposta a 1 il bit 3.
- XOR *inverte* dei bit, e confronta: $x \oplus x = 0$, e $x \oplus y$ ha un 1 dove x e y differiscono. È l'operazione dei codici di Hamming dello stato 0xd.

Combinando scorrimenti e maschere si estrae un campo da una parola. Per esempio, il campo funct3 di un'istruzione (bit 14–12) è (istruzione $\gg 12$)`&0x7`:

```
        srli  t1, t0, 12     # porta i bit 14-12 in fondo
        andi  t1, t1, 0x7    # tiene solo gli ultimi 3 bit
```

APPENDICE

B

Scheda di riferimento RISC-V

Questa appendice raccoglie in poche pagine tutto ciò che occorre per leggere e scrivere programmi RV32IM: registri, formati, codifica delle istruzioni, pseudo-istruzioni, chiamate di sistema e registri di controllo usati nel libro. Le codifiche sono quelle della specifica ufficiale.

B.1 Registri e formati

Tabella B.1 I registri interi e la convenzione di chiamata.

Registri	Nomi	Uso	Chi lo preserva
`x0`	`zero`	sempre 0	—
`x1`	`ra`	indirizzo di ritorno	chi chiama
`x2`	`sp`	puntatore allo stack	chi è chiamato
`x3`	`gp`	dati globali	—
`x4`	`tp`	dati del thread	—
`x5–x7`	`t0–t2`	temporanei	chi chiama
`x8–x9`	`s0/fp`, `s1`	salvati, puntatore al frame	chi è chiamato
`x10–x11`	`a0–a1`	argomenti e risultati	chi chiama
`x12–x17`	`a2–a7`	argomenti	chi chiama
`x18–x27`	`s2–s11`	salvati	chi è chiamato
`x28–x31`	`t3–t6`	temporanei	chi chiama

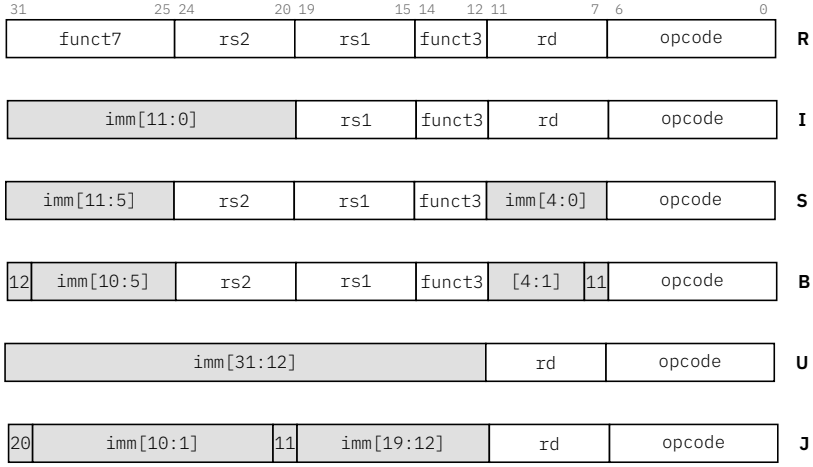


Figura B.1 I sei formati delle istruzioni. In grigio i bit dell'immediato. Negli immediati di B e J il bit 0 non è memorizzato e vale sempre 0.

B.2 Le istruzioni

Nella tabella B.2 `imm` è l'immediato con il segno esteso a 32 bit, `M[a]` la memoria all'indirizzo `a`, e i confronti sono con segno salvo dove indicato. Per gli scorrimenti con immediato (`slli`, `srli`, `srai`) i 7 bit alti dell'immediato fungono da funct7, e i 5 bassi sono la quantità dello scorrimento.

Le istruzioni delle estensioni sono nella tabella B.3; quelle atomiche accettano i suffissi `.aq` e `.rl` (stato 0xf). Infine, le istruzioni privilegiate usate nel libro, tutte con opcode `1110011`: `mret` (`0x30200073`) e `sret` (`0x10200073`) consentono il ritorno da un gestore di eccezioni o interruzioni; `wfi` (`0x10500073`) pone il processore in attesa di un'interruzione; `sfence.vma rs1, rs2` (funct7 `0001001`) svuota il TLB.

Tabella B.2 Le istruzioni di RV32I e la loro codifica.

Istruzione	Fmt	opcode	f3	f7	Effetto
`lui rd, imm`	U	0110111			rd = imm ≪ 12
`auipc rd, imm`	U	0010111			rd = PC + (imm ≪ 12)
`jal rd, off`	J	1101111			rd = PC + 4; PC = PC + off
`jalr rd, off(rs1)`	I	1100111	000		rd = PC + 4; PC = (rs1 + off) con bit 0 a 0
`beq rs1, rs2, off`	B	1100011	000		se rs1 = rs2, PC = PC + off
`bne rs1, rs2, off`	B	1100011	001		se rs1 ≠ rs2
`blt rs1, rs2, off`	B	1100011	100		se rs1 < rs2
`bge rs1, rs2, off`	B	1100011	101		se rs1 ≥ rs2
`bltu rs1, rs2, off`	B	1100011	110		se rs1 < rs2, senza segno
`bgeu rs1, rs2, off`	B	1100011	111		se rs1 ≥ rs2, senza segno
`lb rd, off(rs1)`	I	0000011	000		rd = byte di M[rs1 + off], con segno
`lh rd, off(rs1)`	I	0000011	001		rd = mezza parola, con segno
`lw rd, off(rs1)`	I	0000011	010		rd = parola
`lbu rd, off(rs1)`	I	0000011	100		rd = byte, senza segno
`lhu rd, off(rs1)`	I	0000011	101		rd = mezza parola, senza segno
`sb rs2, off(rs1)`	S	0100011	000		M[rs1 + off] = byte basso di rs2
`sh rs2, off(rs1)`	S	0100011	001		M[rs1 + off] = mezza parola bassa di rs2
`sw rs2, off(rs1)`	S	0100011	010		M[rs1 + off] = rs2
`addi rd, rs1, imm`	I	0010011	000		rd = rs1 + imm
`slti rd, rs1, imm`	I	0010011	010		rd = (rs1 < imm) ? 1 : 0
`sltiu rd, rs1, imm`	I	0010011	011		come `slti`, senza segno
`xori rd, rs1, imm`	I	0010011	100		rd = rs1 XOR imm
`ori rd, rs1, imm`	I	0010011	110		rd = rs1 OR imm
`andi rd, rs1, imm`	I	0010011	111		rd = rs1 AND imm
`slli rd, rs1, sh`	I	0010011	001	0000000	rd = rs1 ≪ sh
`srli rd, rs1, sh`	I	0010011	101	0000000	rd = rs1 ≫ sh, logico
`srai rd, rs1, sh`	I	0010011	101	0100000	rd = rs1 ≫ sh, aritmetico
`add rd, rs1, rs2`	R	0110011	000	0000000	rd = rs1 + rs2
`sub rd, rs1, rs2`	R	0110011	000	0100000	rd = rs1 − rs2
`sll rd, rs1, rs2`	R	0110011	001	0000000	rd = rs1 ≪ rs2[4:0]
`slt rd, rs1, rs2`	R	0110011	010	0000000	rd = (rs1 < rs2) ? 1 : 0
`sltu rd, rs1, rs2`	R	0110011	011	0000000	come `slt`, senza segno
`xor rd, rs1, rs2`	R	0110011	100	0000000	rd = rs1 XOR rs2
`srl rd, rs1, rs2`	R	0110011	101	0000000	rd = rs1 ≫ rs2[4:0], logico
`sra rd, rs1, rs2`	R	0110011	101	0100000	rd = rs1 ≫ rs2[4:0], aritmetico
`or rd, rs1, rs2`	R	0110011	110	0000000	rd = rs1 OR rs2
`and rd, rs1, rs2`	R	0110011	111	0000000	rd = rs1 AND rs2
`fence pred, succ`	I	0001111	000		ordina gli accessi a memoria e dispositivi
`ecall`	I	1110011	000		chiamata all'ambiente (imm = 0)
`ebreak`	I	1110011	000		punto di interruzione (imm = 1)

Tabella B.3 Moltiplicazione e divisione (estensione M), alcune istruzioni atomiche (estensione A) e le istruzioni sui registri di controllo (Zicsr). Le M usano l'opcode `0110011` con funct7 `0000001`; le A l'opcode `0101111` con funct3 `010`, e il tipo di operazione nei 5 bit alti (funct5); le Zicsr l'opcode `1110011`.

Istruzione	f3 / f5	Effetto
`mul rd, rs1, rs2`	`000`	rd = 32 bit bassi di rs1 × rs2
`mulh rd, rs1, rs2`	`001`	rd = 32 bit alti, con segno
`mulhsu rd, rs1, rs2`	`010`	32 bit alti, rs1 con segno, rs2 senza
`mulhu rd, rs1, rs2`	`011`	32 bit alti, senza segno
`div rd, rs1, rs2`	`100`	rd = rs1 / rs2, con segno, troncato
`divu rd, rs1, rs2`	`101`	come `div`, senza segno
`rem rd, rs1, rs2`	`110`	rd = resto di rs1 / rs2, con segno
`remu rd, rs1, rs2`	`111`	come `rem`, senza segno
`lr.w rd, (rs1)`	`00010`	rd = M[rs1] e prenota l'indirizzo
`sc.w rd, rs2, (rs1)`	`00011`	se la prenotazione vale, M[rs1] = rs2 e rd = 0; se no rd ≠ 0
`amoswap.w rd, rs2, (rs1)`	`00001`	atomicamente: t = M[rs1]; M[rs1] = rs2; rd = t
`amoadd.w rd, rs2, (rs1)`	`00000`	atomicamente: t = M[rs1]; M[rs1] = t + rs2; rd = t
`csrrw rd, csr, rs1`	`001`	rd = csr; csr = rs1
`csrrs rd, csr, rs1`	`010`	rd = csr; accende in csr i bit a 1 in rs1
`csrrc rd, csr, rs1`	`011`	rd = csr; spegne in csr i bit a 1 in rs1
`csrrwi`, `csrrsi`, `csrrci`	`101`, `110`, `111`	come sopra, con un immediato da 5 bit al posto di rs1

B.3 Pseudo-istruzioni

Tabella B.4 Le pseudo-istruzioni più usate.

Pseudo	Traduzione	Pseudo	Traduzione
`nop`	`addi x0, x0, 0`	`j off`	`jal x0, off`
`li rd, cost`	`addi` o `lui+addi`	`jal off`	`jal ra, off`
`la rd, et`	`auipc+addi`	`jr rs`	`jalr x0, 0(rs)`
`mv rd, rs`	`addi rd, rs, 0`	`jalr rs`	`jalr ra, 0(rs)`
`not rd, rs`	`xori rd, rs, -1`	`ret`	`jalr x0, 0(ra)`
`neg rd, rs`	`sub rd, x0, rs`	`call et`	`auipc ra+jalr ra`
`seqz rd, rs`	`sltiu rd, rs, 1`	`beqz rs, off`	`beq rs, x0, off`
`snez rd, rs`	`sltu rd, x0, rs`	`bnez rs, off`	`bne rs, x0, off`
`csrr rd, csr`	`csrrs rd, csr, x0`	`bgt rs, rt, off`	`blt rt, rs, off`
`csrw csr, rs`	`csrrw x0, csr, rs`	`ble rs, rt, off`	`bge rt, rs, off`
`csrs csr, rs`	`csrrs x0, csr, rs`	`bgtu rs, rt, off`	`bltu rt, rs, off`
`csrc csr, rs`	`csrrc x0, csr, rs`	`bleu rs, rt, off`	`bgeu rt, rs, off`
`csrwi csr, imm`	`csrrwi x0, csr, imm`	`csrsi csr, imm`	`csrrsi x0, csr, imm`
`csrci csr, imm`	`csrrci x0, csr, imm`		

B.4 Chiamate di sistema

Il numero del servizio si scrive in `a7`, gli argomenti in `a0`, `a1`, `a2`; poi si esegue `ecall`. QtRvSim usa i numeri di Linux; RARS e Ripes hanno numeri propri, e per leggere, scrivere, chiudere un file e terminare usano anche quelli di Linux (tabella B.5).

Tabella B.5 Alcune chiamate di sistema. «Tutti» sta per Linux, QtRvSim, RARS e Ripes.

a7	Servizio	Argomenti	Dove
56	apre un file	`a0` = −100, `a1` nome, `a2` opzioni	Linux, QtRvSim
57	chiude un file	`a0` file	tutti
63	legge byte da un file	`a0` file, `a1` indirizzo, `a2` quanti	tutti
64	scrive byte su un file	`a0` file (1 = schermo), `a1`, `a2`	tutti
93	termina il programma	`a0` codice di uscita	tutti
214	sposta la fine dell'heap	`a0` nuovo limite	Linux, QtRvSim
1	stampa un intero	`a0`	RARS, Ripes
4	stampa una stringa	`a0` indirizzo	RARS, Ripes
5	legge un intero	risultato in `a0`	RARS
10	termina il programma	—	RARS, Ripes
11	stampa un carattere	`a0`	RARS, Ripes

B.5 Registri di controllo

I registri di controllo e di stato (CSR) usati nel libro, al livello *machine* salvo dove indicato:

- `mstatus`: stato del processore; il bit 3 (MIE) abilita le interruzioni, il bit 7 (MPIE) ne conserva il valore durante un gestore (stato 0x10);
- `mtvec`: indirizzo del gestore delle eccezioni e delle interruzioni;
- `mepc`: indirizzo dell'istruzione interrotta o che ha provocato l'eccezione;
- `mcause`: causa dell'eccezione o dell'interruzione (bit più alto a 1 per le interruzioni);
- `mtval`: informazione aggiuntiva, come l'indirizzo che ha causato l'errore;
- `mie` e `mip`: interruzioni abilitate e in attesa, un bit per sorgente (3 software, 7 timer, 11 esterne);
- `satp` (supervisor): modo di traduzione, ASID e pagina fisica della tabella delle pagine (stato 0xe);
- `sepc`, `scause`, `stval` (supervisor): i corrispondenti di `mepc`, `mcause` e `mtval` per il sistema operativo.

APPENDICE

C

Soluzioni

Stato 0x0

1 Unità aritmetica (calcola), unità di controllo (legge le istruzioni e comanda le altre parti), memoria (conserva dati e programma), ingresso e uscita (scambiano informazioni con il mondo esterno).

2 Il programma è codificato in binario e risiede in memoria insieme ai dati. Un programma può quindi leggere e produrre altri programmi: così funzionano compilatori e sistemi operativi.

3 Istruzioni e dati passano dallo stesso canale verso una memoria lenta, un accesso alla volta. Il modello Harvard usa due memorie con due canali, e permette di leggere un'istruzione e un dato nello stesso momento.

4 Entrambe le cose: von Neumann per il programmatore (un solo spazio di indirizzi), Harvard nelle cache di primo livello.

5 L'astrazione offre un'interfaccia più semplice verso la stessa risorsa (i file al posto dei settori del disco). La virtualizzazione offre, allo stesso livello, l'interfaccia di una risorsa diversa (un disco virtuale che è in realtà un file, o un sistema operativo eseguito dentro un altro).

6 In generale no: l'ISA è la stessa, ma l'ABI cambia, perché le chiamate al sistema operativo e le convenzioni sono diverse.

7 Il bus dati trasporta i valori, il bus indirizzi indica dove leggere o scrivere, il bus di controllo indica quale operazione eseguire e quando, e quale componente può usare il bus.

8 Il PC contiene l'indirizzo della prossima istruzione. Lo stato è formato da PC, registri e memoria: gli elementi dotati di memoria.

Stato 0x1

1 Per esempio: istruzioni a lunghezza fissa contro variabile; accesso alla memoria solo con load e store contro istruzioni di calcolo che leggono la memoria. Oggi i processori x86 traducono le istruzioni CISC in micro-operazioni di tipo RISC.

2 $2^5 = 32$. `x0` vale sempre zero e ignora le scritture.

3 `0xbe`: RISC-V è little-endian, quindi all'indirizzo più basso c'è il byte meno significativo.

4 Con `lb`: `0xffffff0`, cioè -16. Con `lbu`: `0x000000f0`, cioè 240.

5 Perché la destinazione dei salti è quasi sempre vicina e un indirizzo completo non trova posto nell'istruzione. All'indietro si salta con una distanza negativa, estesa con il segno e sommata al PC.

6 `lui t0, 0x12345` e poi `addi t0, t0, 0x678`. Qui non è necessaria la correzione, perché il bit 11 di `0x678` vale 0.

7 `jalr x0, 0(ra)`.

8 `0x007302b3`: funct7 0000000_2, rs2 = 7, rs1 = 6, funct3 000_2, rd = 5, opcode 0110011_2.

9 `or x15, x0, x7`, cioè `or a5, zero, t2`: copia `t2` in `a5`.

10 Perché le risorse condivise sono protette dai livelli di privilegio. `ecall` sospende il programma e passa il controllo al sistema operativo, che esegue il servizio indicato in `a7`.

Stato 0x2

1 Il PC è un elemento di stato: conserva il suo valore da un ciclo all'altro. L'ALU è un circuito combinatorio: la sua uscita dipende solo dagli ingressi del momento e non trattiene nulla.

2 $T = 1/f = 1/(2{,}5 \times 10^9)$ s = 0,4 ns, cioè 400 ps.

3 Non lascia traccia: lo stato cambia solo quando qualcosa viene scritto, quindi è come se l'istruzione non fosse mai stata eseguita.

4 I quattro byte da `0x0040001c` a `0x0040001f`. Al ciclo successivo il PC vale `0x00400020`.

5 Nel ciclo singolo ogni componente si usa una sola volta per ciclo, e l'ALU è impegnata dall'istruzione. Un sommatore costa poco e opera in parallelo a tutto il resto.

6 Con 3 ingressi ne servono 2 ($\lceil \log_2 3 \rceil$); con 5 ingressi ne servono 3.

7 È un salto: il risultato diventa l'indirizzo della prossima istruzione. Per questo, in quell'architettura, ogni scrittura in un registro va trattata dall'hardware come un possibile salto.

Stato 0x3

1 Ogni campo occupa sempre gli stessi bit, quindi è sufficiente collegare i fili alla posizione corretta. Se `rs1` cambiasse posizione servirebbe un multiplexer, comandato da un segnale che dipende dal formato: la lettura dei registri dovrebbe attendere la decodifica.

2 Due porte di lettura e una di scrittura: l'istruzione più esigente, di tipo R, legge due registri e ne scrive uno.

3 `x0` non ha flip-flop: all'ingresso 0 dei multiplexer di lettura c'è la costante 0, e l'uscita 0 del decodificatore non è collegata ad alcun registro.

4 Il valore vecchio. Quello nuovo viene scritto al fronte di salita, quando l'istruzione è completata.

5 `0xfe62d6e3` è `bge t0, t1, -20`. Bit 31 = 1 in tutti i bit da 31 a 12, bit 7 = 1 nel bit 11, 111111_2 nei bit 10–5, 0110_2 nei bit 4–1, 0 nel bit 0: `0xffffffec`, cioè −20.

6 Le istruzioni iniziano sempre a indirizzi pari, quindi la distanza di un salto è pari e il suo bit 0 non si memorizza. Il generatore lo riporta a 0, e l'immediato è prodotto già in byte: lo scorrimento è realizzato dal cablaggio.

7 Due registri indicati da bit che in realtà fanno parte dell'immediato, cioè due valori casuali. Non verranno usati, e la lettura incondizionata consente di non attendere la decodifica.

8 L'opcode: per la famiglia sono sufficienti i bit 6–2, mentre i bit 1–0 servono a scartare le parole illegali. Poi funct3 e il bit 30, che servono soprattutto al controllo dell'ALU e alla logica dei salti; per distinguere `ecall` da `ebreak` anche il bit 20.

Stato 0x4

1 Il risultato, su 32 bit, e il segnale zero, su un bit.

2 Perché nelle istruzioni di tipo I il bit 30 fa parte dell'immediato e può

valere 1 per caso, come in `addi t0, t1, 1024`. Solo negli scorrimenti a destra (funct3 = `101`) i bit 31–25 fungono da funct7 e distinguono `srli` da `srai`.

3 `lw` e `sw`: Op1Sel = 0 (`rs1`), Op2Sel = 1 (immediato). `add` e `beq`: Op1Sel = 0, Op2Sel = 0 (`rs1` e `rs2`). `lui`: Op1Sel = 2 (costante 0), Op2Sel = 1.

4 L'indirizzo in cui scrivere: `rs1` più l'immediato di tipo S.

5 Perché nello stesso ciclo l'ALU è occupata a confrontare i due registri. La destinazione è calcolata dal sommatore dei salti.

6 Un SLTU; il salto avviene se il bit 0 del risultato vale 0, cioè se *rs1* $\geq$ *rs2* senza segno.

7 `0x80000000`, che con segno vale -2^{31}: è un overflow, ma RISC-V non segnala nulla.

8 `add t0, t1, t2` seguita da `bltu t0, t1, overflow`.

Stato 0x5

1 $s_i = a_i \oplus b_i \oplus c_i$ e $c_{i+1} = a_i b_i + (a_i \oplus b_i)\, c_i$, cioè il riporto vale 1 quando almeno due ingressi valgono 1.

2 Perché ogni riporto attende il precedente. Il caso peggiore è quello in cui il riporto generato nel bit 0 attraversa tutti i 32 full adder, come in `0xffffffff` + 1.

3 Inverte i bit di b (Binvert) e pone $c_0 = 1$ all'ingresso del primo full adder: $a - b = a + \overline{b} + 1$.

4 Perché in caso di overflow il segno del risultato è sbagliato: `0x7fffffff` − (−1) produce `0x80000000`, negativo. La regola corretta è $less = s_{31} \oplus V$.

5 $g = 0010$, $p = 1111$; $c_1 = 0$, $c_2 = 1$, $c_3 = 1$, $c_4 = 1$. La somma è 0001 con riporto finale 1: 11 + 6 = 17.

6 64 bit: `mul` per i 32 bit bassi; per quelli alti `mulh` (entrambi con segno), `mulhu` (entrambi senza segno) o `mulhsu` (il primo con segno, il secondo senza).

7 Con `10` si sottrae il moltiplicando, con `01` lo si somma; in entrambi i casi poi si esegue uno scorrimento aritmetico a destra.

8 Nessuna eccezione: il quoziente ha tutti i bit a 1, cioè vale −1, e `rem` restituisce il dividendo.

9 $6{,}5 = 110{,}1_2 = 1{,}101_2 \times 2^2$: segno 0, esponente $2 + 127 = 129 = 10000001_2$,

parte frazionaria $101000\ldots0_2$. In esadecimale `0x40d00000`.

10 Perché quasi tutti i numeri sono arrotondati, e gli errori dipendono dal percorso del calcolo. In doppia precisione, per esempio, la somma di 0,1 e 0,2 vale 0,30000000000000004. Si verifica invece che la differenza sia abbastanza piccola.

Stato 0x6

1 Ingressi: l'indirizzo, che arriva dall'ALU; il dato da scrivere, da `rd2`; MemWrite; funct3, per dimensione ed estensione. Uscita: il dato letto. La scrittura avviene al fronte di salita, se MemWrite = 1.

2 Perché leggere un indirizzo qualunque non è innocuo: la lettura di un registro di un dispositivo di I/O può avere effetti, e un indirizzo a caso può generare un'eccezione di memoria virtuale.

3 `lh` legge i byte `be ba`, cioè `0xbabe`, e lo estende con il segno: `0xffffbabe`. `lbu` legge `0xca` e lo estende con zeri: `0x000000ca`.

4 I bit 1–0 la dimensione (00_2 byte, 01_2 mezza parola, 10_2 parola), il bit 2 il tipo di estensione: 0 con il segno, 1 con zeri.

5 Copia il byte in tutte e quattro le posizioni della parola e abilita la scrittura soltanto di quella indicata dai due bit bassi dell'indirizzo.

6 Perché il dato cade a cavallo di due parole: servono due accessi alla memoria e un circuito che ricomponga le parti, mentre nel ciclo singolo la memoria si usa una volta per ciclo.

7 L'indirizzo dall'ALU, che somma `rs1` e l'immediato di tipo S; il dato da `rs2`, attraverso la porta `rd2`.

8 Perché una DRAM risponde in decine di nanosecondi, centinaia di cicli di clock. Si rimedia con una gerarchia di memorie, con le cache vicino al processore (parte IV).

Stato 0x7

1 Dal risultato dell'ALU (WBSel = 0) per OP, OP-IMM, `lui` e `auipc`; dalla memoria (1) per le load; da $PC + 4$ (2) per `jal` e `jalr`.

2 Il filo che riporta il risultato al banco dei registri e l'anello che aggiorna il PC: scrivono in elementi che si trovano prima della fase in cui il valore è pronto. La memoria dati invece si trova proprio nella fase in cui arrivano

indirizzo e dato.

3 La lettura avviene durante il ciclo, la scrittura al fronte di salita: l'ALU calcola la destinazione con il valore vecchio di `ra`, e quello nuovo è disponibile quando il ciclo è concluso.

4 Perché decidono se un'istruzione modifica lo stato: se il loro valore fosse casuale, un registro o la memoria potrebbero essere scritti indebitamente.

5 RegWriteEn = 0, ImmSel = S, Op1Sel = 0, Op2Sel = 1, ALUOp = `00`, MemWrite = 1, WBSel indifferente, Branch, Jal e Jalr a 0.

6 Memoria istruzioni, lettura dei registri, ALU, memoria dati e scrittura del registro: 200 + 100 + 200 + 200 + 100 = 800 ps.

7 La load durerebbe 900 ps, e il periodo di clock dovrebbe salire a 900 ps, circa 1,11 GHz.

8 Perché gli oscillatori operano a frequenza fissa, e soprattutto perché la durata del ciclo andrebbe decisa prima di sapere quale istruzione si esegue, cioè prima di averla letta.

Stato 0x8

1 Perché un passo produce valori che servono al ciclo successivo, e al fronte di salita le uscite dei circuiti combinatori cambiano: quei valori vanno salvati.

2 La seconda memoria e i due sommatori di $PC + 4$ e dei salti. L'istruzione e il dato si leggono in passi diversi, e l'ALU calcola $PC + 4$ e la destinazione dei salti nei passi in cui è libera.

3 Conserva l'indirizzo dell'istruzione: il PC è già stato aggiornato a $PC+4$ nel primo passo, mentre i salti e `auipc` hanno bisogno dell'indirizzo originale.

4 Calcola $PCv + imm$, la destinazione di un eventuale salto. Altrimenti sarebbe inattiva, e se l'istruzione non salta il calcolo non costa nulla.

5 S0, S1, S2 e S5: quattro cicli.

6 4 bit, perché gli stati sono 13: 3 bit ne distinguono solo 8, 4 bit fino a 16.

7 Perché i passi non hanno la stessa durata, e ciascuno occupa un ciclo intero: la lettura e la scrittura dei registri durano 100 ps ma occupano un ciclo da 200. Con 4,125 cicli in media si arriva a 825 ps. Conviene per ridurre l'hardware e quando ci sono istruzioni molto lunghe, come

moltiplicazioni e divisioni, che nel ciclo singolo allungherebbero il clock di tutte.

8 Nel controllo cablato la macchina a stati è costituita da un registro e di logica combinatoria progettata a partire dalle tabelle: è veloce, ma per modificarla occorre riprogettarla. Nel controllo microprogrammato i segnali di ogni passo sono microistruzioni scritte in una memoria di controllo: più flessibile, e modificabile cambiando il contenuto della memoria.

Stato 0x9

1 La latenza è il tempo che una singola istruzione impiega dall'inizio alla fine; il throughput è il numero di istruzioni completate nell'unità di tempo. La pipeline migliora il throughput, mentre la latenza peggiora un po': 1000 ps invece di 800.

2 200 ps, la durata della fase più lenta. L'accelerazione è $800/200 = 4$ e non 5 perché le fasi non sono bilanciate: ID e WB impiegano 100 ps, ma occupano un ciclo intero.

3 $(5 + 100 - 1) \times 200 = 20\,800$ ps, cioè 20,8 ns. Nel ciclo singolo $100 \times 800 = 80\,000$ ps, 80 ns: la pipeline è circa 3,85 volte più veloce, e si avvicina a 4 al crescere del numero di istruzioni.

4 Se saltasse MEM arriverebbe in WB insieme all'istruzione che la precede, e le due dovrebbero scrivere entrambe nel banco dei registri nello stesso ciclo attraverso l'unica porta di scrittura. Con cinque fasi per tutte, in ogni fase c'è al più un'istruzione per ciclo.

5 Il risultato dell'ALU e il segnale zero, il valore di `rs2` da scrivere in memoria, $PC + imm$, $PC + 4$, il numero di `rd` e i campi di controllo M e WB.

6 Perché quando l'istruzione arriva in WB, in IF/ID c'è un'istruzione più giovane: usando il suo campo rd il valore verrebbe scritto nel registro errato. Lo stesso vale per RegWriteEn.

7 MemWrite nel campo M, WBSel nel campo WB. Non entrano ImmSel, usato in ID nello stesso ciclo, e PCSel, calcolato in MEM dalla logica dei salti.

8 In EX: è entrata in IF al ciclo 4, in ID al 5, in EX al 6.

9 Strutturale: con una memoria unica, una load in MEM e un'istruzione in

IF nello stesso ciclo. Sui dati: `add x1, x2, x3` seguita da `sub x4, x1, x5`. Sul controllo: le istruzioni prelevate dopo una `beq` prima che il salto sia deciso.

Stato 0xa

1 Una dipendenza è una proprietà del programma: un'istruzione legge un registro scritto da una precedente. Diventa un conflitto quando, nella pipeline, la lettura avverrebbe prima della scrittura.

2 Quelle di `and` e `or`: leggono `x2` in ID ai cicli 3 e 4, prima che `sub` lo scriva al ciclo 5. `add` lo legge nella seconda metà del ciclo 5, `sw` al ciclo 6.

3 Due: con due `nop` in mezzo, `sub` legge `x1` nello stesso ciclo in cui `add` lo scrive.

4 Dal registro EX/MEM se dipende dall'istruzione precedente; dall'uscita del multiplexer WBSel nella fase WB se dipende da quella due posizioni prima.

5 Le store e i salti condizionati non scrivono registri, ma i bit 11:7 della loro codifica, che contengono parte dell'immediato, potrebbero coincidere con un registro letto. Le scritture in `x0` vengono ignorate, e un'istruzione che legge `x0` deve ottenere zero.

6 Da EX/MEM, cioè dal risultato della seconda istruzione, la più recente: la propagazione da EX/MEM ha la precedenza.

7 Perché il dato esce dalla memoria alla fine di MEM, mentre l'istruzione successiva lo richiede all'inizio di EX, nello stesso ciclo: il valore dovrebbe tornare indietro nel tempo. Dopo uno stallo arriva dall'uscita del multiplexer WBSel nella fase WB.

8 Nel ciclo in cui la load è in EX e l'istruzione dipendente in ID: PCWrite = 0 (il PC non cambia), IF/IDWrite = 0 (l'istruzione resta in ID) e i segnali di controllo verso ID/EX vengono azzerati (in EX entra una bolla). Al ciclo successivo la condizione è falsa e la pipeline riparte.

9 È sufficiente scambiare la prima `addi` con la seconda `lw`: `lw t0, 0(a0)`; `lw t1, 4(a0)`; `addi t0, t0, 1`; `addi t1, t1, 1`. Ogni `addi` è a distanza due dalla sua load: nessuno stallo, e si passa da 10 a 8 cicli.

Stato 0xb

1 Perché il salto si decide in MEM: alla fine del ciclo 4, per un salto prelevato al ciclo 1. Le tre istruzioni prelevate ai cicli 2, 3 e 4 non dovevano essere eseguite e vengono annullate.

2 I salti presi sono il 15% delle istruzioni. Con tre cicli di penalità $CPI = 1 + 0{,}15 \times 3 = 1{,}45$; con uno, $1 + 0{,}15 = 1{,}15$ (trascurando gli altri stalli).

3 Si azzerano i suoi segnali di controllo (RegWriteEn, MemWrite, Branch, Jal, Jalr), così l'istruzione prosegue come una bolla. In RISC-V la parola interamente nulla è un'istruzione illegale: in IF/ID va scritta la codifica di `nop`, `0x00000013`, oppure si azzera un bit di validità.

4 Un circuito di confronto in ID e il sommatore dei salti spostato in ID. Problemi: il confronto di grandezza di `blt` e simili allunga la fase ID; servono propagazione verso ID e nuovi stalli; `jalr` richiede `rs1` nel sommatore.

5 Due stalli (tabella b.1).

6 È un salto che avviene dopo l'istruzione successiva, sempre eseguita (lo *slot* di ritardo, che il compilatore riempie). RISC-V non lo prevede perché vincolerebbe l'ISA a una pipeline particolare: con pipeline più profonde lo slot non nasconde quasi nulla, ma tutti i processori compatibili dovrebbero rispettarlo.

7 Il salto viene preso tre volte e non preso una. Il predittore a un bit sbaglia all'uscita e alla prima iterazione successiva: 2 previsioni corrette su 4. Quello a due bit sbaglia solo all'uscita: 3 su 4.

8 La BHT indica se saltare, ma non la destinazione. Il BTB fornisce già in IF la destinazione, così un salto previsto preso può prelevarla al ciclo successivo senza attendere la decodifica.

9 `mepc` contiene l'indirizzo dell'istruzione che ha provocato l'eccezione (o da cui riprendere dopo un'interruzione), `mcause` il codice della causa, `mtvec` l'indirizzo del gestore. Per una `ecall`, `mepc` punta alla `ecall` stessa: senza il +4, `mret` la rieseguirebbe all'infinito.

10 È un'eccezione dopo la quale tutte le istruzioni precedenti sono completate e quella che l'ha provocata e le successive non hanno cambiato lo stato. Si gestisce per prima quella dell'istruzione più vecchia, la più avanzata nella pipeline.

Stato 0xc

1 Per il muro della potenza: la potenza dinamica cresce come CV^2f, e da quando la tensione ha smesso di scendere ogni aumento di frequenza si è tradotto in calore difficile da smaltire. In più le pipeline molto profonde perdono molti cicli a ogni salto previsto male.

2 Nel VLIW è il compilatore a raggruppare in pacchetti le istruzioni indipendenti; in un superscalare è l'hardware a decidere, durante l'esecuzione.

3 Perché la `addi` del secondo pacchetto ha già diminuito `s1` di 4 quando la `sw` viene eseguita: per scrivere nell'elemento originale serve lo spiazzamento 4.

4 $14/8 = 1{,}75$. Con lo stesso registro, le quattro copie avrebbero dipendenze WAR e WAW fra loro e non si potrebbero sovrapporre.

5 Per mantenere precise le eccezioni e poter scartare i risultati speculativi dopo un salto previsto male: lo stato visibile cambia solo nell'ordine del programma.

6 RAW: da `add` a `sub` su `x1`. WAR: `sub` legge `x1`, che `or` scrive. WAW: `add` e `or` scrivono entrambe `x1`.

7 Ogni scrittura riceve un registro fisico nuovo, quindi non sovrascrive valori che altre istruzioni devono ancora leggere (WAR), e l'ordine delle scritture sullo stesso nome non è più rilevante (WAW). Le RAW restano: il valore deve comunque essere prodotto prima di essere usato.

8 A grana fine si cambia thread a ogni ciclo, ma in ogni ciclo un solo thread emette istruzioni; nel simultaneo, nello stesso ciclo, unità diverse eseguono istruzioni di thread diversi.

9 In ordine operano il prelievo, la predecodifica, la decodifica, la coda, la ridenominazione e il ritiro. Fuori ordine: la stazione di prenotazione unificata e le unità di esecuzione. La ridenominazione si esegue in ordine perché ogni istruzione deve trovare i registri fisici assegnati alle istruzioni che la precedono nel programma.

10 Tre istruzioni: `sbn T, T, i2` ($T = 0$), `sbn T, A, i3` ($T = -A$) e `sbn B, T, i4` ($B = B + A$), dove `i2`, `i3` e `i4` sono le etichette delle istruzioni successive, così che si prosegua in ogni caso.

Stato 0xd

1 Temporale: un indirizzo usato da poco sarà probabilmente usato di nuovo presto. Spaziale: dopo un indirizzo verranno probabilmente usati i suoi vicini. I blocchi di più parole sfruttano la località spaziale.

2 L'offset ha 4 bit (`0x4`), l'indice i 4 bit successivi, il tag gli 8 bit alti. `0x1234` è associato alla riga 3 con tag `0x12`.

3 Offset 4 bit, indice 8, tag 20. Ogni riga occupa $128 + 20 + 1 = 149$ bit, la cache $256 \times 149 = 38\,144$ bit, di cui 32 768 di dati.

4 Perché blocchi consecutivi siano associati a righe diverse. Con l'indice preso dai bit alti, un vettore o un ciclo, che hanno gli stessi bit alti, sarebbero associati tutti alla stessa riga.

5 $65\,536/64 = 1024$ blocchi, divisi in $1024/4 = 256$ insiemi. Offset 6 bit, indice 8, tag $32 - 8 - 6 = 18$. Quattro comparatori, uno per via.

6 Il bit dirty indica se il blocco è stato modificato e va quindi ricopiato in memoria quando viene sostituito. Con l'allocazione in scrittura, la store carica prima il blocco nella cache, poi lo modifica e pone a 1 il bit dirty.

7 Il primo accesso a ciascun blocco è un miss obbligato; tutti i miss successivi sono di conflitto, perché una cache completamente associativa della stessa capacità conterrebbe entrambi i blocchi.

8 $1 + 0{,}01 \times 50 + 0{,}3 \times 0{,}05 \times 50 = 1 + 0{,}5 + 0{,}75 = 2{,}25$.

9 $1 + 15 + 8 \times 1 = 24$ cicli per 32 byte, circa 1,33 byte per ciclo.

10 $C_1 = 1$, $C_2 = 0$, $C_4 = 1$: in memoria va 1011010. Con il bit 6 invertito si legge 1011000, e la sindrome vale $E_4E_2E_1 = 110 = 6$.

11 Indica che ci sono due errori: il controllore non può correggerli, lascia il dato invariato e segnala il problema con un'eccezione.

Stato 0xe

1 Perché il sistema operativo cambia processo ogni pochi millisecondi: copiare a ogni cambio tutta la memoria sul disco e ricaricarla richiederebbe molto più tempo dell'esecuzione stessa.

2 Il numero di processi è limitato dal numero di partizioni, ogni processo dalla dimensione della sua partizione, e ogni processo deve risiedere interamente in memoria: la somma non può superare la memoria fisica.

3 12 bit di offset e 20 di VPN. La tabella avrebbe 2^{20} voci da 4 byte: 4 MiB

per processo.

4 Perché un page fault costa milioni di cicli. La piena associatività elimina i miss di conflitto, e scrivere sul disco a ogni store sarebbe impossibilmente lento.

5 VPN[1] = 2, VPN[0] = 1, offset `0xffc`.

6 Che la voce è valida ma non descrive una pagina: contiene il numero della pagina fisica in cui si trova la tabella del livello successivo.

7 Perché dopo aver caricato la pagina il sistema operativo deve rieseguire l'istruzione che ha causato il fault come se nulla fosse successo: le istruzioni precedenti devono essere complete, quella e le successive senza alcun effetto.

8 Tre: due per le voci della tabella, una per il dato.

9 Indice e offset del blocco non devono superare i 12 bit dell'offset di pagina: 4 KiB per via. Quindi 4 KiB a mappatura diretta, 32 KiB a 8 vie.

10 Perché la cache contiene copie di dati della memoria: quando il sistema operativo rimuove una pagina dalla memoria, ne rimuove anche i blocchi dalle cache e la traduzione dal TLB.

11 Inserisce nelle tabelle dei due processi voci che puntano alle stesse pagine fisiche, con i permessi R e X ma senza W.

Stato 0xf

1 Condividono il codice e tutta la memoria del processo, con la stessa tabella delle pagine. Ognuno ha il proprio PC, i propri registri e il proprio stack.

2 $1/(0{,}2 + 0{,}8/4) = 1/0{,}4 = 2{,}5$ volte con 4 core; con infiniti core al massimo $1/0{,}2 = 5$ volte.

3 Con il write-through la memoria viene aggiornata, ma la copia nella cache dell'altro core resta obsoleta, e le sue letture la trovano con un hit.

4 La coerenza riguarda una singola locazione: tutti i core vedono alla fine lo stesso valore, e le scritture nello stesso ordine. La consistenza riguarda locazioni diverse: quando, e in che ordine, le scritture diventano visibili agli altri core.

5 Con lo snooping ogni richiesta viene trasmessa a tutte le cache, che controllano i propri tag; con una directory una tabella registra quali cache

hanno ogni blocco, e le richieste vanno solo a quelle. Con l'aggiornamento il core che scrive invia il nuovo valore a tutte le copie; con l'invalidazione le altre copie vengono marcate non valide, e il core che le rilegge subisce un miss.

6 La copia del core 0 passa da E a M in silenzio, senza traffico. La lettura del core 1 genera un miss e una richiesta sul bus; il core 0 riconosce l'indirizzo, fornisce il blocco e lo riscrive in memoria, e passa da M a S. La copia del core 1 passa da I a S.

7 Permette di scrivere un blocco letto in precedenza senza alcun traffico sul bus, se nessun altro lo ha: è il caso tipico dei dati privati di un thread. Senza E, la prima scrittura richiederebbe sempre un'invalidazione.

8 Due variabili diverse, scritte da core diversi, si trovano nello stesso blocco: il blocco viene trasferito da una cache all'altra con un miss a ogni scrittura. Si evita collocando le variabili in blocchi diversi, allineandole a 64 byte o separandole con byte di riempimento.

9 Perché la coerenza riguarda i singoli accessi, non la sequenza: entrambi i thread possono leggere lo stesso valore prima che uno dei due scriva. Il problema è risolto da un'istruzione atomica, per esempio `amoadd.w`.

10 Mentre il lucchetto è occupato, la `lw` legge la copia locale in stato S senza generare traffico; la `amoswap`, che è una scrittura e invalida le copie degli altri, viene tentata solo quando il lucchetto sembra libero.

11 Un blocco che non è in L3 non è in nessuna cache del chip; se c'è, i quattro bit di presenza indicano quali core potrebbero averlo, e solo quelli vanno interrogati. Le richieste dagli altri processori possono così essere risolte dalla L3.

Stato 0x10

1 La banda è determinante nella copia di un file grande su un disco o nella riproduzione di un video; la latenza è determinante per il mouse, la tastiera o un gioco in rete.

2 Perché su molti fili paralleli i bit non arrivano tutti nello stesso istante, ed è necessario attendere il più lento; una sola coppia di fili per direzione può operare a frequenze molto più alte, con il clock ricavato dai dati. Il collegamento punto a punto evita il carico di tanti componenti sullo stesso filo.

3 Nel bus sincrono un clock comune scandisce il protocollo; in quello

asincrono ogni passo è confermato dall'altro interlocutore. La memoria disattiva Ack solo dopo aver rilevato la disattivazione di ReadReq: in questo modo ha la certezza che il dispositivo abbia ricevuto la conferma e rilasciato le linee dei dati.

4 Il PCH, che raccoglie i dispositivi lenti: USB, SATA, rete, audio.

5 L'indirizzo appartiene a una zona che la piattaforma assegna al dispositivo, e la scrittura arriva al suo registro invece che alla memoria. I registri cambiano autonomamente e la loro lettura può avere effetti collaterali: una lettura dalla cache restituirebbe un valore obsoleto, e una scrittura potrebbe restare in cache senza mai arrivare al dispositivo.

6 $1000 \times 400 = 400\,000$ cicli al secondo su 2×10^9: lo 0,02%.

7 Quando il bit della sorgente vale 1 sia in `mip` sia in `mie` e, se il processore è al livello machine, il bit MIE di `mstatus` vale 1. All'ingresso nel gestore MIE viene copiato in MPIE e azzerato; `mret` lo ripristina.

8 Perché per un'interruzione `mepc` indica l'istruzione interrotta, che non è ancora stata eseguita e va eseguita al ritorno; per una `ecall` indica la `ecall` stessa, che va saltata.

9 Con il DMA il processore non copia i dati e riceve un'interruzione per ogni blocco di migliaia di byte, invece che per ogni pochi byte.

10 Con la cache: il DMA scrive in memoria lasciando obsolete le copie in cache, o legge dalla memoria dati non ancora riscritti dalla cache; si risolve con l'I/O coerente o svuotando e invalidando la cache via software. Con la memoria virtuale: il DMA usa indirizzi fisici, e le pagine consecutive di un vettore non sono consecutive in memoria; il trasferimento va suddiviso per pagine, le pagine vanno bloccate in memoria, oppure si usa una IOMMU.

11 Un giro dura $60/10\,000 = 6$ ms: in media mezzo giro, 3 ms.

Laboratorio L0

1 Un simulatore come QtRvSim riproduce il circuito: mostra datapath, pipeline e cache, ma è lento. Un emulatore come QEMU traduce le istruzioni RISC-V in istruzioni della macchina che lo ospita e le esegue alla velocità di un processore reale, senza mostrare nulla dell'organizzazione interna. Il primo serve a capire come il processore esegue un programma; il secondo a eseguire programmi grandi, o a verificarne il comportamento sotto Linux.

2 La `add` è definita dall'ISA, che tutti gli strumenti rispettano. I servizi della `ecall` dipendono dall'ambiente, cioè dall'ABI: ogni strumento sceglie quali offrire e con quali numeri.

3 *Pipelined without hazard unit and without cache*: senza stalli né propagazione, un'istruzione che legge un registro scritto da quella appena precedente legge il valore obsoleto.

4 Perché per impostazione predefinita QtRvSim sospende l'esecuzione a ogni chiamata di sistema (l'opzione *Stop on known system call*). Premendo di nuovo *Run*, il programma prosegue fino alla `ecall` numero 93.

5 QtRvSim ignora `.data` e `.text`, e scrive tutto in sequenza dall'indirizzo `0x00000200`, che è anche quello da cui comincia l'esecuzione: il processore tenta di eseguire i byte della stringa. RARS colloca invece i dati nella loro zona, a partire da `0x10010000`. È sufficiente spostare i dati dopo il codice, eventualmente con `.org 0x400`.

6 In QtRvSim 8 byte, perché `li` diventa sempre `lui` più `addi`. In RARS 4: la costante è piccola, ed è sufficiente `addi t0, zero, 5`.

7 Funziona in RARS e in Ripes, dove 1 è il servizio che stampa un intero. QtRvSim emula le chiamate di Linux e non implementa la 1 (in Linux è un servizio che non ha nulla a che fare con la stampa): la simulazione si ferma. Per stampare il numero bisogna convertirlo in caratteri e scriverli con la 64.

8 `-march=rv32im` seleziona le istruzioni: la base a 32 bit più moltiplicazione e divisione, senza virgola mobile né istruzioni compresse, che QtRvSim non supporta. `-mabi=ilp32` fissa le convenzioni: `int`, `long` e puntatori a 32 bit, e nessun registro in virgola mobile per passare gli argomenti.

9 Il file viene caricato in memoria a partire da `0x00010000`, intestazioni comprese: i primi `0x94` byte sono l'intestazione del file ELF e la tabella dei segmenti da caricare, e il codice segue immediatamente.

10 Codice per primo, dopo `.text`; dati dopo `.data`, con le parole prima dei byte e delle stringhe; la prima istruzione con l'etichetta `_start`, dichiarata con `.globl`; solo `.word`, `.byte` e `.asciz`; solo le `ecall` 63, 64 e 93; solo istruzioni di RV32IM; gli indirizzi raggiunti con `la`, mai scritti a mano.

Laboratorio L1

1 A `0x00000200`. I dati si trovano subito dopo l'ultima istruzione: nel programma del laboratorio, a `0x0000022c`.

2 In `auipc t0, 0x0`, che scrive in `t0` l'indirizzo dell'istruzione stessa, `0x200`, e `addi t0, t0, 44`. Il 44 è `0x2c`, la distanza fra la `auipc` e l'etichetta `valore`: `0x22c` − `0x200`.

3 `1e 00 00 00`: il numero 30, con il byte meno significativo all'indirizzo più basso.

4 Lo stack cresce verso il basso, l'heap verso l'alto. Partendo da estremi opposti possono crescere entrambi finché non si incontrano, senza dover decidere in anticipo quanto spazio assegnare a ciascuno.

5 La `read` preleva `ciao` e aggiunge un a capo: in memoria ci sono i 5 byte `63 69 61 6f 0a`, e `a0` vale 5.

6 Manca la `ecall` di uscita. Il processore ha superato l'ultima istruzione e ha tentato di eseguire il primo dato, il numero 20, che in esadecimale è `0x14` e non è un'istruzione valida.

Esercizio L1.1 Tre numeri

La quarta parola vale $7 - 3 + 12 = 16$, cioè `0x00000010`.

```
        .globl _start
        .text
_start: la   t0, dati
        lw   t1, 0(t0)            # primo numero
        lw   t2, 4(t0)            # secondo
        lw   t3, 8(t0)            # terzo
        add  t4, t1, t2
        add  t4, t4, t3
        sw   t4, 12(t0)           # la somma va nella quarta parola
        li   a0, 0
        li   a7, 93
        ecall

        .data
dati:   .word 7, -3, 12, 0
```

Esercizio L1.2 Scambio

Sono sufficienti due registri per i valori, oltre a `t0` per l'indirizzo: si leggono entrambe le parole, poi si scrivono incrociate. Con un registro solo, la prima `sw` distruggerebbe un valore prima di averlo letto.

```
        .globl _start
        .text
_start: la   t0, x
        lw   t1, 0(t0)          # t1 = x
        lw   t2, 4(t0)          # t2 = y
        sw   t2, 0(t0)          # x = vecchio y
        sw   t1, 4(t0)          # y = vecchio x
        li   a0, 0
        li   a7, 93
        ecall

        .data
x:      .word 111
y:      .word 222
```

Esercizio L1.3 Un'espressione

Il risultato è $10 \times 7 = 70$, cioè `0x46`, nella quinta parola.

```
        .globl _start
        .text
_start: la   t0, dati
        lw   t1, 0(t0)          # a
        lw   t2, 4(t0)          # b
        lw   t3, 8(t0)          # c
        lw   t4, 12(t0)         # d
        add  t5, t1, t2         # a + b
        sub  t6, t3, t4         # c - d
        mul  t5, t5, t6         # (a + b) * (c - d)
        sw   t5, 16(t0)
        li   a0, 0
        li   a7, 93
        ecall

        .data
dati:   .word 6, 4, 9, 2, 0
```

Esercizio L1.4 Little-endian

Alla fine `t2` = `0x44`, `t3` = `0x33`, `t4` = `0x22`, `t5` = `0x11`: il byte all'indirizzo più basso è il meno significativo. La `sb` all'indirizzo base più 3 cambia il byte più significativo, e `s0` vale `0xaa223344`.

```
        .globl _start
        .text
_start: la   t0, parola
        li   t1, 0x11223344
        sw   t1, 0(t0)
        lbu  t2, 0(t0)          # byte all'indirizzo più basso
        lbu  t3, 1(t0)
        lbu  t4, 2(t0)
        lbu  t5, 3(t0)          # byte all'indirizzo più alto
        li   t6, 0xaa
        sb   t6, 3(t0)          # cambia solo il byte più alto
        lw   s0, 0(t0)          # rilegge la parola intera
        li   a0, 0
        li   a7, 93
        ecall

        .data
parola: .word 0
```

Esercizio L1.5 Saluto

La `read` restituisce in `a0` il numero di byte letti, a capo compreso; sottraendo 1 si ottiene la lunghezza del nome, che si conserva in `s0` perché le chiamate successive riusano `a0`. La domanda è lunga 16 byte, spazio finale compreso.

```
        .globl _start
        .text
_start: li   a0, 1
        la   a1, domanda
        li   a2, 16
        li   a7, 64             # scrive la domanda
        ecall
        li   a0, 0
        la   a1, nome
        li   a2, 32
        li   a7, 63             # legge il nome
        ecall
        addi s0, a0, -1         # byte letti, senza l'a capo
        li   a0, 1
        la   a1, ciao
        li   a2, 6
        li   a7, 64             # "Ciao, "
        ecall
        li   a0, 1
        la   a1, nome
        mv   a2, s0
        li   a7, 64             # il nome
```

```
        ecall
        li   a0, 1
        la   a1, punto
        li   a2, 2
        li   a7, 64               # "!" e a capo
        ecall
        li   a0, 0
        li   a7, 93
        ecall

        .data
nome:   .space 32
domanda: .asciz "Come ti chiami? "
ciao:   .asciz "Ciao, "
punto:  .asciz "!\n"
```

Esercizio L1.6 Senza `la`

L'istruzione `lw t1, 0x400(zero)` calcola l'indirizzo 0 + `0x400`: il registro `zero` vale sempre 0, e lo spiazzamento diventa un indirizzo assoluto. Funziona perché `0x400` = 1024 è rappresentabile nei 12 bit con segno dello spiazzamento, che raggiunge 2047. L'indirizzo `0x10010000` di RARS è troppo grande: servirebbe prima una `lui` per caricarne la parte alta in un registro.

```
        .globl _start
        .text
_start: lw   t1, 0x400(zero)      # indirizzo 0 + 0x400
        lw   t2, 0x404(zero)
        add  t3, t1, t2
        sw   t3, 0x408(zero)
        li   a0, 0
        li   a7, 93
        ecall

        .org 0x400
valore: .word 10, 20, 0
```

Laboratorio L2

1 Perché il ramo *then* segue immediatamente il salto e ci si arriva quando il salto *non* viene preso. Il salto serve a scavalcarlo, e deve quindi essere preso quando la condizione del C è falsa.

2 Con il controllo in cima e un `j` in fondo, come un `while`, oppure con il controllo in fondo, come un `do–while`, preceduto se serve da un controllo

iniziale. Nei programmi del laboratorio il primo costa 4 istruzioni per iterazione, il secondo 3.

3 Con i byte da 128 a 255: `lb` estende il segno e restituisce un numero negativo, `lbu` riempie con zeri.

4 QtRvSim non accetta i caratteri come immediati: si scrive il codice ASCII, `li t0, 65`, oppure un nome definito con `.equ`.

5 Perché il resto della divisione per 10 fornisce l'ultima cifra. Il buffer si riempie quindi dalla fine verso l'inizio, e alla fine si stampa la porzione riempita, dall'ultima cifra scritta fino in fondo.

6 `neg` lascerebbe `0x80000000`, che con segno è ancora -2^{31}. Con `rem` il primo resto sarebbe -8, e sommando 48 si otterrebbe il carattere 40, una parentesi tonda: al posto delle cifre comparirebbero simboli. Letto senza segno, invece, `0x80000000` vale 2^{31}, e `divu` e `remu` producono le cifre corrette.

Esercizio L2.1 Il massimo di tre

Si parte da `max = a` e si confronta con gli altri due; ogni `if` senza `else` ha un solo salto. Il risultato è 17.

```
        .globl _start
        .text
_start: la   t0, dati
        lw   t1, 0(t0)          # a
        lw   t2, 4(t0)          # b
        lw   t3, 8(t0)          # c
        mv   t4, t1             # max = a
        bge  t4, t2, provac     # if (b > max)
        mv   t4, t2             #     max = b
provac: bge  t4, t3, fine       # if (c > max)
        mv   t4, t3             #     max = c
fine:   sw   t4, 12(t0)
        li   a0, 0
        li   a7, 93
        ecall

        .data
dati:   .word -4, 17, 9, 0      # a, b, c e il posto per il massimo
```

Esercizio L2.2 Contare le vocali

Le vocali sono 20. Per un carattere che non è una vocale si eseguono sette istruzioni di salto: il controllo dello zero, i quattro `beq`, il `bne` che salta il conteggio e il `j` che torna in cima. L'ultimo confronto è negato: se il carattere

è una *u*, il `bne` non salta e l'esecuzione prosegue nel conteggio, senza un salto in più.

```
        .globl _start
        .text
_start: la   t0, testo
        li   a0, 0              # contatore delle vocali
        li   t2, 97             # 'a'
        li   t3, 101            # 'e'
        li   t4, 105            # 'i'
        li   t5, 111            # 'o'
        li   t6, 117            # 'u'
ciclo:  lbu  t1, 0(t0)
        beqz t1, fine
        beq  t1, t2, vocale
        beq  t1, t3, vocale
        beq  t1, t4, vocale
        beq  t1, t5, vocale
        bne  t1, t6, avanti     # l'ultimo confronto è negato
vocale: addi a0, a0, 1
avanti: addi t0, t0, 1
        j    ciclo
fine:   li   a7, 93             # il risultato è il codice di uscita
        ecall

        .data
testo:  .asciz "il processore esegue una istruzione alla volta"
```

Esercizio L2.3 Tutto maiuscolo

Il programma stampa `RISC-V, 32 REGISTRI!`. La lunghezza da stampare è la distanza fra lo zero finale, dove si arresta `t0`, e l'inizio della stringa.

```
        .globl _start
        .text
_start: la   s0, testo
        mv   t0, s0
        li   t2, 97             # 'a'
        li   t3, 122            # 'z'
ciclo:  lbu  t1, 0(t0)
        beqz t1, stampa
        blt  t1, t2, avanti     # prima di 'a': non è minuscola
        bgt  t1, t3, avanti     # dopo 'z': nemmeno
        addi t1, t1, -32        # 'a' - 'A' = 32
        sb   t1, 0(t0)
avanti: addi t0, t0, 1
        j    ciclo
```

```
stampa: li   a0, 1
        mv   a1, s0
        sub  a2, t0, s0          # lunghezza: t0 punta allo zero finale
        li   a7, 64
        ecall
        li   a0, 0
        li   a7, 93
        ecall

        .data
testo:  .asciz "risc-v, 32 registri!\n"
```

Esercizio L2.4 Fattoriale

$12! = 479\,001\,600$ è rappresentabile in 32 bit. $13! = 6\,227\,020\,800$ invece supera 2^{32}, e la `mul` ne conserva solo i 32 bit bassi. Il risultato, 1 932 053 504, è il valore corretto meno 2^{32}; la parte alta del prodotto si otterrebbe con `mulh`.

```
        .globl _start
        .text
_start: la   t0, n
        lw   t1, 0(t0)           # n
        li   a0, 1               # fatt = 1
ciclo:  ble  t1, zero, fine      # while (n > 0)
        mul  a0, a0, t1          #     fatt = fatt * n
        addi t1, t1, -1          #     n = n - 1
        j    ciclo
fine:   sw   a0, 4(t0)
        li   a7, 93
        ecall

        .data
n:      .word 12
fatt:   .word 0
```

Esercizio L2.5 Il doppio

La parte nuova è la lettura. Dopo aver sottratto 48, un carattere è una cifra se il risultato è compreso fra 0 e 9; con un solo confronto *senza segno*, `bltu t2, t3`, si scartano anche i caratteri prima di `0`, che producono numeri negativi e quindi, letti senza segno, enormi. La seconda metà è la stampa del laboratorio. Con `-350` il programma stampa `-700`. Il raddoppio, però, non controlla l'overflow: se $n \geq 2^{30}$ o $n < -2^{30}$, il doppio non è più rappresentabile in 32 bit, e il programma stampa un numero errato senza segnalarlo.

```
        .globl _start
```

```
        .text
# legge un numero dal terminale
_start: li   a0, 0
        la   a1, buf
        li   a2, 12
        li   a7, 63
        ecall
# da testo a numero: n = n * 10 + cifra, finché ci sono cifre
        la   t0, buf
        li   t1, 0              # n
        li   t3, 10
        li   t4, 0              # 1 se c'è il segno meno
        lbu  t2, 0(t0)
        li   t5, 45             # '-'
        bne  t2, t5, leggi
        li   t4, 1
        addi t0, t0, 1
leggi:  lbu  t2, 0(t0)
        addi t2, t2, -48        # da carattere a cifra
        bltu t2, t3, cifra      # fra 0 e 9 (confronto senza segno!)
        j    fatto
cifra:  mul  t1, t1, t3
        add  t1, t1, t2
        addi t0, t0, 1
        j    leggi
fatto:  beqz t4, doppio
        neg  t1, t1
doppio: slli t1, t1, 1          # il doppio
# da numero a testo, come nella stampa del laboratorio
        la   t2, fine_buf
        li   t5, 10
        addi t2, t2, -1
        sb   t5, 0(t2)
        li   t4, 0
        bgez t1, cifre
        li   t4, 1
        neg  t1, t1
cifre:  remu t5, t1, t3
        divu t1, t1, t3
        addi t5, t5, 48
        addi t2, t2, -1
        sb   t5, 0(t2)
        bnez t1, cifre
        beqz t4, scrivi
        li   t5, 45
        addi t2, t2, -1
        sb   t5, 0(t2)
scrivi: li   a0, 1
```

```
        mv   a1, t2
        la   a2, fine_buf
        sub  a2, a2, t2
        li   a7, 64
        ecall
        li   a0, 0
        li   a7, 93
        ecall

        .data
buf:    .space 12               # il testo letto
out:    .space 12               # il testo da scrivere
fine_buf:
```

Esercizio L2.6 Un esercizio d'esame

Servono 111 passi. Per un n positivo, n mod 2 è il bit più basso (`andi`), $n/2$ è uno scorrimento a destra, $3n$ è $2n + n$: uno scorrimento a sinistra e una somma.

```
        .globl _start
        .text
_start: la   t0, n
        lw   t1, 0(t0)          # n
        li   t2, 0              # passi = 0
        li   t3, 1
ciclo:  beq  t1, t3, fine       # while (n != 1) {
        andi t4, t1, 1          #     n % 2: l'ultimo bit
        bnez t4, dispari
        srai t1, t1, 1          #     n = n / 2  (n è positivo)
        j    conta
dispari:
        slli t4, t1, 1          #     n = 3 * n + 1
        add  t1, t1, t4
        addi t1, t1, 1
conta:  addi t2, t2, 1          #     passi++;
        j    ciclo              # }
fine:   sw   t2, 4(t0)
        mv   a0, t2
        li   a7, 93
        ecall

        .data
n:      .word 27
passi:  .word 0
```

Esercizio L2.7 Numeri primi

Il programma stampa 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43 e 47, in 1204 cicli sul processore a ciclo singolo. Il ciclo interno esce in due modi: con `d * d > n`, e allora *n* è primo, o con un resto nullo, e allora non lo è.

```
        .globl _start
        .text
_start: li   s0, 2              # n, il candidato
        li   s1, 50             # limite
esterno:
        bge  s0, s1, fine       # for (n = 2; n < 50; n++)
        li   t0, 2              # d, il divisore
interno:
        mul  t1, t0, t0
        bgt  t1, s0, primo      # d * d > n: nessun divisore, è primo
        remu t1, s0, t0
        beqz t1, prossimo       # n % d == 0: non è primo
        addi t0, t0, 1
        j    interno
primo:  la   t2, fine_buf       # stampa n, come nel laboratorio
        li   t5, 10
        addi t2, t2, -1
        sb   t5, 0(t2)          # a capo
        mv   t1, s0
        li   t3, 10
cifre:  remu t5, t1, t3
        divu t1, t1, t3
        addi t5, t5, 48
        addi t2, t2, -1
        sb   t5, 0(t2)
        bnez t1, cifre
        li   a0, 1
        mv   a1, t2
        la   a2, fine_buf
        sub  a2, a2, t2
        li   a7, 64
        ecall
prossimo:
        addi s0, s0, 1
        j    esterno
fine:   li   a0, 0
        li   a7, 93
        ecall

        .data
buf:    .space 12
fine_buf:
```

Laboratorio L3

1 `0x400` + 7 × 4 = `0x41c`.

2 Perché il puntatore contiene già l'indirizzo dell'elemento: non servono lo scorrimento e la somma che trasformano l'indice in indirizzo. E il confronto con l'indirizzo di fine sostituisce il contatore. Quattro istruzioni per elemento invece di sei.

3 L'undicesima iterazione legge la parola che segue il vettore in memoria, qualunque sia il suo contenuto. Nessun meccanismo lo segnala: il risultato è semplicemente errato, e se il ciclo scrive, altera i dati vicini.

4 Perché RISC-V non ha un'istruzione che moltiplichi e sommi insieme, e le operazioni aritmetiche operano solo sui registri: il prodotto va calcolato in un registro e poi sommato all'accumulatore.

5 $M[2][3]$ e $M[3][3]$ sono sulla stessa colonna, in righe consecutive: distano una riga, $10 \times 4 = 40$ byte. $M[2][3]$ e $M[2][4]$ sono vicini sulla stessa riga: 4 byte.

6 Con `LATO` dispari i due puntatori passano entrambi per l'elemento centrale, che viene sommato due volte. La versione lenta visita ogni elemento una volta sola, e lo somma se si trova su *almeno* una diagonale.

Esercizio L3.1 Il massimo e la sua posizione

Il massimo è 31, in posizione 2. Il confronto `ble` aggiorna il massimo solo se l'elemento è *strettamente* maggiore, quindi a parità resta la prima posizione; con `blt` resterebbe l'ultima, la 4.

```
        .globl _start
        .text
_start: la   t0, v              # p
        addi t1, t0, 40         # fine
        lw   a0, 0(t0)          # max = v[0]
        li   a1, 0              # posizione del massimo
        li   t2, 0              # posizione corrente
ciclo:  addi t0, t0, 4
        addi t2, t2, 1
        beq  t0, t1, fine
        lw   t3, 0(t0)
        ble  t3, a0, ciclo      # non più grande: avanti
        mv   a0, t3             # nuovo massimo
        mv   a1, t2             # e la sua posizione
        j    ciclo
fine:   la   t0, risultato
```

```
        sw   a0, 0(t0)
        sw   a1, 4(t0)
        li   a0, 0
        li   a7, 93
        ecall

        .data
v:      .word 12, -7, 31, 8, 31, 0, 25, -40, 19, 30
risultato: .word 0, 0             # massimo e posizione
```

Esercizio L3.2 Al contrario

Gli elementi indicati dai due puntatori vengono scambiati, e i puntatori si avvicinano; il ciclo termina quando si incontrano o si superano. Con 10 elementi gli scambi sono 5. Il confronto è senza segno perché confronta indirizzi.

```
        .globl _start
        .text
_start: la   t0, v                # p: dal primo elemento
        addi t1, t0, 36           # q: dall'ultimo (9 * 4 byte dopo)
ciclo:  bgeu t0, t1, fine         # finché p < q
        lw   t2, 0(t0)
        lw   t3, 0(t1)
        sw   t3, 0(t0)            # scambio
        sw   t2, 0(t1)
        addi t0, t0, 4            # p avanza
        addi t1, t1, -4           # q arretra
        j    ciclo
fine:   li   a0, 0
        li   a7, 93
        ecall

        .data
v:      .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

Esercizio L3.3 Un esercizio d'esame

Il risultato è 4. Ogni elemento si legge una volta sola: il valore appena letto si conserva in `t2` e all'iterazione successiva funge da `v[i-1]`.

```
        .globl _start
        .text
_start: la   t0, v
        addi t1, t0, 28          # &v[7], l'ultimo
        li   a0, 0               # conta
        lw   t2, 0(t0)           # v[i-1], all'inizio v[0]
ciclo:  addi t0, t0, 4           # for (i = 1; i < 8; i++)
        lw   t3, 0(t0)           # v[i]
        ble  t3, t2, avanti      # if (v[i] > v[i-1])
        addi a0, a0, 1           #     conta++;
avanti: mv   t2, t3              # v[i] sarà il prossimo v[i-1]
        bne  t0, t1, ciclo
        li   a7, 93
        ecall

        .data
v:      .word 5, 8, 8, 3, 9, 12, 1, 4
```

Esercizio L3.4 Somme delle righe

Le somme sono 10, 100 e −10. La formula dell'indirizzo non è necessaria: scorrendo per righe, gli elementi si incontrano nell'ordine in cui sono disposti in memoria, ed è sufficiente un puntatore che avanza di 4.

```
        .globl _start
        .text
_start: la   t0, mat             # scorre la matrice, riga dopo riga
        la   t1, somme           # scorre il vettore delle somme
        li   t2, 3               # righe rimaste
righe:  li   t3, 4               # colonne rimaste
        li   t4, 0               # somma della riga
colonne:
        lw   t5, 0(t0)
        add  t4, t4, t5
        addi t0, t0, 4           # l'elemento dopo è il vicino di destra
        addi t3, t3, -1
        bnez t3, colonne
        sw   t4, 0(t1)
        addi t1, t1, 4
        addi t2, t2, -1
        bnez t2, righe
        li   a0, 0
        li   a7, 93
        ecall

        .data
mat:    .word 1, 2, 3, 4
```

```
        .word 10, 20, 30, 40
        .word -1, -2, -3, -4
somme:  .word 0, 0, 0
```

Esercizio L3.5 La trasposta

Si scambiano solo gli elementi sopra la diagonale, con $j > i$: ciascuno ha il suo compagno sotto la diagonale, e la diagonale resta invariata. La matrice diventa `1 5 9 13 / 2 6 10 14 / 3 7 11 15 / 4 8 12 16`.

```
        .globl _start
        .text
_start: la   s0, mat
        li   s1, 4              # LATO
        li   t0, 0              # i
righe:  addi t1, t0, 1          # j parte da i + 1: sopra la diagonale
colonne:
        bge  t1, s1, prossima
        mul  t2, t0, s1
        add  t2, t2, t1
        slli t2, t2, 2
        add  t2, s0, t2         # &M[i][j]
        mul  t3, t1, s1
        add  t3, t3, t0
        slli t3, t3, 2
        add  t3, s0, t3         # &M[j][i]
        lw   t4, 0(t2)
        lw   t5, 0(t3)
        sw   t5, 0(t2)
        sw   t4, 0(t3)
        addi t1, t1, 1
        j    colonne
prossima:
        addi t0, t0, 1
        blt  t0, s1, righe
        li   a0, 0
        li   a7, 93
        ecall

        .data
mat:    .word  1,  2,  3,  4
        .word  5,  6,  7,  8
        .word  9, 10, 11, 12
        .word 13, 14, 15, 16
```

Esercizio L3.6 Matrice per vettore

Il risultato è $w = (11, -9, 21)$. La matrice si scorre per righe con un solo puntatore; il puntatore a v riparte dall'inizio a ogni riga.

```
        .globl _start
        .text
_start: la   t0, mat            # scorre la matrice
        la   t1, w              # scorre il risultato
        li   t2, 3              # righe rimaste
righe:  la   t3, v              # ogni riga riparte da v[0]
        li   t4, 3
        li   t5, 0              # w[i]
colonne:
        lw   a0, 0(t0)          # M[i][j]
        lw   a1, 0(t3)          # v[j]
        mul  a0, a0, a1
        add  t5, t5, a0
        addi t0, t0, 4
        addi t3, t3, 4
        addi t4, t4, -1
        bnez t4, colonne
        sw   t5, 0(t1)
        addi t1, t1, 4
        addi t2, t2, -1
        bnez t2, righe
        li   a0, 0
        li   a7, 93
        ecall

        .data
mat:    .word 1, 0, 2
        .word 0, 3, -1
        .word 4, 1, 1
v:      .word 5, -2, 3
w:      .word 0, 0, 0
```

Esercizio L3.7 Misurare

Per 8×8 la somma è 520, in 499 cicli con la versione lenta e 84 con quella veloce. Nella versione lenta un elemento fuori dalle diagonali costa 6 istruzioni (il primo confronto, il calcolo e il confronto di $x + y + 1$, l'avanzamento di x e il salto); uno sulla diagonale principale 9, uno sull'altra 12, perché attraversa entrambi i controlli prima di essere sommato. Con 48 elementi fuori diagonale, 8 su ciascuna diagonale, 4 istruzioni per riga e 11 per l'inizializzazione e la terminazione: $48 \times 6 + 8 \times 9 + 8 \times 12 + 8 \times 4 + 11 = 499$.

Laboratorio L4

1 L'indirizzo dell'istruzione che segue la `jal`, PC + 4. La `ret` salta all'indirizzo contenuto in `ra`, che ogni `jal` aggiorna: torna quindi sempre subito dopo l'ultima chiamata.

2 Perché non chiama altre funzioni, quindi `ra` non viene sovrascritto; e se usa solo registri `t` e `a` non ha nulla da salvare.

3 Su `s0`–`s11` e su `sp`, che la funzione chiamata deve restituire intatti. Non può contare sui registri `t` e `a`, né su `ra`.

4 Perché la stessa funzione può essere attiva più volte insieme, come nella ricorsione: ogni attivazione ha bisogno del proprio spazio, e lo stack gliene fornisce uno nuovo a ogni chiamata.

5 Perché l'ABI di RISC-V richiede che `sp` sia sempre multiplo di 16.

6 Al massimo 10 record, uno per livello della catena di chiamate più lunga; le chiamate in tutto sono 177.

Esercizio L4.1 Stampare una stringa

`lunghezza` è una foglia e non salva nulla. `stampa_stringa` chiama `lunghezza`, quindi deve salvare `ra`; e siccome l'indirizzo della stringa le serve dopo la chiamata, lo conserva in `s0`, che a sua volta va salvato.

```
        .globl _start
        .text
_start: la   a0, saluto
        jal  stampa_stringa
        la   a0, nome
        jal  stampa_stringa
        li   a0, 0
        li   a7, 93
        ecall

# lunghezza(a0 = indirizzo): restituisce il numero di caratteri
# prima dello zero finale. Foglia.
lunghezza:
        mv   t0, a0
lu_ciclo:
        lbu  t1, 0(t0)
        beqz t1, lu_fine
        addi t0, t0, 1
        j    lu_ciclo
lu_fine:
        sub  a0, t0, a0           # fine - inizio
```

```
        ret

# stampa_stringa(a0 = indirizzo): scrive la stringa sul terminale.
# Chiama lunghezza, quindi salva ra; l'indirizzo sopravvive in s0.
stampa_stringa:
        addi sp, sp, -16
        sw   ra, 12(sp)
        sw   s0, 8(sp)
        mv   s0, a0
        jal  lunghezza            # a0 = lunghezza
        mv   a2, a0
        mv   a1, s0
        li   a0, 1
        li   a7, 64
        ecall
        lw   s0, 8(sp)
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret

        .data
saluto: .asciz "Ciao, "
nome:   .asciz "Mobius\n"
```

Esercizio L4.2 Potenza

$3^5 = 243$. I record sono al massimo cinque, uno per ogni esponente da 5 a 1: il caso base $e = 0$ ritorna immediatamente, senza costruirne uno.

```
        .globl _start
        .text
_start: li   a0, 3
        li   a1, 5
        jal  potenza              # a0 = 3^5
        li   a7, 93
        ecall

# potenza(a0 = b, a1 = e): restituisce b^e, con e >= 0
potenza:
        bnez a1, po_ric
        li   a0, 1                # b^0 = 1
        ret
po_ric: addi sp, sp, -16
        sw   ra, 12(sp)
        sw   a0, 8(sp)            # b serve dopo la chiamata
        addi a1, a1, -1
        jal  potenza              # a0 = b^(e-1)
        lw   t0, 8(sp)
```

```
        mul  a0, a0, t0          # b * b^(e-1)
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret
```

Esercizio L4.3 Somma ricorsiva

La somma è 108. Il primo elemento si legge prima della chiamata e si conserva nel record; il puntatore al resto del vettore è semplicemente `v + 4`.

```
        .globl _start
        .text
_start: la   a0, v
        li   a1, 6
        jal  somma               # a0 = somma dei 6 elementi
        li   a7, 93
        ecall

# somma(a0 = v, a1 = n): v[0] + somma(&v[1], n - 1); somma(v, 0) = 0
somma:  bnez a1, so_ric
        li   a0, 0
        ret
so_ric: addi sp, sp, -16
        sw   ra, 12(sp)
        lw   t0, 0(a0)           # v[0]
        sw   t0, 8(sp)           # lo conserva sulla pila
        addi a0, a0, 4           # il resto del vettore
        addi a1, a1, -1
        jal  somma
        lw   t0, 8(sp)
        add  a0, a0, t0
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret

        .data
v:      .word 4, 8, 15, 16, 23, 42
```

Esercizio L4.4 Trova l'errore

L'epilogo rilegge anche `a0`, che contiene il risultato, e lo sostituisce con n: la funzione restituisce sempre n. Quindi 2 invece di 1 per $n = 2$, 5 per $n = 5$ (corretto per coincidenza) e 6 invece di 8 per $n = 6$. La correzione consiste nell'eliminare la riga `lw a0, 8(sp)` dall'epilogo, come nella versione della § L4.4.

Esercizio L4.5 Ricerca binaria

Nel vettore `-8, 0, 3, 7, 15, 23, 42, 51, 60, 99` il numero 42 è in posizione 6; un numero assente produce -1. Ogni chiamata dimezza l'intervallo, quindi per 10 elementi sono sufficienti al più quattro livelli.

```
        .globl _start
        .text
_start: la   a0, v
        li   a1, 0
        li   a2, 9
        li   a3, 42              # il valore cercato
        jal  cerca               # a0 = posizione, oppure -1
        li   a7, 93
        ecall

# cerca(a0 = v ordinato, a1 = sinistra, a2 = destra, a3 = x):
# restituisce la posizione di x in v[sinistra..destra], oppure -1
cerca:  ble  a1, a2, ce_ric
        li   a0, -1              # intervallo vuoto: non c'è
        ret
ce_ric: addi sp, sp, -16
        sw   ra, 12(sp)
        add  t0, a1, a2
        srai t0, t0, 1           # centro
        slli t1, t0, 2
        add  t1, a0, t1
        lw   t1, 0(t1)           # v[centro]
        beq  t1, a3, ce_trovato
        blt  a3, t1, ce_sx
        addi a1, t0, 1           # x > v[centro]: cerca a destra
        jal  cerca
        j    ce_esci
ce_sx:  addi a2, t0, -1          # x < v[centro]: cerca a sinistra
        jal  cerca
        j    ce_esci
ce_trovato:
        mv   a0, t0
ce_esci:
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret

        .data
v:      .word -8, 0, 3, 7, 15, 23, 42, 51, 60, 99
```

Esercizio L4.6 Euclide

mcd(1071, 462) = 21. Nella versione qui sotto si salva `ra`, come in ogni funzione che usa `jal`. Ma la chiamata ricorsiva è l'ultima operazione eseguita dalla funzione, e il suo risultato è già il risultato finale: è una *ricorsione in coda*. Al posto di `jal mcd` basterebbe un salto, `j mcd`, e lo stack non sarebbe più necessario: la ricorsione diventa un ciclo. È una trasformazione che i compilatori eseguono automaticamente.

```
        .globl _start
        .text
_start: li   a0, 1071
        li   a1, 462
        jal  mcd                # a0 = mcd(1071, 462)
        li   a7, 93
        ecall

# mcd(a0 = a, a1 = b): Euclide. mcd(a, 0) = a, mcd(a, b) = mcd(b, a % b)
mcd:    bnez a1, mcd_ric
        ret                     # b == 0: il risultato è a, già in a0
mcd_ric:
        addi sp, sp, -16
        sw   ra, 12(sp)
        rem  t0, a0, a1         # a % b
        mv   a0, a1
        mv   a1, t0
        jal  mcd
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret
```

Esercizio L4.7 Un esercizio d'esame

Gli elementi pari sono cinque: 8, −2, 10, 0 e −6. `conta_pari` chiama `pari`, quindi salva `ra`; puntatore, fine e contatore devono sopravvivere alle chiamate e sono conservati in `s0`–`s2`, salvati anch'essi. `pari` è una foglia: l'ultimo bit indica se il numero è dispari, e uno XOR con 1 lo inverte.

```
        .globl _start
        .text
_start: la   a0, v
        li   a1, 8
        jal  conta_pari         # a0 = quanti elementi pari
        li   a7, 93
        ecall

# conta_pari(a0 = v, a1 = n)
```

```
conta_pari:
        addi sp, sp, -16
        sw   ra, 12(sp)
        sw   s0, 8(sp)            # puntatore
        sw   s1, 4(sp)            # fine
        sw   s2, 0(sp)            # c
        mv   s0, a0
        slli s1, a1, 2
        add  s1, s0, s1           # fine = v + n * 4
        li   s2, 0                # c = 0
cp_ciclo:
        beq  s0, s1, cp_fine      # for (i = 0; i < n; i++)
        lw   a0, 0(s0)
        jal  pari                 #     if (pari(v[i]))
        beqz a0, cp_avanti
        addi s2, s2, 1            #         c++;
cp_avanti:
        addi s0, s0, 4
        j    cp_ciclo
cp_fine:
        mv   a0, s2               # return c;
        lw   s2, 0(sp)
        lw   s1, 4(sp)
        lw   s0, 8(sp)
        lw   ra, 12(sp)
        addi sp, sp, 16
        ret

# pari(a0 = x): restituisce 1 se x è pari, 0 altrimenti. Foglia.
pari:   andi a0, a0, 1
        xori a0, a0, 1
        ret

        .data
v:      .word 3, 8, -2, 7, 10, 0, 13, -6
```

Laboratorio L5

1. Il banco dei registri scrive nella prima metà del ciclo e legge nella seconda. Un'istruzione a distanza 3 è in ID mentre quella che scrive è in WB, e legge il valore appena scritto; una a distanza 2 è in ID quando quella che scrive è ancora in MEM, e legge il valore vecchio.
2. Perché il dato di una load è disponibile solo alla fine di MEM, mentre l'istruzione successiva ne ha bisogno all'inizio di EX, un ciclo prima. La propagazione può accorciare il percorso, non anticipare il tempo.

3 Tre cicli. Il salto si decide in MEM, e nel frattempo sono entrate nella pipeline le tre istruzioni che lo seguono in memoria: se il salto è preso, vanno eliminate.

4 Perché il predittore a un bit, all'uscita dal ciclo interno, cambia previsione e sbaglia anche al rientro: due errori per ogni iterazione del ciclo esterno. Quello a due bit, dopo un solo errore, indebolisce la previsione senza cambiarla, e al rientro prevede correttamente.

5 Per righe, ogni miss carica in cache un blocco di quattro parole consecutive, e le tre letture successive sono hit. Per colonne, due letture consecutive cadono in blocchi diversi, e quando il ciclo tornerebbe a usare un blocco, nella cache minuscola è già stato sostituito.

6 Perché un miss legge un blocco intero di quattro parole, e senza accesso a burst ogni parola costa quanto un accesso singolo: 40 cicli per un blocco, contro i 10 di una sola lettura senza cache. La cache torna conveniente quando le parole successive alla prima costano poco.

7 Perché un'interruzione può arrivare appena viene abilitata. Se il gestore aggiorna un contatore che il programma azzera solo dopo, l'azzeramento cancella il lavoro già fatto.

Esercizio L5.1 Prevedere gli stalli

Con la propagazione, 2 stalli: uno fra la prima `lw` e la `add`, uno fra la seconda `lw` e la `sw`, che ha bisogno del dato appena caricato (QtRvSim non dispone di una propagazione dall'uscita della memoria dati all'ingresso della store successiva). La `addi` non attende: il risultato della `add` le viene fornito per propagazione. Con i soli stalli sono 10, due per ogni dipendenza a distanza 1: all'interno della `la` (fra `auipc` e `addi`), fra la `la` e la prima `lw`, fra `lw` e `add`, fra `add` e `addi`, fra la seconda `lw` e la `sw`.

Esercizio L5.2 Riordinare il prodotto scalare

La versione del laboratorio L3 subisce uno stallo per iterazione: 71 cicli e 5 stalli. Spostando le tre istruzioni di aggiornamento fra le load e la `mul` gli stalli spariscono e i cicli scendono a 66.

```
        .globl _start
        .text
_start: la   t0, va              # puntatore al vettore A
        la   t1, vb              # puntatore al vettore B
        li   t2, 5               # elementi rimasti
        li   a0, 0               # accumulatore
ciclo:  lw   t3, 0(t0)           # A[i]
```

```
        lw   t4, 0(t1)          # B[i]
        addi t0, t0, 4          # tre istruzioni indipendenti
        addi t1, t1, 4          # fra la lw e la mul che usa t4
        addi t2, t2, -1
        mul  t5, t3, t4
        add  a0, a0, t5
        bnez t2, ciclo
        li   a7, 93             # A . B come codice di uscita
        ecall

        .data
va:     .word 1, 2, 3, 4, 5
vb:     .word 6, 7, 8, 9, 10
```

Esercizio L5.3 Il conto dei salti

Sul ciclo singolo si eseguono 4 istruzioni prima del ciclo, 3 per iterazione e l'`ebreak`: $4 + 3 \times 100 + 1 = 305$ cicli. Sulla pipeline si aggiungono 3 cicli per ognuno dei 99 salti presi e 3 cicli per riempire la pipeline: $305 + 297 + 3 = 605$.

Esercizio L5.4 Un bit o due

Il predittore a un bit sbaglia due volte per ogni iterazione del ciclo esterno, all'uscita dal ciclo interno e al rientro; quello a due bit una volta sola, all'uscita. I totali sono gli stessi della versione con 4 iterazioni, 22 e 13 errori (più o meno gli errori iniziali, finché i predittori non si adattano): il numero di errori dipende da quante volte si entra e si esce dal ciclo interno, non da quante iterazioni esegue.

Esercizio L5.5 Blocchi più grandi

Con blocchi di b parole, una scansione in ordine produce un miss ogni b letture: tasso di hit 0, 50, 75 e 87,5% per $b = 1$, 2, 4 e 8. Con blocchi di una parola la cache non sfrutta la località spaziale, e in questo programma, che legge ogni dato una volta sola, non porta alcun vantaggio.

Esercizio L5.6 Otto blocchi, quattro cache

Solo la cache completamente associativa trova degli hit, 48 su 64: gli 8 blocchi di una colonna (quattro colonne consecutive usano gli stessi 8 blocchi) vi trovano posto tutti. Nelle altre l'insieme di un blocco dipende dalla sua posizione: le righe distano 2 blocchi, e le 8 righe di una colonna si concentrano in pochi insiemi. Con 2 insiemi da 4 vie sono associate tutte allo stesso insieme, 8 blocchi per 4 posti; con 4 insiemi da 2 vie in due insiemi, con 8 insiemi a mappatura diretta in quattro. In tutti i casi i blocchi si sostituiscono a vicenda prima di essere riusati. La cache a mappatura diretta più piccola che trova

hit ha 16 insiemi, cioè 256 byte, tanto quanto la matrice: le 8 righe sono allora associate a 8 insiemi diversi.

Esercizio L5.7 Un semaforo

I colori sono memorizzati in un piccolo vettore; un indice da 0 a 2 li scorre, e un ciclo vuoto funge da pausa. Con 200 000 iterazioni la pausa è di 400 000 istruzioni.

```
        .equ SPILED,   0xffffc100
        .equ LED_RGB1, 0x10
        .globl _start
        .text
_start: li   s0, SPILED
        la   s1, colori          # i tre colori, uno dopo l'altro
        li   s2, 0               # quale colore (0, 1, 2)
ciclo:  slli t0, s2, 2
        add  t0, s1, t0
        lw   t0, 0(t0)           # colori[s2]
        sw   t0, LED_RGB1(s0)
        li   t1, 200000          # attesa: un ciclo che non fa nulla
attendi:
        addi t1, t1, -1
        bnez t1, attendi
        addi s2, s2, 1           # prossimo colore
        li   t2, 3
        blt  s2, t2, ciclo
        li   s2, 0               # dopo il rosso, di nuovo verde
        j    ciclo

        .data
colori: .word 0x00ff00, 0xffc000, 0xff0000   # verde, giallo, rosso
```

Esercizio L5.8 Contare con le interruzioni

Il programma principale si limita a eseguire un ciclo vuoto: ogni carattere scritto nella casella *Input* interrompe il ciclo, e il gestore aggiorna il conteggio e la fila di LED. Il registro `s1`, con l'indirizzo delle periferiche, si prepara prima di abilitare le interruzioni, perché il gestore lo usa.

```
        .equ SERIALE,  0xffffc000
        .equ RX_STATO, 0x00
        .equ RX_DATO,  0x04
        .equ SPILED,   0xffffc100
        .equ LED_LINE, 0x04
        .globl _start
        .text
```

```
_start: li   s0, 0                # caratteri ricevuti
        li   s1, SPILED
        sw   zero, LED_LINE(s1)
        la   t0, gestore
        csrw mtvec, t0
        li   t0, SERIALE
        li   t1, 2
        sw   t1, RX_STATO(t0)     # interruzione alla ricezione
        li   t1, 0x10000
        csrrs zero, mie, t1
        csrrsi zero, mstatus, 8
attesa: j    attesa               # tutto il lavoro lo fa il gestore

gestore:
        li   t2, SERIALE
        lw   t3, RX_DATO(t2)      # legge: la richiesta si spegne
        addi s0, s0, 1
        sw   s0, LED_LINE(s1)     # il conteggio sui LED
        mret
```

Laboratorio L6

1 Si assegnano i registri agli argomenti e al risultato, come prescrive l'ABI; si assegnano le variabili locali a registri temporanei, o a registri `s` salvati nel prologo se la funzione ne chiama altre; si trasformano gli indici in puntatori, con l'indirizzo di fine; si traducono `if` e cicli con gli schemi noti, negando la condizione nel salto; si scrive il risultato in `a0` e si ritorna con `ret`.

2 Perché senza ottimizzazioni il compilatore traduce ogni riga separatamente, senza tener conto dei valori già presenti nei registri: la memoria è l'unico posto in cui una variabile ha sicuramente il suo valore all'inizio e alla fine di ogni riga. È anche ciò che occorre a un debugger, che può mostrare e modificare qualunque variabile a qualunque riga.

3 Con `slt`, che scrive `1` se `x < v[i]` e `0` altrimenti: il risultato si somma a `c`, e sommare zero equivale a non incrementare.

4 Perché `-O2` riordina le istruzioni in modo che nessuna usi il dato di una load appena eseguita: i 1000 stalli load-use spariscono, e con loro 1000 cicli.

5 Perché sulla pipeline senza predittore ogni salto preso costa tre cicli in più, e srotolando di otto i salti presi passano da 2000 a 250: si risparmiano 3500 istruzioni e $1750 \times 3 = 5250$ cicli di salti, 8750 in tutto. Con il predittore

i salti costano già poco, e il guadagno si riduce alle 3500 istruzioni.

6 È una nota che l'assemblatore lascia nel file oggetto accanto a un'istruzione che usa un simbolo di indirizzo ancora ignoto, per esempio una funzione definita in un altro file. È il linker a risolverla, quando assegna gli indirizzi definitivi e completa il campo; nel collegamento dinamico, alcune si risolvono durante l'esecuzione.

7 Prima: il sistema operativo carica l'eseguibile e crea il processo, prepara lo stack con gli argomenti della riga di comando e salta al punto d'ingresso; il codice di avvio prepara `gp`, inizializza la libreria e chiama `main(argc, argv)`. Dopo: il valore restituito da `main` diventa il codice di uscita e viene passato a `exit`, che nella libreria reale svuota anche i buffer di `printf` prima di chiedere al sistema operativo di terminare il processo.

Esercizio L6.1 Un esercizio d'esame

`massimo` non chiama altre funzioni, quindi `m` è assegnato a un temporaneo, `t0`. L'indice diventa un puntatore che parte da `&v[1]` e si ferma all'indirizzo di fine $v + 4n$; il confronto fra indirizzi è senza segno. La `ble` salta l'aggiornamento quando $v[i] \leq m$, la condizione negata dell'`if`. Con il vettore di prova il risultato è 14; con $n = 1$ il ciclo non esegue alcuna iterazione e la funzione restituisce `v[0]`.

```
        .globl _start
        .text
_start: la   a0, v
        li   a1, 8
        jal  massimo
        li   a7, 93
        ecall

# int massimo(int *v, int n): v in a0, n in a1; m in t0
massimo:
        lw   t0, 0(a0)            # m = v[0]
        slli t1, a1, 2
        add  t1, a0, t1           # fine
        addi a0, a0, 4            # i = 1
ma_ciclo:
        bgeu a0, t1, ma_fine      # for (; i < n; i++)
        lw   t2, 0(a0)
        ble  t2, t0, ma_avanti    #     if (v[i] > m)
        mv   t0, t2               #         m = v[i];
ma_avanti:
        addi a0, a0, 4
        j    ma_ciclo
```

```
ma_fine:
        mv   a0, t0               # return m;
        ret

        .data
v:      .word -3, 14, 7, 14, -20, 9, 0, 11
```

Esercizio L6.2 Contare un carattere

Le a sono 7. Un carattere si legge con `lbu`: nell'ABI di RISC-V il tipo `char` è senza segno, ed è proprio l'istruzione che usa il compilatore. Il ciclo ha due salti condizionati: la `beqz`, che esce al terminatore, e la `bne`, che salta l'incremento quando il carattere è diverso da `c`.

```
        .globl _start
        .text
_start: la   a0, frase
        li   a1, 97               # 'a'
        jal  conta
        li   a7, 93
        ecall

# int conta(char *s, char c): s in a0, c in a1; n in t0
conta:  li   t0, 0                # n = 0
co_ciclo:
        lbu  t1, 0(a0)            # *s
        beqz t1, co_fine          # while (*s != '\0')
        bne  t1, a1, co_avanti    #     if (*s == c)
        addi t0, t0, 1            #         n++;
co_avanti:
        addi a0, a0, 1            #     s++;
        j    co_ciclo
co_fine:
        mv   a0, t0               # return n;
        ret

        .data
frase:  .asciz "la casa sulla cascata"
```

Esercizio L6.3 Una funzione ricorsiva

La somma delle cifre di 98765 è 35. Il caso base non costruisce nessun record di attivazione: *n* è già in `a0`, ed è sufficiente `ret`. Il caso ricorsivo chiama, quindi salva `ra`; e `n % 10` deve sopravvivere alla chiamata, quindi va salvato nello stack (in alternativa, in `s0`, salvato nel prologo). Il record è di 16 byte perché l'ABI richiede che `sp` sia multiplo di 16. `rem` e `div` sono le versioni con

segno, come le operazioni del C su `int`.

```
        .globl _start
        .text
_start: li   a0, 98765
        jal  somma_cifre
        li   a7, 93
        ecall

# int somma_cifre(int n): n < 10 ? n : n % 10 + somma_cifre(n / 10)
somma_cifre:
        li   t0, 10
        blt  a0, t0, sc_base      # if (n < 10) return n;
        addi sp, sp, -16
        sw   ra, 12(sp)
        rem  t1, a0, t0           # n % 10
        sw   t1, 8(sp)            # serve dopo la chiamata
        div  a0, a0, t0           # n / 10
        jal  somma_cifre
        lw   t1, 8(sp)
        add  a0, a0, t1
        lw   ra, 12(sp)
        addi sp, sp, 16
sc_base:
        ret
```

Esercizio L6.4 Che cosa calcola?

`t0` parte da 0 e accumula, `t1` parte da 1 e cresce di 1; il ciclo esce quando $t1 > a0$, la negazione di $t1 \leq a0$, con un confronto con segno; il corpo somma $t1 \times t1$. La funzione restituisce la somma dei quadrati da 1 a n, e $f(5) = 1 + 4 + 9 + 16 + 25 = 55$:

```
int f(int n) {
    int s = 0;
    for (int i = 1; i <= n; i++)
        s += i * i;
    return s;
}
```

Esercizio L6.5 E questa?

La funzione salva `ra` e `s0` perché chiama, e conserva n in `s0`, che la chiamata non può cambiare. Restituisce la somma dei numeri da 1 a n, e $g(5) = 15$:

```
int g(int n) {
    if (n == 0)
        return 0;
    return g(n - 1) + n;
}
```

Quando $n = 0$ il risultato è zero, e `a0`, che contiene n, vale già zero: la `beqz` salta direttamente all'epilogo, e `a0` torna com'è. Con n negativo, invece, la ricorsione non termina mai: ogni chiamata decrementa n di uno senza mai raggiungere lo zero, finché lo stack non invade il resto della memoria. Lo stesso accadrebbe in C.

Esercizio L6.6 Il compilatore al lavoro

Ecco la funzione prodotta da GCC 13 con `-O2`, con un programma principale per provarla:

```
        .globl _start
        .text
_start: la   a0, v
        li   a1, 8
        jal  massimo
        li   a7, 93
        ecall

# massimo come lo scrive GCC 13 con -O2 (etichette rinominate)
massimo:
        li   a5, 1
        mv   a3, a0              # a3 = v
        lw   a0, 0(a0)           # m = v[0], già in a0
        ble  a1, a5, fine        # n <= 1: il ciclo non si esegue
        slli a1, a1, 2
        addi a5, a3, 4           # a5 = &v[1]
        add  a3, a3, a1          # a3 = &v[n]
ciclo:  lw   a4, 0(a5)
        addi a5, a5, 4           # avanza prima di usare a4
        bge  a0, a4, salta       # if (v[i] > m)
        mv   a0, a4              #     m = v[i];
salta:  bne  a5, a3, ciclo       # il test è in fondo
fine:   ret

        .data
v:      .word -3, 14, 7, 14, -20, 9, 0, 11
```

L'`if` resta un salto, la `bge`. L'accorgimento di `conta_maggiori` qui non funziona: non c'è da sommare 0 o 1, ma da scegliere fra due valori, e RV32IM non ha un'istruzione per farlo senza saltare. Con l'estensione Zbb, che QtRvSim

non ha, sì: compilando con `-march=rv32im_zbb`, GCC usa `max a0,a0,a4`, e il ciclo non ha più salti nel corpo.

Il caso $n = 1$ è trattato da un controllo all'inizio, `ble a1,a5,fine`: se $n \leq 1$ la funzione restituisce `v[0]`, che è già in `a0`. Superato il controllo, il ciclo esegue almeno un'iterazione, e il compilatore colloca il controllo in fondo, con una `bne` verso l'indirizzo di fine. Altri due dettagli: `m` risiede direttamente in `a0`, e l'avanzamento del puntatore si trova fra la `lw` e la `bge`, che altrimenti attenderebbe il dato. Sul vettore di prova la versione del compilatore esegue 46 istruzioni contro 51, e sulla pipeline con propagazione costa 91 cicli senza stalli, contro 109 con 7 stalli.

Esercizio L6.7 Srotolare a mano

Se gli elementi sono dispari, il primo si tratta separatamente prima del ciclo; poi il ciclo esegue $n/2$ iterazioni, ciascuna su due elementi. Il risultato resta 130. Le load si trovano tutte all'inizio dell'iterazione e le `mul` seguono, così nessuna istruzione attende un dato, nemmeno nel caso dispari, dove le due `addi` separano le load dalla `mul`.

```
        .globl _start
        .text
_start: la   t0, va              # puntatore al vettore A
        la   t1, vb              # puntatore al vettore B
        li   t2, 5               # elementi
        li   a0, 0               # accumulatore
        andi t3, t2, 1           # il numero di elementi è dispari?
        beqz t3, coppie
        lw   t3, 0(t0)           # sì: il primo elemento va da solo
        lw   t4, 0(t1)
        addi t0, t0, 4           # in mezzo, per non aspettare t4
        addi t1, t1, 4
        mul  a0, t3, t4
coppie: srli t2, t2, 1           # coppie di elementi da fare
        beqz t2, fine
ciclo:  lw   t3, 0(t0)           # A[i]
        lw   t4, 0(t1)           # B[i]
        lw   t5, 4(t0)           # A[i+1]
        lw   t6, 4(t1)           # B[i+1]
        mul  t3, t3, t4
        mul  t5, t5, t6
        add  a0, a0, t3
        add  a0, a0, t5
        addi t0, t0, 8           # due elementi per giro
        addi t1, t1, 8
        addi t2, t2, -1
        bnez t2, ciclo
```

```
fine:   li   a7, 93              # A . B come codice di uscita
        ecall

        .data
va:     .word 1, 2, 3, 4, 5
vb:     .word 6, 7, 8, 9, 10
```

Sulla pipeline senza predittore il programma passa da 71 a 50 cicli: i salti presi scendono da 4 a 1, gli stalli da 5 a 0, le istruzioni da 51 a 44. Con soli cinque elementi incidono maggiormente le istruzioni fuori dal ciclo; con 100 elementi si passa da 1211 a 768 cicli: per ogni elemento 8 istruzioni, uno stallo e i tre cicli di un salto preso, 12 in tutto, diventano 6 istruzioni e mezzo salto, 7,5 cicli.

APPENDICE

D

Letture consigliate

Questo libro è un inizio. Ecco i testi per proseguire lo studio, divisi per scopo. Molti dei testi in inglese costituiscono il riferimento internazionale della materia.

D.1 I manuali

David A. Patterson, John L. Hennessy, *Struttura e progetto dei calcolatori. Progettare con RISC-V*, Zanichelli, seconda edizione italiana. Originale: *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, Morgan Kaufmann, seconda edizione, 2020.

Il testo di riferimento dei corsi di architettura, e la fonte di molti esempi classici. Copre tutto il programma, con più dettagli su virgola mobile, prestazioni e processori paralleli.

Sarah L. Harris, David Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2021.

Procede dalle porte logiche fino al processore RISC-V, descritto anche in un linguaggio per l'hardware. Ideale per chi vuole realizzare concretamente il datapath degli stati 0x2–0x7, per esempio su una FPGA.

John L. Hennessy, David A. Patterson, *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann; sesta edizione 2017, settima edizione 2025, con Christos Kozyrakis.

Il seguito avanzato del primo: pipeline profonde, esecuzione fuori ordine, gerarchie di memoria, multiprocessori e acceleratori, sempre con un approccio quantitativo.

D.2 RISC-V

David Patterson, Andrew Waterman, *The RISC-V Reader: An Open Architecture Atlas*, Strawberry Canyon, 2017.

Un libro sottile scritto da due dei padri di RISC-V: spiega l'ISA istruzione per istruzione e, soprattutto, il perché di ogni scelta.

RISC-V International, *The RISC-V Instruction Set Manual*, Volume I (*Unprivileged Architecture*) e Volume II (*Privileged Architecture*), gratuiti su `riscv.org`.

La specifica ufficiale. È più leggibile di quanto si pensi, ed è piena di note che spiegano le scelte dei progettisti. Gli esempi di Sv32, delle istruzioni atomiche e delle interruzioni di questo libro derivano da qui.

D.3 Approfondimenti

William A. Wulf, Sally A. McKee, *Hitting the Memory Wall: Implications of the Obvious*, ACM SIGARCH Computer Architecture News, 1995.

Poche pagine che hanno dato un nome al divario fra processori e memoria (stati 0x6 e 0xd).

Gene M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*, AFIPS Spring Joint Computer Conference, 1967.

L'articolo, senza formule, da cui nasce la legge di Amdahl (stato 0xf).

John L. Hennessy, David A. Patterson, *A New Golden Age for Computer Architecture*, Communications of the ACM, febbraio 2019.

La conferenza per il premio Turing dei due autori: la fine della legge di Moore, le architetture specializzate e il ruolo delle ISA aperte come RISC-V.

Michael E. Thomadakis, *The Architecture of the Nehalem Processor and Nehalem-EP SMP Platforms*, Texas A&M University, 2011.

Il rapporto tecnico da cui provengono i dati su Nehalem degli stati da 0xc a 0xf.

D.4 Per il piacere di leggere

Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Microsoft Press, 1999; seconda edizione 2022.

Dal codice Morse a un processore completo, una porta logica alla volta, senza dare nulla per scontato.

Tracy Kidder, *The Soul of a New Machine*, Little, Brown, 1981.

Il racconto, premiato con il Pulitzer, di una squadra di ingegneri che progetta un calcolatore a tempo di record. Si legge come un romanzo.

Richard W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*, Gordon and Breach, 1997; ristampa Stripe Press, 2020.

Le lezioni dell'inventore dei codici dello stato 0xd su come si conduce la ricerca, e su come si sceglie un problema importante.



Glossario

I termini italiani usati nel libro, con l'equivalente inglese che si trova nei manuali, nelle specifiche e nei simulatori, e lo stato (o il laboratorio) in cui vengono spiegati. Segue l'elenco delle sigle.

Termini

abilitazioni di byte *byte enable* 0x6
accesso diretto alla memoria *direct memory access* 0x10
aggiornamento *write update* 0xf
allocazione in scrittura *write allocate* 0xd
annullamento *flush* 0xb
assemblatore *assembler* L0
avvio *boot* 0x0
banco dei registri *register file* 0x1
banda *bandwidth* 0xd
blocco, linea *block, line* 0xd
buffer delle destinazioni *branch target buffer* 0xb
buffer di scrittura *write buffer* 0xd
cache a mappatura diretta *direct mapped cache* 0xd
cache associativa a insiemi *set associative cache* 0xd
cache completamente associativa *fully associative cache* 0xd
calcolatore a un'istruzione *one instruction set computer* 0xc
catena di compilazione *toolchain* L0
collegamento dinamico *dynamic linking* L6
collegamento statico *static linking* L6
compilatore *compiler* L0
conflitto *hazard* 0x9
controllo ciclico *polling* 0x10
controllore delle interruzioni *interrupt controller* 0x10
corsa critica *race condition* 0xf
corsia *lane* 0x10
decodifica *instruction decode* 0x3
decodificatore *decoder* 0x3
dipendenza *dependence* 0xa
emissione multipla *multiple issue* 0xc
emulatore *emulator* L0
epilogo *epilogue* L4
errore transitorio *soft error* 0xd
esecuzione *execute* 0x4
eseguibile *executable* L0
falsa condivisione *false sharing* 0xf
funzione foglia *leaf function* L4
gestore *handler* 0xb
insieme di lavoro *working set* 0xe
interruzione *interrupt* 0xb
invalidazione *write invalidate* 0xf

I/O mappato in memoria *memory-mapped I/O* 0x10
latenza *latency* 0x9
lucchetto *lock* 0xf
maschera delle interruzioni *interrupt mask* 0x10
memoria interfogliata *interleaved memory* 0xd
mezza parola *halfword* 0x1
miss obbligato, di capacità, di conflitto *compulsory, capacity, conflict miss* 0xd
modello di memoria *memory consistency model* 0xf
muro della memoria *memory wall* 0x6
numero di pagina virtuale, fisica *virtual, physical page number* 0xe
pagina fisica *frame* 0xe
parola *word* 0x1
parola critica per prima *critical word first* 0xd
penalità di miss *miss penalty* 0xd
periferica *peripheral, device* 0x10
posizionamento (disco) *seek* 0x10
prelievo *fetch* 0x2
prelievo anticipato *prefetching* 0xd
prenotazione *reservation* 0xf
prologo *prologue* L4
propagazione *forwarding, bypassing* 0xa
puntatore *pointer* L3
punto d'ingresso *entry point* L0
record di attivazione *activation record, stack frame* L4
registri del dispositivo *device registers* 0x10
riavvio anticipato *early restart* 0xd
richiesta di proprietà *request for ownership* 0xf
ridenominazione dei registri *register renaming* 0xc
rilassamento *linker relaxation* L6
rilocazione *relocation* L6
ritiro *retirement, commit* 0xc
salto condizionato *branch* 0x1
scambio condizionato *compare and swap* 0xf
scorrimento *shift* 0x1
scrittura (fase) *write back* 0x7
scrittura differita *write-back* 0xd
scrittura immediata *write-through* 0xd
sezione critica *critical section* 0xf
shifter a barile *barrel shifter* 0x5
simulatore *simulator* L0
sindrome *syndrome* 0xd
sommatore a propagazione del riporto *ripple carry adder* 0x5
spiazzamento *offset* 0x1
srotolamento dei cicli *loop unrolling* 0xc
stack (pila) *stack* L4
stallo, bolla *stall, bubble* 0xa
tabella della storia dei salti *branch history table* 0xb
tabella delle pagine *page table* 0xe
tasso di hit, di miss *hit rate, miss rate* 0xd
tempo medio di accesso *average memory access time* 0xd
transazione *bus transaction* 0x10
unità di rilevamento dei conflitti *hazard detection unit* 0xa
vie *ways* 0xd
voce della tabella delle pagine *page table entry* 0xe

Sigle

ABI *application binary interface* 0x0
ALU *arithmetic logic unit* 0x0
AMAT *average memory access time* 0xd
AMO *atomic memory operation* 0xf
API *application programming interface* 0x0
BHT *branch history table* 0xb
BTB *branch target buffer* 0xb

CISC *complex instruction set computer* 0x1
CPI *cycles per instruction* 0x7
CPU *central processing unit* 0x0
CSR *control and status register* 0xb
CU *control unit* 0x0
DDR *double data rate* 0xd
DMA *direct memory access* 0x10
DRAM *dynamic random access memory* 0x6
ECC *error correcting code* 0xd
ELF *executable and linkable format* L0
GCC *GNU compiler collection* L0
IOMMU *input-output memory management unit* 0x10
IPC *instructions per cycle* 0xc
ISA *instruction set architecture* 0x0
LRU *least recently used* 0xd
MESI *modified, exclusive, shared, invalid* 0xf
NRU *not recently used* 0xd
NUMA *non-uniform memory access* 0xf
PCH *platform controller hub* 0x10
PLIC *platform-level interrupt controller* 0x10
PPN *physical page number* 0xe
PTE *page table entry* 0xe
RARS *RISC-V assembler and runtime simulator* L0
RAS, CAS *row, column access strobe* 0xd
RAW, WAR, WAW *read after write, write after read, write after write* 0xa, 0xc
RISC *reduced instruction set computer* 0x1
RVWMO *RISC-V weak memory ordering* 0xf
SDRAM *synchronous DRAM* 0xd
SECDED *single error correction, double error detection* 0xd
SMT *simultaneous multithreading* 0xc
SRAM *static random access memory* 0x6
SSD *solid-state drive* 0x10
TLB *translation lookaside buffer* 0xe
VIPT *virtually indexed, physically tagged* 0xe
VLIW *very long instruction word* 0xc
VPN *virtual page number* 0xe
WSL *Windows subsystem for Linux* L0



Ringraziamenti

Se sei arrivato fin qui, grazie. Spero che queste pagine ti abbiano aiutato a capire un po' meglio come funziona la macchina simbolo della terza rivoluzione industriale, e che ti abbiano trasmesso almeno una parte della passione che ho per questo mondo. Spero anche che i retroscena storici comparsi qua e là lungo il libro ti abbiano divertito quanto hanno divertito me mentre li cercavo.

Non sono un professore, né un professionista del settore, e non ho ancora una laurea. Questo libro è nato in modo semplice: alla fine del corso ho voluto riordinare i miei appunti e, davanti alla mole di pagine accumulate fra lezioni e approfondimenti personali, ho capito che potevano diventare qualcosa di più. *STATI* è il risultato.

Ringrazio il docente del corso di Architettura dei Calcolatori dell'Università di Genova, dalle cui lezioni tutto è cominciato.

Ringrazio Garre, che si è prestato, e si presta tuttora, a fare da cavia per queste pagine mentre prendevano forma.

E grazie a te, lettore, per aver dato fiducia al libro di uno sconosciuto che si firma con uno pseudonimo un po' goliardico. I miei contatti sono nelle prime pagine: scrivimi per dirmi che cosa ne pensi, che cosa cambieresti e, se ti va, se hai superato l'esame e con quale voto.

Buono studio a tutti.

MOBIUS

«Ciò che non posso creare, non lo capisco.»

What I cannot create, I do not understand.
RICHARD FEYNMAN, SULLA SUA LAVAGNA, 1988

«Il modo migliore per predire il futuro è inventarlo.»

The best way to predict the future is to invent it.
ALAN KAY, 1971

«Parlare costa poco. Mostrami il codice.»

Talk is cheap. Show me the code.
LINUS TORVALDS, 2000

«Possiamo vedere solo poco più avanti, ma vediamo molte cose che restano da fare.»

We can only see a short distance ahead, but we can see plenty there that needs to be done.
ALAN TURING, *COMPUTING MACHINERY AND INTELLIGENCE*, 1950



Indice analitico

accelerazione, 112
accesso diretto alla memoria, 219
allineamento, 21
allocazione in scrittura, 171
ALU, 4, 56
ALUCtl, 56
ALUOp, 57
AMO, 204
annullamento (flush), 135
antidipendenza, 151
assemblatore, 228
astrazione, 8

banchi di memoria, 176
banco dei registri (register file), 19, 46
banda, 212
big-endian, 20
bit, 309
bit di parità, 177
bit di validità, 164
bit dirty, 170
blocchi, 163
bolla, 123
buffer delle destinazioni (BTB), 140
buffer di scrittura, 170
bus, 11
bus asincrono, 213
bus sincrono, 213

cablato, 103
cache
    inclusiva, 174
cache a mappatura diretta, 164
cache associativa a insiemi, 168
cache completamente associativa, 168
carry lookahead, 71
catena di compilazione, 228
ciclo di clock, 38
ciclo singolo, 36
circuito combinatorio, 36
CISC, 15
clock, 38
coerente, 200
collegamento dinamico, 304
collegamento statico, 304
collo di bottiglia di von Neumann, 6
compilatore, 228
complemento a 2, 311
conflitto, 117
    load-use, 127
    sui dati, 122
consistenza, 200

controllo (control path), 36
controllo ciclico, 217
controllore della cache, 163
core, 198
corsa critica, 204
CPU, 5
CU, 4

datapath, 36
decodifica (decode), 12, 45
decodificatore, 47
dipendenza, 122
    di uscita, 151
direttiva, 30
distanza di Hamming, 177
divisione con ripristino, 74
DRAM, 85

ecall, 29
ECC (memoria), 180
eccezione, 141
    precisa, 142
elemento di stato, 37
ELF, formato, 228
emissione multipla, 147
    statica, 148
emulatore, 228
epilogo, 274
esadecimale, 310
esecuzione (execute), 55
esecuzione fuori ordine, 150
esecuzione speculativa, 151
eseguibile, 228
estensione del segno, 22
etichetta, 30

falsa condivisione, 203
fence (istruzione), 206
flip-flop, 37
formato di istruzione, 26
frame pointer, 276
frequenza, 38
fronte di salita, 38
full adder, 68
funzione foglia, 272

generatore degli immediati, 48
gerarchia, 162

handshake, 213
hit, 163

I/O mappato in memoria, 216
IEEE 754, 75
immediato, 22
ImmSel, 48
indice, 164
indirizzi fisici, 184
indirizzi virtuali, 184
interfaccia, 8
interoperabilità, 8
interruzione, 141, 217
invalidazione, 201
IPC, 147

jump and link, 24

latenza, 111, 212
legacy, 17
legge di Amdahl, 198
linker, 228
little-endian, 20
load/store, 16
località spaziale, 163
località temporale, 163
logica dei salti, 61
LRU (least recently used), 170
lucchetto, 205

macchina a stati finiti, xiii
macchina virtuale, 8

maschera, 312
memoria dati, 82
memoria istruzioni, 40
MemWrite, 82
MESI (protocollo), 202
MESIF (protocollo), 207
micro-operazione, 17
microarchitettura, 10
microprogramma, 104
miss, 163
    tre C, 171
modello Harvard, 6
multiplexer, 42
multiprogrammazione, 184
multithreading, 153

notazione posizionale, 309
NRU (not recently used), 170
numero di pagina virtuale, 185

offset, 164
OISC, 156
Op1Sel, 58
Op2Sel, 58
overflow, 62, 311

page fault, 186, 190
pagine, 185
parola, 20
partizioni, 184
PC, 12, 39
PCSel, 42
penalità
    di un salto, 134
penalità di miss, 163
percorso critico, 94
periferiche, 212
periodo, 38
pipeline, 110
politica di sostituzione, 170
potenza, muro della, 147
prelievo (fetch), 12
previsione dei salti, 138
    dinamica, 139
privilegio, livelli di, 29
processo, 184
processore, 5
program counter, 12
programma, 184
programma memorizzato, 6
prologo, 274
propagazione, 124
pseudo-istruzioni, 25
puntatore, 264
punto d'ingresso, 228

RAS e CAS, 175
record di attivazione, 274
refresh, 176
registri del dispositivo, 216
registro, 37
registro base, 184
registro di pipeline, 113
registro intermedio, 98
RegWriteEn, 47
reset, 3
ridenominazione dei registri, 152
rilassamento, 303
rilocazione, 303
RISC, 16
RISC-V, 18
ritardo di propagazione, 37

salti, 23
salto condizionato (branch), 23
salto ritardato, 137
satp (registro), 189
scorrimento (shift), 23
scrittura (write back), 89
scrittura differita, 170

scrittura immediata, 170
SDRAM, 176
SECDED, 180
shifter a barile, 70
simulatore, 227
sindrome, 179
snooping, 201
sommatore a propagazione del riporto, 68
sommatore dei salti, 60
SRAM, 85
srotolamento dei cicli, 148
stack, 274
stallo, 123
stato, 13
superscalari, 150
Sv32, 187

tabella delle pagine, 185
tag, 164
tasso di miss, 163
tempo medio di accesso, 172
thrashing, 190
thread, 198
throughput, 111
TLB (translation lookaside buffer), 191
transazione, 213
transizione, xiii

unità
  di rilevamento dei conflitti, 127
unità di controllo, 50

VIPT (cache), 193
virtualizzazione, 8

WBSel, 90

zero, 56